The Ultimate React Course 2024: React, Next.js, Redux & More

Section 1: Welcome, Welcome, Welcome!

Course Roadmap and Projects

1. We can build any application we can imagine
2. We can become a skilled React developer
3. Overview
    1. Part 01: Fundamentals
        1. Components
        2. JSX
        3. Props
        4. State
        5. Forms
    2. Part 02: Intermediate
        1. Data fetching
        2. useEffect hook
        3. Custom hooks
        4. How React works behind the scenes
    3. Part 03: Advanced
        1. Reducers
        2. Performance optimization
        3. SPA
            1. React router
        4. Redux
        5. Modern Redux toolkit
    4. Part 04: Professional Dev
        1. Two professional applications
            1. Industry standard libraries and tools
                1. Redux
                2. Tailwind
                3. React Query
                4. Supabase
                5. ...

Building Our First React App!

1. Small React App (tiny app)
    1. Fundamental ingredients that all React apps are built upon
2. We don't need a development environment
    1. codesandbox.io
    2. react.new
        1. A new app is created in coesandbox.io
3. Code:

```
import { useEffect, useState } from "react";

export default function App() {
  const [advice, setAdvice] = useState(""); // returns an array
  const [count, setCount] = useState(0);
  // [value, setterFunction]
  // setterFunction - used to update the state

  async function getAdvice() {
    const res = await fetch("https://api.adviceslip.com/advice");
    const data = await res.json();
    console.log(data);
    setAdvice(data.slip.advice);
    setCount((c) => c + 1);
  }

  useEffect(function () {
    getAdvice();
  }, []);
  // Function that we want to get executed at the beginning (when the component is loaded)
  // [] - dependency array

  return (
    <div>
      <h1>Alhamdulillah!</h1>
      <h2>{advice}</h2>
      <button onClick={getAdvice}>Get advice</button>
      <Message count={count} />
    </div>
  );
}

function Message(props) {
  // Uppercase is convention for component
  return (
    <p>
      You have read <strong>{props.count}</strong> pieces of advice
    </p>
  );
}
```

Watch Before You Start!

1. Some quick considerations before we start...
    1. This course is for everyone! So please don't write a bad review.
    2. To make the course perfect for YOU: rewatch lectures, jumpt to other sections, watch the source with slower or faster speed, or ask questions
2. Don't have to watch the entire course
3. Don't worry about best-practices
4. Make sure to understand the earlier sections before moving to the subsequent sections
    1. Review the notes
    2. Review the code
    3. Review the project
    4. Write your own code for practice
5. Have fun while coding

Read Before You Start!

1. Starter code
2. Join community
3. Resources page

Downloading Course Material

4. Starter code
    1. Code > Download ZIP

Section 2: Part 1: REACT FUNDAMENTALS [4 PROJECTS]

Introduction to Part 1

1. Skills
    1. Setting up React Apps
    2. React components
    3. Props
    4. JSX
    5. Handling events
    6. State & State management (to make apps dynamic)

Useful Resources for Part 1

1. React Documentation
2. Create React App
    1. Setup
3. Vite: Getting Started
    1. Real-World React apps
4. Adding React URL to an HTML Document
    1. Pure React lecture

Section 3: A First Look at React

Section Overview

1. Topics:
    1. Why React?
    2. What is React?
    3. React vs. vanilla JS
    4. Setting up new React apps
        1. Options for creation
            1. CreateReactApp

Why Do Front-End Frameworks Exist?

1. Why not use vanilla JS?

The Rise of Single-Page Applications

1. Before 2010:
    1. Server-side rendering (the old way (but now getting modern again))
        1. View (template) + Data -> Render Webpage
            1. Browser takes the page and paints it on the screen
        2. Example:
            1. Wordpress
                1. JS - only to add simple dynamics to the page (simple animations, hover effects, ...)
                    1. jQuery was used for this
                        1. Makes JS work the same way across all browsers
    2. Developers started writing more and more JS code, which led to the rise of Single-Page web-applications
        1. Single-Page Web-Applications: Rendered in the browser (not on the server)
            1. Client-Side rendering:
                1. Browser + View -> Render Webpage -> Browser
            2. They are called web-applications
                1. If web-applications need data, it comes from server APIs
                    1. Application consumes API data and renders the screen for each view of the application
            3. SPAs makes us feel like we were using a native desktop or phone applications
                1. We can click on links and submit forms without the page reloading

1. Technically we are on the same page
2. Server-side rendering is making a comeback
    1. Frameworks built on top of modern client-side rendering frameworks
        1. Next.js
        2. Remix
        3. ...

Single-Page Applications with Vanilla Javascript?

1. There are many problems with using vanilla JS to build large-scale applications
2. Front-end web applications are about:
    1. **Handling** data + **displaying** data in a user interface
        1. Recieves data
        2. Changes data (as user uses the app)
        3. Displays the current data on the screen
    2. User interface needs to **stay in sync** with data

        ```
        DATA <-> UI
        ```

        1. UI must always display the current state of the data
            1. **Very hard problem** to solve!
                1. Example: Airbnb
                    1. Search bar
                    2. Filters
                    3. Search as we move the map
                2. Example: If we change location or date, list of apartments needs to be updated
                    1. Map must also be updated
                    2. When map is moved (assuming user has selected this option), the list of apartments needs to be updated
        2. Piece of data = Piec of state
            1. Keeping UI and data in sync would be **virtually impossible with just vanilla JS**
                1. Why?
                    1. Problems with JS and jQuery
                        1. Requires lots of direct DOM manipulation and traversing (imperative) "Spaghetti code"
                            1. We might end up with a huge mess
                        2. Data (state) is usually stored in the DOM (in HTML elements), shared across entire app - Hard to reason + bugs
                            1. Many parts of the app can access and modify DOM state directly (may introduce many bugs)
                    2. If we handle the above problems, we might end up constructing our own framework
                        1. It is better to use a well-established, and battle tested framework instead

Why do Front-End Frameworks Exist?

1. JavaScript front-end frameworks exist because...
    1. Keeping a user interface in sync with data is **really hard** and a **lot of work**
    2. Front-end frameworks **solve this problem** and take hard work away from developers
        1. Users can only focus on data & building UIs
            1. Different frameworks have different approaches
                1. They are similar in keeping the UI and data in sync
2. They enforce a "correct" way of structuring and writing code (therefore contributing to solving the problem of "spaghetti code")
    1. Other developers can follow the conventions
3. They give developers and teams a **consistent** way of building front-end applications
    1. Everyone on the team will build their part of the app in the exact same style as everyone else
        1. Consistent code-base & product

React vs. Vanilla JavaScript

1. Comparison of an application
    1. React: (more automatic)
        1. Even HTML is done in JS (inside JS)
        2. No manual selection of elements
        3. We attach event-listeners using attributes
        4. As soon as we update the state, the UI is automatically kept in sync
    2. Vanilla JavaScript: (more manual)
        1. HTML is in charge
            1. JS is inside HTML
        2. We need to manually select elements in JS
            1. No class required
        3. We need to attach event-listeners manually in JS
        4. Updating state will not automatically update UI
            1. UI needs to be updated manually

What is React?

What is React?

1. It is a JavaScript library for building UIs
2. **Extremely popular**, **declarative**, **component-based**, **state-driven** JavaScript **library for building user interfaces**, constructed by Facebook
    1. Based on components
        1. Buttons, input-fields, search-bars, ...
        2. They are the building-blocks of UIs in React
        3. React takes the components and draws them on a webpage
        4. We build complex UI by building components and combining components like lego pieces
            1. Example:

                1. Navbar
                2. Results pannel
                3. Map
                4. Search bar
            5. We also re-use components
                1. Multiple elements on the page can look similar
                    1. Example: Listing component can be defined once and re-used multiple times on the page by adjusting data
    2. Declarative
        1. Each component must have all the information about what it looks like
            1. Solution: We **describe** how components look like and how they work using a **declarative syntax called JSX**
                1. **Declarative**: Telling React what a component should look like, **based on current data/state**
        2. React is **abstraction** away from DOM: we **never touch the DOM** (directly, as with vanilla JS)
            1. We tell react what we want to do when data changes but not how to do it
        3. JSX:
            1. A syntax that **combines** HTML, CSS, JavaScript as well as referencing **other React components**
    3. State-driven
        1. How does React update the UI?
            1. Main goal of React is to keep UI in sync with data
                1. The data is called state in React
        2. Example:
            1. **State**: An array of apartments on Airbnb
        3. Based on the initial state, React will render UI with Components we wrote using JSX
        4. Based on an event (button-click say), the state might change (more data might be loaded from an API)
            1. Whenever state changes, we **manually** update the state in the React app
                1. As a result React will **automatically** re-render the UI to reflect the latest state
                    1. **React reacts to state changes by re-rendering the UI** (hence it is called React)
    4. JavaScript library
        1. Is React a library or a framework?
            1. It is just a library
                1. **Because React is only the "view" layer**. We need to pick multiple external libraries to build a complete application
                    1. Routing
                    2. Data-fetching
                2. Solution: Frameworks built on top of React (includes all functionalities that React is missing out-of-the-box)
                    1. Next.js
                    2. Remix
    5. Extremely popular
        1. Biggest reason
            1. Most used framework so far
        2. Many large companies have adopted React
            1. Smaller companies are following their footsteps
                1. Huge job market with high demand for React developers
                    1. Well paying jobs, too
        3. Large and vibrant React developer community
            1. Tutorials
            2. Stackoverflow Q&A
            3. Lot of support given to React developers
        4. Gigantic third-party library ecosystem
            1. Many useful libraries and tools into our React applications
    6. Constructed by Facebook
        1. React was built in 2011 by Jordan Walke, an engineer working at Facebook at the time
            1. Was first used for news-feed
            2. Then used for chat-app
            3. Instagram
        2. React was open-sourced in 2013, and has since then completely transformed front-end web development
            1. Many modern frameworks were created on top of React
3. Summary:
    1. Rendering components on a Webpage (UI) based on their current state
    2. Keeping the UI in sync with state, by re-rendering (reacting) when state changes
        1. Implemented using:
            1. Virtual DOM
            2. Fibre-tree
            3. One-way data-flow
            4. ...

Setting Up Our Development Environment

1. Download and install VS Code
2. Download and install Google Chrome
3. Download and install Node.js
    1. The tools run on node.js
        1. = v18 (LTS)
4. VS Code config:
    1. Extensions
        1. ESLint (code linter)
            1. Finds errors or best practices we are violating
        2. Prettier (code formatter)
        3. Color theme: one monokai
        4. material icon theme
    2. Settings:
        1. Auto Save: onFocusChange
        2. default formatter: Prettier - Code formatter
        3. format on save: Check
        4. ESlint run: onSave
    3. Terminal
        1. `node -v`

4. Snippets
    1. Starter files: 00-setup
        1. snippets.json - open in VSCode
            1. Copy everything
        2. User snippets:
            1. New Global Snippets file (or open existing one)
                1. Paste

Pure React

1. Inside HTML file (no modern tooling)
2. Folder: 01-pure-react
    1. Open it as project folder in VSCode
        1. index.html
            1. ! (enter)
        2. react.dev
            1. Learn
                1. Installation
                    1. Try React locally
                        1. download this HTML page
                            1. Copy-paste the `<script>`s
                                1. `react.development.js` - core React library
                                    1. Works with components, state, etc. (how to interact with React)
                                2. `react-dom.development.js` - Rendering layer which can render the React components into the DOM
                                    1. React can be rendered in other environments (As Native applications using ReactNative)
3. Code:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Hello React!</title>
</head>

<body>
  <div id="root"></div>

  <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
  <script>
    function App() {
      // We cannot use JSX because we don't have the tooling to convert it into JS

      // const time = new Date().toLocaleTimeString();
      const [time, setTime] = React.useState(new Date().toLocaleTimeString());

      React.useEffect(function () {
        setInterval(function () {
          setTime(new Date().toLocaleTimeString());
        }, 1000);
      }, [])

      return React.createElement("header", null, `Hello React! It's ${time}`); // `null` - props, `'Hello React!'` is content
    } // We need to manually reload the page when we make changes

    const root = ReactDOM.createRoot(document.getElementById('root')); // <div id="root"></div> will become the root of the application. Applica
    root.render(React.createElement(App)); // React renders by adding the App component
  </script>
</body>

</html>
```

A Quick Look at React's Official Documentation

1. react.dev
    1. Learn
        1. Getting started
        2. Installation
        3. Learning React
            1. Fundamentals
        4. **Escape Hatches**
            1. Useful
                1. After the course
    2. Reference
        1. Every function in React
            1. `useState` - more examples, more information
            2. `useLayoutEffect`

Setting Up a New React Project: The Options

1. Options to setup a react project

The Two Options for Setting up a React Project

1. Create-React-App
    1. Complete "starter kit" for React applications
        1. To make it easy for developers to scaffold new React apps

2. Everything is **already configured**: ESLit, Prettier, Jest, etc.
    1. Common dev tools are preconfigured
        1. Development server
        2. Webpack (for module bundling)
        3. Dev tools
            1. ESLint
            2. Prettier
            3. Jest (Testing)
            4. Babel (To enable latest JS features)
    3. Problem:
        1. It was developed many years ago.
            1. Uses **slow and outdated** technologies (i.e. webpack)
        2. They stopped innovating
    4. Usecases:
        1. Use for tutorials or experiments
            1. To get started quickly
                1. Used in the course
        2. Don't use for a real-world app
2. Vite (Build tool)
    1. Usecase: Modern real-world apps
    2. **Modern build tool** that contains a starter **template** for setting up React applications
    3. Need to manually set up **ESLint**, Prettier, Testing library, ...
        1. Most painful - Setting up ESLint to play right with React
    4. Why?
        1. Extremely fast hot module replacement (HMR) (fast to automatically refresh the page) and bundling (extremely fast - good for large-scale applications. Page will update almost instantly)

What About React Frameworks?

1. The React team now advises to use a "React Framework" for new projects (Built on top of React library)
    1. Next.js
        1. Additional features:
            1. Routing
            2. Data-fetching
            3. Server-side rendering
    2. Remix
2. Many people think that this is not the best idea: **"vanilla" React aps are important too**
3. This only makes sense for building actual products, **not for learning React**
    1. We need to learn React itself

Setting Up a Project With Create-React-App

1. Create-React-App - CLI tool

    1. CMD:

    ```
    cd <folder>/ultimate-react-course-main
    npx create-react-app@5 pizza-menu
    ```

    2. It is a regular npm project (`npm init` could have been used)

        1. package.json

        2. node_modules

            1. react
            2. react-dom
            3. src
                1. Development happens here
            4. public
                1. All assets that will end up in the final application go
                    1. Images
                    2. index.html
                        1. `<div id="root"></div>`
                    3. index.js

        3. Starting:

        ```
        npm start
        ```

        4. Open terminal in VS Code

            1. Run the command
                1. A web-server is started and App is run in that
                2. It has hot-module replacement

Section 4: [Optional] Review of Essential JavaScript for React

Section Overview

Destructuring Objects and Arrays

Rest/Spread Operator

Template Literals

Ternaries Instead of if/else Statements

Arrow Functions

Short-Circuiting And Logical Operators: &&, ||, ??

Optional Chaining

The Array map Method

The Array filter Method

The Array reduce Method

The Array sort Method

Working With Immutable Arrays

Asynchronous JavaScript: Promises

Asynchronous JavaScript: Async/Await

Section 5: Working With Components, Props, and JSX

Section Overview

1. Three important concepts
    1. Components
        1. Building blocks of React App
    2. Props
        1. To share data
    3. JSX
2. Constructing and reusing components
3. Rendering lists
4. Conditional rendering
5. Start writing code on your own

Rendering the Root Component and Strict Mode

1. `src/index.js`
    1. Why?
        1. Webpack (module bundler) expects entry point to be called `index.js`
2. `react` and `read-dom` - imported
3. `App` - name is a convention
    1. `React.StrictMode` - Good idea to activate it
        1. It renders component twice (to check for bugs) durnig development
        2. Checks if we are using outdated parts of the React API

Before We Start Coding: Debugging

1. Techniques to deal with errors and problems
    1. If page doesn't update
        1. Ensure the application is running
            1. Stop the app and restart
            2. Reload on the browser (HOT reload breaks sometimes)
        2. Keep the terminal open
        3. Open Console
            1. React Dev tools (later)
        4. React also displays errors on the screen
            1. If we don't understand the error, copy and paste in Google
                1. Stackoverflow
        5. Work with ESLint
            1. Hover to see the error
            2. They appear in Problems tab in VS Code
        6. If prettier stops working:
            1. Go to output tab
            2. Select Prettier (process running in VS Code)
                1. It shows error sometimes
        7. If ESLint stops working:
            1. Go to output tab
            2. Select ESLint
                1. If you see errors, fix them
        8. Compare with working code (final folder)

Components as Building Blocks

Components as Building Blocks

1. Components:
    1. React applications are entirely made out of components
        1. There is nothing that is not a component or that is inside a component
        2. Components are building-blocks of any UI

1. React takes components and draws them onto the page
    1. React renders a view for each component, and all the views make up the UI
2. Piece of UI that has its own **data**, **logic**, and **appearance** (how it works and looks)
3. We build complex UIs by **building multiple components** and **combining** them
    1. Example:
        1. VideoPlayer
        2. Menu
        3. RefineQuestions
        4. QuestionList
        5. ...
        6. Filters
            1. It is inside RefineQuestions component
        7. Question
            1. It is inside QuestionList component
            2. We can re-use it for multiple questions by passing different data (Props)
                1. Whenever we want duplication in UI, we define a component and re-use it as many times as necessary
4. Components can be **reused**, **nested** inside each other, and **pass data** between them
    1. Key advantages:
        1. Nestability
        2. Reusability
5. A crucial skill:
    1. Breaking a UI into components
        1. One thing that helps with that is: Component Tree
            1. Shows the hierarchy that exists between components that make up the UI
                1. Makes it easy to understand how the components are nested inside each other & how they relate to one-another (relationships - parent-child)
                    1. Parent component: RefineQuestions
                    2. Child component: Filters

Constructing And Reusing a Component

1. 03-pizza-menu
    1. Copy the everything and paste in `public` folder
        1. Move `index.css` to `src` folder
2. create-react-app adds a git repository automatically
    1. Settings
        1. Search: diff decorations
            1. none
3. Code:

```
import React from "react";
import ReactDOM from "react-dom/client";

const pizzaData = [
  {
    name: "Focaccia",
    ingredients: "Bread with italian olive oil and rosemary",
    price: 6,
    photoName: "pizzas/focaccia.jpg",
    soldOut: false,
  },
  {
    name: "Pizza Margherita",
    ingredients: "Tomato and mozarella",
    price: 10,
    photoName: "pizzas/margherita.jpg",
    soldOut: false,
  },
  {
    name: "Pizza Spinaci",
    ingredients: "Tomato, mozarella, spinach, and ricotta cheese",
    price: 12,
    photoName: "pizzas/spinaci.jpg",
    soldOut: false,
  },
  {
    name: "Pizza Funghi",
    ingredients: "Tomato, mozarella, mushrooms, and onion",
    price: 12,
    photoName: "pizzas/funghi.jpg",
    soldOut: false,
  },
  {
    name: "Pizza Salamino",
    ingredients: "Tomato, mozarella, and pepperoni",
    price: 15,
    photoName: "pizzas/salamino.jpg",
    soldOut: true,
  },
  {
    name: "Pizza Prosciutto",
    ingredients: "Tomato, mozarella, ham, aragula, and burrata cheese",
    price: 18,
    photoName: "pizzas/prosciutto.jpg",
    soldOut: false,
  },
];

function App() {
  return (
    <div>
      <h1>Alhamdulillah!</h1>
```

```
        <Pizza />
        <Pizza />
        <Pizza />
      </div>
    );
}

// Two rules:
// 1: Starts with capital letter
// 2: returns a markup (exactly one element)
// 3: Nesting declaration inside another declaration works but it must not be done
function Pizza() {
  return (
    <div>
      <h2>Pizza Spinaci</h2>
      <p>Tomato, mozarella, spinach, and ricotta cheese</p>
      <img src="pizzas/spinaci.jpg" alt="spinaci" />
    </div>
  );
}

// React v18
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

// React before 18
// React.render(<App />, document.getElementById("root")); // What to render, where to render
```

What is JSX?

1. Component
    1. Contains:
        1. Data
        2. Logic
        3. Appearance
2. JSX:
    1. **Declarative** syntax to **describe** what components **look like** and **how they work** (based on their data and logic)
        1. All about component's appearance
    2. Components must **return** a block of JSX
        1. React will use it to render on the UI
    3. Extension of JavaScript that allows us to **embed JavaScript, CSS and React components into HTML**
        1. Embedding of JS - Example: Referencing JS variables using `{}` syntax
        2. We can embed other React components
    4. There is a simple way of converting JSX into JS:

```
<header>
    <h1 style="color: red">
        Hello React!
    </h1>
</header>

    |
    |Babel (automatically included in the application)
    |
    v

React.createElement(
    'header',
    null,
    React.createElement(
        'h1',
        { style: { color: 'red' } },
        'Hello React!'
    )
);
```

        1. Each JSX element is **converted** to a `React.createElement` function call
            1. The function calls construct the HTML on the screen
        2. Conversion is necessary because Browsers understand only HTML, CSS, and JS
        3. We could use React **without JSX**
            1. Hard to read and understand
            2. Hard to write

JSX is Declarative

1. Imperative (How to do things)
    1. If we use vanilla JS, we use an imperative approach
        1. Manual DOM element selections and DOM traversing
        2. Step-by-step DOM mutations until we reach the desired UI
    2. Doing the above is infeasible for complex apps
2. Declarative
    1. Describe what UI should look like using JSX, **based on current data** (Props and State)
        1. No DOM manipulation at all

        1. No event-listeners
        2. No query-selectors
        3. No classlist
        4. No `textContent` properties
    2. React is an **abstration** away from DOM: **we never touch the DOM**
    3. Instead, we think of the UI as a **reflection of the current data**
        1. React automatically synchronizes with the data
    4. We just tell React using JSX, what we want to see on the screen, but not achieve it step-by-step

Constructing More Components

1. Code:

```
function App() {
  return (
    <div>
      <Header />
      <Menu />
      <Footer />
    </div>
  );
}

function Header() {
  return (
    <div>
      <h1>Fast React Pizza Co.</h1>
      <Pizza />
      <Pizza />
      <Pizza />
    </div>
  );
}

function Menu() {
  return <h2>Our menu</h2>;
}

function Footer() {
  // return React.createElement("footer", null, "We're currently open");
  return <footer>{new Date().toLocaleTimeString()} W're currently open</footer>;
}
```

JavaScript Logic in Components

1. Components are just JS functions, so any JS code can be executed in the functions
    1. The code is executed as soon as the function is called
        1. As soon as the component is initialized
2. Code:

```
function Footer() {
  const hour = new Date().getHours();
  // console.log(hour);
  const openHour = 7;
  const closeHour = 22;
  const isOpen = hour >= openHour && hour <= closeHour;
  console.log(isOpen);

  // if (hour >= openHour && hour <= closeHour) {
  //   alert("We're currently open");
  // } else {
  //   alert("Sorry we're closed");
  // }

  // return React.createElement("footer", null, "We're currently open");
  return <footer>{new Date().toLocaleTimeString()} W're currently open</footer>;
}
```

Separation of Concerns

1. JSX combines HTML, CSS, and JS, in a single block of code
    1. Why?

Separation of Concerns?

1. Rise of interactive SPAs
    1. Before SPAs
        1. One file for HTML, CSS, and JS
            1. Traditional separation of concerns
    2. As pages became more interactive, they became SPAs (JS started to determine UI and content)
        1. JavaScript became more in charge of the HTML
            1. HTML and JS are tightly coupled together (HTML doesn't make sense without the JS)
    3. Logic and UI are tightly coupled:
        1. Why keep them separated?
            1. Solution: React Components + JSX
                1. Hence why a React component contains Data, logic, and appearance of one piece of the UI
                    1. Everything is mixed together (content & logic are co-located (things that change together))
                        1. We have one component per file instead of one technology per file
2. Separation of Concerns:
    1. It is one concern per component (instead of one concern per file)
        1. Each component is **concerned with one piece of the UI**
            1. New paradigm

1. Having all info about a certain component in one single place

Styling React Applications

1. We have different ways of styling
    1. React doesn't care
        1. It is just a library (it doesn't have a preferred way of styling components, applications)
            1. We can choose many options
                1. Inline styling
                    1. We cannot use "" as in HTML, but we need to pass a JS object using {}
                        1. We need to enter JS mode to pass a JS object using {}
                2. External CSS & SASS
                    1. We can include external CSS (like in any other webpage)
                    2. They are global styles
                3. CSS Modules
                4. Styled components
                5. Tailwind CSS
2. External CSS Example:

```
import React from "react";
import ReactDOM from "react-dom/client";
import "./index.css"; // Webpack automatically included styles in our application

const pizzaData = [
  {
    name: "Focaccia",
    ingredients: "Bread with italian olive oil and rosemary",
    price: 6,
    photoName: "pizzas/focaccia.jpg",
    soldOut: false,
  },
  {
    name: "Pizza Margherita",
    ingredients: "Tomato and mozarella",
    price: 10,
    photoName: "pizzas/margherita.jpg",
    soldOut: false,
  },
  {
    name: "Pizza Spinaci",
    ingredients: "Tomato, mozarella, spinach, and ricotta cheese",
    price: 12,
    photoName: "pizzas/spinaci.jpg",
    soldOut: false,
  },
  {
    name: "Pizza Funghi",
    ingredients: "Tomato, mozarella, mushrooms, and onion",
    price: 12,
    photoName: "pizzas/funghi.jpg",
    soldOut: false,
  },
  {
    name: "Pizza Salamino",
    ingredients: "Tomato, mozarella, and pepperoni",
    price: 15,
    photoName: "pizzas/salamino.jpg",
    soldOut: true,
  },
  {
    name: "Pizza Prosciutto",
    ingredients: "Tomato, mozarella, ham, aragula, and burrata cheese",
    price: 18,
    photoName: "pizzas/prosciutto.jpg",
    soldOut: false,
  },
];

// `class` is a reserved keyword in JS
function App() {
  return (
    <div className="container">
      <Header />
      <Menu />
      <Footer />
    </div>
  );
}

// `style` - also available in HTML
// In HTML, we don't use it due to separation of concerns
// In React, it is fine (because all are mixed per component)
function Header() {
  // const style = { color: "red", fontSize: "48px", textTransform: "uppercase" };
  const style = {};

  return (
    <header className="header">
      <h1 style={style}>Fast React Pizza Co.</h1>
    </header>
  );
}

function Menu() {
  return (
    <main className="menu">
      <h2>Our menu</h2>
      <Pizza />
```

```
      <Pizza />
      <Pizza />
    </main>
  );
}

function Footer() {
  const hour = new Date().getHours();
  // console.log(hour);
  const openHour = 7;
  const closeHour = 22;
  const isOpen = hour >= openHour && hour <= closeHour;
  console.log(isOpen);

  // if (hour >= openHour && hour <= closeHour) {
  //   alert("We're currently open");
  // } else {
  //   alert("Sorry we're closed");
  // }

  // return React.createElement("footer", null, "We're currently open");
  return <footer>{new Date().toLocaleTimeString()} W're currently open</footer>;
}

// Two rules:
// 1: Starts with capital letter
// 2: returns a markup (exactly one element)
// 3: Nesting declaration inside another declaration works but it must not be done
function Pizza() {
  return (
    <div>
      <img src="pizzas/spinaci.jpg" alt="spinaci" />
      <h3>Pizza Spinaci</h3>
      <p>Tomato, mozarella, spinach, and ricotta cheese</p>
    </div>
  );
}

// React v18
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

// React before 18
// React.render(<App />, document.getElementById("root")); // What to render, where to render
```

Passing and Receiving Props

1. Props:
    1. How we pass data between components
        1. In particular, between parent component and child components
    2. It is a communication channel between parent and child component
2. Code:

```
function Header() {
  // const style = { color: "red", fontSize: "48px", textTransform: "uppercase" };
  const style = {};

  return (
    <header className="header">
      <h1 style={style}>Fast React Pizza Co.</h1>
    </header>
  );
}

// We can pass arrays, objects, react components, etc as props
function Menu() {
  return (
    <main className="menu">
      <h2>Our menu</h2>
      <Pizza
        name="Pizza Spinaci"
        ingredients="Tomato, mozarella, spinach, and ricotta cheese"
        photoName="pizzas/spinaci.jpg"
        price={10}
      />

      <Pizza
        name="Pizza Funghi"
        ingredients="Tomato, mushrooms"
        price={12}
        photoName="pizzas/funghi.jpg"
      />
    </main>
  );
}

// Two rules:
// 1: Starts with capital letter
// 2: returns a markup (exactly one element)
// 3: Nesting declaration inside another declaration works but it must not be done
function Pizza(props) {
  console.log(props); // {name: 'Pizza Spinacci', ingredients: 'Tomato, ...'}

  return (
    <div className="pizza">
      <img src={props.photoName} alt={props.name} />
```

```
          <div>
            <h3>{props.name}</h3>
            <p>{props.ingredients}</p>
            <span>{props.price + 3}</span>
          </div>
        </div>
      );
    }
```

Props, Immutability, and One-Way Data Flow

Reviewing Props

1. Props
    1. Props are used to pass data from **parent components** to **child components (down the component tree)**
    2. Props are essential React tool to **configure** and **customize** components (like function parameters)
    3. With props, parent components **control** how child components look and work
        1. They are just like arguments we pass to JS functions
            1. Since we can pass anything to JS functions, we can pass anything to JSX as props
                1. Single values
                2. Arrays
                3. Objects
                4. Functions
                5. **Other components**
                6. ...
        2. Anything can be passed as props

Props are Read-Only

1. Component:
    1. Data
        1. What is this?
            1. Props + State
                1. Props
                    1. Props is data coming from the **outside**, and can **only** be updated by the **parent component** (owner)
                        1. It cannot be modified by he child component
                    2. Props are read-only, they are **immutable**! This is one of React's strict rules.
                        1. Why?
                            1. Mutating props would affect parent, causing **side effects** (not pure) (That is how objects work in JS)
                                1. If we copy an object and mutate the copy, the original object is mutated
                                    1. If we change an object outside the component function, the function has caused a side-effect
                                        1. Whenever we change data that is outside the current function
                                        2. React is about pure functions (about component's data)
                                2. Components have to be **pure functions** in terms of props and state
                                    1. Allows React to optimize apps, avoid bugs (when we manipulate external data), make apps predictable
                                        1. **General rule: A component should never mutate it's data outside its function scope**

                                        ```
                                        let x = 7;

                                        function Component() {
                                            x = 23;
                                            return <h1>Number {x}</h1>
                                        }
                                        ```
                2. State
                    1. State is internal component data that can be updated by the **component's** logic
                    2. This is for data that changes over time (mutable)
        2. Logic
        3. Appearance

One-Way Data Flow

1. In React applications, data can only be passed from parent to child components. (happens by using props)
    1. Never from children to parent
        1. Angular has **two-way** data flow
2. Why?
    1. Makes applications more predictable and easier to understand
        1. Easier to understand where the data is coming from if it flows only in one direction
    2. Makes applications easier to debug, as we have more control over the data
    3. Is more performant
        1. Two-way data-binding is less efficient to implement
3. What if we wanted to pass data to parent component
    1. Later

CHALLENGE #1: Profile Card (v1)

1. Browser: react.new
    1. Uses `create-react-app` under-the-hood
2. Code:

```
import "./App.css";

function App() {
  return (
    <div className="card">
      <Avatar />
      <div className="data">
        <Intro />
        <SkillList />
      </div>
    </div>
  );
}

function Avatar() {
  return (
    <div className="avatar">
      <img src="avatar_hero.jpg" alt="Avatar Hero" width="100%" />
    </div>
  );
}

function Intro() {
  return (
    <div className="intro">
      <h2>Abdullah Mutthoju</h2>
      <p>
        Java Full-stack web developer and architect. When not doing serious
        work, I like to play with my daughter, travel, and shop.
      </p>
    </div>
  );
}

function SkillList() {
  return (
    <div className="skill-list">
      <Skill style={{ backgroundColor: "red" }} emoji="⚡" skill="Java" />
      <Skill style={{ backgroundColor: "green" }} emoji="💰" skill="Azure" />
      <Skill
        style={{ backgroundColor: "yellow" }}
        emoji="✈️"
        skill="Kubernetes"
      />
      <Skill style={{ backgroundColor: "cyan" }} emoji="📈" skill="React" />
    </div>
  );
}

function Skill(props) {
  return (
    <span className="skill" style={props.style}>
      {props.skill} {props.emoji}
    </span>
  );
}

export default App;
```

The Rules of JSX

1. Rules:
    1. General JSX Rules
        1. JSX works essentially like HTML, but we can enter "JavaScript mode" by using `{}` (for text or attributes)
            1. Anywhere in the markup
            2. Where text or attribute is expected
        2. We can place **JavaScript expressions** (anything that produces a value) inside `{}`. Examples:
            1. Reference variables
            2. Construct arrays or objects.
            3. `[].map()`.
            4. Ternary operator
        3. Statements are **not allowed** (if/else, for, switch)
        4. JSX produces a **JavaScript expression**

```
const el = <h1>Hello React!</h1>;
// same as
const el = React.createElement('h1', null, 'Hello React!'); // converted to this expression
```

            1. We can place **other pieces of JSX** inside `{}`
                1. Since we can put any JS expression inside `{}` (JS mode)
            2. We can write JSX **anywhere** inside a component (in `if`/`else`, assign to variables, pass it into functions)
        5. A piece of JSX can only have **one root element**. If you need more, use `<React.Fragment>` (or the short `<>`) (later)

2. Differences between JSX and HTML
    1. `className` instead of HTML's class
    2. `htmlFor` instead of HTML's `for`
    3. Every tag needs to be **closed**. Exmaples: `<img />`, `<br />`
    4. All event handlers and other properties need to be **camelCased**. Examples: `onClick` or `onMouseOver`
    5. **Exception:** `aria-*` and `data-*` are written with dashes like in HTML
    6. CSS inline styles are written like this: `{{<style>}}` (to reference a variable, and then an object)
    7. CSS property names are also **camelCased**
    8. Comments need to be in `{}` (because they are JS)

Rendering Lists

1. One of the most common things we do in almost all React applications
2. How to render list?
3. What is rendering a list?
    1. When we have an array, and we want to construct a component for each element of the array
    2. Each child in a list must have a unique `key` property (prop that is internal to React - for performance optimization (later))
4. Code:

```
function Menu() {
  return (
    <main className="menu">
      <h2>Our menu</h2>

      <ul className="pizzas">
        {/* Array is converted to list of components. forEach will not work (it doesn't return a new array) */}
        {pizzaData.map((pizza) => (
          <Pizza pizzaObj={pizza} key={pizza.name} />
        ))}
      </ul>

      {/* <Pizza
        name="Pizza Spinaci"
        ingredients="Tomato, mozarella, spinach, and ricotta cheese"
        photoName="pizzas/spinaci.jpg"
        price={10}
      />
      <Pizza
        name="Pizza Funghi"
        ingredients="Tomato, mushrooms"
        price={12}
        photoName="pizzas/funghi.jpg"
      /> */}
    </main>
  );
}

// Two rules:
// 1: Starts with capital letter
// 2: returns a markup (exactly one element)
// 3: Nesting declaration inside another declaration works but it must not be done
function Pizza(props) {
  console.log(props); // {name: 'Pizza Spinacci', ingredients: 'Tomato, ...'}

  return (
    <li className="pizza">
      <img src={props.pizzaObj.photoName} alt={props.pizzaObj.name} />
      <div>
        <h3>{props.pizzaObj.name}</h3>
        <p>{props.pizzaObj.ingredients}</p>
        <span>{props.pizzaObj.price}</span>
      </div>
    </li>
  );
}
```

Conditional Rendering With &&

1. Rendering JSX (a piece or entire component) based on a condition
2. Code:

```
function Menu() {
  const pizzas = pizzaData;
  const numPizzas = pizzas.length;

  return (
    <main className="menu">
      <h2>Our menu</h2>
      {/* It is a truthy value. React renders a 0. Never use number */}
      {numPizzas > 0 && (
        <ul className="pizzas">
          {/* Array is converted to list of components. forEach will not work (it doesn't return a new array) */}
          {pizzas.map((pizza) => (
            <Pizza pizzaObj={pizza} key={pizza.name} />
          ))}
        </ul>
      )}

      {/* <Pizza
        name="Pizza Spinaci"
        ingredients="Tomato, mozarella, spinach, and ricotta cheese"
        photoName="pizzas/spinaci.jpg"
```

```
          price={10}
        />
        <Pizza
          name="Pizza Funghi"
          ingredients="Tomato, mushrooms"
          price={12}
          photoName="pizzas/funghi.jpg"
        /> */}
      </main>
    );
  }
```

Conditional Rendering With Ternaries

1. Code:

```
function Menu() {
  const pizzas = pizzaData;
  const numPizzas = pizzas.length;

  return (
    <main className="menu">
      <h2>Our menu</h2>
      {/* It is a truthy value. React renders a 0. Never use number.
      We can only write JS expressions that return values within `{}` */}
      {numPizzas > 0 ? (
        <ul className="pizzas">
          {/* Array is converted to list of components. forEach will not work (it doesn't return a new array) */}
          {pizzas.map((pizza) => (
            <Pizza pizzaObj={pizza} key={pizza.name} />
          ))}
        </ul>
      ) : (
        <p>We're still working on our menu. Please come back later.</p>
      )}
    </main>
  );
}

...

function Footer() {
  const hour = new Date().getHours();
  // console.log(hour);
  const openHour = 9;
  const closeHour = 22;
  const isOpen = hour >= openHour && hour <= closeHour;
  console.log(isOpen);

  // if (hour >= openHour && hour <= closeHour) {
  //   alert("We're currently open");
  // } else {
  //   alert("Sorry we're closed");
  // }

  // return React.createElement("footer", null, "We're currently open");
  return (
    <footer>
      {isOpen ? (
        <div className="order">
          <p>We're open until {closeHour}:00. Come visit us or order online.</p>
          <button className="btn">Order</button>
        </div>
      ) : (
        <p>
          We're happy to welcome you between {openHour}:00 and {closeHour}:00
        </p>
      )}
    </footer>
  ); // short circuiting. React doesn't render `true` or `false` into the DOM
}
```

Extracting JSX Into a New Component

1. Multiple returns (based on a condition)
2. Code:

```
function Pizza(props) {
  console.log(props); // {name: 'Pizza Spinacci', ingredients: 'Tomato, ...'}

  if (props.pizzaObj.soldOut) return null; // nothing is rendered

  return (
    <li className="pizza">
      <img src={props.pizzaObj.photoName} alt={props.pizzaObj.name} />
      <div>
        <h3>{props.pizzaObj.name}</h3>
        <p>{props.pizzaObj.ingredients}</p>
        <span>{props.pizzaObj.price}</span>
      </div>
    </li>
  );
}

function Footer() {
  const hour = new Date().getHours();
  // console.log(hour);
  const openHour = 9;
  const closeHour = 22;
  const isOpen = hour >= openHour && hour <= closeHour;
  console.log(isOpen);
```

```
  // if (hour >= openHour && hour <= closeHour) {
  //   alert("We're currently open");
  // } else {
  //   alert("Sorry we're closed");
  // }

  // return React.createElement("footer", null, "We're currently open");
  if (!isOpen)
    return (
      <footer className="footer">
        <p>
          We're happy to welcome you between {openHour}:00 and {closeHour}:00
        </p>
      </footer>
    );

  return (
    <footer className="footer">
      {
        <div className="order">
          <p>We're open until {closeHour}:00. Come visit us or order online.</p>
          <button className="btn">Order</button>
        </div>
      }
    </footer>
  ); // short circuiting. React doesn't render `true` or `false` into the DOM
}
```

3. Code:

```
function Footer() {
  const hour = new Date().getHours();
  // console.log(hour);
  const openHour = 7;
  const closeHour = 22;
  const isOpen = hour >= openHour && hour <= closeHour;
  console.log(isOpen);

  return (
    <footer className="footer">
      {isOpen ? (
        <Order closeHour={closeHour} />
      ) : (
        <p>
          We're happy to welcome you between {openHour}:00 and {closeHour}:00
        </p>
      )}
    </footer>
  ); // short circuiting. React doesn't render `true` or `false` into the DOM
}

function Order(props) {
  return (
    <div className="order">
      <p>
        We're open until {props.closeHour}:00. Come visit us or order online.
      </p>
      <button className="btn">Order</button>
    </div>
  );
}
```

Destructuring Props

1. Props are always passed into child components (even if it is empty)
2. Code:

```
function Pizza({ pizzaObj }) {
  // Shows what props will be received by the function. We don't have to go to where the component is being used
  if (pizzaObj.soldOut) return null; // nothing is rendered

  return (
    <li className="pizza">
      <img src={pizzaObj.photoName} alt={pizzaObj.name} />
      <div>
        <h3>{pizzaObj.name}</h3>
        <p>{pizzaObj.ingredients}</p>
        <span>{pizzaObj.price}</span>
      </div>
    </li>
  );
}

...

function Order({ closeHour, openHour }) {
  // If we try to destructure a property that doesn't exist, it is `undefined`
  return (
    <div className="order">
      <p>
        We're open from {openHour}:00 until {closeHour}:00. Come visit us or
        order online.
      </p>
      <button className="btn">Order</button>
    </div>
  );
}
```

React Fragments

1. It allows us to have more than one element in a piece of JSX
2. Using nothing:

```
<>...</>
```

3. Using `React.Fragment` **(M)**

```
<React.Fragment key="abcd">
   ...
</Readt.Fragment>
```

Setting Classes and Text Conditionally

1. Conditionally set text, and class names
2. Code:

```
function Pizza({ pizzaObj }) {
  // Shows what props will be received by the function. We don't have to go to where the component is being used
  // if (pizzaObj.soldOut) return null; // nothing is rendered

  // We don't have to use classList and imperative code to set class
  return (
    <li className={`pizza ${pizzaObj.soldOut ? "sold-out" : ""}`}>
      <img src={pizzaObj.photoName} alt={pizzaObj.name} />
      <div>
        <h3>{pizzaObj.name}</h3>
        <p>{pizzaObj.ingredients}</p>
        <span>{pizzaObj.soldOut ? "SOLD OUT" : pizzaObj.price}</span>
      </div>
    </li>
  );
}

function Footer() {
  const hour = new Date().getHours();
  const openHour = 7;
  const closeHour = 22;
  const isOpen = hour >= openHour && hour <= closeHour;
  console.log(isOpen);

  return (
    <footer className="footer">
      {isOpen ? (
        <Order openHour={openHour} closeHour={closeHour} />
      ) : (
        <p>
          We're happy to welcome you between {openHour}:00 and {closeHour}:00
        </p>
      )}
    </footer>
  ); // short circuiting. React doesn't render `true` or `false` into the DOM
}

function Order({ closeHour, openHour }) {
  // If we try to destructure a property that doesn't exist, it is `undefined`
  return (
    <div className="order">
      <p>
        We're open from {openHour}:00 until {closeHour}:00. Come visit us or
        order online.
      </p>
      <button className="btn">Order</button>
    </div>
  );
}
```

Section Summary

1. Components are the building blocks of a UI in React
2. Each component is a self-contained piece of the UI which includes
    1. Data
    2. JS Logic
    3. Appearance: JSX
        1. JSX: A declarative syntax
            1. It is this block of JSX that is returned from a component (This is what we see on the screen)
            2. It can contain markup
                1. HTML
                2. CSS
                3. JS inside {} (entering the JS mode)
3. A complete application is composed of multiple components organized in a component tree
    1. Component tree
        1. Components at the top (parent component) use the components below them (child components)
            1. Example:
                1. App
                    1. Header
                    2. Menu
                        1. Pizza
                        2. Pizza
                        3. Pizza
                    3. Footer
                        1. Order
    2. To share data between components
        1. Parent components can pass data into child components using **Props**
            1. For each value we pass, we define one prop (property)

        2. Using props, we can configure components as we wish
        2. Props can only be passed down a tree (from parent to child) (never the other way around)
    3. List: Rendering multiple components at once using the JavaScript `.map()` method
        1. There is nothing special in React for loops
            1. We can use JS to construct lists of components of the same type
    4. Components can be conditionally rendered using JS tools: &&, ?, and multipler `return`
4. Theory:
    1. Difference between declarative and imperative programming
    2. Separation of concerns
    3. Why we cannot mutate props
    4. ...

CHALLENGE #2: Profile Card (v2)

1. Code:

```
import "./App.css";

const skills = [
  {
    skill: "Java",
    level: "advanced",
    color: "#C3DCAF",
  },
  {
    skill: "Spring Boot",
    level: "advanced",
    color: "#E84F33",
  },
  {
    skill: "React",
    level: "beginner",
    color: "#FF3B00",
  },
  {
    skill: "Azure",
    level: "intermediate",
    color: "#2662EA",
  },
  {
    skill: "JavaScript",
    level: "intermediate",
    color: "#EFD81D",
  },
];

function App() {
  return (
    <div className="card">
      <Avatar />
      <div className="data">
        <Intro />
        <SkillList />
      </div>
    </div>
  );
}

function Avatar() {
  return (
    <div className="avatar">
      <img src="avatar_hero.jpg" alt="Avatar Hero" width="100%" />
    </div>
  );
}

function Intro() {
  return (
    <div className="intro">
      <h2>Abdullah Mutthoju</h2>
      <p>
        Java Full-stack web developer and architect. When not doing serious
        work, I like to play with my daughter, travel, and shop.
      </p>
    </div>
  );
}

function SkillList() {
  return (
    <div className="skill-list">
      {skills.map((skill) => {
        const emoji =
          skill.level === "advanced"
            ? "⚡"
            : skill.level === "intermediate"
            ? "💰"
            : "❤️";

        return (
          <Skill
            style={{ backgroundColor: skill.color }}
            emoji={emoji}
            skill={skill.skill}
          />
        );
      })}
    </div>
  );
}
```

```
function Skill(props) {
  return (
    <span className="skill" style={props.style}>
      {props.skill} {props.emoji}
    </span>
  );
}

export default App;
```

Section 6: State, Events, and Forms: Interactive Components

Section Overview

1. Making components interactive
    1. Handling **events** (in React)
    2. **State** to update the UI
    3. Building **forms** the "React way"
        1. Using state
    4. **Controlled** elements
2. Flashcards application

Let's Build a Steps Component

1. Events & State
2. Static part of the component
    1. New project
3. Code:

```
const messages = [
  "Learn React ⚛️",
  "Apply for jobs 💼",
  "Invest your new income 🤑",
];

export default function App() {
  const step = 1;

  return (
    <div className="steps">
      <div className="numbers">
        <div className={`${step >= 1 ? "active" : ""}`}>1</div>
        <div className={`${step >= 2 ? "active" : ""}`}>2</div>
        <div className={`${step >= 3 ? "active" : ""}`}>3</div>
      </div>

      <p className="message">
        Step {step}: {messages.at(step - 1)}
      </p>

      <div className="buttons">
        <button style={{ backgroundColor: "#7950f2", color: "#fff" }}>
          Previous
        </button>
        <button style={{ backgroundColor: "#7950f2", color: "#fff" }}>
          Next
        </button>
      </div>
    </div>
  );
}
```

Handling Events the React Way

1. It is straight-forward
2. Declarative approach
    1. Similar to HTML inline event-listener
        1. We listen for event right on the element where it will happen
3. Code:

```
export default function App() {
  const step = 1;

  function handlePrevious() {
    // `handle<Name>` is standard
    alert("Previous");
  }

  function handleNext() {
    // `handle<Name>` is standard
    alert("Next");
  }

  return (
    <div className="steps">
      <div className="numbers">
        <div className={`${step >= 1 ? "active" : ""}`}>1</div>
        <div className={`${step >= 2 ? "active" : ""}`}>2</div>
        <div className={`${step >= 3 ? "active" : ""}`}>3</div>
```

```
          </div>

          <p className="message">
            Step {step}: {messages.at(step - 1)}
          </p>

          <div className="buttons">
            <button
              style={{ backgroundColor: "#7950f2", color: "#fff" }}
              onClick={handlePrevious}
            >
              Previous
            </button>
            <button
              style={{ backgroundColor: "#7950f2", color: "#fff" }}
              onClick={handleNext}
            >
              Next
            </button>
          </div>
        </div>
      );
    }
```

What is State in React?

What we need to Learn

1. **State** is the most important concept in React
    1. (So we will keep learning about state throughout the entire course...)
        1. Everything in React revolves around state
2. What React developers need to learn about state:
    1. **What is state** and **why** do we need it?
    2. How to use state in **practice**?
        1. `useState`
        2. `useReducer`
        3. Context API
        4. Redux (external tool)
    3. **Thinking** about state (how to)
        1. When to use state
        2. Where to place state
        3. Types of state

What is State?

1. What if a component needs to hold its own data and need to hold it over time?
    1. Unlike props that are passed from its parent
2. What if we want to make our app interactive, changing UI as a result of an action?
    1. Solution: State
3. **State**:
    1. Data that a component **can hold over time**, necessary for information that it needs to **remember** throughout the app's lifecycle
    2. It is the **component's memory**
    3. Examples:
        1. Notification count
        2. Text content of an input field
        3. Active tab (in a tabbed component)
        4. Contents of a shopping cart (more complext)
    4. The user can change the valus in a state
        1. When a user reads a notification, the count will go down by one
        2. When a user clicks on another tab, that tab will become active
    5. Piece of state: Each of the pieces of information is a piece of state
        1. State - general term
        2. **State variable/ piece of state**: A single variable in a component (component state)
        3. **State**: The entire state a component is in (the entire condition at a certain point in time)
            1. State = all the pieces of state together
    6. Updating **component state** triggers React to **re-render the component** (in the UI)
        1. Whenever we update a piece of state in a component, this will make React re-render the component in the UI
            1. A new update Component View (Component visually rendered on the screen in the UI) is created
                1. **When one single component is rendered, we call that a view**
                    1. All views together make up the final UI
        2. State is the mechanism used by React to keep UI in sync with data
            1. If we change the state, we change the UI
            2. State allows developers to:
                1. Update the component's view (by re-rendering it)
    7. State allows developers to:
        1. Update the component's view (by re-rendering it)
        2. Persist local variables between multiple renders (and re-renders)
    8. State is a tool (most powerful in React). **Mastering (the mechanics) state will unlock the power of React development**

Constructing a State Variable With useState

1. `useState(<initial-value>)` - It is a hook in React
    1. All functions that start with `use` are react hooks (`useEffect`, `useReducer`, `useState`, ...)
    2. We can only call hooks on the top level of the function (Component function)
        1. **We cannot call it inside `if` statements, another function, or loop**
2. We must only update state using setter function only (not manually)
3. Code:

```
export default function App() {
  // const step = 1;
  // 1. Add a new state variables
  // 2. We use it in code (usually in JSX)
  // 3. We update the state in an event-handler
  const [step, setStep] = useState(1); // takes a default value

  function handlePrevious() {
    // `handle<Name>` is standard
    if (step > 1) setStep(step - 1);
  }

  function handleNext() {
    // `handle<Name>` is standard
    if (step < 3) setStep(step + 1);
  }

  return (
    <div className="steps">
      <div className="numbers">
        <div className={step >= 1 ? "active" : ""}>1</div>
        <div className={step >= 2 ? "active" : ""}>2</div>
        <div className={step >= 3 ? "active" : ""}>3</div>
      </div>

      <p className="message">
        Step {step}: {messages.at(step - 1)}
      </p>

      <div className="buttons">
        <button
          style={{ backgroundColor: "#7950f2", color: "#fff" }}
          onClick={handlePrevious}
        >
          Previous
        </button>
        <button
          style={{ backgroundColor: "#7950f2", color: "#fff" }}
          onClick={handleNext}
        >
          Next
        </button>
      </div>
    </div>
  );
}
```

Don't Set State Manually!

1. Breaking React:

```
let [step, setState] = useState(1);
// ...
step = step = 1; // React doesn't know that this state variable is updated. Nothing happens after this
```

    1. `setState` updates the state value without mutating it
        1. The update operation is possible only through this function
    2. Using array or object:

```
const [test] = useState({ name: "Jonas" });
//
test.name = 'Fred'; // It works
```

        1. It is a really bad-practice (React doesn't want us to do this)
        2. Better way:

```
setTest({ name: 'Fred' });
```

The Mechanics of State

1. We don't do direct DOM manipulations (because React is **declarative**) (we don't touch the DOM)
    1. How is a component view updated them?
        1. **In React, a view is updated by re-rendering the (entire) component** (important React principle) (whenever the underlying data changes)
            1. How?
                1. **React calls the component function again** (each time the component is rendered)
                    1. This is like:
                        1. React removes the existing view & re-renders the updated view when re-render needs to happen
                2. **State is preserved throughout re-renders**
                    1. Even if component is re-rendered again and again, the state is not reset (unless the component disappears from the UI entirely (unmounting))
                    2. **When the state is updated, the component is automatically re-rendered**
2. Flow:
    1. If a user clicks a button
    2. An event handler is triggered
    3. The handler (manually) updates a piece of state (using the `set` function coming from the `useState` hook)
    4. React sees that the state has changed and it will automatically re-render the component
        1. Results in an updated view of the component
3. **Whenever we want to update a view, we update the state**
    1. React will react to the update and do it's thing
        1. React is called React because:
            1. React reacts to state changes by re-rendering the UI

Adding Another Piece of State

1. Code:

```
export default function App() {
  // const step = 1;
  // 1. Add a new state variables
  // 2. We use it in code (usually in JSX)
  // 3. We update the state in an event-handler
  const [step, setStep] = useState(1); // takes a default value
  const [isOpen, setIsOpen] = useState(true);

  function handlePrevious() {
    // `handle<Name>` is standard
    if (step > 1) setStep(step - 1);
  }

  function handleNext() {
    // `handle<Name>` is standard
    if (step < 3) setStep(step + 1);
  }

  return (
    <>
      <button className="close" onClick={() => setIsOpen(!isOpen)}>
        &times;
      </button>
      {isOpen && (
        <div className="steps">
          <div className="numbers">
            <div className={step >= 1 ? "active" : ""}>1</div>
            <div className={step >= 2 ? "active" : ""}>2</div>
            <div className={step >= 3 ? "active" : ""}>3</div>
          </div>

          <p className="message">
            Step {step}: {messages.at(step - 1)}
          </p>

          <div className="buttons">
            <button
              style={{ backgroundColor: "#7950f2", color: "#fff" }}
              onClick={handlePrevious}
            >
              Previous
            </button>
            <button
              style={{ backgroundColor: "#7950f2", color: "#fff" }}
              onClick={handleNext}
            >
              Next
            </button>
          </div>
        </div>
      )}
    </>
  );
}
```

React Developer Tools

1. Open console:
    1. Click on link to download Dev-Tools
        1. Click: Chrome web-store
        2. Two tabs are added:
            1. Components
            2. Profiler
2. Components:
    1. It is for showing the component tree
    2. We can also see state
        1. props
            1. Props the component currently selected is receiving
        2. hooks
            1. One entry for each state hook
                1. We can manipulate the variables to experiment with state
                    1. We can toggle boolean values
                    2. We can change values

Updating State Based on Current State

1. We update a state variable based on the current value of the state
    1. How to do that?
        1. We take the state value returned and return modified state
2. We must not directly update state as we have done using `setStep` function. We pass a function that receives the current value of the state:

```
setStep(s => s - 1);
setStep(s => s + 1);
```

    1. If we use it twice, the state is updated twice

```
setStep(s => s + 1); // Preferred approach if we want to set state based on the current state. This is the more safe way
setStep(s => s + 1);
```

        1. If we don't want to set state based on the current state, we can directly pass the value

```
setStep(25);
```

More Thoughts About State + State Guidelines

One Component,One State

1. Each component has and manages **its own state**, no matter how many times we render the same component
    1. The multiple component instances will operate independent of one another
        1. Example: Multiple instances of the `counter` component
            1. Counters in each component are independent and will not affect the other components
    2. State is isolated inside each component

UI as a Function of State

1. UI = f(state)
    1. UI is representation of all the states of all the components
    2. The application is about changing state over time
        1. Correctly displaying state at all times
2. With state, we view UI as a **reflection of data changing over time**
    1. We **describe that reflection** of data using:
        1. State
        2. Event handlers, and
        3. JSX
3. We describe the UI, and React takes care of the rest

In Practical Terms

1. Practical guidelines about state
    1. Use a state variable for any data that the component should keep track of ("remember") over time. **This is data that will change at some point**. In Vanilla JS, That's a `let` variable, or an `[]`, or `{}` (that we mutate during the application life-cycle)
    2. Whenever you want something in the component to be **dynamic**, construct a piece of state related to that "thing", and update the state when the "thing" should change (aka "be dynamic")
        1. Example: A modal window can be open or closed. So we define a state variable `isOpen` that tracks whether the modal is open or not. On `isOpen = true` we display the window, on `isOpen = false` we hide it.
    3. If you want to change the way a component looks, or the data it displays, **update its state**. This usually happens in an **event handler** function
    4. When building a component, imagine its view as a **reflection of state changing over time**
    5. For data that should not trigger component re-render, **don't use state**. Use a regular variable instead. This is common **beginner mistake** to use state for any variable

A Vanilla JavaScript Implementation

1. Vanilla JS
    1. JS inside HTML
    2. Manual selection of DOM elements
    3. Variables are updated inside event-handler functions
        1. We need to call the function which does DOM manipulation (to keep it in sync with the state)
            1. Imperative code (step by step)
                1. We need DOM manipulation

CHALLENGE #1: Date Counter (v1)

1. Code:

```
import { useState } from "react";
import "./App.css";

function App() {
  return <Counter />;
}

function Counter() {
  const [step, setStep] = useState(1);
  const [count, setCount] = useState(1);
  const [date, setDate] = useState(new Date());

  function handleDecreaseStep() {
    if (step > 0) setStep((step) => step - 1);
  }

  function handleIncreaseStep() {
    setStep((step) => step + 1);
  }

  function handleDecreaseCount() {
    setCount((count) => count - step);
    setDate((date) => {
      const newDate = new Date(date);
      newDate.setDate(newDate.getDate() - step);
      return newDate;
    });
  }

  function handleIncreaseCount() {
    setCount((count) => count + step);
    setDate((date) => {
      const newDate = new Date(date);
      newDate.setDate(newDate.getDate() + step);
      return newDate;
    });
  }

  return (
    <div>
      <div>
```

```
        <button onClick={handleDecreaseStep}>-</button>
        <span>Step: {step}</span>
        <button onClick={handleIncreaseStep}>+</button>
      </div>
      <div>
        <button onClick={handleDecreaseCount}>-</button>
        <span>Count: {count}</span>
        <button onClick={handleIncreaseCount}>+</button>
      </div>
      <div>
        {count} days from today is: {date.toString()}
      </div>
    </div>
  );
}

export default App;
```

Starting a New Project: The "Far Away" Travel List

1. Components:
    1. Logo
    2. Form
    3. PackingList
        1. Item
    4. Stats

Building the Layout

1. Starter file: 05-travel-list/index.css
2. Code:

```
export default function App() {
  return (
    <div className="app">
      <Logo />
      <Form />
      <PackingList />
      <Stats />
    </div>
  );
}

function Logo() {
  return <h1>🌴 Far Away 🧳</h1>;
}

function Form() {
  return (
    <div className="add-form">
      <h3>What do you need for your trip?</h3>
    </div>
  );
}

function PackingList() {
  return <div className="list"></div>;
}

function Stats() {
  return (
    <footer className="stats">
      <em>💼 You have X items on your list, and you already packed X (X%)</em>
    </footer>
  );
}
```

Rendering the Items List

1. Code:

```
const initialItems = [
  { id: 1, description: "Passports", quantity: 2, packed: false },
  { id: 2, description: "Socks", quantity: 12, packed: true },
  { id: 2, description: "Charger", quantity: 1, packed: false },
];

export default function App() {
  return (
    <div className="app">
      <Logo />
      <Form />
      <PackingList />
      <Stats />
    </div>
  );
}

function Logo() {
  return <h1>🌴 Far Away 🧳</h1>;
}

function Form() {
  return (
    <div className="add-form">
      <h3>What do you need for your trip?</h3>
    </div>
  );
```

```
}

function PackingList() {
  return (
    <div className="list">
      <ul className="list">
        {initialItems.map((item) => (
          <Item item={item} />
        ))}
      </ul>
    </div>
  );
}

function Item({ item }) {
  return (
    <li>
      <span style={item.packed ? { textDecoration: "line-through" } : {}}>
        {item.quantity} {item.description}
      </span>
      <button>❌</button>
    </li>
  );
}

function Stats() {
  return (
    <footer className="stats">
      <em>💼 You have X items on your list, and you already packed X (X%)</em>
    </footer>
  );
}
```

Building a Form and Handling Submissions

1. Code:

```
function Form() {
  function handleSubmit(e) {
    e.preventDefault();
    console.log(e);
  }

  return (
    <form className="add-form" onSubmit={handleSubmit}>
      <h3>What do you need for your trip?</h3>
      <select>
        {Array.from({ length: 20 }, (_, i) => i + 1).map((num) => (
          <option key={num} value={num}>
            {num}
          </option>
        ))}
      </select>
      <input type="text" placeholder="Item..." />
      {/* <button onClick={handleClick}>Add</button> */}
    </form>
  );
}
```

Controlled Elements

1. By default, input fields maintain their own state inside the DOM (in the element itself) (it makes it hard to read the values) (values are left in the elements - which is not preferred)
    1. Solution: In React, all state is kept in a central place (inside React app)
        1. Implementation technique: **Controlled elements**
            1. React controls and owns the state of input fields (not the DOM)
                1. Form data needs to be maintained in a state
2. Steps:
    1. Construct a piece of state
    2. Use the state in the JSX as a value: Example: `{description}`
    3. We want to connect the state to the value we type in the form
        1. Since we set the `description` to empty, the input form will stay empty even if we type into it

        ```
        <input
          type="text"
          placeholder="Item..."
          value={description}
          onChange={(e) => setDescription(e.target.value)}
        />
        ```

            1. If we enter anything, it will get synchronized
                1. Whenever we change content in the input fields, `onChange` event is fired-off
3. Code:

```
import { useState } from "react";

const initialItems = [
  { id: 1, description: "Passports", quantity: 2, packed: false },
  { id: 2, description: "Socks", quantity: 12, packed: true },
  { id: 3, description: "Charger", quantity: 1, packed: false },
];
```

```
export default function App() {
  return (
    <div className="app">
      <Logo />
      <Form />
      <PackingList />
      <Stats />
    </div>
  );
}

function Logo() {
  return <h1>🌴 Far Away 👛</h1>;
}

function Form() {
  const [description, setDescription] = useState("");
  const [quantity, setQuantity] = useState(1);

  function handleSubmit(e) {
    e.preventDefault();

    if (!description) return;

    const newItem = { description, quantity, packed: false, id: Date.now() };

    console.log(newItem);

    // Allowing React to keep state in sync with Form elements
    setDescription("");
    setQuantity(1);
  }

  return (
    <form className="add-form" onSubmit={handleSubmit}>
      <h3>What do you need for your trip?</h3>
      <select
        value={quantity}
        onChange={(e) => setQuantity(Number(e.target.value))}
      >
        {Array.from({ length: 20 }, (_, i) => i + 1).map((num) => (
          <option key={num} value={num}>
            {num}
          </option>
        ))}
      </select>
      <input
        type="text"
        placeholder="Item..."
        value={description}
        onChange={(e) => setDescription(e.target.value)}
      />
      <button>Add</button>
      {/* <button onClick={handleClick}>Add</button> */}
    </form>
  );
}

function PackingList() {
  return (
    <div className="list">
      <ul className="list">
        {initialItems.map((item) => (
          <Item item={item} key={item.id} />
        ))}
      </ul>
    </div>
  );
}

function Item({ item }) {
  return (
    <li>
      <span style={item.packed ? { textDecoration: "line-through" } : {}}>
        {item.quantity} {item.description}
      </span>
      <button>❌</button>
    </li>
  );
}

function Stats() {
  return (
    <footer className="stats">
      <em>💼 You have X items on your list, and you already packed X (X%)</em>
    </footer>
  );
}
```

State vs. Props

1. The difference between State and Props
2. Differences:
    1. State:
        1. **Internal** data, owned by component
            1. Component's memory
                1. It can hold data over time (across multiple re-renders)
        2. Can be updated by the component itself

    1. Causes component to be re-rendered by React
3. Updating state causes component to re-render
4. Used to make components interactive

2. Props:

    1. **External** data, owned by parent component

        1. Similar to function parameters
            1. As a communication channel between parent and child components (parents can pass data into children)

    2. Similar to function parameters

    3. Read-only

        1. They cannot be modified by the components receiving them

    4. **Receiving new props causes component to re-render**.

        1. Usually when the parent's state has been updated

```
function Question() {
    const [upvotes, setUpvotes] = useState(0);

    return (
        <div>
            {/* ... */}
            <Button upvotes={upvotes} bgColor="blue" />
        </div>
    );
}

function Button(upvotes, bgColor) {
    const [hovered, setHovered] = useState(false);

    return (
        <div>
            {/* ... */}
            <button
                onMouseEnter={() => setHovered(true)}
                onMouseLeave={() => setHovered(false)}
                style={{ background: bgColor }}>
                {hovered} ? "Upvote" : `${upvotes}`}
            </button>
        </div>
    );
}
```

            1. `upvotes` is state maintained in parent
            2. If `upvotes` is updated, the parent component is re-rendered
            3. Since `upvotes` is passed as a prop to the child component, the child component is also re-rendered (otherwise, the child component will not be in sync with the state)

        2. Whenever a piece of state is passed as a prop, when the state updates, both the parent and child components are re-rendered

            1. **A connection between state & props**

    5. Used by parent to configure child component (prosp are "settings")

EXERCISE #1: Flashcards

1. How to use state in practice in certain situations can be tricky
2. Code:

```
import "./styles.css";
import { useState } from "react";

export default function App() {
  return (
    <div className="App">
      <FlashCards />
    </div>
  );
}

const questions = [
  {
    id: 3457,
    question: "What language is React based on?",
    answer: "JavaScript",
  },
  {
    id: 7336,
    question: "What are the building blocks of React apps?",
    answer: "Components",
  },
  {
    id: 8832,
    question: "What's the name of the syntax we use to describe a UI in React?",
    answer: "JSX",
  },
  {
    id: 1297,
    question: "How to pass data from parent to child components?",
    answer: "Props",
  },
  {
```

```
    id: 9103,
    question: "How to give components memory?",
    answer: "useState hook",
  },
  {
    id: 2002,
    question:
      "What do we call an input element that is completely synchronised with state?",
    answer: "Controlled element",
  },
];

function FlashCards() {
  const [selectedId, setSelectedId] = useState(null);

  function handleClick(id) {
    setSelectedId(id !== selectedId ? id : null);
  }

  return (
    <div className="flashcards">
      {questions.map((question) => (
        <div
          key={question.id}
          onClick={() => handleClick(question.id)}
          className={question.id === selectedId ? "selected" : ""}
        >
          {question.id === selectedId ? question.answer : question.question}
        </div>
      ))}
    </div>
  );
}
```

CHALLENGE #2: Date Counter (v2)

1.

Section 7: Thinking In React: State Management

Section Overview

1. Core skills:
    1. **Thinking** in React
    2. **State** management
    3. **When** and **where** to construct state?
    4. **Derived** state (when and where)
    5. **Lifting** up state
        1. How to communicate between parent and child components (by lifting the state up)

What is "Thinking in React"?

Thinking in React is a Core Skill

1. Building React applications requires completely different mindset
    1. It is different from building applications from vanilla JS
2. React Mindset:
    1. How to work with React API
        1. Functions (`useState`)
    2. Thinking in React
    3. Intersection: This is where professional React apps are built
3. Thinking in React means?
    1. Thinking about a very good mental model of how, and when to use all the React tools like:
        1. Components
        2. State
        3. Props
        4. Data flow
        5. Effects
        6. etc.
    2. Thinking in **state transitions**, not element mutations

Thinking in React as a Process

1. The "Thinking in React" Process: (helps us build apps in a more structured way)
    1. Break the desired UI into **components** and establish the **component tree** (how the components are related to one-another)
        1. Thinking about re-usability and composability of components
    2. Build a **static** version in React (without state and interactivity)
        1. We do most of the coding up-front, before thinking about state and interactivity
    3. Think about **state**: (state management)
        1. When to use state
        2. Types of state: local vs. global
        3. Where to place each piece of state
    4. Establish **data flow**: Thinking about (state management)
        1. One-way data flow
        2. Child-to-parent communication
        3. Accessing global state
2. There is always back-and-forth among the steps (it is not linear)
3. When you know how to "think in React", you will be able to answer:
    1. How to break up a UI design into components?
    2. How to make some components reusable?

3. How to assemble UI from reusable components?
4. What pieces of state do I need for interactivity?
5. Where to place? (What component should "own" each piece of state?)
6. What types of state can or should I use?
7. How to make data flow through app?
4. We will master the skills by practice and writing lots of code

Fundamentals of State Management

1. Managing state is the most important aspect when it comes to thinking in React

What is State Management?

1. We can use `useState` to construct multiple pieces of state to track data that changes over the life-cycle of an application
    1. Example: Udemy
        1. `searchQuery`
        2. `shoppingCart`
        3. `coupons`
        4. `notifications`
        5. `language`
        6. `isOpen`
        7. `user`
    2. Questions:
        1. How do we know we need these pieces of state?
        2. How do we know where to place them (inside the code)?
    3. Answer: State management
        1. It can be defined in different ways by different people
        2. A definition:
            1. **State management:** (Giving each piece of state a **home** - it becomes important as the application grows)
                1. Deciding **when** to define pieces of state
                2. What **types** of state are necessary
                3. **Where** to place each piece of state
                4. How data **flows** through the app

Types of State: Local vs. Global State

1. Local State
    1. State needed **only by one or few components** (child or sibling components)
    2. State that is defined in a component and **only that component and child components** have access to it (by passing via props)
        1. Using `useState` inside a component
            1. Example: Input text in the search-bar (only that component needs to know about the state)
2. Global State
    1. State that **many components** (in the app) might need
    2. **Shared** state that is accessible to **every component** in the entire application
        1. Shared between all components
    3. Implementation:
        1. Context API (React's)
        2. Redux
            1. External global state management library
    4. Example:
        1. Udemy: Shopping cart
            1. Almost all components needs access to this state
3. **Important guideline: We should always start with local state**
    1. Move it to Global state if we truly need it

State: When and Where?

1. If we realize that we: Need to store data
    1. When does this happen?
        1. Ask: Will data change at some point?
            1. If No: Regular `const` variable
            2. If Yes:
                1. Can it be computed from existing state/props?
                    1. Yes: **Derive state** (calculate it based on already existing state or prop)
                    2. No:
                        1. Should it re-render component? (because, updating state always re-renders a component)
                            1. No: Ref (`useRef` (it persists data over time, but does not re-render a component, later)
                            2. Yes:
                                1. **Place a new piece of state in component** (Using `useState`) (in the component we are building)
                                    1. Always start with local state
    2. Where to place state?
        1. Is state used by only this component?
            1. Yes: Leave it in the component
            2. No:
                1. Also used by a child component?
                    1. Yes: Pass the state to child via props
                    2. No:
                        1. Is the state used by one or a few sibling components (or parent component)?
                            1. Yes: **Lift state up to first common parent component**
                                1. **Lifting state up**
                            2. No (Used by many other components):
                                1. Probably **global state**. Global state management later in the course
2. The above becomes intuitive later

Thinking About State and Lifting State Up

1. Flow chart for items:
    1. Will data change at some point?
        1. Yes: Each time we add a new item to the items list
    2. Can the data be computed from existing state/props?
        1. No: We cannot derive the state
    3. Should it re-render component (whenever it is updated)?
        1. Yes
    4. **Place a new piece of state in the component**
    5. Was it only used by this component?
        1. No
    6. Is the state also needed by a child component?
        1. No: It is used by a sibling component
            1. **Lift state up to first common parent**

2. Code:

```
import { useState } from "react";

const initialItems = [
  { id: 1, description: "Passports", quantity: 2, packed: false },
  { id: 2, description: "Socks", quantity: 12, packed: true },
  { id: 3, description: "Charger", quantity: 1, packed: false },
];

export default function App() {
  const [items, setItems] = useState([]);

  // The state and logic to update the state are in this component (the home of the state)
  // The function is passed as a prop
  // This is lifting-up a state
  function handleAddItem(item) {
    setItems((items) => [...items, item]);
  }

  return (
    <div className="app">
      <Logo />
      <Form onAddItem={handleAddItem} />
      <PackingList items={items} />
      <Stats />
    </div>
  );
}

function Logo() {
  return <h1>🌴 Far Away 👛</h1>;
}

function Form({ onAddItem }) {
  const [description, setDescription] = useState("");
  const [quantity, setQuantity] = useState(1);

  function handleSubmit(e) {
    e.preventDefault();

    if (!description) return;

    const newItem = { description, quantity, packed: false, id: Date.now() };

    console.log(newItem);

    onAddItem(newItem);

    // Allowing React to keep state in sync with Form elements
    setDescription("");
    setQuantity(1);
  }

  return (
    <form className="add-form" onSubmit={handleSubmit}>
      <h3>What do you need for your trip?</h3>
      <select
        value={quantity}
        onChange={(e) => setQuantity(Number(e.target.value))}
      >
        {Array.from({ length: 20 }, (_, i) => i + 1).map((num) => (
          <option key={num} value={num}>
            {num}
          </option>
        ))}
      </select>
      <input
        type="text"
        placeholder="Item..."
        value={description}
        onChange={(e) => setDescription(e.target.value)}
      />
      <button>Add</button>
      {/* <button onClick={handleClick}>Add</button> */}
    </form>
  );
}

function PackingList({ items }) {
  return (
    <div className="list">
      <ul className="list">
        {items.map((item) => (
          <Item item={item} key={item.id} />
        ))}
```

```
        </ul>
      </div>
    );
  }

  function Item({ item }) {
    return (
      <li>
        <span style={item.packed ? { textDecoration: "line-through" } : {}}>
          {item.quantity} {item.description}
        </span>
        <button>❌</button>
      </li>
    );
  }

  function Stats() {
    return (
      <footer className="stats">
        <em>💼 You have X items on your list, and you already packed X (X%)</em>
      </footer>
    );
  }
```

Reviewing "Lifting Up State"

Problem: Sharing State with Sibling Component

1. Example:
    1. Checkout
        1. Promotions (user can input a coupon codes - they will be added to a list of coupons) (It is local to promotions component along with `setCoupons` function)
            1. coupons
            2. setCoupons
        2. Total
            1. Needs access to `coupons` state
                1. To know what discounts to apply and what price to display
                2. How to give access to `coupons` state?
                    1. Solution: Lift it up
2. Lift state up:
    1. To place a state in a component that is a parent of both components that need the state in question
        1. Example: We lift `coupons` and `setState` to `Checkout` component (state was lifted up to the closest common parent)
    2. Giving access to the state to both the components is just passing it using props to them
    3. By lifting state up, we have successfully shared one piece of state with multiple components in different positions in the component tree
    4. **We must really get used to this pattern**
        1. We need this pattern because of `React`'s one-way data flow
3. What will happen if user adds a new coupon?
    1. How to update the `coupons` state?
        1. We cannot mutate props
        2. If data flows from parent to children, how can `Promotions` (child) update state in `Checkout` (parent)?
            1. Solution: Pass `setCoupons` function as a prop to the component that needs to update the state 2. The child component can call the function to update the state in the parent component
    2. **Child-to-parent communication (inverse data flow):** child updating parent state (data "flowing" **up**)

Deleting an Item: More Child-to-Parent Communication!

1. Code:

```
import { useState } from "react";

const initialItems = [
  { id: 1, description: "Passports", quantity: 2, packed: false },
  { id: 2, description: "Socks", quantity: 12, packed: true },
  { id: 3, description: "Charger", quantity: 1, packed: false },
];

export default function App() {
  const [items, setItems] = useState([]);

  // The state and logic to update the state are in this component (the home of the state)
  // The function is passed as a prop
  // This is lifting-up a state
  function handleAddItem(item) {
    setItems((items) => [...items, item]);
  }

  function handleDeleteItem(id) {
    setItems((items) => items.filter((item) => item.id !== id));
  }

  return (
    <div className="app">
      <Logo />
      <Form onAddItem={handleAddItem} />
      <PackingList items={items} onDeleteItem={handleDeleteItem} />
      <Stats />
    </div>
  );
}

function Logo() {
  return <h1>🌴 Far Away 👜</h1>;
}

function Form({ onAddItem }) {
  const [description, setDescription] = useState("");
```

```
  const [quantity, setQuantity] = useState(1);

  function handleSubmit(e) {
    e.preventDefault();

    if (!description) return;

    const newItem = { description, quantity, packed: false, id: Date.now() };

    console.log(newItem);

    onAddItem(newItem);

    // Allowing React to keep state in sync with Form elements
    setDescription("");
    setQuantity(1);
  }

  return (
    <form className="add-form" onSubmit={handleSubmit}>
      <h3>What do you need for your trip?</h3>
      <select
        value={quantity}
        onChange={(e) => setQuantity(Number(e.target.value))}
      >
        {Array.from({ length: 20 }, (_, i) => i + 1).map((num) => (
          <option key={num} value={num}>
            {num}
          </option>
        ))}
      </select>
      <input
        type="text"
        placeholder="Item..."
        value={description}
        onChange={(e) => setDescription(e.target.value)}
      />
      <button>Add</button>
      {/* <button onClick={handleClick}>Add</button> */}
    </form>
  );
}

function PackingList({ items, onDeleteItem }) {
  return (
    <div className="list">
      <ul className="list">
        {items.map((item) => (
          <Item item={item} key={item.id} onDeleteItem={onDeleteItem} />
        ))}
      </ul>
    </div>
  );
}

function Item({ item, onDeleteItem }) {
  return (
    <li>
      <span style={item.packed ? { textDecoration: "line-through" } : {}}>
        {item.quantity} {item.description}
      </span>
      <button onClick={() => onDeleteItem(item.id)}>❌</button>
    </li>
  );
}

function Stats() {
  return (
    <footer className="stats">
      <em>💼 You have X items on your list, and you already packed X (X%)</em>
    </footer>
  );
}
```

Updating an Item: Complex Immutable Data Operation

1. Code:

```
import { useState } from "react";

const initialItems = [
  { id: 1, description: "Passports", quantity: 2, packed: false },
  { id: 2, description: "Socks", quantity: 12, packed: true },
  { id: 3, description: "Charger", quantity: 1, packed: false },
];

export default function App() {
  const [items, setItems] = useState([]);

  // The state and logic to update the state are in this component (the home of the state)
  // The function is passed as a prop
  // This is lifting-up a state
  function handleAddItem(item) {
    setItems((items) => [...items, item]);
  }

  function handleDeleteItem(id) {
    setItems((items) => items.filter((item) => item.id !== id));
  }

  function handleToggleItem(id) {
    setItems((items) =>
      items.map((item) =>
```

```
          item.id === id ? { ...item, packed: !item.packed } : item
        )
      );
    }

    return (
      <div className="app">
        <Logo />
        <Form onAddItem={handleAddItem} />
        <PackingList
          items={items}
          onDeleteItem={handleDeleteItem}
          onToggleItem={handleToggleItem}
        />
        <Stats />
      </div>
    );
  }

  function Logo() {
    return <h1>🌴 Far Away 👜</h1>;
  }

  function Form({ onAddItem }) {
    const [description, setDescription] = useState("");
    const [quantity, setQuantity] = useState(1);

    function handleSubmit(e) {
      e.preventDefault();

      if (!description) return;

      const newItem = { description, quantity, packed: false, id: Date.now() };

      console.log(newItem);

      onAddItem(newItem);

      // Allowing React to keep state in sync with Form elements
      setDescription("");
      setQuantity(1);
    }

    return (
      <form className="add-form" onSubmit={handleSubmit}>
        <h3>What do you need for your trip?</h3>
        <select
          value={quantity}
          onChange={(e) => setQuantity(Number(e.target.value))}
        >
          {Array.from({ length: 20 }, (_, i) => i + 1).map((num) => (
            <option key={num} value={num}>
              {num}
            </option>
          ))}
        </select>
        <input
          type="text"
          placeholder="Item..."
          value={description}
          onChange={(e) => setDescription(e.target.value)}
        />
        <button>Add</button>
        {/* <button onClick={handleClick}>Add</button> */}
      </form>
    );
  }

  function PackingList({ items, onDeleteItem, onToggleItem }) {
    return (
      <div className="list">
        <ul className="list">
          {items.map((item) => (
            <Item
              item={item}
              key={item.id}
              onDeleteItem={onDeleteItem}
              onToggleItem={onToggleItem}
            />
          ))}
        </ul>
      </div>
    );
  }

  function Item({ item, onDeleteItem, onToggleItem }) {
    return (
      <li>
        <input
          type="checkbox"
          value={item.packed}
          onChange={() => {
            onToggleItem(item.id);
          }}
        />
        <span style={item.packed ? { textDecoration: "line-through" } : {}}>
          {item.quantity} {item.description}
        </span>
        <button onClick={() => onDeleteItem(item.id)}>❌</button>
      </li>
    );
  }

  function Stats() {
```

```
    return (
      <footer className="stats">
        <em>💼 You have X items on your list, and you already packed X (X%)</em>
      </footer>
    );
  }
```

Derived State

1. Derived state: state that is computed from an existing piece of state or from props
    1. Example: Three separate pieces of state, even though `numItems`, and `totalPrice` depend on `cart`
    2. Problems:
        1. Need to keep them in sync (update together)
            1. Whenever we update `cart`, we need to update `numItems`, and `totalPrice`
        2. 3 state updates will cause 3 re-renders
    3. Solution:
        1. We can derive `numItems`, and `totalPrice` from `cart` and store them in separate variables

        ```
        const numItems = cart.length
        const totalPrice = cart.reduce((acc, cur) => acc + cur.price, 0);
        ```

    4. Just regular variables, no `useState`
    5. `cart` state is the **single source of truth** for this related data
        1. Everything will stay in sync
            1. If `cart` is updated, the component is re-rendered, the function is called again as a result, and the other variables are updated
    6. Works because re-rendering component will **automatically re-calculate** derived state
    7. If one state can be computed from another, prefer derived state

Calculating Statistics as Derived State

1. Code:

```
function Stats({ items }) {
  if (!items.length)
    return (
      <p className="stats">
        <em>Start adding some items to your packing list 🚀</em>
      </p>
    );

  const numItems = items.length;
  const numPacked = items.filter((item) => item.packed).length;
  const percentage = Math.round((numPacked / numItems) * 100);

  return (
    <footer className="stats">
      <em>
        {percentage === 100
          ? "You got everything! Ready to go ✈️"
          : `💼 You have ${numItems} items on your list, and you already packed ${numPacked} (${percentage}%)`}
      </em>
    </footer>
  );
}
```

Sorting Items

1. Code:

```
function PackingList({ items, onDeleteItem, onToggleItem }) {
  const [sortBy, setSortBy] = useState("input");

  let sortedItems;

  if (sortBy === "input") sortedItems = items;
  if (sortBy === "description")
    sortedItems = items
      .slice()
      .sort((a, b) => a.description.localeCompare(b.description));

  if (sortBy === "packed")
    sortedItems = items
      .slice()
      .sort((a, b) => Number(a.packed) - Number(b.packed));

  return (
    <div className="list">
      <ul className="list">
        {sortedItems.map((item) => (
          <Item
            item={item}
            key={item.id}
            onDeleteItem={onDeleteItem}
            onToggleItem={onToggleItem}
          />
        ))}
      </ul>
```

```
        <div className="actions">
          <select value={sortBy} onChange={(e) => setSortBy(e.target.value)}>
            <option value="input">Sort by input order</option>
            <option value="description">Sort by description</option>
            <option value="packed">Sort by packed status</option>
          </select>
        </div>
      </div>
    );
  }
```

Clearing the List

1. Code:

```
import { useState } from "react";

const initialItems = [
  { id: 1, description: "Passports", quantity: 2, packed: false },
  { id: 2, description: "Socks", quantity: 12, packed: true },
  { id: 3, description: "Charger", quantity: 1, packed: false },
];

export default function App() {
  const [items, setItems] = useState([]);

  // The state and logic to update the state are in this component (the home of the state)
  // The function is passed as a prop
  // This is lifting-up a state
  function handleAddItem(item) {
    setItems((items) => [...items, item]);
  }

  function handleDeleteItem(id) {
    setItems((items) => items.filter((item) => item.id !== id));
  }

  function handleToggleItem(id) {
    setItems((items) =>
      items.map((item) =>
        item.id === id ? { ...item, packed: !item.packed } : item
      )
    );
  }

  function handleClearItems() {
    const confirmed = window.confirm(
      "Are you sure you want to delete all items?"
    );
    if (confirmed) setItems(() => []);
  }

  return (
    <div className="app">
      <Logo />
      <Form onAddItem={handleAddItem} />
      <PackingList
        items={items}
        onDeleteItem={handleDeleteItem}
        onToggleItem={handleToggleItem}
        onClearItems={handleClearItems}
      />
      <Stats items={items} />
    </div>
  );
}

function Logo() {
  return <h1>🌴 Far Away 👜</h1>;
}

function Form({ onAddItem }) {
  const [description, setDescription] = useState("");
  const [quantity, setQuantity] = useState(1);

  function handleSubmit(e) {
    e.preventDefault();

    if (!description) return;

    const newItem = { description, quantity, packed: false, id: Date.now() };

    console.log(newItem);

    onAddItem(newItem);

    // Allowing React to keep state in sync with Form elements
    setDescription("");
    setQuantity(1);
  }

  return (
    <form className="add-form" onSubmit={handleSubmit}>
      <h3>What do you need for your trip?</h3>
      <select
        value={quantity}
        onChange={(e) => setQuantity(Number(e.target.value))}
      >
        {Array.from({ length: 20 }, (_, i) => i + 1).map((num) => (
          <option key={num} value={num}>
            {num}
          </option>
        ))}
```

```
        </select>
        <input
          type="text"
          placeholder="Item..."
          value={description}
          onChange={(e) => setDescription(e.target.value)}
        />
        <button>Add</button>
        {/* <button onClick={handleClick}>Add</button> */}
      </form>
    );
  }

  function PackingList({ items, onDeleteItem, onToggleItem, onClearItems }) {
    const [sortBy, setSortBy] = useState("input");

    let sortedItems;

    if (sortBy === "input") sortedItems = items;
    if (sortBy === "description")
      sortedItems = items
        .slice()
        .sort((a, b) => a.description.localeCompare(b.description));

    if (sortBy === "packed")
      sortedItems = items
        .slice()
        .sort((a, b) => Number(a.packed) - Number(b.packed));

    return (
      <div className="list">
        <ul className="list">
          {sortedItems.map((item) => (
            <Item
              item={item}
              key={item.id}
              onDeleteItem={onDeleteItem}
              onToggleItem={onToggleItem}
            />
          ))}
        </ul>

        <div className="actions">
          <select value={sortBy} onChange={(e) => setSortBy(e.target.value)}>
            <option value="input">Sort by input order</option>
            <option value="description">Sort by description</option>
            <option value="packed">Sort by packed status</option>
          </select>
          <button onClick={onClearItems}>Clear List</button>
        </div>
      </div>
    );
  }

  function Item({ item, onDeleteItem, onToggleItem }) {
    return (
      <li>
        <input
          type="checkbox"
          value={item.packed}
          onChange={() => {
            onToggleItem(item.id);
          }}
        />
        <span style={item.packed ? { textDecoration: "line-through" } : {}}>
          {item.quantity} {item.description}
        </span>
        <button onClick={() => onDeleteItem(item.id)}>❌</button>
      </li>
    );
  }

  function Stats({ items }) {
    if (!items.length)
      return (
        <p className="stats">
          <em>Start adding some items to your packing list 🚀</em>
        </p>
      );

    const numItems = items.length;
    const numPacked = items.filter((item) => item.packed).length;
    const percentage = Math.round((numPacked / numItems) * 100);

    return (
      <footer className="stats">
        <em>
          {percentage === 100
            ? "You got everything! Ready to go ✈️"
            : `💼 You have ${numItems} items on your list, and you already packed ${numPacked} (${percentage}%)`}
        </em>
      </footer>
    );
  }
```

Moving Components Into Separate Files

1. Select the component
    1. Right click > Refactor > Move to a new file
2. We use Default exports for components

EXERCISE #1: Accordion Component (v1)

1. Code:

```
import "./styles.css";
import { useState } from "react";

const faqs = [
  {
    title: "Where are these chairs assembled?",
    text: "Lorem ipsum dolor sit amet consectetur, adipisicing elit. Accusantium, quaerat temporibus quas dolore provident nisi ut aliquid ratio
  },
  {
    title: "How long do I have to return my chair?",
    text: "Pariatur recusandae dignissimos fuga voluptas unde optio nesciunt commodi beatae, explicabo natus.",
  },
  {
    title: "Do you ship to countries outside the EU?",
    text: "Excepturi velit laborum, perspiciatis nemo perferendis reiciendis aliquam possimus dolor sed! Dolore laborum ducimus veritatis facere
  },
];

export default function App() {
  return (
    <div>
      <Accordion data={faqs} />
    </div>
  );
}

function Accordion({ data }) {
  return (
    <div className="accordion">
      {data.map((el, i) => (
        <AccordionItem num={i} title={el.title} text={el.text} key={el.title} />
      ))}
    </div>
  );
}

function AccordionItem({ num, title, text }) {
  const [isOpen, setIsOpen] = useState(false);

  function handleToggle() {
    setIsOpen((isOpen) => !isOpen);
  }

  return (
    <div className={`item ${isOpen ? "open" : ""}`} onClick={handleToggle}>
      <p className="number">{num < 9 ? `0${num + 1}` : num + 1}</p>
      <p className="text">{title}</p>
      <p className="icon">{isOpen ? "-" : "+"}</p>
      {isOpen && <div className="content-box">{text}</div>}
    </div>
  );
}
```

The "children" Prop: Making a Reusable Button

1. `children` prop is a prop that each React component automatically receives.
    1. The value of `children` is the one between opening and closing tag of the component
        1. One of the most useful features
            1. Makes the component truly re-usable
        2. The component will just take the JSX and render it
            1. A hole that can be filled by us passing the content into the component

The Children Prop

1. Button:
    1. props.children
        1. An empty **"hole"** that can be filled by **any JSX the component receives** as children
    2. How do we pass in the children?
        1. JSX in between opening and closing tags of the component
            1. Example: Children of `Button`, accessible through `props.children`
2. The children prop allow us **to pass JSX into an element** (besides regular props)
3. Essential tools to make **reusable** and **configurable** components (especially component **content**)
    1. Example, we want a button with diferent emojis and text
        1. Pass in a different JSX, and the button will get completely different content
4. Really useful for **generic** components that **don't know their content** before being used (e.g. modal)
    1. Generic Slider
    2. Generic Button
    3. ...

More Reusable With the "children" Prop

1. Code:

```
import { useState } from "react";

const messages = [
  "Learn React ⚛️",
  "Apply for jobs 💼",
  "Invest your new income 🤑",
];

export default function App() {
```

```
  return (
    <div>
      <Steps />
      <StepMessage step={1}>
        <p>Pass in content</p>
        <p>🏋️</p>
      </StepMessage>
    </div>
  );
}

function Steps() {
  // const step = 1;
  // 1. Add a new state variables
  // 2. We use it in code (usually in JSX)
  // 3. We update the state in an event-handler
  const [step, setStep] = useState(1); // takes a default value
  const [isOpen, setIsOpen] = useState(true);

  function handlePrevious() {
    // `handle<Name>` is standard
    if (step > 1) setStep((s) => s - 1);
  }

  function handleNext() {
    // `handle<Name>` is standard
    if (step < 3) setStep((s) => s + 1);
  }

  return (
    <div>
      <button className="close" onClick={() => setIsOpen((io) => !io)}>
        &times;
      </button>
      {isOpen && (
        <div className="steps">
          <div className="numbers">
            <div className={step >= 1 ? "active" : ""}>1</div>
            <div className={step >= 2 ? "active" : ""}>2</div>
            <div className={step >= 3 ? "active" : ""}>3</div>
          </div>

          <StepMessage step={step}>
            {messages[step - 1]}
            <div className="buttons">
              <Button
                bgColor="#e7e7e7"
                textColor="#333"
                onClick={() => alert(`Learn how to ${messages[step - 1]}`)}
              >
                Learn how
              </Button>
            </div>
          </StepMessage>

          <div className="buttons">
            <Button bgColor="#7950f2" textColor="#fff" onClick={handlePrevious}>
              <span>⬅️</span> Previous
            </Button>
            <Button bgColor="#7950f2" textColor="#fff" onClick={handleNext}>
              Next <span>➡️</span>
            </Button>
          </div>
        </div>
      )}
    </div>
  );
}

function StepMessage({ step, children }) {
  return (
    <div className="message">
      <h3>Step {step}</h3>
      {children}
    </div>
  );
}

function Button({ textColor, bgColor, onClick, children }) {
  return (
    <button
      style={{ backgroundColor: bgColor, color: textColor }}
      onClick={onClick}
    >
      {children}
    </button>
  );
}
```

EXERCISE #2: Accordion Component (v2)

1. Code:

```
import { useState } from "react";

const messages = [
  "Learn React ⚛️",
  "Apply for jobs 💼",
  "Invest your new income 🤑",
];

export default function App() {
```

```
  return (
    <div>
      <Steps />
      <StepMessage step={1}>
        <p>Pass in content</p>
        <p>🌴</p>
      </StepMessage>
    </div>
  );
}

function Steps() {
  // const step = 1;
  // 1. Add a new state variables
  // 2. We use it in code (usually in JSX)
  // 3. We update the state in an event-handler
  const [step, setStep] = useState(1); // takes a default value
  const [isOpen, setIsOpen] = useState(true);

  function handlePrevious() {
    // `handle<Name>` is standard
    if (step > 1) setStep((s) => s - 1);
  }

  function handleNext() {
    // `handle<Name>` is standard
    if (step < 3) setStep((s) => s + 1);
  }

  return (
    <div>
      <button className="close" onClick={() => setIsOpen((io) => !io)}>
        &times;
      </button>
      {isOpen && (
        <div className="steps">
          <div className="numbers">
            <div className={step >= 1 ? "active" : ""}>1</div>
            <div className={step >= 2 ? "active" : ""}>2</div>
            <div className={step >= 3 ? "active" : ""}>3</div>
          </div>

          <StepMessage step={step}>
            {messages[step - 1]}
            <div className="buttons">
              <Button
                bgColor="#e7e7e7"
                textColor="#333"
                onClick={() => alert(`Learn how to ${messages[step - 1]}`)}
              >
                Learn how
              </Button>
            </div>
          </StepMessage>

          <div className="buttons">
            <Button bgColor="#7950f2" textColor="#fff" onClick={handlePrevious}>
              <span>⬅️</span> Previous
            </Button>
            <Button bgColor="#7950f2" textColor="#fff" onClick={handleNext}>
              Next <span>➡️</span>
            </Button>
          </div>
        </div>
      )}
    </div>
  );
}

function StepMessage({ step, children }) {
  return (
    <div className="message">
      <h3>Step {step}</h3>
      {children}
    </div>
  );
}

function Button({ textColor, bgColor, onClick, children }) {
  return (
    <button
      style={{ backgroundColor: bgColor, color: textColor }}
      onClick={onClick}
    >
      {children}
    </button>
  );
}
```

CHALLENGE #1: Tip Calculator

1. Code:

```
import { useState } from "react";
import "./App.css";

function App() {
  const tipValues = [0, 5, 10, 20];
  const [bill, setBill] = useState(0);
  const [firstTip, setFirstTip] = useState(tipValues[0]);
  const [secondTip, setSecondTip] = useState(tipValues[0]);
  let tip = (firstTip + secondTip) * bill * 0.005;
  let total = bill + tip;
```

```
  function handleReset() {
    setBill(0);
    setFirstTip(tipValues[0]);
    setSecondTip(tipValues[0]);
  }

  return (
    <div>
      <BillInput bill={bill} onBillChange={setBill} />
      <SelectPercentage
        text="How did you like the service?"
        tipValues={tipValues}
        tip={firstTip}
        onTipChange={setFirstTip}
      />
      <SelectPercentage
        text="How did your friend like the service?"
        tipValues={tipValues}
        tip={secondTip}
        onTipChange={setSecondTip}
      />
      {total > 0 ? (
        <Output bill={bill} tip={tip} total={total} />
      ) : (
        <h2> </h2>
      )}
      <button onClick={handleReset}>Reset</button>
    </div>
  );
}

function BillInput({ bill, onBillChange }) {
  function handleBillChange(bill) {
    onBillChange(Number(bill));
  }

  return (
    <div>
      <span>How much was the bill?</span>
      <input
        type="text"
        value={bill}
        onChange={(e) => handleBillChange(e.target.value)}
      ></input>
    </div>
  );
}

function SelectPercentage({ text, tipValues, tip, onTipChange }) {
  function handleTipChange(newTip) {
    onTipChange(Number(newTip));
  }

  return (
    <div>
      <span>{text}</span>
      <select value={tip} onChange={(e) => handleTipChange(e.target.value)}>
        <option value={tipValues[0]}>Dissatisfied (0%)</option>
        <option value={tipValues[1]}>It was okay (5%)</option>
        <option value={tipValues[2]}>It was good (10%)</option>
        <option value={tipValues[3]}>Absolutely Amazing (20%)</option>
      </select>
    </div>
  );
}

function Output({ bill, tip, total }) {
  return <h2>{`You pay $${total} ($${bill} + $${tip} tp)`}</h2>;
}

export default App;
```

Section 8: [Optional] Practice Project: Eat-'N-Split

Section Overview

1. Topics:
    1. **Review** everything we learned
    2. **No** new concepts!
    3. This practice is **extremely important for your progress**

Project Setup

1. `npx create-reate-app@5 eat-n-split`
2. We can go to dinner or lunch and split the bill
3. Split UI into components
    1. It is subjective
        1. We have some techniques
    2. `FriendList`
        1. `Friend`
    3. `Form`
    4. `BillData`

Building the Static App: List of Friends

1. Code:

```
const initialFriends = [
  {
    id: 118836,
    name: "Clark",
    image: "https://i.pravatar.cc/48?u=118836",
    balance: -7,
  },
  {
    id: 933372,
    name: "Sarah",
    image: "https://i.pravatar.cc/48?u=933372",
    balance: 20,
  },
  {
    id: 499476,
    name: "Anthony",
    image: "https://i.pravatar.cc/48?u=499476",
    balance: 0,
  },
];

export default function App() {
  return (
    <div className="app">
      <div className="sidebar">
        <FriendsList />
      </div>
    </div>
  );
}

function FriendsList() {
  const friends = initialFriends;

  return (
    <ul>
      {friends.map((friend) => (
        <Friend friend={friend} key={friend.id} />
      ))}
    </ul>
  );
}

function Friend({ friend }) {
  return (
    <li key={friend.id}>
      <img src={friend.image} alt={friend.name} />
      <h3>{friend.name}</h3>
      {friend.balance < 0 && (
        <p className="red">
          You owe {friend.name} {Math.abs(friend.balance)}💶
        </p>
      )}
      {friend.balance > 0 && (
        <p className="green">
          {friend.name} owes you {Math.abs(friend.balance)}💶
        </p>
      )}
      {friend.balance === 0 && <p>You and {friend.name} are even</p>}

      <button className="button">Select</button>
    </li>
  );
}
```

Building the Static App: Forms

1. Code:

```
const initialFriends = [
  {
    id: 118836,
    name: "Clark",
    image: "https://i.pravatar.cc/48?u=118836",
    balance: -7,
  },
  {
    id: 933372,
    name: "Sarah",
    image: "https://i.pravatar.cc/48?u=933372",
    balance: 20,
  },
  {
    id: 499476,
    name: "Anthony",
    image: "https://i.pravatar.cc/48?u=499476",
    balance: 0,
  },
];

export default function App() {
  return (
    <div className="app">
      <div className="sidebar">
        <FriendsList />
        <FormAddFriend />
        <Button>Add Friend</Button>
      </div>

      <FormSplitBill />
    </div>
  );
}
```

```
function FriendsList() {
  const friends = initialFriends;

  return (
    <ul>
      {friends.map((friend) => (
        <Friend friend={friend} key={friend.id} />
      ))}
    </ul>
  );
}

function Friend({ friend }) {
  return (
    <li key={friend.id}>
      <img src={friend.image} alt={friend.name} />
      <h3>{friend.name}</h3>
      {friend.balance < 0 && (
        <p className="red">
          You owe {friend.name} {Math.abs(friend.balance)}💶
        </p>
      )}
      {friend.balance > 0 && (
        <p className="green">
          {friend.name} owes you {Math.abs(friend.balance)}💶
        </p>
      )}
      {friend.balance === 0 && <p>You and {friend.name} are even</p>}

      <Button>Select</Button>
    </li>
  );
}

function Button({ children }) {
  return <button className="button">{children}</button>;
}

function FormAddFriend() {
  return (
    <form className="form-add-friend">
      <label>🎉 Friend name</label>
      <input type="text" />

      <label>🌄 Image URL</label>
      <input type="text" />

      <Button>Add</Button>
    </form>
  );
}

function FormSplitBill() {
  return (
    <form className="form-split-bill">
      <h2>Split a bill with X</h2>

      <label>💶 Bill Value</label>
      <input type="text" />

      <label>🧍 Your expense</label>
      <input type="text" />

      <label>🧳 X's expense</label>
      <input type="text" disabled />

      <label>🪙 Who is paying the bill</label>
      <select>
        <option value="user">You</option>
        <option value="friend">X</option>
      </select>

      <Button>Split Bill</Button>
    </form>
  );
}
```

Displaying the New Friend Form

1. If we want UI to re-render, we might need a state
2. Code:

```
function Button({ children, onClick }) {
  return (
    <button className="button" onClick={onClick}>
      {children}
    </button>
  );
}

export default function App() {
  const [showAddFriend, setShowAddFriend] = useState(false);

  function handleShowAddFriend() {
    setShowAddFriend((show) => !show);
  }

  return (
    <div className="app">
      <div className="sidebar">
```

```
        <FriendsList />
        {showAddFriend && <FormAddFriend />}
        <Button onClick={handleShowAddFriend}>
          {showAddFriend ? "Close" : "Add Friend"}
        </Button>
      </div>

      <FormSplitBill />
    </div>
  );
}
```

Adding a New Friend

1. Code:

```
import { useState } from "react";

const initialFriends = [
  {
    id: 118836,
    name: "Clark",
    image: "https://i.pravatar.cc/48?u=118836",
    balance: -7,
  },
  {
    id: 933372,
    name: "Sarah",
    image: "https://i.pravatar.cc/48?u=933372",
    balance: 20,
  },
  {
    id: 499476,
    name: "Anthony",
    image: "https://i.pravatar.cc/48?u=499476",
    balance: 0,
  },
];

function Button({ children, onClick }) {
  return (
    <button className="button" onClick={onClick}>
      {children}
    </button>
  );
}

export default function App() {
  const [friends, setFriends] = useState(initialFriends);
  const [showAddFriend, setShowAddFriend] = useState(false);

  function handleShowAddFriend() {
    setShowAddFriend((show) => !show);
  }

  function handleAddFriend(friend) {
    setFriends((friends) => [...friends, friend]);
    setShowAddFriend(false);
  }

  return (
    <div className="app">
      <div className="sidebar">
        <FriendsList friends={friends} />
        {showAddFriend && <FormAddFriend onAddFriend={handleAddFriend} />}
        <Button onClick={handleShowAddFriend}>
          {showAddFriend ? "Close" : "Add Friend"}
        </Button>
      </div>

      <FormSplitBill />
    </div>
  );
}

function FriendsList({ friends }) {
  return (
    <ul>
      {friends.map((friend) => (
        <Friend friend={friend} key={friend.id} />
      ))}
    </ul>
  );
}

function Friend({ friend }) {
  return (
    <li key={friend.id}>
      <img src={friend.image} alt={friend.name} />
      <h3>{friend.name}</h3>
      {friend.balance < 0 && (
        <p className="red">
          You owe {friend.name} {Math.abs(friend.balance)}💶
        </p>
      )}
      {friend.balance > 0 && (
        <p className="green">
          {friend.name} owes you {Math.abs(friend.balance)}💶
        </p>
      )}
      {friend.balance === 0 && <p>You and {friend.name} are even</p>}

      <Button>Select</Button>
```

```
      </li>
    );
  }

  function FormAddFriend({ onAddFriend }) {
    const [name, setName] = useState("");
    const [image, setImage] = useState("https://i.pravatar.cc/48");

    function handleSubmit(e) {
      e.preventDefault();

      if (!name || !image) return;

      const id = crypto.randomUUID();
      const newFriend = {
        name,
        image: `${image}?=${id}`,
        balance: 0,
        id,
      };

      onAddFriend(newFriend);

      setName("");
      setImage("https://i.pravatar.cc/48");
    }

    return (
      <form className="form-add-friend" onSubmit={handleSubmit}>
        <label>🎊 Friend name</label>
        <input
          type="text"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />

        <label>🌄 Image URL</label>
        <input
          type="text"
          value={image}
          onChange={(e) => setImage(e.target.value)}
        />

        <Button>Add</Button>
      </form>
    );
  }

  function FormSplitBill() {
    return (
      <form className="form-split-bill">
        <h2>Split a bill with X</h2>

        <label>💶 Bill Value</label>
        <input type="text" />

        <label>💰 Your expense</label>
        <input type="text" />

        <label>🗝️ X's expense</label>
        <input type="text" disabled />

        <label>🪙 Who is paying the bill</label>
        <select>
          <option value="user">You</option>
          <option value="friend">X</option>
        </select>

        <Button>Split Bill</Button>
      </form>
    );
  }
```

Selecting a Friend

1. Code:

```
import { useState } from "react";

const initialFriends = [
  {
    id: 118836,
    name: "Clark",
    image: "https://i.pravatar.cc/48?u=118836",
    balance: -7,
  },
  {
    id: 933372,
    name: "Sarah",
    image: "https://i.pravatar.cc/48?u=933372",
    balance: 20,
  },
  {
    id: 499476,
    name: "Anthony",
    image: "https://i.pravatar.cc/48?u=499476",
    balance: 0,
  },
];

function Button({ children, onClick }) {
  return (
```

```
      <button className="button" onClick={onClick}>
        {children}
      </button>
    );
}

export default function App() {
  const [friends, setFriends] = useState(initialFriends);
  const [showAddFriend, setShowAddFriend] = useState(false);
  const [selectedFriend, setSelectedFriend] = useState(null);

  function handleShowAddFriend() {
    setShowAddFriend((show) => !show);
  }

  function handleAddFriend(friend) {
    setFriends((friends) => [...friends, friend]);
    setShowAddFriend(false);
  }

  function handleSelection(friend) {
    // setSelectedFriend(friend);
    setSelectedFriend((cur) => (cur?.id === friend.id ? null : friend));
    setShowAddFriend(false);
  }

  return (
    <div className="app">
      <div className="sidebar">
        <FriendsList
          friends={friends}
          onSelection={handleSelection}
          selectedFriend={selectedFriend}
        />
        {showAddFriend && <FormAddFriend onAddFriend={handleAddFriend} />}
        <Button onClick={handleShowAddFriend}>
          {showAddFriend ? "Close" : "Add Friend"}
        </Button>
      </div>

      {selectedFriend && <FormSplitBill selectedFriend={selectedFriend} />}
    </div>
  );
}

function FriendsList({ friends, onSelection, selectedFriend }) {
  return (
    <ul>
      {friends.map((friend) => (
        <Friend
          friend={friend}
          key={friend.id}
          onSelection={onSelection}
          selectedFriend={selectedFriend}
        />
      ))}
    </ul>
  );
}

function Friend({ friend, onSelection, selectedFriend }) {
  const isSelected = selectedFriend?.id === friend.id;

  return (
    <li key={friend.id} className={isSelected ? "selected" : ""}>
      <img src={friend.image} alt={friend.name} />
      <h3>{friend.name}</h3>
      {friend.balance < 0 && (
        <p className="red">
          You owe {friend.name} {Math.abs(friend.balance)}💶
        </p>
      )}
      {friend.balance > 0 && (
        <p className="green">
          {friend.name} owes you {Math.abs(friend.balance)}💶
        </p>
      )}
      {friend.balance === 0 && <p>You and {friend.name} are even</p>}

      <Button onClick={() => onSelection(friend)}>
        {isSelected ? "Close" : "Select"}
      </Button>
    </li>
  );
}

function FormAddFriend({ onAddFriend }) {
  const [name, setName] = useState("");
  const [image, setImage] = useState("https://i.pravatar.cc/48");

  function handleSubmit(e) {
    e.preventDefault();

    if (!name || !image) return;

    const id = crypto.randomUUID();
    const newFriend = {
      name,
      image: `${image}?=${id}`,
      balance: 0,
      id,
    };

    onAddFriend(newFriend);
```

```
    setName("");
    setImage("https://i.pravatar.cc/48");
  }

  return (
    <form className="form-add-friend" onSubmit={handleSubmit}>
      <label>🎉 Friend name</label>
      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />

      <label>🌄 Image URL</label>
      <input
        type="text"
        value={image}
        onChange={(e) => setImage(e.target.value)}
      />

      <Button>Add</Button>
    </form>
  );
}

function FormSplitBill({ selectedFriend }) {
  return (
    <form className="form-split-bill">
      <h2>Split a bill with {selectedFriend.name}</h2>

      <label>💶 Bill Value</label>
      <input type="text" />

      <label>🧍 Your expense</label>
      <input type="text" />

      <label>🧑‍🤝‍🧑 {selectedFriend.name}'s expense</label>
      <input type="text" disabled />

      <label>🤑 Who is paying the bill</label>
      <select>
        <option value="user">You</option>
        <option value="friend">{selectedFriend.name}</option>
      </select>

      <Button>Split Bill</Button>
    </form>
  );
}
```

Constructing Controlled Elements

1. Code:

```
import { useState } from "react";

const initialFriends = [
  {
    id: 118836,
    name: "Clark",
    image: "https://i.pravatar.cc/48?u=118836",
    balance: -7,
  },
  {
    id: 933372,
    name: "Sarah",
    image: "https://i.pravatar.cc/48?u=933372",
    balance: 20,
  },
  {
    id: 499476,
    name: "Anthony",
    image: "https://i.pravatar.cc/48?u=499476",
    balance: 0,
  },
];

function Button({ children, onClick }) {
  return (
    <button className="button" onClick={onClick}>
      {children}
    </button>
  );
}

export default function App() {
  const [friends, setFriends] = useState(initialFriends);
  const [showAddFriend, setShowAddFriend] = useState(false);
  const [selectedFriend, setSelectedFriend] = useState(null);

  function handleShowAddFriend() {
    setShowAddFriend((show) => !show);
  }

  function handleAddFriend(friend) {
    setFriends((friends) => [...friends, friend]);
    setShowAddFriend(false);
  }

  function handleSelection(friend) {
    // setSelectedFriend(friend);
```

```
    setSelectedFriend((cur) => (cur?.id === friend.id ? null : friend));
    setShowAddFriend(false);
  }

  return (
    <div className="app">
      <div className="sidebar">
        <FriendsList
          friends={friends}
          onSelection={handleSelection}
          selectedFriend={selectedFriend}
        />
        {showAddFriend && <FormAddFriend onAddFriend={handleAddFriend} />}
        <Button onClick={handleShowAddFriend}>
          {showAddFriend ? "Close" : "Add Friend"}
        </Button>
      </div>

      {selectedFriend && <FormSplitBill selectedFriend={selectedFriend} />}
    </div>
  );
}

function FriendsList({ friends, onSelection, selectedFriend }) {
  return (
    <ul>
      {friends.map((friend) => (
        <Friend
          friend={friend}
          key={friend.id}
          onSelection={onSelection}
          selectedFriend={selectedFriend}
        />
      ))}
    </ul>
  );
}

function Friend({ friend, onSelection, selectedFriend }) {
  const isSelected = selectedFriend?.id === friend.id;

  return (
    <li key={friend.id} className={isSelected ? "selected" : ""}>
      <img src={friend.image} alt={friend.name} />
      <h3>{friend.name}</h3>
      {friend.balance < 0 && (
        <p className="red">
          You owe {friend.name} {Math.abs(friend.balance)}💶
        </p>
      )}
      {friend.balance > 0 && (
        <p className="green">
          {friend.name} owes you {Math.abs(friend.balance)}💶
        </p>
      )}
      {friend.balance === 0 && <p>You and {friend.name} are even</p>}

      <Button onClick={() => onSelection(friend)}>
        {isSelected ? "Close" : "Select"}
      </Button>
    </li>
  );
}

function FormAddFriend({ onAddFriend }) {
  const [name, setName] = useState("");
  const [image, setImage] = useState("https://i.pravatar.cc/48");

  function handleSubmit(e) {
    e.preventDefault();

    if (!name || !image) return;

    const id = crypto.randomUUID();
    const newFriend = {
      name,
      image: `${image}?=${id}`,
      balance: 0,
      id,
    };

    onAddFriend(newFriend);

    setName("");
    setImage("https://i.pravatar.cc/48");
  }

  return (
    <form className="form-add-friend" onSubmit={handleSubmit}>
      <label>🎷 Friend name</label>
      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />

      <label>🌄 Image URL</label>
      <input
        type="text"
        value={image}
        onChange={(e) => setImage(e.target.value)}
      />

      <Button>Add</Button>
```

```
      </form>
    );
  }

  function FormSplitBill({ selectedFriend }) {
    const [bill, setBill] = useState(""); // input text elements have ""
    const [paidByUser, setPaidByUser] = useState("");
    const paidByFriend = bill ? bill - paidByUser : ""; // Reset each time state is changed and component is re-rendered and the function is execu
    const [whoIsPaying, setWhoIsPaying] = useState("user");

    return (
      <form className="form-split-bill">
        <h2>Split a bill with {selectedFriend.name}</h2>

        <label>💶 Bill Value</label>
        <input
          type="text"
          value={bill}
          onChange={(e) => setBill(Number(e.target.value))}
        />

        <label>💰 Your expense</label>
        <input
          type="text"
          value={paidByUser}
          onChange={(e) =>
            setPaidByUser(
              Number(e.target.value) > bill ? paidByUser : Number(e.target.value)
            )
          }
        />

        <label>🗝️ {selectedFriend.name}'s expense</label>
        <input type="text" disabled value={paidByFriend} />

        <label>🪙 Who is paying the bill</label>
        <select
          value={whoIsPaying}
          onChange={(e) => setWhoIsPaying(e.target.value)}
        >
          <option value="user">You</option>
          <option value="friend">{selectedFriend.name}</option>
        </select>

        <Button>Split Bill</Button>
      </form>
    );
  }
```

1. If we click on another user, the component is not re-rendered but a component is rendered at the same place (state will remain the same)

Splitting a Bill

1. Code:

```
import { useState } from "react";

const initialFriends = [
  {
    id: 118836,
    name: "Clark",
    image: "https://i.pravatar.cc/48?u=118836",
    balance: -7,
  },
  {
    id: 933372,
    name: "Sarah",
    image: "https://i.pravatar.cc/48?u=933372",
    balance: 20,
  },
  {
    id: 499476,
    name: "Anthony",
    image: "https://i.pravatar.cc/48?u=499476",
    balance: 0,
  },
];

function Button({ children, onClick }) {
  return (
    <button className="button" onClick={onClick}>
      {children}
    </button>
  );
}

export default function App() {
  const [friends, setFriends] = useState(initialFriends);
  const [showAddFriend, setShowAddFriend] = useState(false);
  const [selectedFriend, setSelectedFriend] = useState(null);

  function handleShowAddFriend() {
    setShowAddFriend((show) => !show);
  }

  function handleAddFriend(friend) {
    setFriends((friends) => [...friends, friend]);
    setShowAddFriend(false);
  }

  function handleSelection(friend) {
    // setSelectedFriend(friend);
```

```
    setSelectedFriend((cur) => (cur?.id === friend.id ? null : friend));
    setShowAddFriend(false);
  }

  function handleSplitBill(value) {
    setFriends((friends) =>
      friends.map((friend) =>
        friend.id === selectedFriend.id
          ? { ...friend, balance: friend.balance + value }
          : friend
      )
    );

    setSelectedFriend(null);
  }

  return (
    <div className="app">
      <div className="sidebar">
        <FriendsList
          friends={friends}
          onSelection={handleSelection}
          selectedFriend={selectedFriend}
        />
        {showAddFriend && <FormAddFriend onAddFriend={handleAddFriend} />}
        <Button onClick={handleShowAddFriend}>
          {showAddFriend ? "Close" : "Add Friend"}
        </Button>
      </div>

      {selectedFriend && (
        <FormSplitBill
          selectedFriend={selectedFriend}
          onSplitBill={handleSplitBill}
        />
      )}
    </div>
  );
}

function FriendsList({ friends, onSelection, selectedFriend }) {
  return (
    <ul>
      {friends.map((friend) => (
        <Friend
          friend={friend}
          key={friend.id}
          onSelection={onSelection}
          selectedFriend={selectedFriend}
        />
      ))}
    </ul>
  );
}

function Friend({ friend, onSelection, selectedFriend }) {
  const isSelected = selectedFriend?.id === friend.id;

  return (
    <li key={friend.id} className={isSelected ? "selected" : ""}>
      <img src={friend.image} alt={friend.name} />
      <h3>{friend.name}</h3>
      {friend.balance < 0 && (
        <p className="red">
          You owe {friend.name} {Math.abs(friend.balance)}💶
        </p>
      )}
      {friend.balance > 0 && (
        <p className="green">
          {friend.name} owes you {Math.abs(friend.balance)}💶
        </p>
      )}
      {friend.balance === 0 && <p>You and {friend.name} are even</p>}

      <Button onClick={() => onSelection(friend)}>
        {isSelected ? "Close" : "Select"}
      </Button>
    </li>
  );
}

function FormAddFriend({ onAddFriend }) {
  const [name, setName] = useState("");
  const [image, setImage] = useState("https://i.pravatar.cc/48");

  function handleSubmit(e) {
    e.preventDefault();

    if (!name || !image) return;

    const id = crypto.randomUUID();
    const newFriend = {
      name,
      image: `${image}?=${id}`,
      balance: 0,
      id,
    };

    onAddFriend(newFriend);

    setName("");
    setImage("https://i.pravatar.cc/48");
  }
```

```
    return (
      <form className="form-add-friend" onSubmit={handleSubmit}>
        <label>🧑‍🤝‍🧑 Friend name</label>
        <input
          type="text"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />

        <label>🌄 Image URL</label>
        <input
          type="text"
          value={image}
          onChange={(e) => setImage(e.target.value)}
        />

        <Button>Add</Button>
      </form>
    );
  }

  function FormSplitBill({ selectedFriend, onSplitBill }) {
    const [bill, setBill] = useState(""); // input text elements have ""
    const [paidByUser, setPaidByUser] = useState("");
    const paidByFriend = bill ? bill - paidByUser : ""; // Reset each time state is changed and component is re-rendered and the function is execu
    const [whoIsPaying, setWhoIsPaying] = useState("user");

    function handleSubmit(e) {
      e.preventDefault();

      if (!bill || !paidByUser) return;

      onSplitBill(whoIsPaying === "user" ? paidByFriend : -paidByUser);
    }

    return (
      <form className="form-split-bill" onSubmit={handleSubmit}>
        <h2>Split a bill with {selectedFriend.name}</h2>

        <label>💶 Bill Value</label>
        <input
          type="text"
          value={bill}
          onChange={(e) => setBill(Number(e.target.value))}
        />

        <label>🧍 Your expense</label>
        <input
          type="text"
          value={paidByUser}
          onChange={(e) =>
            setPaidByUser(
              Number(e.target.value) > bill ? paidByUser : Number(e.target.value)
            )
          }
        />

        <label>🗝️ {selectedFriend.name}'s expense</label>
        <input type="text" disabled value={paidByFriend} />

        <label>🪙 Who is paying the bill</label>
        <select
          value={whoIsPaying}
          onChange={(e) => setWhoIsPaying(e.target.value)}
        >
          <option value="user">You</option>
          <option value="friend">{selectedFriend.name}</option>
        </select>

        <Button>Split Bill</Button>
      </form>
    );
  }
```

Section 9: PART 2: INTERMEDIATE REACT [2 PROJECTS]

Introduction to Part 2

1. Build a few small projects
2. Topics:
    1. How react works behind the scenes
    2. Project
        1. UIs and layouts
        2. Setting up effects
        3. Fetching data
        4. custom hooks
        5. ...

Useful Resources for Part 2

1. Resources:
    1. Writing Resilient Components - Dan Abramov - React Team
    2. Things I think about when I write React code - GitHub repository
    3. A (Mostly) Complete Guide to React Rendering Behavior - Mark Erikson (Redux team)
    4. A Visual Guide to React Rendering - Multi-part series, check out the other ones
    5. Inside Fiber: in-depth overview of the new reconcilliation algorithm in React
    6. A Cartoon Intro to Fiber - YouTube video

7. What is React Fiber? React.js Deep Dive - YouTube video
8. The React and React Native Event System Explained
9. Under the hood of event listeners in React
10. A DIY guide to build your own React
11. useSynExternalStore First Look
12. Under the hood of React's hooks system
13. Why Do React Hooks Rely on Call Order? - By Dan Abramov
14. So you think you know everything about React refs
15. react-use: Reusable React Hook Library - Github repository
16. react-hookz: React hooks done right - GitHub repository

Section 10: Thinking in React: Components, Composition, and Reusability

Section Overview

1. Topics:
    1. How to **think** about components
    2. Composition
    3. Reusability
    4. How to **split** a component
    5. Building **layouts**
2. What are we going to study:
    1. How to split a UI into components
    2. What type of components we can use
    3. How to implement in React
    4. Simple app layout using composition

Setting Up the "usePopcorn" Project

1. `npx create-react-app@5 usepopcorn`
    1. usepopcorn.netlify.app
        1. It is all about hooks
2. Topics:
    1. Component reusability

How to Split a UI into Components

1. Important questions:
    1. How do we split UI into components?
    2. When should be build new components?

Component Size Matters

1. Component size axis
    1. Small to huge (none of them are ideal)
        1. Huge:
            1. Too many **responsibilities**
                1. If it does too many multiple things, we must break it up into multiple components
            2. Might need too many **props**
                1. 10-15 props
            3. Hard to **reuse**
            4. **Complex** code, hard to understand (and use)
        2. Small:
            1. We end up with 100s of mini-components
            2. Confusing codebase
            3. Too **abstracted**
                1. To define something new to hide the implementation details of the thing
                    1. Since, each component is an abstraction
        3. Right balance: Generally, we need to find the right balance between too specific and too broad

How to Split a UI into Components

1. How?
    1. Logical separation
        1. Each component has a well-defined responsibility
            1. Display price
            2. Display rating
    2. Some are reusable
        1. Heart button
        2. Superhost label
    3. Low complexity
        1. Not overly complex
2. The 4 Criteria for splitting a UI into components:
    1. Logical separation of content/layout
        1. Components define a logical separation
    2. Reusability
        1. We need to make some of the components re-usable
    3. Responsibilities/ Complexity
        1. Each component should have a single, well-defined responsibility
    4. Personal coding style
        1. Some people like smaller components while other prefer larger components
            1. Choose a component size that works best for me (to be as productive as possible)

Framework: When to Construct a New Component?

1. Criterion 1:
    1. **Suggestion:** When in doubt, start with a relatively big component, then split it into smaller components as it becomes necessary
        1. When does it become necessary?
            1. Use 4 criteria:
                1. Logical separation of content/layout
                2. Reusability
                3. Responsibilities/complexity
                4. Personal coding style
    2. Skip the above step if you're sure you need to reuse.
        1. Otherwise, you don't need to focus on reusability and complexity early on
2. Criterion 2:
    1. Does the component contain pieces of content or layout that **don't belong together**?
        1. You might need a new component
    2. Is it possible to reuse part of the component?
        1. Do you **want** or **need** to reuse it?
            1. You might need a new component
3. Criterion 3:
    1. Is the component doing too **many different things**?
    2. Does the component rely on too **many props**
    3. Does the component have too **many pieces of state** and/or effects?
    4. Is the code, including JSX, too **complex/confusing**?
4. Criterion 4:
    1. Do you prefer **smaller** functions/components?
5. The above are all **guidelines**... it will become intuitive over time!

Some More General Guidelines

1. Be aware that contructing a new component **defines a new abstraction**. Abstractions have a **cost**, because **more abstractions require more mental energy** to switch back and forth between components. So try not to construct new components too early.
2. Name a component according to **what it does** or **what it displays**. Don't be afraid to using long component names
3. Never declare a new component **inside another component**
    1. Co-locate related components inside the same file. Don't seperate components into different files too early
4. It's completely normal that an app has components of **many different sizes**, including very small and huge ones (See next slide...)

Any App Has Components of Different Sizes and Reusability

1. Some very small components are necessary
    1. Examples:
        1. Heart button
        2. Superhost
        3. Rarefind
        4. ...
    2. Reasons:
        1. Highly reusable
        2. Very low complexity
2. **Most apps will have a few huge components**
    1. Not meant to be reused (**not a problem!**)
    2. Examples:
        1. Page component
            1. Contains the entire layout of the page
                1. It is not meant to be re-used
                    1. Don't worry about reusability or needing to split the component up (?)
3. Reusability range is similar to the size range:
    1. Small - most reusable
    2. Huge - non-reusable
        1. Not all components are meant to be re-usable
    3. Components with all different sizes
        1. Different degrees of:
            1. Size
            2. Reusability
            3. Responsibility
            4. Complexity

Splitting Components in Practice

1. 07-usepopcorn
    1. Mixing JSX and native HTML elements might be ugly
2. Code:

```
import { useState } from "react";

const tempMovieData = [
  {
    imdbID: "tt1375666",
    Title: "Inception",
    Year: "2010",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BMjAxMzY3NjcxNF5BMl5BanBnXkFtZTcwNTI5OTM0Mw@@._V1_SX300.jpg",
  },
  {
    imdbID: "tt0133093",
    Title: "The Matrix",
    Year: "1999",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BNzQzOTk3OTAtNDQ0Zi00ZTVkLWI0MTEtMDllZjNkYzNjNTc4L2ltYWdlXkEyXkFqcGdeQXVyNjU0OTQ0OTY@._V1_SX300.jp
  },
  {
```

```
    imdbID: "tt6751668",
    Title: "Parasite",
    Year: "2019",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BYWZjMjk3ZTItODQ2ZC00NTY5LWE0ZDYtZTI3MjcwN2Q5NTVkXkEyXkFqcGdeQXVyODk4OTc3MTY@._V1_SX300.jpg",
  },
];

const tempWatchedData = [
  {
    imdbID: "tt1375666",
    Title: "Inception",
    Year: "2010",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BMjAxMzY3NjcxNF5BMl5BanBnXkFtZTcwNTI5OTM0Mw@@._V1_SX300.jpg",
    runtime: 148,
    imdbRating: 8.8,
    userRating: 10,
  },
  {
    imdbID: "tt0088763",
    Title: "Back to the Future",
    Year: "1985",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BZmU0M2Y1OGUtZjIxNi00ZjBkLTg1MjgtOWIyNThiZWIwYjRiXkEyXkFqcGdeQXVyMTQxNzMzNDI@._V1_SX300.jpg",
    runtime: 116,
    imdbRating: 8.5,
    userRating: 9,
  },
];

function Navbar() {
  return (
    <nav className="nav-bar">
      <Logo />
      <Search />
      <NumResults />
    </nav>
  );
}

function Logo() {
  return (
    <div className="logo">
      <span role="img">🍿</span> {/* No need to break */}
      <h1>usePopcorn</h1>
    </div>
  );
}

function Search() {
  const [query, setQuery] = useState("");
  return (
    <input
      className="search"
      type="text"
      placeholder="Search movies..."
      value={query}
      onChange={(e) => setQuery(e.target.value)}
    />
  );
}

function NumResults() {
  return (
    <p className="num-results">
      Found <strong>X</strong> results
    </p>
  );
}

function Main() {
  return (
    <main className="main">
      <ListBox />
      <WatchedBox />
    </main>
  );
}

function ListBox() {
  const [isOpen1, setIsOpen1] = useState(true);

  return (
    <div className="box">
      <button
        className="btn-toggle"
        onClick={() => setIsOpen1((open) => !open)}
      >
        {isOpen1 ? "–" : "+"}
      </button>
      {isOpen1 && <MovieList />}
    </div>
  );
}

function MovieList() {
  const [movies, setMovies] = useState(tempMovieData);

  return (
    <ul className="list">
      {movies?.map((movie) => (
        <Movie movie={movie} />
      ))}
```

```
      </ul>
    );
  }

  function Movie({ movie }) {
    return (
      <li key={movie.imdbID}>
        <img src={movie.Poster} alt={`${movie.Title} poster`} />
        <h3>{movie.Title}</h3>
        <div>
          <p>
            <span>🗓</span>
            <span>{movie.Year}</span>
          </p>
        </div>
      </li>
    );
  }

  function WatchedBox() {
    const [watched, setWatched] = useState(tempWatchedData);
    const [isOpen2, setIsOpen2] = useState(true);

    return (
      <div className="box">
        <button
          className="btn-toggle"
          onClick={() => setIsOpen2((open) => !open)}
        >
          {isOpen2 ? "–" : "+"}
        </button>
        {isOpen2 && (
          <>
            <WatchedSummary watched={watched} />

            <WatchedMovieList watched={watched} />
          </>
        )}
      </div>
    );
  }

  function WatchedSummary({ watched }) {
    const avgImdbRating = average(watched.map((movie) => movie.imdbRating));
    const avgUserRating = average(watched.map((movie) => movie.userRating));
    const avgRuntime = average(watched.map((movie) => movie.runtime));

    return (
      <div className="summary">
        <h2>Movies you watched</h2>
        <div>
          <p>
            <span>#️⃣</span>
            <span>{watched.length} movies</span>
          </p>
          <p>
            <span>⭐️</span>
            <span>{avgImdbRating}</span>
          </p>
          <p>
            <span>🌟</span>
            <span>{avgUserRating}</span>
          </p>
          <p>
            <span>⏳</span>
            <span>{avgRuntime} min</span>
          </p>
        </div>
      </div>
    );
  }

  function WatchedMovieList({ watched }) {
    return (
      <ul className="list">
        {watched.map((movie) => (
          <WatchedMovie movie={movie} key={movie.imdbID} />
        ))}
      </ul>
    );
  }

  function WatchedMovie({ movie }) {
    return (
      <li key={movie.imdbID}>
        <img src={movie.Poster} alt={`${movie.Title} poster`} />
        <h3>{movie.Title}</h3>
        <div>
          <p>
            <span>⭐️</span>
            <span>{movie.imdbRating}</span>
          </p>
          <p>
            <span>🌟</span>
            <span>{movie.userRating}</span>
          </p>
          <p>
            <span>⏳</span>
            <span>{movie.runtime} min</span>
          </p>
        </div>
      </li>
    );
```

```
}

const average = (arr) =>
  arr.reduce((acc, cur, i, arr) => acc + cur / arr.length, 0);

export default function App() {
  return (
    <>
      <Navbar />
      <Main />
    </>
  );
}
```

Component Categories

1. Most of your components will **naturally** (we shouldn't force our components into one of the categories) all into **one of three categories**:
    1. Stateless/presentational components
        1. **No state**
        2. Can receive props and simply present received data or other content
        3. Usually **small and reusable**
            1. Example:
                1. Logo
                2. NumResults
                3. Movie
    2. Stateful components
        1. **Have state**
        2. Can still be **reusable**
            1. Example: Search
    3. Structural components
        1. **Pages**, **Layouts**, or **Screens** of the app
        2. Result of **composition** (many smaller components together)
        3. Can be **huge and non-reusable** (but don't have to)
        4. **They are responsible for providing some sort of structure to the application**
            1. Pages
            2. Layouts
            3. Screens
            4. ...
        5. Might not be present in small apps
            1. Can be found in bigger apps

Prop Drilling

1. Classification of components:
    1. Structural:
        1. `App`
        2. `NavBar`
        3. `Main`
    2. Presentational:
        1. `Logo` - Stateless
        2. `NumResults`
        3. `Movie`
        4. `WatchedMoviesList`
        5. `WatcheMovie`
    3. Stateful:
        1. `Search`
        2. `ListBox`
        3. `MovieList`
        4. `WatchedBox`
2. Prop drilling:
    1. We need to pass a prop into several nested child components in order to get the data into a deeply nested component
        1. The intermediate nested components do not need the prop
    2. Not good if we need to pass it very deep

Component Composition

1. Principle: Component composition

What is Component Composition?

1. "Using" A component
    1. Modal component using Success component (using nesting)

```
// It can be called a `SuccessModal` because we cannot use it for anything else
function Modal() {
    return (
        <div>
            <Success />
        </div>
    );
}

function Success() {
    return <p>Well done!</p>;
}
```

        1. Problem:
            1. `Success` is inside `Modal`: We can NOT reuse Modal (deeply linked together in the JSX)

1. `Modal` cannot be used to display any other message (`Error` message say)

2. Solution: Component Composition

    1. Using `children` prop

    ```
    function Modal({ children }) {
        return (
            <div className="modal">
                {children}
            </div>
        );
    }
    ```

        1. We can pass `Success` or `Error` between opening and closing tags when we use `Modal`

        ```
        <Modal>
            <Success />
        </Modal>
        ```

            1. We can REUSE `Modal`

        2. **We leave an empty slot ready to be filled by any other component**

        3. `Error` message:

        ```
        <Modal>
            <Error />
        </Modal>
        ```

2. **Component Composition:** Combining different components using the `children` prop (or explicitly defined props)

3. We use component composition, we can:

    1. Construct highly reusable and flexible components
    2. Fix prop dilling (great for layouts)
        1. Good for layouts

4. It is possible because components don't need to know their children in advance

Fixing Prop Drilling With Composition (And Building a Layout)

1. The entire layout is seen in one place (in the `App` component)

Using Composition to Make a Reusable Box

1. Code:

```
import { useState } from "react";

const tempMovieData = [
  {
    imdbID: "tt1375666",
    Title: "Inception",
    Year: "2010",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BMjAxMzY3NjcxNF5BMl5BanBnXkFtZTcwNTI5OTM0Mw@@._V1_SX300.jpg",
  },
  {
    imdbID: "tt0133093",
    Title: "The Matrix",
    Year: "1999",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BNzQzOTk3OTAtNDQ0Zi00ZTVkLWI0MTEtMDllZjNkYzNjNTc4L2ltYWdlXkEyXkFqcGdeQXVyNjU0OTQ0OTY@._V1_SX300.jp
  },
  {
    imdbID: "tt6751668",
    Title: "Parasite",
    Year: "2019",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BYWZjMjk3ZTItODQ2ZC00NTY5LWE0ZDYtZTI3MjcwN2Q5NTVkXkEyXkFqcGdeQXVyODk4OTc3MTY@._V1_SX300.jpg",
  },
];

const tempWatchedData = [
  {
    imdbID: "tt1375666",
    Title: "Inception",
    Year: "2010",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BMjAxMzY3NjcxNF5BMl5BanBnXkFtZTcwNTI5OTM0Mw@@._V1_SX300.jpg",
    runtime: 148,
    imdbRating: 8.8,
    userRating: 10,
  },
  {
    imdbID: "tt0088763",
    Title: "Back to the Future",
    Year: "1985",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BZmU0M2Y1OGUtZjIxNi00ZjBkLTg1MjgtOWIyNThiZWIwYjRiXkEyXkFqcGdeQXVyMTQxNzMzNDI@._V1_SX300.jpg",
    runtime: 116,
    imdbRating: 8.5,
    userRating: 9,
  },
];

function Navbar({ children }) {
  return (
```

```
      <nav className="nav-bar">
        <Logo />
        {children}
      </nav>
    );
}

function Logo() {
  return (
    <div className="logo">
      <span role="img">🍿</span> {/* No need to break */}
      <h1>usePopcorn</h1>
    </div>
  );
}

function Search() {
  const [query, setQuery] = useState("");
  return (
    <input
      className="search"
      type="text"
      placeholder="Search movies..."
      value={query}
      onChange={(e) => setQuery(e.target.value)}
    />
  );
}

function NumResults({ movies }) {
  return (
    <p className="num-results">
      Found <strong>{movies.length}</strong> results
    </p>
  );
}

function Main({ children }) {
  return <main className="main">{children}</main>;
}

function Box({ children }) {
  const [isOpen, setIsOpen] = useState(true);

  return (
    <div className="box">
      <button className="btn-toggle" onClick={() => setIsOpen((open) => !open)}>
        {isOpen ? "–" : "+"}
      </button>
      {isOpen && children}
    </div>
  );
}

function MovieList({ movies }) {
  return (
    <ul className="list">
      {movies?.map((movie) => (
        <Movie movie={movie} />
      ))}
    </ul>
  );
}

function Movie({ movie }) {
  return (
    <li key={movie.imdbID}>
      <img src={movie.Poster} alt={`${movie.Title} poster`} />
      <h3>{movie.Title}</h3>
      <div>
        <p>
          <span>🗓</span>
          <span>{movie.Year}</span>
        </p>
      </div>
    </li>
  );
}

// function WatchedBox() {
//   const [watched, setWatched] = useState(tempWatchedData);
//   const [isOpen2, setIsOpen2] = useState(true);

//   return (
//     <div className="box">
//       <button
//         className="btn-toggle"
//         onClick={() => setIsOpen2((open) => !open)}
//       >
//         {isOpen2 ? "–" : "+"}
//       </button>
//       {isOpen2 && (
//         <>
//           <WatchedSummary watched={watched} />

//           <WatchedMovieList watched={watched} />
//         </>
//       )}
//     </div>
//   );
// }

function WatchedSummary({ watched }) {
  const avgImdbRating = average(watched.map((movie) => movie.imdbRating));
```

```
  const avgUserRating = average(watched.map((movie) => movie.userRating));
  const avgRuntime = average(watched.map((movie) => movie.runtime));

  return (
    <div className="summary">
      <h2>Movies you watched</h2>
      <div>
        <p>
          <span>#️⃣</span>
          <span>{watched.length} movies</span>
        </p>
        <p>
          <span>⭐️</span>
          <span>{avgImdbRating}</span>
        </p>
        <p>
          <span>🌟</span>
          <span>{avgUserRating}</span>
        </p>
        <p>
          <span>⏳</span>
          <span>{avgRuntime} min</span>
        </p>
      </div>
    </div>
  );
}

function WatchedMovieList({ watched }) {
  return (
    <ul className="list">
      {watched.map((movie) => (
        <WatchedMovie movie={movie} key={movie.imdbID} />
      ))}
    </ul>
  );
}

function WatchedMovie({ movie }) {
  return (
    <li key={movie.imdbID}>
      <img src={movie.Poster} alt={`${movie.Title} poster`} />
      <h3>{movie.Title}</h3>
      <div>
        <p>
          <span>⭐️</span>
          <span>{movie.imdbRating}</span>
        </p>
        <p>
          <span>🌟</span>
          <span>{movie.userRating}</span>
        </p>
        <p>
          <span>⏳</span>
          <span>{movie.runtime} min</span>
        </p>
      </div>
    </li>
  );
}

const average = (arr) =>
  arr.reduce((acc, cur, i, arr) => acc + cur / arr.length, 0);

export default function App() {
  const [movies, setMovies] = useState(tempMovieData);
  const [watched, setWatched] = useState(tempWatchedData);

  return (
    <>
      <Navbar>
        <Search />
        <NumResults movies={movies} />
      </Navbar>
      <Main>
        <Box>
          <MovieList movies={movies} />
        </Box>
        <Box>
          <WatchedSummary watched={watched} />
          <WatchedMovieList watched={watched} />
        </Box>
      </Main>
    </>
  );
}
```

Passing Elements as Props (Alternative to children)

1. Passed as an element prop:

```
<Main>
  <Box element={<MovieList movies={movies} />} />
  <Box
    element={
      <>
        <WatchedSummary watched={watched} />
        <WatchedMovieList watched={watched} />
      </>
    }
  />
</Main>
```

```
...

function Box({ element }) {
  const [isOpen, setIsOpen] = useState(true);

  return (
    <div className="box">
      <button className="btn-toggle" onClick={() => setIsOpen((open) => !open)}>
        {isOpen ? "–" : "+"}
      </button>
      {isOpen && element}
    </div>
  );
}
```

1. Similar to passing `children` prop (inside React both must be the same)
2. Using `children` is the preferred way

Building a Reusable Star Rating Component

1. Isolated component - to be used anywhere
    1. Users can decide the number of stars

Constructing the Stars

Handling Hover Events

1. Code:

```
import { useState } from "react";

const containerStyle = {
  display: "flex",
  alignItems: "center",
  gap: "16px",
};
// This doesn't change
// This object doesn't have to be re-generated by JS each time the component re-rendered (because the function is called again)

const starContainerStyle = {
  display: "flex",
};

const textStyle = {
  lineHeight: "1",
  margin: "0",
};

export default function StarRating({ maxRating = 5 }) {
  const [rating, setRating] = useState(0);
  const [tempRating, setTempRating] = useState(0);

  function handleRating(rating) {
    setRating(rating);
  }

  return (
    <div style={containerStyle}>
      <div style={starContainerStyle}>
        {Array.from({ length: maxRating }, (_, i) => (
          <Star
            key={i}
            onRate={() => handleRating(i + 1)}
            full={tempRating ? tempRating >= i + 1 : rating >= i + 1}
            onHoverIn={() => setTempRating(i + 1)}
            onHoverOut={() => setTempRating(0)}
          />
        ))}
      </div>
      <p style={textStyle}>{tempRating || rating || ""}</p>
    </div>
  );
}

const starStyle = {
  width: "48px",
  height: "48px",
  display: "block",
  cursor: "pointer",
};

function Star({ onRate, full, onHoverIn, onHoverOut }) {
  return (
    <span
      role="button"
      style={starStyle}
      onClick={onRate}
      onMouseEnter={onHoverIn}
      onMouseLeave={onHoverOut}
    >
      {full ? (
        <svg
          xmlns="http://www.w3.org/2000/svg"
          viewBox="0 0 20 20"
          fill="#000"
          stroke="#000"
        >
          <path d="M9.049 2.927c.3-.921 1.603-.921 1.902 0l1.07 3.292a1 1 0 00.95.69h3.462c.969 0 1.371 1.24.588 1.81l-2.8 2.034a1 1 0 00-.364 1
        </svg>
      ) : (
```

```
            <svg
              xmlns="http://www.w3.org/2000/svg"
              fill="none"
              viewBox="0 0 24 24"
              stroke="#000"
            >
              <path
                strokeLinecap="round"
                strokeLinejoin="round"
                strokeWidth="{2}"
                d="M11.049 2.927c.3-.921 1.603-.921 1.902 0l1.519 4.674a1 1 0 00.95.69h4.915c.969 0 1.371 1.24.588 1.81l-3.976 2.888a1 1 0 00-.363 1
              />
            </svg>
          )}
        </span>
      );
    }
```

Props as a Component API

Props as an API

1. Any component is created by someone and consumed by someone
    1. They might be different developers
    2. We must think about two different entities
        1. Author decides what props the component can accept
        2. Consumer uses the component in an application by specifying values for the props
    3. Why?
        1. Component props = Public API (of the component)
            1. When an author chooses what props to pass to the component, we are choosing the public interface of the component
            2. We are also choosing, how much complexity of the component we want to expose to the consumer of the API
                1. We need to find a good balance on how strict we want to be
                    1. How many props we want to enable for configuration
                        1. Example: Weather component
                            1. Too few props: Extremely simple: Only allowing one prop (location for which consumer wants a weather)
                                1. Not **flexible** enough
                                2. Might not be **useful**
                            2. Too many props: URL, number of days, daily/hourly, how many days, which temperature unit, what data to be displayed, etc...
                                1. Too **hard** to use
                                    1. Exposing too much **complexity**
                                    2. **Hard-to-write** code
                2. We need to find the right balance between too little and too many props, that works for both the consumer and the author (based on project needs)
                    1. We can provide default values for many of them.
    4. A component is just an abstraction that encapsulates UI and logic

Improving Reusability With Props

1. If we want to re-use in many applications (or share it with npm)
2. Consumers want to define:
    1. Colors of the stars
    2. Sizes of the stars
    3. Text
3. Note: Never initialize a state from prop is true if we want state variable to stay in sync with passed in prop
    1. If prop value is updated, state value also gets updated
        1. However, we are using prop as seed data (just as an initial value)
4. Other props that we can allow:
    1. Color to change based on ratings
    2. Different spacing between stars
    3. Which side the text label appears
5. Another most important point:
    1. The consumer might need the `rating` state outside the component

PropTypes

1. Adding type-checking
    1. It is better to use TypeScript, instead of JS
    2. If we don't want to switch to TypeScript, we can use this approach:

Code

1. Index.js

```
import React, { useState } from "react";
import ReactDOM from "react-dom/client";
// import "./index.css";
// import App from "./App";

import StarRating from "./StarRating";

function Test() {
  const [movieRating, setMovieRating] = useState(0);

  return (
    <div>
      <StarRating color="blue" maxRating={10} onSetRating={setMovieRating} />
      <p>This movie was rated {movieRating} stars</p>
    </div>
  );
}
```

```
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    {/* <App /> */}
    <StarRating
      maxRating={5}
      messages={["Terrible", "Bad", "Okay", "Good", "Amazing"]}
    />
    <StarRating
      maxRating={10}
      size={24}
      color="red"
      className="text"
      defaultRating={3}
    />
    {/* <StarRating maxRating={10} />
    <StarRating /> */}
    <Test />
  </React.StrictMode>
);
```

2. StarRating.js

```
import { useState } from "react";
import PropTypes from "prop-types"; // create-react-app comes with this pre-installed

const containerStyle = {
  display: "flex",
  alignItems: "center",
  gap: "16px",
};
// This doesn't change
// This object doesn't have to be re-generated by JS each time the component re-rendered (because the function is called again)

const starContainerStyle = {
  display: "flex",
};

// JS throws warning
StarRating.propTypes = {
  maxRating: PropTypes.number,
  defaultRating: PropTypes.number,
  color: PropTypes.string,
  size: PropTypes.number,
  messages: PropTypes.array,
  className: PropTypes.string,
  onSetRating: PropTypes.func,
};

export default function StarRating({
  maxRating = 5,
  color = "#fcc419",
  size = 48,
  className = "",
  messages = [],
  defaultRating = 0,
  onSetRating,
}) {
  const textStyle = {
    lineHeight: "1",
    margin: "0",
    color,
    fontSize: `${size / 1.5}px`,
  };

  const [rating, setRating] = useState(defaultRating);
  const [tempRating, setTempRating] = useState(0);

  function handleRating(rating) {
    setRating(rating);
    onSetRating && onSetRating(rating);
  }

  return (
    <div style={containerStyle}>
      <div style={starContainerStyle} className={className}>
        {Array.from({ length: maxRating }, (_, i) => (
          <Star
            key={i}
            onRate={() => handleRating(i + 1)}
            full={tempRating ? tempRating >= i + 1 : rating >= i + 1}
            onHoverIn={() => setTempRating(i + 1)}
            onHoverOut={() => setTempRating(0)}
            color={color}
            size={size}
          />
        ))}
      </div>
      <p style={textStyle}>
        {messages.length === maxRating
          ? messages[tempRating ? tempRating - 1 : rating - 1]
          : tempRating || rating || ""}
      </p>
    </div>
  );
}

function Star({ onRate, full, onHoverIn, onHoverOut, color, size }) {
  const starStyle = {
    width: `${size}px`,
    height: `${size}px`,
    display: "block",
    cursor: "pointer",
```

```
  };

  return (
    <span
      role="button"
      style={starStyle}
      onClick={onRate}
      onMouseEnter={onHoverIn}
      onMouseLeave={onHoverOut}
    >
      {full ? (
        <svg
          xmlns="http://www.w3.org/2000/svg"
          viewBox="0 0 20 20"
          fill={color}
          stroke={color}
        >
          <path d="M9.049 2.927c.3-.921 1.603-.921 1.902 0l1.07 3.292a1 1 0 00.95.69h3.462c.969 0 1.371 1.24.588 1.81l-2.8 2.034a1 1 0 00-.364 1
        </svg>
      ) : (
        <svg
          xmlns="http://www.w3.org/2000/svg"
          fill="none"
          viewBox="0 0 24 24"
          stroke={color}
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            strokeWidth="{2}"
            d="M11.049 2.927c.3-.921 1.603-.921 1.902 0l1.519 4.674a1 1 0 00.95.69h4.915c.969 0 1.371 1.24.588 1.81l-3.976 2.888a1 1 0 00-.363 1
          />
        </svg>
      )}
    </span>
  );
}
```

CHALLENGE #1: Text Expander Component

1. MUST DO

How React Works Behind the Scenes

Section Overview

1. Topics:
    1. How things work **inside** React
    2. You'll become a **better** and more **confident** React developer
    3. Will be a bit intense...
    4. Watch at least the **final lecture**!
2. Benefit of learning it:
    1. We can become a better and confident React developer
    2. We can ace common React questions
    3. This puts in top 5% of all React developers

Project Setup and Walkthrough

1. Copy `08-how-react-works/starter` and rename it to `how-react-works`
    1. `npm i`
    2. `npm start`

Components, Instances, and Elements

1. Conceptual differences between React components, component instances, and React elements

Component vs. Instance vs. Element

1. Component:
    1. **Description** of a piece of UI
    2. A component is a JS function that **returns React elements** (element tree), usually written as JSX (the elements)
    3. It is a "Blueprint" or "Template"
        1. Using the blueprint/template, React instantiates one or more Component Instances
2. Component Instance
    1. Component instances are created when we "use" components
        1. Example:

```
function App() {
    return (
        <div className='tabs'>
            <Tab item={content[0]} />
            <Tab item={content[1]} />
            <Tab item={content[2]} />
        </div>
    );
}
```

            1. 3 instances of `Tab` components are placed in the component tree (in the actual application)
                1. Since React calls `Tab()` function 3 times

        1. One time for each instance
    2. An instance is the actual "pysical manifestation" of a component living in a component tree
        1. Component is just a function we wrote before it is called
    3. Each instance has its own state and props
    4. **Each instance has a lifecycle** (can "be born", "live", and "die")
        1. We usually use component and component-instance interchangeably

2. As React executes the code in each of the instances, each of them will return one or more React elements

3. React Element

    1. Behind the scenes, JSX gets converted to multiple `React.createElement` function calls
    2. As React calls `React.createElement` functions, the result will be a React element
    3. A React element is the **result of the function calls (using components in React code)**
        1. **It is a big, immutable JS object that React keeps in memory**
    4. What is it?
        1. It contains all the information that is necessary to construct DOM elements for the current component instance
    5. The **React Element** is ultimately converted into **DOM Element (HTML)**
        1. Painted onto the screen by the Browser
    6. DOM Element is the actual **visual representation** of the component instance in the browser
        1. It is not React Element that is rendered to the DOM
        2. **React elements just live inside React app** (they have nothing to do the DOM)
            1. React elements are converted into DOM when they are painted onto the screen in the final step

Instances and Elements in Practice

1. Code:

```
console.log(<DifferentContent test={23} />); // React uses the object created to convert it into DOM elements
// Has `Symbol`s: Primitives that cannot be transmitted via JSON. It cannot come from an API call. If a hacker tries to send a fake React elemen

console.log(DifferentContent()); // `type`: `div` instead of `DifferentComponent`. React doesn't see it as a component. It sees raw React elemen
// We want React to see the component instance

function Tabbed({ content }) {
  const [activeTab, setActiveTab] = useState(0);

  return (
    <div>
      <div className="tabs">
        <Tab num={0} activeTab={activeTab} onClick={setActiveTab} />
        <Tab num={1} activeTab={activeTab} onClick={setActiveTab} />
        <Tab num={2} activeTab={activeTab} onClick={setActiveTab} />
        <Tab num={3} activeTab={activeTab} onClick={setActiveTab} />
      </div>

      {activeTab <= 2 ? (
        <TabContent item={content.at(activeTab)} />
      ) : (
        <DifferentContent />
      )}

      {TabContent({ item: content.at(0) })}
      {/* It works. Component tree doesn't show it
      The state of the component is managed inside the parent component
      */}
    </div>
  );
}
```

How Rendering Works: Overview

Quick Recap Before We Get Started

1. We build components
2. We use the components as many times as we want to construct one or more instances of the components
    1. The component instances are actual physical components which live in our application and hold state and props
3. Each JSX will produce a bunch of `React.createElement` function calls which will produce a React element for each component instance
4. The React elements will be transformed to DOM elements and displayed as UI on the screen
5. **How do the React elements get transformed into DOM elements and displayed on the screen?**

Overview: How Components are Displayed on the Screen

1. Phases involved in displaying components to the screen

    1. We zoom into each of the phases to learn how the entire process works internally behind the scenes

2. Process:

```
Render is Triggered
The process is started by React each time a new render is triggered (By updating state somewhere)
    |
    v
Render Phase
React calls component functions and figures out how DOM should be updated (to reflect the latest state changes)
    DOM is not updated in this phase:
        In React, rendering is NOT updating the DOM or displaying elements on the screen. Rendering only happens internally inside React, it
    |
    v
Commit Phase
    React actually writes to the DOM, updating, inserting, and deleting elements (to reflect the correct state of the application)
        This phase is responsible for actual rendering (traditional definition)
    |
    v
```

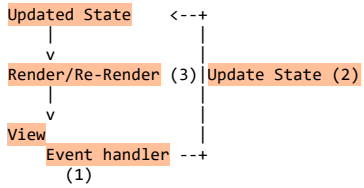
```
Browser Paint
    Browser identifies that the DOM is updated, and it repaints the screen (not part of React)
```

How Renders are Triggered

1. The two situations that trigger renders:
    1. **Initial render** of the application (first time the application renders)
    2. **State is updated** in one or more component instances (**re-render**) (somewhere in the application)
2. The render process is triggered for the entire application (not for just a single component)
    1. The entire DOM is not updated
        1. Rendering is only calling out component functions and figuring out what needs to change in the DOM
    2. **In practice**, it looks like React only re-renders the component where the state update happens, but that's not how it **works behind the scenes**
        1. **React looks at the entire tree whenever a render happens**
    3. Renders are **not** triggered immediately, but **scheduled** for when the JS engine has some "free time" (the difference is usually a few milliseconds that we won't notice). There is also batching of multiple `setState` calls (in the same function) in event handlers (later)

How Rendering Works: The Render Phase

Review: The Mechanics of State in React




```
1. Not True #1: Rendering is updating the screen/DOM
    1. Rendering in React is just about calling the component functions and figuring out what needs to change in the DOM tree.
2. Not True #2: React completely discards old view (DOM) on re-render (technically)
    1. DOM will not be updated for the entire component instance
```

The Render Phase

1. When render phase begins,

```
Component instances that triggered re-render
    React will go through the entire component tree and takes all components instances that triggered re-render and renders them (calls correspo
    |
    v
React Elements
    Result of re-render
    |
    v
New Virtual DOM
    All React elements together is called a Virtual DOM
    |
    v
Reconciliation + Diffing <- Current Fiber Tree (before state update)
    The New Virtual DOM gets reconciled with the current Fiber tree (as it exists before the state update)
    The reconciliation is done by the Reconsiler called "Fiber"
    |
    v
Updated Fiber Tree
    The result of the reconciliation process is an updated Fiber tree
        It is the tree that will eventually be used to write to the DOM
    |
    v
List of DOM updates
    Result of the render phase ("list of effects")
```

The Virtual DOM (React Element Tree)

1. On the initial render:
    1. React takes the component tree and transforms it into a **React Element Tree** (Virtual DOM)
    2. **Virtual DOM**:
        1. Tree of all React elements created from all instances in the component tree
        2. Cheap and fast to construct multiple trees
            1. Even if we need multiple iterations
                1. Why? It is just a JS object
        3. It has nothing to do with "shadow DOM"
            1. Shadow DOM: A browser technology used in things like web-components
2. Suppose there is a state update in a component:
    1. It triggers a re-render
        1. React calls function of the component again
        2. The new react element thus generated is placed in the React Element Tree (Virtual DOM)
        3. **Rendering a component will cause all of its child components to be rendered as well (no matter if props changed or not)**
            1. If updated components return one or more other components, the nested components will be re-rendered as well
                1. If we update the highest component, the entire application will be re-rendered
                    1. Necessary because React doesn't know whether children will be affected (React plays it safe)
                        1. It does not update the entire DOM
                            1. Only Virtual DOM is re-created (not a big problem for small or medium sized applications)

What is Reconciliation and Why do we Need it?

1. Why not update the entire DOM whenever state changes somewhere in the app?
    1. Creation of Virtual DOM (React Element Tree) is cheap and fast (it is just a JS object)
        1. Writing to DOM is inefficient and wasteful (to always write the entire Virtual DOM to the real DOM, each time a render is triggered):

1. Writing to the DOM is (relatively) slow
2. Usually only a **small part of the DOM** needs to be updated
    1. The rest of the DOM can be re-used
2. For initial render, there is no way of constructing the entire DOM from scratch
    1. It doesn't make any sense doing it again after that
        1. Example: If we click show modal button, `showModal = true`
            1. Only the new DOM related to the modal need to be inserted into the DOM
                1. The rest of the DOM must stay the same
                    1. **React reuses as much of the existing DOM as possible** (tries to be as efficient as possible)
                        1. How does it do that?
                            1. **Reconciliation:** Deciding which DOM elements actually need to be inserted, deleted, or updated, in order to reflect the latest state changes
                                1. **The result of the reconciliation process is going to be a list of DOM operations necessary to update the current DOM with the new state**
                                    1. It is processed by a reconsiler
                                        1. It is the engine of React (heart) - it allows us to never touch the DOM directly. It tells the React what the next snapshot of UI should look like based on state

The Reconciler: Fiber

1. Process:
    1. During the initial render of the application, fiber takes the entire element tree (virtual DOM) and builds another tree: Fiber tree
        1. Fiber tree: Special internal tree that has a "fiber" for each component instance and DOM element (in the app)
            1. Unlike React elements in Virtual DOM, **Fibers are NOT re-created on every render**
                1. It is never destroyed (immutable data-structure)
                    1. Once it created in the initial render, it is mutated over and over again in future reconciliation steps
            2. Fibers are perfect to keep track of:
                1. Current state
                2. Props
                3. Side effects
                4. List of used hooks
                5. ...
            3. The actual state and props of any component instance are internally stored inside the corresponding fiber in the fiber tree
            4. Each fiber contains a queue of work to do
                1. Updating state
                2. Updating refs
                3. Running registered side-effects
                4. Performing DOM updates
                5. ...
            5. **Fiber is defined as a unit of work**
            6. Fibers are arranged in a different way than the elements in the React Element tree

```
App
 |
Video---Modal---Btn
          |
        Overlay
          |
         h3----button
```

                1. Each first child has a link to the parent
                2. All the other children have a link to the previous sibling
                    1. Linked List
                        1. Makes it easier for React to process the work associated with each fiber
                        2. Both React Element Tree and Fiber tree include DOM elements as well
                    2. Both trees are the entire representation of entire DOM structure (not just React components)
            7. Work can be done **asynchronously**
                1. Rendering process (what reconciler does) can be split into chunks, tasks can be prioritized (over others), and work can be **paused**, **reused**, or **thrown away** (if not valid anymore)
                    1. Automatic
                2. Practices uses of asynchronous rendering
                    1. Enables **concurrent features** like `Suspense` or transitions (React 18)
                    2. Long renders **won't block** JS engine
                        1. Allows rendering to pause and resume later
                            1. Especially good for large application
                    3. Possible because render phase does not produce any visible output to the DOM yet

Reconciliation in Action

1. Steps:
    1. Suppose the state `showModal = true`
    2. If we change `showModal = false`, it triggers a re-render
        1. The `Modal` and all its children are gone
    3. A new virtual DOM is created
        1. State update in `App`, so **all remaining children** are re-rendered
    4. The New vDOM needs to be reconciled with the fiber tree

        1. Results in `workInProgress` tree
    5. Whenever reconciliation happens, fiber walks through the entire tree (step by step) and analyzes what needs to change in the current fiber tree and the updated fiber tree
        1. Uses Virtual DOM
        2. **Comparing elements based on their position in the tree** - diffing
        3. It has the following marking is done in fiber tree:
            1. DOM Update (change of text in Btn)
            2. DOM Deletions (They are no longer in vDOM but are present in fiber tree)
            3. No updates (didn't change in the vDOM)
        6. After diffing, the DOM manipulations will be placed in a list called "list of effects"
            1. Used in commit phase to mutate the DOM

How Rendering Works: The Commit Phase

1. List of DOM updates (output of render phase)
    1. This list is used in the commit phase
2. **React writes to the DOM**: insertions, deletions, and updates (list of DOM updates are "flushed" to the DOM)
    1. React goes through the effects list and applies them one-by-one to the actual DOM elements (that were already existing in the DOM tree)
3. **Committing is synchronous**: DOM is updated in one go, it can't be interrupted. This is necessary so that the DOM never shows partial results, ensuring a consistent UI (in sync with state at all times).
    1. It cannot be paused or interrupted
4. After the commit phase completes, the `workInProgress` fiber tree becomes the `current` tree **for the next render cycle**
    1. Fiber trees are never discarded and never recreated from scratch
        1. They are reused to save precious rendering time
5. Browser paint:
    1. It re-paints the screen when it has idle time
        1. Updated UI on the screen
            1. We need to understand how browsers work internally
6. **Commit Phase** implementation:
    1. It is done my `ReactDOM` (and not `React`)
        1. `React` does not touch the DOM. React **only renders**. It doesn't know where the render result will go
            1. Why? React can be used on different platforms ("hosts")
                1. React Native - used to build native applications for iOS and Android
                2. Remotion - used to build videos using React
                3. Other renderers (they only commit but not render) - Word documents, PDFs, Figma designs, ...
    2. vDOM is only for web-applications, React Element Tree is preferred because React can be used for other kinds of applications
        1. That is why we import `React` and `ReactDOM`

Recap: Putting It All Together

1. Trigger <- happens only on inital render and state updates
2. Render Phase <- Does not produce any visual output
    1. Renders all component instances that needs rendering to re-render
    2. Steps:
        1. Updated React Elements will be placed in a new Virtual DOM (tree of React elements)
            1. All child components will be re-rendered (because React doesn't know if child components have been affected or not)
        2. Virtual DOM is reconciled with the Current Fiber Tree (representation of Element Tree before the state update)
            1. Why? It is slow and inefficient to rebuild the DOM tree, each time something on the screen must be updated
            2. Reconciliation finds the smallest number of DOM updates that reflects the latest state updates on the screen
                1. Done by a reconciler called fiber
        3. For each React element and a DOM element, there is a fiber
            1. Fiber - holds actual component state, props, and a queue of work
                1. Queue of work (after reconciliation) contains DOM updates needed for the element
            2. The computation for DOM updates is performed by a diffing algorithm
                1. It compares step-by-step the elements in the new virtual DOM with the elements in the current fiber tree (to know what has changed)
            3. The result of reconciliation and diffing process is a second updated fiber tree & a list of DOM updates
            4. **Asynchronous:** Fiber's work can be split, prioritized, paused, resumed
                1. Necessary for concurrent features
                2. Necessary to prevent JS engine from being blocked by complex render processes
        4. List of DOM updates will be written to the DOM in the commit phase
            1. Renderer (ReactDOM say) will insert, delete, and update DOM elements
                1. We end up with updated DOM that reflects the new state of the application
            2. **Synchronous**: DOM updates are written in one go, to keep UI consistent
        5. Once the browser realizes that the DOM has been updated, it starts a new browser paint to visually update UI on the screen
3. Some of the things discussed have practical implications, and implications on performance

How Diffing Works

1. Diffing uses 2 fundamental assumptions (rules):

    1. Two elements of different types will **produce different trees**
    2. Elements with a stable `key` prop (key that is consistent over time) **stay the same across renders**
        1. The assumptions are obvious, but allows React to go from (1Billion [O(n^3))] to 1000 [O(n)] operations per 1000 processed elements

2. Two situations that need to be considered

    1. Same position, **different** elements (between two renders)

        1. Examples:

```
<div>
    <SearchBar />
</div>
<main>...</main>

    |
    v
```

```
<header>
    <SearchBar />
</header>
<main>...</main>
```

1. React assumes **entire sub-tree is no longer valid**
    1. The element and its children will be destroyed and removed from the DOM (**including state**)
        1. The `div` is removed and a new `header` and `SearchBar` component instances are created
            1. If there were components with state, **that state will not be recovered**
                1. State is reset
    2. It works the same for React elements (`SearchBar` changing to `ProfileMenu` component)
        1. State is destroyed

2. Same position, **same** elements

    1. Example:

```
<div>
    <SearchBar wait={1} />
</div>
<main>...</main>
    |
    v
<div>
    <SearchBar wait={5} />
</div>
<main>...</main>
```

    1. Element will be kept (as well as child elements), **including state**
        1. Implications:
            1. Preserves state of both DOM elements and React elements

    2. Changes:

        1. React Elements: **New props** are passed if they changed between renders
        2. DOM Elements: React will just mutate the DOM element **attributes**
            1. React tries to be as efficient as possible, so the DOM elements will stay the same (they will not be removed from the DOM)
                1. State is also not destroyed
        3. Sometimes this is **not** what we want... Then we can use the `key` prop

Diffing Rules in Practice

1. Example:
    1. If we click on different tabs, the state of `TabContent` does not change because it remains in the same position in the Component Tree
    2. If the `TabContent` is replaced with `DifferentContent` component, the state is destroyed. Once we navigate back the `TabContent` the state is reset

The Key Prop

What is the Key Prop?

1. Key Prop
    1. Special prop that we can use to tell the diffing algorithm that an element is **unique** (by giving a unique id)
        1. Works for both DOM elements and React elements
    2. Allows React to **distinguish** between multiple instances of the same component type
        1. Why do we need to do this?
            1. When a key **stays the same across renders**, the element will be kept in the DOM (even if the position in the tree changes)
                1. This is why we need to **use keys in lists**
    3. When a key **changes between renders**, the element will be destroyed and a new one will be created (even if the position in the tree is the same as before)
        1. **Using keys to reset state** - use-case

Keys in Lists [Stable Key]

1. Example:

```
<ul>
    <Question question={q[1]} />
    <Question question={q[2]} />
</ul>
    | Adding new list item
    v
<ul>
    <Question question={q[0]} />
    <Question question={q[1]} />
    <Question question={q[2]} />
</ul>
```

    1. The original items appear in different positions in the React element tree (no longer 1st and 2nd children, but 2nd and 3rd children)

    2. Same elements, but **different position in tree**, so they are removed and recreated in the DOM (bad for performance)

        1. Wasted work (React doesn't know that)

    3. Solution: Keys (keys help us uniquely identify elements)

```
<ul>
    <Question key='q1' question={q[1]} />
    <Question key='q2' question={q[2]} />
</ul>
    | Adding new list item
    v
<ul>
    <Question key='q0' question={q[1]} />
```

```
            <Question key='q1' question={q[1]} />
            <Question key='q2' question={q[1]} />
        </ul>
```

1. The elements have stabe keys (keys that stay the same across renders)
2. The elements will be kept in the DOM
3. **Different position in the tree, but the key stays the same**, so the elements will be kept in the DOM
    1. Always use keys!
        1. If we have multiple child elements of the same type

Key Prop to Reset State (Changing Key)

1. Example:

```
<QuestionBox>
    <Question
        question={{
            title: 'React vs JS',
            body: 'Why should we use React?',
        }}
    />
</QuestionBox>
    |
    v
<QuestionBox>
    <Question
        question={{
            title: 'Best course ever',
            body: 'This is THE React course!',
        }}
    />
</QuestionBox>
```

    1. `Question` state (`answer`):
        1. Set to 'React allows us to build apps faster'
    2. Now `Question` changes to 'This is THE React course!'
        1. Diffing rule: If we have the same element at the same position in the tree, the **DOM element and state** will be kept
            1. State was preserved. NOT what we want
    3. Solution:

```
<QuestionBox>
    <Question
        question={{
            title: 'Best course ever',
            body: 'This is THE React course!',
        }}
        key="q23"
    />
</QuestionBox>
```

        1. `key` prop uniquely identifies the question
            1. When a new question appears, we can give it a different key

```
<QuestionBox>
    <Question
        question={{
            title: 'Best course ever',
            body: 'This is THE React course!',
        }}
        key="q89"
    />
</QuestionBox>
```

                1. React identifies it to be a different component instance, so React instantiates a new React element
                    1. State will be reset (`answer` is set to empty)

Resetting State With the Key Prop

1. Fixing Tabbed component

```
<TabContent
  item={content.at(activeTab)}
  key={content.at(activeTab).summary}
/>
```

Using the Key Prop to Fix Our Eat-'N-Split App

1. Code:

```
{selectedFriend && (
  <FormSplitBill
    selectedFriend={selectedFriend}
    onSplitBill={handleSplitBill}
    key={selectedFriend.id}
  />
)}
```

Rules for Render Logic: Pure Components

1. Rules
2. What is render logic?

The Two Types of Logic in React Components

1. Render Logic
    1. Code that lives at the **top level** of the component function
    2. Participates in **describing** how the component view (of an instance) looks like
    3. Example:

```
function Question({ question }) {
    const [newAnswer, setNewAnswer] = useState(''); // Participates in describing how component view looks like
    const numAnswers = question.answers.length ?? 0; // Participates in describing how component view looks like

    const handleNewAnswer = function (e) {
        if (question.closed) return;
        setNewAnswer(e.target.value);
    }

    const createList = function () {
        // Participates in describing how component view looks like
        return (
            <ul>
                {question.answers.map((q) => (
                    <li>{q}</li>
                ))}
            </ul>
        );
    }

    // Participates in describing how component view looks like
    return (
        <div>
            <h3>{question.title}</h3>
            <p>{question.body}</p>
            {question.hasAnswer ? (
                createList()
            ) : (
                <input
                    value={newAnswer}
                    onChange={handleNewAnswer}
                />
            )
        </div>
    )
}
```

        1. Render logic: It is the code that is executed **every time** the component renders (each time the function is called)

2. Event Handler Functions
    1. Executed as a **consequence of the event** that the handler is listening for (`change` event in the example)
        1. `handleNewAnswer` - event handler function
    2. Code that actually **does things**: update state, perform an HTTP request, read an input field, navigate to another page, etc (all things that change and manipulate the application in some way).
        1. Code that makes things happen in the application
3. React requires that all components are pure when it comes to render logic for the app to work as expected

Refresher: Functional Programming Principles

1. **Side effect:** Dependency on or modification of any data outside the function scope. **"Interaction with the outside world"**. Examples: mutating external variables, HTTP requests, writing to DOM, setting timers

```
const areas = {};

function circleArea(r) {
    areas.circle = 3.14 * r * r;
}
```

    1. Side effect: Outside variable mutation

2. **Pure function:** A function that has **no** side effects
    1. Does **not** change any variables outside its scope
    2. Give the **same input**, a pure function always returns the **same output**

```
function circleArea(r) {
    return 3.14 * r * r;
}
```

        1. Makes it predictable

    3. Unpredictable functions:

```
function circleArea(r) {
    const date = Date.now();
    const area = 3.14 * r * r;
    return `${date}: ${area}`
}
```

        1. The output will always be different

3. **Side effects are not bad!**: A program can only be useful if it has some interaction with the outside world
    1. If an app doesn't fetch any data or never writes to the DOM is useless

1. Solution: To make useful and bug-free applications, we need to know when and how to cause side-effects

Rules for Render Logic

1. **Components must be pure when it comes to render logic:** given the same props (input), a component instance should always return the same JSX (output)
    1. **Render logic must produce no side effects**: no interaction with the "outside world" is allowed. So, in render logic:
        1. Do NOT perform **network requests** (API calls)
        2. Do NOT start **timers**
        3. Do NOT directly **use the DOM API**
        4. Do NOT **mutate objects or variables** outside of the function scope
            1. This is why we can't mutate props (hard rule)
                1. Doing so is a side effect
        5. Do NOT **update sate (or refs):** this will cause an infinite loop!
            1. They are not side effects though
        6. Other side effects may be allowed because they are not harmless:
            1. `console.log`
            2. Generating random numbers
    2. Side effects are allowed (and encouraged) in **event handler functions!** There is also a special hook to **register side effects** (`useEffect`)
        1. If we need to cause side effects as soon as a function executes, we can register that side effect using a special hook called `useEffect`

State Update Batching

How State Updates are Batched

1. Renders are **not** triggered immediately, but **scheduled** for when the JS engine has some "free time". There is also batching of multiple `setState` calls in event handlers
2. Example:

```
const [answer, setAnswer] = useState('');
const [best, setBest] = useState(true);
const [solved, setSolved] = useState(false);

const reset = function () {
    setAnswer('');
    console.log(answer);
    setBest(true);
    setSolved(false);
}

return (
    <div>
        <button onClick={reset}>Reset</button>
        {/* ... */}
    </div>
);
```

    1. Event handler function: `reset`
        1. How are the 3 pieces of state updated behind the scenes
            1. React does not update one state at a time (which are in the same event handler)
                1. The state updates will be batched into one state update for the event-handler
                2. **React triggers one re-render and commit per event handler**
                    1. The state changes represent one view, and hence React updates screen only once (we may not be interested in partial render)
                        1. Better performance
        2. What is the value of `answer` be at this point?
            1. State is stored in the Fiber tree during **render phase**
            2. At this point, re-render has **not happened yet**
                1. React is reading the function line by line and didn't update the state yet, and not re-rendered yet
                    1. The `answer` still contains **current state**, **not** the updated state ('')
                    2. The state is stale
        3. Updating state in React is asynchronous
            1. Because updated state variables are **not** immediately available after `setState` call, **but only after the re-render**.
            2. This also applies when **only one** piece of state is updated
            3. If we need to update state **based on pervious update**, we use `setState` with callback (`setAnswer(answer => ...)`) (instead of a single value)

Batching Beyond Event Handler Functions

1. Before React 18, React only did automatic batching only in event-handlers (not in situations in which a browser event has alreay happened)
    1. There are situations in which we need to update the state long after a browser event has already happened
        1. Examples:
            1. Timeouts (running `reset` function a second after the `click` event)

               ```
               setTimeout(reset, 1000);
               ```

            2. Promises (running `reset` function only after some data is fetched)

               ```
               fetchStuff().then(reset);
               ```

            3. Native events

               ```
               el.addEventListener('click', reset);
               ```

        2. Before React 18, state updates inside a function that is called after timeout or primises were not batched.
            1. React would update state variables one by one
        3. With latest React version, we get automatic batching all the time everywhere in the code

4. In certain situations, in which automatic batching can be problematic
    1. Solution: We can **opt out** of automatic batching by wrapping a state update in `ReactDOM.flushSync()` (but you will never need this)

State Update Batching in Practice

1. If the new state is equal to the current state, the component is not re-rendered (nor the state is updated)
2. If we are updating state based on the previous state, we must always use a callback function
3. Code:

```
function handleInc() {
  // setLikes(likes + 1);
  setLikes((likes) => likes + 1);
}

function handleTripleInc() {
  // Increased only once. `likes` is 0 in all the three cases. The state is stale
  // setLikes(likes + 1);
  // setLikes(likes + 1);
  // setLikes(likes + 1);

  // In each callback, we get access to the latest updated state
  setLikes((likes) => likes + 1); // 0 + 1
  setLikes((likes) => likes + 1); // 1 + 1
  setLikes((likes) => likes + 1); // 2 + 1
}
```

    1. Batching works with other functions as well (not only event-handlers)

```
function handleUndoLater() {
  setTimeout(handleUndo, 2000); // handleUndo - is just another function that is called at a later time
}
```

How Events Work in React

1. How React handles events and how they work behind the scenes

DOM Refresher: Event Propagation and Delegation

1. Example:

```
Document <html> <- DOM tree (not Fiber tree or React element tree)
    |
<body>
    |
<div>
#root
    |
<header>
.topbar
    |
<div>
.options
    |
+---+------+---------+
|           |         |
<button>    <button>  <button>
.btn        .btn      .btn
```

    1. As soon as an event is fired, a new event object will be created
        1. It is not created where the event occurred
            1. The object will be created at the root of the document (top of the tree)
    2. The event object travels down the tree until it reaches the target element in the capturing phase
        1. Target element: Element on which the event was first triggered
        2. At the target, we can place an event handler on the element
    3. After the event object reached the target element, it travels up the tree upto the root of the tree in the bubbling phase
2. During capturing and bubbling phases, an event object goes through every single parent and child element, one by one
    1. It seems the event has originated in each of the DOM elements
    2. By default, event handlers listen to events on the target **and during the bubbling phase**
        1. Every single event-handler in a parent will be executed during the bubbling phase (if they are listening to the same type of event)
            1. We can **prevent bubbling** with `e.stopPropagation()` function (works in vanila JS, and React)
3. Event Delegation:
    1. Handling events for multiple elements centrally in **one single parent element**
        1. If there are 1000 buttons, if each button has an event handler, it would be problematic for the app's performance and memory usage
            1. Example: Add handler to **parent** (`.options`)
                1. When a click happens on one of the buttons, the event will bubble up to the `.options` element.
                2. We can check for **target** element (`e.target`)
                3. If **target** is one of the `<button>`s, handle the event
                    1. Very common in vanilla JS apps, but no so much in React apps
                        1. Why talk about it?
                            1. Sometimes we might find strange behavior related to events in our application (which might have to do with event bubbling)
                            2. React does this behind the scenes

How React handles Events

1. If we want to attach an event handler to one of the buttons or to another DOM element

```
<button
    className="btn"
    onClick={() => setLoading(true)}
/>
```

1. What React does is the following:

```
document
    .querySelector('#root')
    .addEventListener(
        'click',
        () => setLoading(true)
    );
```

1. `#root` container is the DOM element in which the React app is displayed
2. React registers all event handlers on the **root DOM container**. This is where **all events are handled**
    1. React does it in a more complex way
    2. React registers **one event handler function per event type** at the root node of the fiber tree during the render phase
        1. If we have multiple event-handler functions per event type, React bundles them together into one handler function in the root node of the fiber tree
        2. Behind the scenes, React performs **event delegation** for all events in our applications
            1. To the root DOM container where they are handled
    3. Flow:
        1. When an event occurs (button is clicked), and event is created in the Document root and it descends down until it is captured by the target.
        2. The event object will bubble back up as soon as the event reaches the root container (where React registered all the handlers), the event is handled (if a handler matches the target event type)
            1. Usually `div#root`, but can be **any** DOM element
        3. After the event is handled, it continues to bubble up and disappears

Synthetic Events

1. How event objects work behind the scenes
    1. Whenever we declare an event handler, React gives us access to the event object that was created (just like JS)
        1. In React the event object is different
            1. In vanilla JS, we get access to the native DOM event object
                1. `PointerEvent`
                2. `MouseEvent`
                3. `KeyboardEvent`
                4. ...
            2. React gives us **SyntheticEvent** (a thin wrapper around DOM's native event object)
                1. They are similar to native event objects but they add or change some functionality on top of them
                2. They have **same interface** as native event objects, like `stopPropagation()`, and `preventDefault()`
                3. Fixes browser inconsistencies, so that events work in the exact **same way in all browsers**
                4. **Most synthetic events bubble** (including focus, blur, and change), except for scroll
2. Event Handlers in React vs JS
    1. Attributes for event handlers are named used `camelCase` (prop-names) (`onClick` instead of `onclick` or `click` (if we use `addEventListener`))
    2. In vanilla JS, to stop the default behavior of the browser in response to an event, we can return `false` from an event handler function
        1. Example: Preventing browser from reloading the page when we click on `submit`
        2. Default behavior can **not** be prevented by returning `false` (only by using `preventDefault()` on synthetic event object)
    3. Attach `Capture` to the event handler name, if you need to handle during **capture phase** (example: `onClickCapture`)

Libraries vs. Frameworks & The React Ecosystem

First, an Analogy

1. Making a sushi for the first time
    1. All-in-one Kit
        1. Ease of mind: All ingredients are included
        2. No choice
            1. You're stuck with the kit's ingredients (we may not like one or more of them)
    2. Separate Ingredients
        1. Freedom: You can choose the best ingredients
        2. Decision fatigue: You need to research and buy all ingredients separately
2. Front-end technologies
    1. All-in-one kit
        1. Angular
        2. Vue
        3. Swelt
    2. Separate Ingredients
        1. React

Framework vs Library

1. Framework
    1. Easy of mind: Everything you need to build a complete application **is included** in the framework ("batteries included")
        1. A complete structure that includes everything we need to build a complete large scale application
    2. Frameworks include everything
        1. Angular:
            1. HTTP Requests
            2. Styling
            3. Routing
            4. Form management
    3. No choice: We're stuck with the framework's tools and conventions (which is not always bad!)
2. Library
    1. Pieces of code that developers share for other developers to use
        1. Example: React
            1. React - View library

        1. It just draws components on a UI (view)
    2. To build a large-scale SPA, we need to include many third-party libraries for
        1. HTTP Requests
        2. Styling
        3. Routing
        4. Form management
        5. ...
    3. We can build React apps with just React (without libraries)
        1. But only for small apps
        2. When learning React
    4. Freedom: You can (or need to) **choose multiple 3rd-party libraries** to build a complete application
        1. We can choose the one we like the most
        2. Every app has different requirements and can have a different combination of libraries
    5. Decision fatigue: You need to **research**, **download**, **learn**, and **stay-up-to-date** with multiple external libraries
        1. We will have a very good understanding of the most important libraries that we usually include in most React projects

React 3rd-Party Library Ecosystem

1. React's popularity has led to a huge ecosystem of 3rd-party libraries which can be included in our app for different needs
    1. Routing (for SPAs):
        1. **React Router**
        2. React Location
    2. HTTP requests:
        1. **JS** `fetch()`
        2. Axios
    3. Remote state management:
        1. **React Query**
        2. SWR
        3. Apollo
    4. Global state management:
        1. **Context API**
        2. **Redux**
        3. Zustand
    5. Styling
        1. **CSS modules**
        2. **Styled components**
        3. **Tailwindcss**
    6. Form management
        1. **React Hook Form**
        2. Formik
    7. Animations/Transitions
        1. Motion
        2. react-spring
    8. UI components
        1. MJ
        2. Chakra
        3. Mantine
2. Many developers were overwhelmed by the many decisions that need to be made for choosing 3rd-party libraries, which led to the development of opinionated React frameworks such as:
    1. Nextjs
    2. Remix
    3. Gatsby

Frameworks Build on Top of React

1. Next.js, Remix, and Gatsby are React frameworks because they are built on top of React
    1. They extend React functionality
    2. Opinionated: Other developers have included their own opinions on how to handle stuff like the following into their frameworks
        1. Routing
        2. State-management
        3. Styling
2. In Vanilla React app, we need to make the decisions on what external libraries to use
3. In Next.js, Remix, and Gatsby, some of the decisions have already been taken
    1. Makes project development much easier, faster, and lead to better overall developer experience
4. Different frameworks specialize in different aspects
    1. All of them offload much of the setup work from us
5. **React frameworks offer many other features:** server-side rendering (SSR), static site generation (SSG), better developer experience (DX), etc.
    1. Many can be described as full-stack frameworks
        1. We can build full-stack apps with them (using React as the base layer)
6. **We can learn the frameworks only if we master React, and the third-party frameworks**

Section Summary: Practical Takeaways

Practical Summary

1. Practical takeaways and conclusions (we need them in the future)
2. A **component** is like a blueprint for a piece of UI that will eventually exist on the screen. When we "use" a component, React constructs a **component instance**, which is like an actual physical manifestation of a component, containing props, state, and more. A component instance, when rendered, will return a **React element**
    1. Similar to a blueprint of a house, and actual houses are built using the blueprint
3. "Rendering" only means **calling component functions** and calculating what DOM elements need to be inserted, deleted, or updated. It has nothing to do with writing to the DOM. Therefore, **each time a component instance is rendered and re-rendered, the function is called again**
    1. Writing to the DOM is "committing" in React language
4. Only the **initial app render** and **state updates** can cause a render, which happens for the **entire application**, not just one single component
    1. The Render process is execute for all components

5. When a component instance gets re-rendered, **all its children will get re-rendered as well**. This doesn't mean that all children will get updated in the DOM, thanks to reconciliation, which checks which elements have actually changed between two renders. But all this re-rendering can still have an impact on performance (more on that later in the course)
6. Diffing is how React decides which DOM elements needs to be added or modified. If, between renders, a certain React element **stays at the same position in the element tree**, the corresponding DOM element and component state will stay the same. If the element **changed to a different position**, or if it's a **different element type**, the DOM element and state will be destroyed.
    1. They will be reset
7. Giving elements a key prop allows React to distinguish between multiple component instances. **When a key stays the same across renders**, the element is kept in the DOM. This is why we need to use keys in lists. **When we change the key between renders**, the DOM element will be destroyed and rebuilt. We use this as a **trick to reset state**
8. **Never declare a new component inside another component!** Doing so will re-construct the nested component every time the parent component re-renders. React will always see the nested component (like a new variable) as **new**, and therefore **reset its state** each time the parent state is updated
    1. Always declare components at the top-level of the file (never inside another component)
9. The logic that produces JSX output for a component instance ("render logic") is **not allowed to produce any side effects**: no API calls, no timers, no object or variable mutations, no state updates. **Side effects are allowed in event handlers** and **useEffect** (next section)
10. The DOM is updated in the commit phase, **but not by React, but by a "renderer" called ReactDOM**. That's why we always need to include both libraries in a React web app project. We can use other renderers to use React on different platforms, for example to build mobile or native apps (with React Native)
11. Multiple state updates inside an event handler function are **batched**, so they happen all at once, **causing only one re-render** (for performance). This means we can **not access a state variable immediately after updating it**: state updates are **asynchronous**. Since React 18, batching also happens in timeouts, promises, and native event handlers.
12. When using events in event handlers, we get access to a **synthetic event object**, not the browser's native object, so that **events work the same way across all browsers**. The difference is that **most synthetic events bubble**, including focus, blur, and change, which do not bubble as native browser events. Only the scroll event does not bubble.
13. **React is a library, not a framework**. This means that you can assemble your application using your favorite third-party libraries (for flexibility and freedom). The downside is that you need to find and learn all these additional libraries. No problem, as you will learn about the most commonly used libraries in this course.

Section 12: Effects and Data Fetching

Section Overview

1. Topics:
    1. Data fetching is essential
    2. Effects with `useEffect` hook
        1. `usePopcorn` project
        2. Side effects
        3. How and when effects are executed
    3. Effect cleanup
    4. Real-world application!

The Component Lifecycle

1. Lifecyle of components

Component (Instance) Lifecycle

1. Encompasses the different phases, a specific component can go through over time
2. Phases:
    1. Mount/Initial Render
        1. Component instance is rendered for the **first time**
        2. Fresh state and props are **created** (for the component instance)
            1. Component is born in this phase
    2. Re-Render (unlimited number of times) (optional)
        1. Happens when:
            1. **State** changes
                1. This is only for the entire application
            2. **Props** change
            3. **Parent** re-renders
            4. **Context** changes
        2. All components do not go through this phase
            1. Some components are simply mounted and unmounted
    3. Unmount (when no longer needed) (can happen when a user navigates to a new section or page or close the app)
        1. Component instance is **destroyed** and **removed**
        2. State and props are **destroyed**
3. **We can hook into the lifecycle**
    1. We can define code to be executed at these specific points in time
        1. We do so using the `useEffect` hook

How NOT to Fetch Data in React

1. Loading data
    1. Doing it the wrong way
        1. We must never update state in the Render logic
            1. If we break the rule?
2. Movies:
    1. OMDB (Open source version of IMDB)
        1. Generate API Key (for free)
    2. Fetching movies data:

```
fetch(`http://www.omdbapi.com/?apikey=${KEY}&s=interstellar`)
  .then((res) => res.json())
```

```
.then((data) => setMovies(data.Search));
```

1. Runs infinite number of requests (never stops)
    1. Why?
        1. Setting state causes component to re-render
        2. Re-render will execute the function again
        3. The function fetches data again and sets the set
        4. Thus the infinite loop
            1. Hence, we must not set state in the render logic
    2. Another example:

```
setWatched([]); // Causes too many re-renders
```

3. How to set state?
    1. In the `useEffect` hook

useEffect to the Rescue

1. It gives us a place where we can safely write side effects
    1. Side effects registered with the `useEffect` hook will only be executed **after certain renders**
        1. Example: **Only after the initial render**
    2. `useEffect` hook is used to register an effect (a side effect). The effect is the function we pass into the hook.
        1. **After it has been painted onto the screen** (after render)
2. Code:

```
useEffect(function () {
  // function contains code that we want to run as a side-effect
  // It is registered to be run at a certain point in time
  fetch(`http://www.omdbapi.com/?apikey=${KEY}&s=interstellar`)
    .then((res) => res.json())
    .then((data) => setMovies(data.Search));
}, []); // It doesn't return anything
// Second argument is dependency array
// `[]` - means that the hook will run only on mount (inital render)
// Good if we want to fetch data as soon as the application loads
```

A First Look at Effects

1. What is an effect, and how is it different from an event-handler function?

Where to Define Side Effects

1. Review: A **side effect** is basically any "interaction between a React component and the world outside the component". We can also think of a side effect as "code that actually does something" (useful to happen). Examples: Data fetching, setting up subscriptions, setting up timers, manually accessing the DOM, etc.
2. Side Effect:
    1. We need side effects all the time (when we build React apps). They make our applications **do something**. **Not** in render logic (side effects should not occur **during** the execution of the render logic. It can be after)!
    2. We can define side-effects in two different places in React
        1. In Event Handlers
            1. Event handlers are functions triggered by events: `onClick`, `onSubmit`, etc.
            2. Sometimes this is **not enough** for the application's needs
                1. We might need code that executes automatically **as soon as a component renders** (triggered by rendering)
                    1. Solution: Effects (`useEffect`)
        2. Using Effects
            1. Effects allow us to write code that will run at **different moments**:
                1. Mount
                2. Re-render
                3. Unmount

Event Handlers vs. Effects

1. When? (Both produce the same result, but at different moments)
    1. Event Handlers
        1. Executed when the **corresponding event happens**
        2. Used to **react** to an event
        3. **Preferred way of defining side effects!**
    2. Effects (`useEffect`)
        1. Executed **after the component mounts** (initial render), and **after subsequent re-renders** (according to dependency array)
            1. We can use the dependency array to tell the component to run `useEffect` after a component re-renders
        2. Each effect can returns a `cleanup` function (a function which will be called before the component re-renders or unmounts)

```
useEffect(function () {
    fetch(...)
        .then(...)
        .then(...);

    return () => console.log('Cleanup');
}, []);
```

            1. We should not think about lifecycle, but about synchronization.

1. Used to keep a component **synchronized with some external system** (in this example, with the API movie data)
2. We must not over-use `useEffect` hook
    1. Everything that can handed using event-handlers must be handled in event-handlers

Using an async Function

1. If we use `async` function inside `useEffect`, we get a warning that effect callbacks are synchronous to prevent race-conditions (we cannot return a promise from a `useEffect` callback).
2. In React 18, in strict mode, the effects will run twice
    1. This is only true in development mode.
        1. If we remove from String mode, effect is called only once
    2. This doesn't happen in production

Adding a Loading State

1. If moves are loaded in the background, we want to display a loading indicator

    1. Throttle network using Slow 3G
    2. Solution: More state
        1. A state variable that indictes that the data is still being loaded
            1. When the data is loaded, we can display the data and not the loading indicator

2. Code:

```
import { useEffect, useState } from "react";

const tempMovieData = [
  {
    imdbID: "tt1375666",
    Title: "Inception",
    Year: "2010",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BMjAxMzY3NjcxNF5BMl5BanBnXkFtZTcwNTI5OTM0Mw@@._V1_SX300.jpg",
  },
  {
    imdbID: "tt0133093",
    Title: "The Matrix",
    Year: "1999",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BNzQzOTk3OTAtNDQ0Zi00ZTVkLWI0MTEtMDllZjNkYzNjNTc4L2ltYWdlXkEyXkFqcGdeQXVyNjU0OTQ0OTY@._V1_SX300.jp
  },
  {
    imdbID: "tt6751668",
    Title: "Parasite",
    Year: "2019",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BYWZjMjk3ZTItODQ2ZC00NTY5LWE0ZDYtZTI3MjcwN2Q5NTVkXkEyXkFqcGdeQXVyODk4OTc3MTY@._V1_SX300.jpg",
  },
];

const tempWatchedData = [
  {
    imdbID: "tt1375666",
    Title: "Inception",
    Year: "2010",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BMjAxMzY3NjcxNF5BMl5BanBnXkFtZTcwNTI5OTM0Mw@@._V1_SX300.jpg",
    runtime: 148,
    imdbRating: 8.8,
    userRating: 10,
  },
  {
    imdbID: "tt0088763",
    Title: "Back to the Future",
    Year: "1985",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BZmU0M2Y1OGUtZjIxNi00ZjBkLTg1MjgtOWIyNThiZWIwYjRiXkEyXkFqcGdeQXVyMTQxNzMzNDI@._V1_SX300.jpg",
    runtime: 116,
    imdbRating: 8.5,
    userRating: 9,
  },
];

function Navbar({ children }) {
  return (
    <nav className="nav-bar">
      <Logo />
      {children}
    </nav>
  );
}

function Logo() {
  return (
    <div className="logo">
      <span role="img">🍿</span> {/* No need to break */}
      <h1>usePopcorn</h1>
    </div>
  );
}

function Search() {
  const [query, setQuery] = useState("");
  return (
    <input
      className="search"
      type="text"
      placeholder="Search movies..."
      value={query}
```

```
        onChange={(e) => setQuery(e.target.value)}
      />
    );
}

function NumResults({ movies }) {
  return (
    <p className="num-results">
      Found <strong>{movies.length}</strong> results
    </p>
  );
}

function Main({ children }) {
  return <main className="main">{children}</main>;
}

function Box({ element }) {
  const [isOpen, setIsOpen] = useState(true);

  return (
    <div className="box">
      <button className="btn-toggle" onClick={() => setIsOpen((open) => !open)}>
        {isOpen ? "–" : "+"}
      </button>
      {isOpen && element}
    </div>
  );
}

function MovieList({ movies }) {
  return (
    <ul className="list">
      {movies?.map((movie) => (
        <Movie movie={movie} />
      ))}
    </ul>
  );
}

function Movie({ movie }) {
  return (
    <li key={movie.imdbID}>
      <img src={movie.Poster} alt={`${movie.Title} poster`} />
      <h3>{movie.Title}</h3>
      <div>
        <p>
          <span>🗓</span>
          <span>{movie.Year}</span>
        </p>
      </div>
    </li>
  );
}

// function WatchedBox() {
//   const [watched, setWatched] = useState(tempWatchedData);
//   const [isOpen2, setIsOpen2] = useState(true);

//   return (
//     <div className="box">
//       <button
//         className="btn-toggle"
//         onClick={() => setIsOpen2((open) => !open)}
//       >
//         {isOpen2 ? "–" : "+"}
//       </button>
//       {isOpen2 && (
//         <>
//           <WatchedSummary watched={watched} />

//           <WatchedMovieList watched={watched} />
//         </>
//       )}
//     </div>
//   );
// }

function WatchedSummary({ watched }) {
  const avgImdbRating = average(watched.map((movie) => movie.imdbRating));
  const avgUserRating = average(watched.map((movie) => movie.userRating));
  const avgRuntime = average(watched.map((movie) => movie.runtime));

  return (
    <div className="summary">
      <h2>Movies you watched</h2>
      <div>
        <p>
          <span>#️⃣</span>
          <span>{watched.length} movies</span>
        </p>
        <p>
          <span>⭐</span>
          <span>{avgImdbRating}</span>
        </p>
        <p>
          <span>🌟</span>
          <span>{avgUserRating}</span>
        </p>
        <p>
          <span>⏳</span>
          <span>{avgRuntime} min</span>
        </p>
      </div>
```

```
      </div>
    );
  }

  function WatchedMovieList({ watched }) {
    return (
      <ul className="list">
        {watched.map((movie) => (
          <WatchedMovie movie={movie} key={movie.imdbID} />
        ))}
      </ul>
    );
  }

  function WatchedMovie({ movie }) {
    return (
      <li key={movie.imdbID}>
        <img src={movie.Poster} alt={`${movie.Title} poster`} />
        <h3>{movie.Title}</h3>
        <div>
          <p>
            <span>⭐</span>
            <span>{movie.imdbRating}</span>
          </p>
          <p>
            <span>🌟</span>
            <span>{movie.userRating}</span>
          </p>
          <p>
            <span>⌛</span>
            <span>{movie.runtime} min</span>
          </p>
        </div>
      </li>
    );
  }

  const KEY = "4256c7"; // If it is inside the component, it will get re-created each time the component is re-rendered

  const average = (arr) =>
    arr.reduce((acc, cur, i, arr) => acc + cur / arr.length, 0);

  export default function App() {
    const [movies, setMovies] = useState(tempMovieData);
    const [watched, setWatched] = useState(tempWatchedData);
    const [isLoading, setIsLoading] = useState(false);
    const query = "interstellar";

    useEffect(function () {
      // function contains code that we want to run as a side-effect
      // It is registered to be run at a certain point in time

      async function fetchMovies() {
        setIsLoading(true);
        const res = await fetch(
          `http://www.omdbapi.com/?apikey=${KEY}&s=${query}`
        );
        const data = await res.json();
        setMovies(data.Search);
        console.log(movies); // empty array, because setting state is asynchronous
        setIsLoading(false);
      }

      fetchMovies();
    }, []); // It doesn't return anything
    // Second argument is dependency array
    // `[]` - means that the hook will run only on mount (inital render)
    // Good if we want to fetch data as soon as the application loads

    // setWatched([]); // Causes too many re-renders

    return (
      <>
        <Navbar>
          <Search />
          <NumResults movies={movies} />
        </Navbar>
        <Main>
          <Box element={isLoading ? <Loader /> : <MovieList movies={movies} />} />
          <Box
            element={
              <>
                <WatchedSummary watched={watched} />
                <WatchedMovieList watched={watched} />
              </>
            }
          />
          {/* <Box>
            <MovieList movies={movies} />
          </Box>
          <Box>
            <WatchedSummary watched={watched} />
            <WatchedMovieList watched={watched} />
          </Box> */}
        </Main>
      </>
    );
  }

  function Loader() {
    return <p className="loader">Loading...</p>;
  }
```

Handling Errors

1. When we do data fetching in web applications, and dealing with async data, we must assume that something might go wrong.
    1. What can go wrong?
        1. Users losing their internet connections
            1. Slow 3G
            2. Once movies are loading, we select offline
                1. Console log: Failed to fetch
                    1. Users lost internet connection
                        1. We want to display error message on the screen
                            1. Reacting to errors is not built into `fetch`

The useEffect Dependency Array

What's the USEEFFECT Dependency Array?

1. The dependency array
    1. By default, effects run **after every render**. We can prevent that by passing a **dependency array**
        1. Why does `useEffect` need an array of dependencies?
            1. Without the dependency array, React doesn't know **when** to run the effect.
                1. **Each time one of the dependencies changes, the effect will be executed again**
        2. What are dependencies?
            1. Every **state variable** and **prop** used inside the effect **MUST be included in the dependency array**
2. Example:

```
const title = props.move.Title;
const [userRating, setUserRating] = useState('');

useEffect(
    function () {
        if (title) return;
        document.title = `${title} ${userRating && (Rated ${userRating} )
        }`;

        return () => (document.title = 'userPopcorn');
    },
    [title, userRating]
);
```

    1. Both `title` and `userRating` must be in the dependency array, since we are using both in the `useEffect` hook.
        1. Otherwise, if the `title` or `userRating` changes, React will not know about the change to execute the `useEffect` code
            1. The bug is called: "stale closure" (later)

UseEffect is a Synchronization Mechanism

1. The machanics of effects:
    1. `useEffect` is like an **event listener** that is listening for one dependency to change. **Whenever a dependency changes, it will execute the effect again**
2. Dependencies:
    1. `[title, userRating]`
        1. `title` changes -> Effect is executed again
        2. `userRating` changes -> Effect is executed again
3. Effects **react** to updates to state and props used inside the effect (the dependencies). So **effects are "reactive"** (like state updates re-render the UI)
    1. Extremely useful and powerful
4. **Effects are used to keep a component synchronized with external system**
    1. Some system that lives outside our React-based code

```
Component state/props -> synchronize with -> external system (side effect)
```

        1. Example, if we update the `title`, the `title` of the document will get updated
    2. `useEffect` is a synchronization mechanism
        1. **A mechanism to sync the effect with the state of the application**

Synchronization and Lifecycle

1. Dependency (State or Prop) Changes
    1. -> Effect is Executed Again
    2. -> Component is re-rendered
2. Effects and component lifecycle are deeply connected (it is not a lifecycle hook, but a synchronization mechanism)
3. Whe canuse the dependency array to run effects **when the component renders or re-renders**
4. Types of dependency arrays:
    1. `useEffect(fn, [x, y, z]):
        1. Synchronization:
            1. Effect synchronizes with x, y, and z
        2. Lifecycle
            1. Runs on **mount** and **re-renders** triggered by updating x, y, or z
                1. If any other piece of prop or state is updated, this effect will not be executed
    2. `useEffect(fn, [])`:
        1. Synchronization:
            1. Effect synchronizes with **no state/props**

        1. It doesn't use any values relevant for rendering the component.
            1. It is safe to be executed only once
    2. Lifecycle:
        1. Runs only on **mount** (initial render)
3. `useEffect(fn)`:
    1. Synchronization:
        1. Effect synchronizes with **everything**
            1. Every state and every prop will be dependencies in this case
    2. Lifecycle
        1. Runs on **every render** (usually bad)

When are Effects Executed?

1. Timeline:

```
|
- Mount (Initial Render) `<MovieDetails />`
|
- Commit
|
- Browser Paint
|
- Effect (after browser paint)
    Because effects may contain long running processes (fetching data say)
        Otherwise it would block the component, and will see old version of the component for too long
    If effect sets state, an additional render will be required
        Hence we must not overuse effects
|
- `title` changes
|
- Re-render
|
- Commit
|
- Layout Effect (Another type of effect that is **very rarely** necessary (`useLayoutEffect`) - discouraged
|
- Browser paint
|
- [more effects]
|
- Effect (since `title` is part of the effect dependency)
|
- ... (this can repeat again until component unmounts
|
- Unmount
```

Synchronizing Queries With Movie Data

1. Code examples:

```
useEffect(function () {
  console.log("After initial render"); // Appears second - because it appears before the other effects.
}, []);

useEffect(function () {
  console.log("After every render");
}); // Appears last
// If `query` is updated, this effect is executed because it runs after every render.

useEffect(
  function () {
    console.log("Synchronized with `query` state variable");
  },
  [query]
);
```

Selecting a Movie

Loading Movie Details

1. Code: Loading movie details

```
useEffect(
  function () {
    async function getMovieDetails() {
      setIsLoading(true);
      const res = await fetch(
          `http://www.omdbapi.com/?apikey=${KEY}&i=${selectedId}`
      );
      const data = await res.json();
      console.log(data);
      setMovie(data);
      setIsLoading(false);
    }
    getMovieDetails();
  },
  [selectedId]
);
```

Adding a Watched Movie

Adding a New Effect: Changing Page Title

1. Changing the title is a side effect
    1. We want to register a side effect using the `useEffect` hook

The useEffect Cleanup Function

1. We want the title to return to original text if we unmount the movie-details component
    1. We want to execute some code when a component unmounts
        1. Solution: Clean up function

        . . .

        - BROWSER PAINT (after every re-render) |
        - CLEANUP (before the next effect is executed again) |
        - EFFECT |
        - UNMOUNT |
        - CLEANUP | v

The Cleanup Function

1. Useeffect Cleanup Function
    1. Function that we can **return from an effect** (optional)
    2. Runs on two different occasions:
        1. Before the effect is **executed again** (to cleanup the results of the previous side-effect)
        2. After a component has **unmounted** (to reset the side-effect created if it is necessary)
2. Flow:

```
Component renders
    |
    v
Execute effect if
dependency array
includes updated data

Component unmounts
    |
    v
Execute cleanup function
```

3. When do we need a clean-up function?
    1. Necessary whenever the side effect **keeps happening after the component has been re-rendered or unmounted**
        1. Example: HTTP request in an effect
            1. If component is rerendered while the first request is still running, a new second request would be fired-off
                1. Can result in race-condition
                    1. Solution: Cancel request (potential cleanup) whenever the component re-renders or unmounts
        2. Example: API subscription -> Cancel subscription
        3. Example: Start timer -> Stop timer
        4. Example: Add event listener -> Remove listener
4. Each effect should do **only one thing**! Use **one useEffect hook for each side effect.** This makes effects easier to clean up.
    1. If we need to use multiple effects, we need to define multiple `useEffect` hooks
        1. Easier to understand
        2. Easier to define cleanup up function

Clean Up the Title

1. Code:

```
useEffect(
  function () {
    if (!title) return;
    document.title = `Movie | ${title}`;

    return function () {
      // clean-up function - runs after each re-render, and when component unmounts
      document.title = "usePopcorn";
      console.log(`Clean up effect for movie ${title}`); // `title` - remembered by the function because of closure concept
        // closure - a function will always remember the variables that were present at the time and place the function was created
        // The function closed over the `title` variable, and will remember it in the future
    };
  },
  [title]
);
```

Clean Up Data Fetching

1. As we type search keyword, we send a request for each letter
    1. Problems:
        1. Having too many requests will slow them down
        2. We will end up downloading too much data (most of which we may not be using)
        3. If an intermediate request takes longer than the others
            1. It would be the last one to arrive
                1. It would be this result that will be stored in the end in the state and used in the UI
                    1. A race condition
2. Code:

```
useEffect(
  function () {
```

```
    // function contains code that we want to run as a side-effect
    // It is registered to be run at a certain point in time

    const controller = new AbortController(); // Browser API

    async function fetchMovies() {
      try {
        setIsLoading(true);
        setError("");
        const res = await fetch(
          `http://www.omdbapi.com/?apikey=${KEY}&s=${query}`,
          { signal: controller.signal }
        );

        if (!res.ok)
          throw new Error("Something went wrong with fetching movies");

        const data = await res.json();

        if (data.Response === "False") throw new Error("Movie not found");

        setMovies(data.Search);
        console.log(data); // empty array, because setting state is asynchronous
      } catch (err) {
        console.error(err.message);
        if (err.name !== "AbortError") setError(err.message);
      } finally {
        setIsLoading(false);
      }
    }

    if (query.length < 3) {
      setMovies([]);
      setError("");
      return;
    }

    fetchMovies();

    return function () {
      controller.abort();
    };
  },
  [query]
);
```

One More Effect: Listening to a Keypress

1. If we want to do DOM manipulation, we can use `useEffect`
2. Code:

```
useEffect(
  function () {
    function callback(e) {
      if (e.code === "Escape") {
        onCloseMovie();
        console.log("CLOSING");
      }
    }

    document.addEventListener("keydown", callback); // DOM manipulation. Stepping outside React. Hence, React team calls the `useEffect` hook an
    // Too many event listeners can cause memory problem

    return function () {
      document.removeEventListener("keydown", callback);
    };
  },
  [onCloseMovie]
);
```

3. Full Code:

```
import { useEffect, useState } from "react";
import StarRating from "./StarRating";

const tempMovieData = [
  {
    imdbID: "tt1375666",
    Title: "Inception",
    Year: "2010",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BMjAxMzY3NjcxNF5BMl5BanBnXkFtZTcwNTI5OTM0Mw@@._V1_SX300.jpg",
  },
  {
    imdbID: "tt0133093",
    Title: "The Matrix",
    Year: "1999",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BNzQzOTk3OTAtNDQ0Zi00ZTVkLWI0MTEtMDllZjNkYzNjNTc4L2ltYWdlXkEyXkFqcGdeQXVyNjU0OTQ0OTY@._V1_SX300.jp
  },
  {
    imdbID: "tt6751668",
    Title: "Parasite",
    Year: "2019",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BYWZjMjk3ZTItODQ2ZC00NTY5LWE0ZDYtZTI3MjcwN2Q5NTVkXkEyXkFqcGdeQXVyODk4OTc3MTY@._V1_SX300.jpg",
  },
];

const tempWatchedData = [
  {
```

```
    imdbID: "tt1375666",
    Title: "Inception",
    Year: "2010",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BMjAxMzY3NjcxNF5BMl5BanBnXkFtZTcwNTI5OTM0Mw@@._V1_SX300.jpg",
    runtime: 148,
    imdbRating: 8.8,
    userRating: 10,
  },
  {
    imdbID: "tt0088763",
    Title: "Back to the Future",
    Year: "1985",
    Poster:
      "https://m.media-amazon.com/images/M/MV5BZmU0M2Y1OGUtZjIxNi00ZjBkLTg1MjgtOWIyNThiZWIwYjRiXkEyXkFqcGdeQXVyMTQxNzMzNDI@._V1_SX300.jpg",
    runtime: 116,
    imdbRating: 8.5,
    userRating: 9,
  },
];

function Navbar({ children }) {
  return (
    <nav className="nav-bar">
      <Logo />
      {children}
    </nav>
  );
}

function Logo() {
  return (
    <div className="logo">
      <span role="img">🍿</span> {/* No need to break */}
      <h1>usePopcorn</h1>
    </div>
  );
}

function Search({ query, setQuery }) {
  return (
    <input
      className="search"
      type="text"
      placeholder="Search movies..."
      value={query}
      onChange={(e) => setQuery(e.target.value)}
    />
  );
}

function NumResults({ movies }) {
  return (
    <p className="num-results">
      Found <strong>{movies.length}</strong> results
    </p>
  );
}

function Main({ children }) {
  return <main className="main">{children}</main>;
}

function Box({ element }) {
  const [isOpen, setIsOpen] = useState(true);

  return (
    <div className="box">
      <button className="btn-toggle" onClick={() => setIsOpen((open) => !open)}>
        {isOpen ? "–" : "+"}
      </button>
      {isOpen && element}
    </div>
  );
}

function MovieList({ movies, onSelectMovie }) {
  return (
    <ul className="list list-movies">
      {movies?.map((movie) => (
        <Movie movie={movie} key={movie.imdbID} onSelectMovie={onSelectMovie} />
      ))}
    </ul>
  );
}

function Movie({ movie, onSelectMovie }) {
  return (
    <li key={movie.imdbID} onClick={() => onSelectMovie(movie.imdbID)}>
      <img src={movie.Poster} alt={`${movie.Title} poster`} />
      <h3>{movie.Title}</h3>
      <div>
        <p>
          <span>🗓</span>
          <span>{movie.Year}</span>
        </p>
      </div>
    </li>
  );
}

// function WatchedBox() {
//   const [watched, setWatched] = useState(tempWatchedData);
//   const [isOpen2, setIsOpen2] = useState(true);
```

```
//   return (
//     <div className="box">
//       <button
//         className="btn-toggle"
//         onClick={() => setIsOpen2((open) => !open)}
//       >
//         {isOpen2 ? "–" : "+"}
//       </button>
//       {isOpen2 && (
//         <>
//           <WatchedSummary watched={watched} />

//           <WatchedMovieList watched={watched} />
//         </>
//       )}
//     </div>
//   );
// }

function MovieDetails({ selectedId, onCloseMovie, onAddWatched, watched }) {
  const [movie, setMovie] = useState({});
  const [isLoading, setIsLoading] = useState(false);
  const [userRating, setUserRating] = useState("");

  const isWatched = watched.map((movie) => movie.imdbID).includes(selectedId);
  const watchedUserRating = watched.find(
    (movie) => movie.imdbID === selectedId
  )?.userRating;

  const {
    Title: title,
    Year: year,
    Poster: poster,
    Runtime: runtime,
    imdbRating,
    Plot: plot,
    Released: released,
    Actors: actors,
    Director: director,
    Genre: genre,
  } = movie;

  console.log(title, year);
  function handleAdd() {
    const newWatchedMovie = {
      imdbID: selectedId,
      title,
      year,
      poster,
      imdbRating: Number(imdbRating),
      runtime: Number(runtime.split(" ").at(0)),
    };

    onAddWatched(newWatchedMovie);
    onCloseMovie();
  }

  useEffect(
    function () {
      function callback(e) {
        if (e.code === "Escape") {
          onCloseMovie();
        }
      }

      document.addEventListener("keydown", callback); // DOM manipulation. Stepping outside React. Hence, React team calls the `useEffect` hook
      // Too many event listeners can cause memory problem

      return function () {
        document.removeEventListener("keydown", callback);
      };
    },
    [onCloseMovie]
  );

  useEffect(
    function () {
      async function getMovieDetails() {
        setIsLoading(true);
        const res = await fetch(
          `http://www.omdbapi.com/?apikey=${KEY}&i=${selectedId}`
        ); // connects controller to `fetch`
        const data = await res.json();
        console.log(data);
        setMovie(data);
        setIsLoading(false);
      }
      getMovieDetails();
    },
    [selectedId]
  );

  useEffect(
    function () {
      if (!title) return;
      document.title = `Movie | ${title}`;

      return function () {
        // clean-up function - runs after each re-render, and when component unmounts
        document.title = "usePopcorn";
        console.log(`Clean up effect for movie ${title}`); // `title` - remembered by the function because of closure concept
        // closure - a function will always remember the variables that were present at the time and place the function was created
        // The function closed over the `title` variable, and will remember it in the future
```

```
      };
    },
    [title]
  );

  return (
    <div className="details">
      {isLoading ? (
        <Loader />
      ) : (
        <>
          <header>
            <button className="btn-back" onClick={onCloseMovie}>
              &larr;
            </button>
            <img src={poster} alt={`Poster of ${movie} movie`} />
            <div className="details-overview">
              <h2>{genre}</h2>
              <p>
                {released} &bull; {runtime}
              </p>
              <p>
                <span></span>
                {imdbRating} IMDb rating
              </p>
            </div>
          </header>

          <section>
            <div className="rating">
              {!isWatched ? (
                <>
                  <StarRating
                    maxRating={10}
                    size={24}
                    onSetRating={setUserRating}
                  />
                  {userRating > 0 && (
                    <button className="btn-add" onClick={handleAdd}>
                      + Add to list
                    </button>
                  )}
                </>
              ) : (
                <p>
                  You rated the movie {watchedUserRating} <span>⭐</span>
                </p>
              )}
            </div>
            <p>
              <em>{plot}</em>
            </p>
            <p>Staring {actors}</p>
            <p>directed by {director}</p>
          </section>
        </>
      )}
    </div>
  );
}

function WatchedSummary({ watched }) {
  const avgImdbRating = average(watched.map((movie) => movie.imdbRating));
  const avgUserRating = average(watched.map((movie) => movie.userRating));
  const avgRuntime = average(watched.map((movie) => movie.runtime));

  return (
    <div className="summary">
      <h2>Movies you watched</h2>
      <div>
        <p>
          <span>#️⃣</span>
          <span>{watched.length} movies</span>
        </p>
        <p>
          <span>⭐</span>
          <span>{avgImdbRating.toFixed(2)}</span>
        </p>
        <p>
          <span>🌟</span>
          <span>{avgUserRating.toFixed(2)}</span>
        </p>
        <p>
          <span>⏳</span>
          <span>{avgRuntime} min</span>
        </p>
      </div>
    </div>
  );
}

function WatchedMovieList({ watched, onDeleteWatched }) {
  return (
    <ul className="list">
      {watched.map((movie) => (
        <WatchedMovie
          movie={movie}
          key={movie.imdbID}
          onDeleteWatched={onDeleteWatched}
        />
      ))}
    </ul>
  );
}
```

```
function WatchedMovie({ movie, onDeleteWatched }) {
  return (
    <li key={movie.imdbID}>
      <img src={movie.poster} alt={`${movie.title} poster`} />
      <h3>{movie.Title}</h3>
      <div>
        <p>
          <span>⭐</span>
          <span>{movie.imdbRating}</span>
        </p>
        <p>
          <span>🌟</span>
          <span>{movie.userRating}</span>
        </p>
        <p>
          <span>⌛</span>
          <span>{movie.runtime} min</span>
        </p>

        <button
          className="btn-delete"
          onClick={() => onDeleteWatched(movie.imdbID)}
        >
          X
        </button>
      </div>
    </li>
  );
}

const KEY = "4256c7"; // If it is inside the component, it will get re-created each time the component is re-rendered

const average = (arr) =>
  arr.reduce((acc, cur, i, arr) => acc + cur / arr.length, 0);

export default function App() {
  const [query, setQuery] = useState("");
  const [movies, setMovies] = useState(tempMovieData);
  const [watched, setWatched] = useState(tempWatchedData);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState("");
  const [selectedId, setSelectedId] = useState(null);

  // useEffect(function () {
  //   console.log("After initial render"); // Appears second - because it appears before the other effects.
  // }, []);

  // useEffect(function () {
  //   console.log("After every render");
  // }); // Appears last
  // // If `query` is updated, this effect is executed because it runs after every render.

  // useEffect(
  //   function () {
  //     console.log("Synchronized with `query` state variable");
  //   },
  //   [query]
  // );

  function handleSelectMovie(id) {
    setSelectedId((selectedId) => (selectedId === id ? null : id));
  }

  function handleCloseMovie() {
    setSelectedId(null);
  }

  function handleAddWatched(movie) {
    setWatched((watched) => [...watched, movie]);
  }

  function handleDeleteWatched(id) {
    setWatched((watched) => watched.filter((movie) => movie.imdbID !== id));
  }

  console.log("During render"); // Appears first - effects run after browser paint. This statement is executed during render
  // If `query` is updated, this line executes during render

  useEffect(
    // Can be converted to an event-listener in this case
    function () {
      // function contains code that we want to run as a side-effect
      // It is registered to be run at a certain point in time

      const controller = new AbortController(); // Browser API

      async function fetchMovies() {
        try {
          setIsLoading(true);
          setError("");
          const res = await fetch(
            `http://www.omdbapi.com/?apikey=${KEY}&s=${query}`,
            { signal: controller.signal }
          );

          if (!res.ok)
            throw new Error("Something went wrong with fetching movies");

          const data = await res.json();

          if (data.Response === "False") throw new Error("Movie not found");

          setMovies(data.Search);
```

```
          console.log(data); // empty array, because setting state is asynchronous
        } catch (err) {
          console.log(err.message);
          if (err.name !== "AbortError") setError(err.message);
        } finally {
          setIsLoading(false);
        }
      }

      if (query.length < 3) {
        setMovies([]);
        setError("");
        return;
      }

      handleCloseMovie();
      fetchMovies();

      return function () {
        controller.abort();
      };
    },
    [query]
  ); // It doesn't return anything
  // Second argument is dependency array
  // `[]` - means that the hook will run only on mount (inital render)
  // Good if we want to fetch data as soon as the application loads

  // setWatched([]); // Causes too many re-renders

  return (
    <>
      <Navbar>
        <Search query={query} setQuery={setQuery} />
        <NumResults movies={movies} />
      </Navbar>
      <Main>
        <Box
          element={
            (isLoading && <Loader />) ||
            (!isLoading && !error && (
              <MovieList movies={movies} onSelectMovie={handleSelectMovie} />
            )) ||
            (error && <ErrorMessage message={error} />)
          }
        />
        <Box
          element={
            selectedId ? (
              <MovieDetails
                selectedId={selectedId}
                onCloseMovie={handleCloseMovie}
                onAddWatched={handleAddWatched}
                watched={watched}
              />
            ) : (
              <>
                <WatchedSummary watched={watched} />
                <WatchedMovieList
                  watched={watched}
                  onDeleteWatched={handleDeleteWatched}
                />
              </>
            )
          }
        />
        {/* <Box>
          <MovieList movies={movies} />
        </Box>
        <Box>
          <WatchedSummary watched={watched} />
          <WatchedMovieList watched={watched} />
        </Box> */}
      </Main>
    </>
  );
}

function Loader() {
  return <p className="loader">Loading...</p>;
}

function ErrorMessage({ message }) {
  return (
    <p className="error">
      <span>⛔ {message}</span>
    </p>
  );
}
```

CHALLENGE #1: Currency Converter

1. Currency conversion using an API that contains real-time currency conversion data
2.

Section 13: Custom Hooks, Refs, and More State

Section Overview

1. Topics:

1. Hooks are easy to **learn**, hard to **master**
    1. A solid understanding of hooks is crucial to all React developers
2. **Rules** of hooks
    1. And why they exist
3. **Deep** dive into `useState`
4. `useRef`
5. **Custom** hooks
    1. Own hooks

React Hooks and Their Rules

What are React Hooks?

1. React Hooks
    1. Special built-in functions that allow us to **hook into React internals**
        1. They are APIs that expose internal React functionalities such as
            1. Contructing and accessing **state** from Fiber tree
            2. Registering **side effects** in Fiber tree
                1. Fiber tree is deep inside React and is usually not accessible
                    1. Using `useState` or `useEffect` hook, we can hook into the internal mechanism
            3. Manual **DOM selections**
            4. Access context
            5. ...
    2. Always start with "use" (`useState`, `useEffect`, etc.)
        1. To distinguish from other functions
    3. Enable easy **reusing of non-visual logic**: we can compose multiple hooks into our own **custom hooks**
        1. reusing of non-visual logic: logic that is not UI
            1. Class-based components had some problems, hence we have function components with hooks
    4. Give **function components** the ability to own state and run side effects at different lifecycle points (before v16.8 only avaialable in class **components**)

Overview of All Built-In Hooks

1. Most used:
    1. `useState`
    2. `useEffect`
    3. `useReducer`
    4. `useContext`
2. Less used:
    1. `useRef`
    2. `useCallback`
    3. `useMemo`
    4. `useTransition`
    5. `useDeferredValue`
    6. `useLayoutEffect`
    7. `useDebugValue`
    8. `useImperativeHandle`
    9. `useId`
3. Only for libraries (for library authors)
    1. `useSyncExternalStore`
    2. `useInsertionEffect`

The Rules of Hooks

1. Rules:
    1. Only call hooks at the top level
        1. Do **NOT** call hooks inside **conditionals, loops, nested functions,** or after an **early return** (similar to a condition)
            1. Why?
                1. This is necessary to ensure that hooks are always called in the **same order** (hooks rely on this)
    2. Only call hooks from React functions
        1. Only call hooks inside a **function components** or a **custom hook**
            1. Not from regular functions
            2. Not from class components
2. The rules are automatically enforced by React's ESLint rules

Hooks Rely on Call Order

1. Whenever an application is rendered, React constructs a tree of React elements (vDOM)
    1. On initial render, React constructs a fiber tree out of vDOM, where each element is a fiber
        1. Each fiber contains
            1. Received props
            2. List of work
            3. **Linked list of all the hooks that were used in the component instance**
                1. Example: Hypothetical example:

```
const [A, setA] = useState(23)

if (A === 23)
    const [B, setB] = useState('');

useEffect(fnZ, []);
```

1. List built based on hooks call order
    1. `useState(23)` (has reference to second hook)
    2. `useState('')` (has reference to third hook)
    3. `useEffect(...)`
2. Suppose the component is re-rendered because the state changed from 23 to 7
    1. Problem:
        1. The condition is false
            1. `useState('')` does not exist after the render
            2. The first hook is still pointing to the `useState('')` hook that no longer exists
            3. Nothing is pointing to the third hook (`useEffect(...)`)
        2. A === 23 is now false, so after re-render, this hook would no longer exist, **destroying the linked list**
            1. It works this way because fibers are not re-created on every render
                1. Hence, the list is also not re-created
                    1. If a hook disappears from the list, then the list gets completely broken
                    2. Condition will completely mess up the list of hooks (React will be confused will not be able to track the list of hooks)
                    3. Hence, hooks will be called in the same order after every render (hence the first rule)

2. Why have linked-list in the first place?

    1. Linked list relies on a hook called order is the simplest way to associate a hook with its value
        1. The order in which the hook is called uniquely identifies the hook
            1. Hook 1 has a state of 23, and hook 2 has a state of '', ...
                1. Values are associated with call order
                    1. Developers don't have to manually assign names to each hook (there are multiple problems)

The Rules of Hooks in Practice

1. In Dev Tools:

    1. `MovieDetails`
        1. hooks
            1. Hooks are numbered (React identifies by order number (not by name))

2. Code:

```
/* eslint-disable */
if (imdbRating > 8) [isTop, setIsTop] = useState(true);
```

    1. If the condition is met, React stops working and throws an error

    2. Console:

```
Warning: React has detected a change in the order of Hooks called by MovieDetails. This will lead to bugs and errors if not fixed. For more

   Previous render            Next render
   ------------------------------------------------------
1. useState                   useState
2. useState                   useState
3. useState                   useState
4. useEffect                  useState
   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

    3. Another example:

```
  if (imdbRating > 8) return <p>Greatest ever!</p>; // Same problem occurs. Fewer hooks were rendered (rest of the hooks did not get render
```

    4. The initial values we pass to `useState` only matter on the initial render

        1. Example:

```
const [isTop, setIsTop] = useState(imdbRating > 8);
console.log(isTop); // always `false`
```

            1. What we passed is only initial state, and React will look at the initial state only on the initial render. (when the component first mounts)

                1. `imdbRating` is `undefined` on initial render
                2. The initialization is not executed on re-render

            2. How to overcome this issue?

                1. `useEffect`

```
useEffect(function () {
    setIsTop(imdbRating > 8);
}, [imdbRating])
```

                    1. Alternative:

```
const isTop = imdbRating > 8; // derived state. It gets updated, each time the component re-renders
console.log(isTop);
```

3. State updates are asynchronous

```
setAvgRating(Number(imdbRating));
// alert(avgRating); // We do not get access to the updated state soon after we update the state. We will get access to the updated state after

setAvgRating((avgRating + userRating) / 2); // `avgRating` is still 0. It is asynchronous state setting
```

More Details of useState

1. Persisting watchlist in local storage
    1. Local storage:

1. A simple key/value storage available in the browser, where we can store some data for each domain (per URL)
2. Where to check:
    1. Dev tools
        1. Application tab
            1. Local Storage

Initializing State With a Callback (Lazy Initial State)

1. Code:

```
const [watched, setWatched] = useState(function () {
  const storedValue = localStorage.getItem("watched");
  return JSON.parse(storedValue);
}); // React will call the function when the component mounts and uses the value returned by the function as the initial value of the state. The

...

function handleAddWatched(movie) {
  setWatched((watched) => [...watched, movie]);

  // localStorage.setItem("watched", JSON.stringify([...watched, movie]));
}

function handleDeleteWatched(id) {
  setWatched((watched) => watched.filter((movie) => movie.imdbID !== id));
}

useEffect(
  function () {
    localStorage.setItem("watched", JSON.stringify(watched));
  },
  [watched]
); // When we add or delete a movie in the watched list, the local storage is synchronized.
```

useState Summary

Summary of Defining and Updating State

1. Constructing State
    1. Simple way

       ```
       const [count, setCount] = useState(23);
       ```

    2. Based on a callback function (lazy evaluation)

       ```
       const [count, setCount] = useState(
           () => localStorage.getItem('count');
       );
       ```

        1. Whenever the initial state requires some computation, for instance to retrieve data from local storage, we can pass in a callback function instead of a single value
            1. Lazy evaluation
            2. The function must be pure and accept no arguments. Called only on initial render
                1. For sebsequent renders, the function will be ignored.
2. Updating State
    1. Simple way
        1. By passing a single value into the setter function as the next state

           ```
           setCount(1000);
           ```

    2. Based on current state

       ```
       setCount((c) => c + 1);
       ```

        1. Function must be pure and return next state (based on the current state)
            1. Preferred way whenever it makes sense
    3. Make sure to **NOT** mutate objects or arrays, but to **replace** them
        1. A new array or object must be created which incorporates the changes we want to make
            1. Pass the new object into the setter function

How NOT to Select DOM Elements in React

1. Why do we need refs in React
2. Code example:

```
function Search({ query, setQuery }) {
  useEffect(function () {
    const el = document.querySelector(".search");
    console.log(el);
    el.focus();
  }, []);
...
```

    1. React is about declarative.
        1. Manually selecting a DOM element is not React way of doing things
            1. Not inline with the rest of the React code

1. We must not manually add classes, event-listeners, etc... directly to DOM elements, and not add classes to JSX elements just for selecting them.
2. If we have a dependency, we will select the element over and over again

2. Solution: refs

Introducing Another Hook: useRef

1. `useRef` - easiest hook of all

What are Refs?

1. Ref with `useRef`
    1. "Box" (object) with a **mutable** `.current` property that is **persisted across renders** ("normal" variables are always reset)
        1. We can write to and read from the ref using `.current`

           ```
           const myRef = useRef(23);

           myRef.current = 1000; // mutable
           ```

        2. Two big use cases:
            1. Creation of variable that stays the same between renders (e.g. previous state, `setTimeout` id, etc)
            2. Selecting and storing DOM elements
                1. We want to store DOM elements and preserve across renders
2. Refs are for **data that is NOT rendered:** usually only appear in event handlers or effects, not in JSX (otherwise use state)
    1. For visual appearance, use state instead
    2. Do **NOT** read write or read `.current` in render logic (like state)
        1. Can cause undesirable side-effects
        2. The mutations are usually done in a `useEffect` hook

State vs. Refs

1. Refs are like state but with less powers
    1. Both refs and state are persisted across renders
        1. Component remembers values even after re-renders
        2. Updating state causes re-render of component
            1. State is used only if we want a component to re-render on update
        3. Updating refs does not cause re-render of component
            1. Refs are used only if we want the component to remember data over time (never re-render)
                1. Examples: preserving previous state, or storing the id of `setTimeout`
    2. State is immutable but refs are not
    3. State is updated asynchronously but refs are not updated asynchronously

Refs to Select DOM Elements

1. Three steps:
    1. Step 1:

       ```
       const inputEl = useRef(null);
       ```

    2. Step 2:

       ```
       <input
           ...
           ref={inputEl}
       />
       ```

        1. Connects the element to the ref
            1. `inputEl` contains the `input` element

    3. Step 3:

       ```
       useEffect(function () {
           inputEl.current.focus(); // `inputEl.current` contains the DOM element
       }, []);
       ```

        1. Ref gets added to the DOM element after the DOM has already loaded. It can be accessed only in `useEffect` which runs after the DOM is loaded.
2. Code:

```
useEffect(
  function () {
    function callback(e) {
      if (document.activeElement === inputEl.current) return;

      if (e.code === "Enter") {
        inputEl.current.focus();
        setQuery("");
      }
    }

    document.addEventListener("keydown", callback);
    return () => document.addEventListener("keydown", callback);
  },
  [setQuery]
);
```

Refs to Persist Data Between Renders

1. A variable that is persisted across renders without triggering a re-render
2. Code:

```
function MovieDetails({ selectedId, onCloseMovie, onAddWatched, watched }) {
// ...

  const countRef = useRef(0);

  useEffect(
    function () {
      if (userRating) countRef.current++;
    },
    [userRating]
  ); // Because we cannot mutate state in render logic

// ...

  function handleAdd() {
    const newWatchedMovie = {
      // ...
      countRatingDecisions: countRef.current,
    };
```

What are Custom Hooks? When to Construct One?

Reusing Logic with Custom Hooks

1. It is about re-usability
    1. We can re-use two kinds of things in React
        1. Piece of UI
            1. Using a component
        2. Piece of logic
            1. Questions to ask:
                1. Does logic contain any **hooks**?
                    1. No
                        1. Regular function
                            1. Inside or outside a component
                    2. Yes
                        1. We cannot extract logic that contains a hook into a regular function
                            1. Solution: Custom hook
                                1. Allows us to reuse **non-visual logic** in multiple components (stateful logic among multiple components, or any logic)
                                2. One custom hook shoud have **one purpose**, to make it **reusable**, and **portable** (even across multiple projects)
                                    1. Developers have also started to share their custom hooks
                                        1. We can download from `npm` and use it
                                3. The rules of hooks applies to custom hooks as well.
2. Example of a custom hook:

```
function useFetch (url) {
    const [data, setData] = useState([]);
    const [isLoading, setIsLoading] = useState(false);

    useEffect(function () {
        fetch(url)
            .then((res) => res.json())
            .then((res) => setData(res));
    }, []);

    return [data, isLoading]
}
```

    1. Unline components, it can receive and return **any relevant data** (usually array [] or object {})
        1. Components can only receive props and must return JSX
        2. A custom hook uses one or more react hooks
            1. `useState`, `useEffect`
    2. Function name must start with `use`

Constructing our First Custom Hook: useMovies

1. Convention:
    1. Use default exports for components
    2. Use named exports for custom hooks
2. `useMovies.js`

```
import { useEffect, useState } from "react";

const KEY = "4256c7"; // If it is inside the component, it will get re-created each time the component is re-rendered

export function useMovies(query, callback) {
  const [movies, setMovies] = useState([]);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState("");

  useEffect(
    // Can be converted to an event-listener in this case
    function () {
```

```
      callback?.(); // optional chaining
      // function contains code that we want to run as a side-effect
      // It is registered to be run at a certain point in time

      const controller = new AbortController(); // Browser API

      async function fetchMovies() {
        try {
          setIsLoading(true);
          setError("");
          const res = await fetch(
            `http://www.omdbapi.com/?apikey=${KEY}&s=${query}`,
            { signal: controller.signal }
          );

          if (!res.ok)
            throw new Error("Something went wrong with fetching movies");

          const data = await res.json();

          if (data.Response === "False") throw new Error("Movie not found");

          setMovies(data.Search);
          console.log(data); // empty array, because setting state is asynchronous
        } catch (err) {
          console.log(err.message);
          if (err.name !== "AbortError") setError(err.message);
        } finally {
          setIsLoading(false);
        }
      }

      if (query.length < 3) {
        setMovies([]);
        setError("");
        return;
      }

      // handleCloseMovie();
      fetchMovies();

      return function () {
        controller.abort();
      };
    },
    [query]
  ); // It doesn't return anything
  // Second argument is dependency array
  // `[]` - means that the hook will run only on mount (inital render)
  // Good if we want to fetch data as soon as the application loads

  return { movies, error, isLoading };
}
```

3. App.js

```
const { movies, error, isLoading } = useMovies(query, handleCloseMovie);
```

Constructing useLocalStorageState

1. Code:

```
import { useState, useEffect } from "react";

export function useLocalStorageState(initialState, key) {
  const [value, setValue] = useState(function () {
    const storedValue = localStorage.getItem(key);
    return storedValue ? JSON.parse(storedValue) : initialState;
  });

  useEffect(
    function () {
      localStorage.setItem(key, JSON.stringify(value));
    },
    [value, key]
  ); // When we add or delete a movie in the watched list, the local storage is synchronized.

  return [value, setValue];
}
```

    1. Custom hook setter function updates both state and local storage

Construct useKey

1. Code:

```
import { useEffect } from "react";

export function useKey(key, action) {
  useEffect(
    function () {
      function callback(e) {
        if (e.code.toLowerCase() === key.toLowerCase()) {
          action();
        }
      }

      document.addEventListener("keydown", callback); // DOM manipulation. Stepping outside React. Hence, React team calls the `useEffect` hook
      // Too many event listeners can cause memory problem

      return function () {
```

```
      document.removeEventListener("keydown", callback);
    };
  },
  [key, action]
);
}
```

1. App.js

```
useKey("Escape", onCloseMovie);

// ...

useKey("Enter", function () {
  if (document.activeElement === inputEl.current) return;

  inputEl.current.focus();
  setQuery("");
});
```

CHALLENGE #1: useGeolocate

1. MUST DO!

Section 14: [Optional] React Before Hooks: Class-Based React

Section Overview

Our First Class Component

Working With Event Handlers

Class Components vs. Function Components

Starting the "Classy Weather" App

Fetching Weather Data

Displaying the Weather

Removing Boilerplate Code With Class Fields

Child to Parent Communication

Lifecycle Methods

Section: 15: PART 3: ADVANCED REACT + REDUX [4 PROJECTS]

Introduction to Part 3

1. Advanced state management with
    1. `useReducer` hook
    2. Context API
    3. Redux
2. Projects
    1. Using veet
    2. Using React router library
3. Deep into React cosmos for performance optimization
    1. `useMemo` hook
    2. `useCallback` hook

Useful Resources for Part 3

1. High-quality resources
    1. Tao of React - Software Design, Architecture & Best Practices
    2. The new wave of React state management - Excellent read!
    3. A Visual Guide to React Rendering - useMemo
    4. React as a UI Runtime - By Dan Abramov from the React team
    5. You Might Not Need an Effect (Official React docs)
    6. A Complete Guide to useEffect (By Dan Abramov)
    7. useEffect sometimes fires before paint
    8. Making setInterval Declarative with React Hooks (By Dan Abramov)
    9. Redux - Not Dead Yet! (By Mark Erikson from the Redux team)
    10. Why React Context is Not a "State Management" Tool (By Mark Erikson)
2. Library documentation:
    1. Vite (Why Vite is so fast)
    2. CSS Modules
    3. React Router
    4. React Leaflet: Installation
    5. Redux: Style Guide (A must-read for Redux users!)
    6. Redux Toolkit
    7. React Redux

Section 16: The Advanced useReducer Hook

Section Overview

1. Topics:
    1. Another **powerful** way of managing state
    2. Mastering `useReducer` is important to understand **Redux**
    3. Half the section in essentially another **practice project**
        1. `useReducer` re-inforces all that we learn't so far

Yet Another Hook: useReducer

1. High-level overview
    1. Another project:
        1. `npx create-react-app@5 react-quiz`
2. `useReducer` hook:
    1. It is a more complex and more advanced way of managing state than the `useState` hook
    2. It works with a reducer function
        1. Reducer function: It is a pure function that takes previous state, and an action as argument, and returns the next state
3. Code:

```
function reducer(state, action) {
  console.log(state, action);
  // return state + action;
  if (action.type === "inc") return state + 1;
  if (action.type === "dec") return state - 1;
  if (action.type === "setCount") return action.payload;
  // What the reducer returns will become the next state
}

...
function DateCounter() {
  // const [count, setCount] = useState(0);

  const [count, dispatch] = useReducer(reducer, 0); // returns the current state, and dispatch function (which is used to update the state)

  // ...

  const dec = function () {
    // dispatch(-1);
    dispatch({ type: "dec" }); // standard convention to have `type`, and `payload` properties. Similar in Redux
    // setCount((count) => count - 1);
    // setCount((count) => count - step);
  };

  const inc = function () {
    // dispatch(1); // 1 - action in the reducer. This is called dispatching an action
    dispatch({ type: "inc" });
    // setCount((count) => count + 1);
    // setCount((count) => count + step);
  };

  const defineCount = function (e) {
    // setCount(Number(e.target.value));
  };

  const defineStep = function (e) {
    // setStep(Number(e.target.value));
    dispatch({ type: "setCount", payload: Number(e.target.value) });
  };
```

Managing Related Pieces of State

1. We use reducer if we have complex state to manage (not just one single value)
    1. State is usually an object (not just a single value)
2. Advantage of reducer function:
    1. We can have all the possible state updates that can happen in the application in one central place
        1. It makes it easy to understand the application without going into every component, and every function.
3. Code:

```
import { useReducer } from "react";

const initialState = { count: 0, step: 1 };

// The real logic of updating the state is centralized in one location in the reducer
function reducer(state, action) {
  console.log(state, action);

  switch (action.type) {
    case "dec":
      return { ...state, count: state.count - state.step };
    case "inc":
      return { ...state, count: state.count + state.step };
    case "setCount":
      return { ...state, count: action.payload };
    case "setStep":
      return { ...state, step: action.payload };
    case "reset":
      return initialState; // We can update multiple pieces of state at the same time
    default:
      throw new Error("Unknown action");
```

```
  }
}

function DateCounter() {
  const initialState = { count: 0, step: 1 };
  const [state, dispatch] = useReducer(reducer, initialState); // returns the current state, and dispatch function (which is used to update the
  const { count, step } = state;

  // This mutates the date object.
  const date = new Date("june 21 2027");
  date.setDate(date.getDate() + count);

  const dec = function () {
    dispatch({ type: "dec" }); // standard convention to have `type`, and `payload` properties. Similar in Redux
  };

  const inc = function () {
    dispatch({ type: "inc" });
  };

  const defineCount = function (e) {
    dispatch({ type: "setCount", payload: Number(e.target.value) });
  };

  const defineStep = function (e) {
    dispatch({ type: "setStep", payload: Number(e.target.value) });
  };

  const reset = function () {
    dispatch({ type: "reset" });
  };

  return (
    <div className="counter">
      <div>
        <input
          type="range"
          min="0"
          max="10"
          value={step}
          onChange={defineStep}
        />
        <span>{step}</span>
      </div>

      <div>
        <button onClick={dec}>-</button>
        <input value={count} onChange={defineCount} />
        <button onClick={inc}>+</button>
      </div>

      <p>{date.toDateString()}</p>

      <div>
        <button onClick={reset}>Reset</button>
      </div>
    </div>
  );
}
export default DateCounter;
```

Managing State With useReducer

Why useReducer?

1. State management with `useState` is not enough in certain situations:
    1. As components, and state become more complex, using `useState` is not enough
        1. Example: When components have a **lot of state variables and state updates,** spread across many event handlers **all over the component** (maybe even multiple components)
            1. Can be overwhelming and hard to manage
    2. When **multiple state updates** need to happen **at the same time** (as a reaction to the same event, like "starting a game" (set score to 0, set isPlaying status, start a timer at the same time)
    3. When updating one piece of state that **depends on one or multiple other peices of state**
        1. Becomes challenging when there is a lot of state
2. Solution: In all the above situations, `useReducer` can be of great help

Managing State with useReducer

1. State with useReducer
    1. An alternative way of setting state, ideal for **complex state** and **related pieces of state**
        1. Example:

            ```
            const [state, dispatch] = useReducer(reducer, initialState);
            ```

    2. Stores related pieces of state in a **state** object (that is returned from the `useReducer` hook
    3. `useReducer` needs a **reducer**: function containing **all logic to update state. Decouples state logic from component** (makes components cleaner and readable) (It is like `setState()` with superpowers)
    4. **reducer:** pure function (**no side effects!**) that takes current `state` and `action`, **and returns the next state**
        1. Reducer is not allowed to mutate the state
    5. **action:** object that describes **how to update state**

        1. It contains an action type, and a payload (input data)
            1. Used by reducer to determine how to define the next state

    6. **dispatch:** function to trigger state updates, by **"sending" actions** from **event handlers** to the **reducer**

How Reducers Update State

1. Let us say we are in an event-handler of a component

```
Updating state in a component
    |
    v
dispatch -action(type = 'updateDay', payload = 23)-> reducer
                                                       |
                                                    Returns
                                                       |
                                                       v
                                                  Next state
                                                       |
                                                    triggers
                                                       |
                                                       v
                                                  Re-render
```

    1. Action: Object taht contains information on how the reducer should update the state
        1. Reducer will set the day's value to 23
            1. The shape of action doesn't have to be the one mentioned
    2. Why is it called a reducer?
        1. Just like `array.reduce()` accumulates all array values into a single value, React reducer accumulates ("reduce") **actions over time** into a single state
    3. In the background, the `dispatch` function has access to the reducer, because we passed it to the `useReducer` hook
        1. Dispatcher coordinates the whole thing and gives access to the reducer, the current state

2. `useReducer` vs `useState`

    1. `useState` - we get back a setter function

        1. To update state, we pass updated state value. React will update the state

            1. This triggers a re-render

```
setState
    |
    update
    |
    v
Next (updated) state
    |
    v
Re-render
```

3. Analogy

    1. Real-world task: Withdrawing $5000 form your bank account

        1. I go to a bank and approach a teller
        2. We tell the person, the amount, and from which account
        3. Teller will check the account
        4. Teller goes to bank's vault and gets money
        5. Teller hands it over to me

    2. `useReducer`:

        1. State: vault (**what** needs to be updated)

            1. Where money is stored and updated

        2. Reducer: Teller

            1. **Who makes** the update
            2. Teller knows much more.
                1. How to request loan
                2. How to deposit money
                3. ...

        3. Dispatcher: Customer

            1. **Who requests** the update
                1. Doing so by going to a person and withdrawing the money

        4. Action: The amount, and account info

            1. **How** to make the update

```
{
    type: 'withdraw',
    payload: {
        amount: 5000,
        account: 453423,
    },
}
```

                1. Reducer decouples and abstracts state update logic from the client
                    1. Easy to understand components

The "React Quiz" App

Loading Questions from a Fake API

1. Code:

```
import { useEffect, useReducer } from "react";
import Header from "./Header";
import Main from "./Main";

const initialState = {
  questions: [],

  // 'loading', 'error', 'ready', 'active', 'finished'
  status: "loading",
};

function reducer(state, action) {
  switch (action.type) {
    case "dataReceived":
      return {
        ...state,
        questions: action.payload,
        status: "ready",
      };
    case "dataFailed":
      return {
        ...state,
        status: "error",
      };
    default:
      throw new Error("Action is unknown");
  }
}

export default function App() {
  const [state, dispatch] = useReducer(reducer, initialState);

  useEffect(function () {
    fetch("http://localhost:8000/questions")
      .then((res) => res.json())
      .then((data) => dispatch({ type: "dataReceived", payload: data }))
      .catch((err) => dispatch({ type: "dataFailed" }));
  }, []);

  // `dataReceived` can be considered as an event

  return (
    <div className="app">
      <Header />
      <Main>
        <p>1/15</p>
        <p>Question?</p>
      </Main>
    </div>
  );
}
```

Handling Loading, Error, and Ready Status

1. StartScreen.js

```
function StartScreen({ numQuestions }) {
  return (
    <div className="start">
      <h2>Welcome to the React Quiz!</h2>
      <h3>{numQuestions} questions to test your React mastery</h3>
      <button className="btn btn-ui">Let's start</button>
    </div>
  );
}

export default StartScreen;
```

2. App.js

```
export default function App() {
  const [{ questions, status }, dispatch] = useReducer(reducer, initialState);

  const numQuestions = questions.length;

// ...

  return (
    <div className="app">
      <Header />
      <Main>
        {status === "loading" && <Loader />}
        {status === "error" && <Error />}
        {status === "ready" && <StartScreen numQuestions={numQuestions} />}
      </Main>
    </div>
  );
}
```

Starting a New Quiz

1. We need to pass `dispatch` functions around (similar to passing an event handler)
    1. We are using a state reducer, so we can just pass `dispatch` function

2. StartScreen.js:

```
function StartScreen({ numQuestions, dispatch }) {
  return (
    <div className="start">
      <h2>Welcome to the React Quiz!</h2>
      <h3>{numQuestions} questions to test your React mastery</h3>
      <button
        className="btn btn-ui"
        onClick={() => dispatch({ type: "start" })}
      >
        Let's start
      </button>
    </div>
  );
}

export default StartScreen;
```

3. App.js

```
import { useEffect, useReducer } from "react";
import Header from "./Header";
import Main from "./Main";
import Loader from "./Loader";
import Error from "./Error";
import StartScreen from "./StartScreen";
import Question from "./Question";

const initialState = {
  questions: [],

  // 'loading', 'error', 'ready', 'active', 'finished'
  status: "loading",
};

function reducer(state, action) {
  switch (action.type) {
    case "dataReceived":
      return {
        ...state,
        questions: action.payload,
        status: "ready",
      };
    case "dataFailed":
      return {
        ...state,
        status: "error",
      };
    case "start":
      return {
        ...state,
        status: "active",
      };
    default:
      throw new Error("Action is unknown");
  }
}

export default function App() {
  const [{ questions, status }, dispatch] = useReducer(reducer, initialState);

  const numQuestions = questions.length;

  useEffect(function () {
    fetch("http://localhost:8000/questions")
      .then((res) => res.json())
      .then((data) => dispatch({ type: "dataReceived", payload: data }))
      .catch((err) => dispatch({ type: "dataFailed" }));
  }, []);

  // `dataReceived` can be considered as an event

  return (
    <div className="app">
      <Header />
      <Main>
        {status === "loading" && <Loader />}
        {status === "error" && <Error />}
        {status === "ready" && (
          <StartScreen numQuestions={numQuestions} dispatch={dispatch} />
        )}
        {status === "active" && <Question />}
      </Main>
    </div>
  );
}
```

Displaying Questions

1. Question.js

```
import Options from "./Options";

function Question({ question }) {
  return (
    <div>
      <h4>{question.question}</h4>
      <Options question={question} />
    </div>
  );
}
```

```
export default Question;
```

2. Options.js

```
function Options({ question }) {
  return (
    <div className="options">
      {question.options.map((option) => (
        <button className="btn btn-option" key={option}>
          {option}
        </button>
      ))}
    </div>
  );
}

export default Options;
```

3. StartScreen.js

```
// ...
<button
  className="btn btn-ui"
  onClick={() => dispatch({ type: "start" })}
>
// ...
```

Handling New Answers

1. It is better to have as much logic as possible that updates state inside the reducer (than in the place where the event is first handled)

Moving to the Next Question

Displaying Progress

1. Progress.js

```
function Progress({ index, numQuestions, points, maxPossiblePoints, answer }) {
  return (
    <header className="progress">
      <progress max={numQuestions} value={index + Number(answer !== null)} />

      <p>
        Question <strong>{index + 1}</strong> / {numQuestions}
      </p>
      <p>
        <strong>{points}</strong> / {maxPossiblePoints}
      </p>
    </header>
  );
}

export default Progress;
```

Finishing a Quiz

Restarting a Quiz

1. By reading the reducer, we will know what the application does

Setting Up a Timer With useEffect

1. We start a timer, and the game will end when the timer stops
2. If state lives in the App component, if it changes, all the components will re-render
    1. A performance issue with very large applications
3. Other features:
    1. Allow user to select a certain number of questions
    2. Allow user to filter for the dificulty of questions
    3. We can store the highscore using the API and fetch when the application is opened again
    4. We could store all answers in an array
        1. User can go back in time, and review the answers

Section Summary: useState vs. useReducer

1. Usestate vs Usereducer

2. `useState`

    1. Ideal for **single**, **independent pieces of state** (numbers, strings, single arrays, etc)
    2. Logic to update state is placed directly in event handlers or effects, **spread all over one or multiple components**
        1. All over a component
        2. Even child components if we use lifted state
    3. State is updated by calling `setState` (setter returned from `useState`)
    4. **Imperative** state updates
    5. **Easy** to understand and to use

3. `useReducer`

    1. Ideal for multiple **related pieces of state** and **complex state** (e.g. object with many values and nested objects or arrays)

    2. Logic to update state lives in **one central place, decoupled from components:** the reducer

1. The logic is decoupled from components

3. State is updated by **dispatch an action** to a reducer

    1. Reducer knows how to perform state updates for different actions
    2. Reducers map state transitions to actions with well-defined names

4. **Declarative** state updates: complex state transitions are **mapped** to actions

    1. Instead of setting the following three states, we can dispatch an action

    ```
    setScore(0);
    setPlaying(true);
    setTimeSec(0);

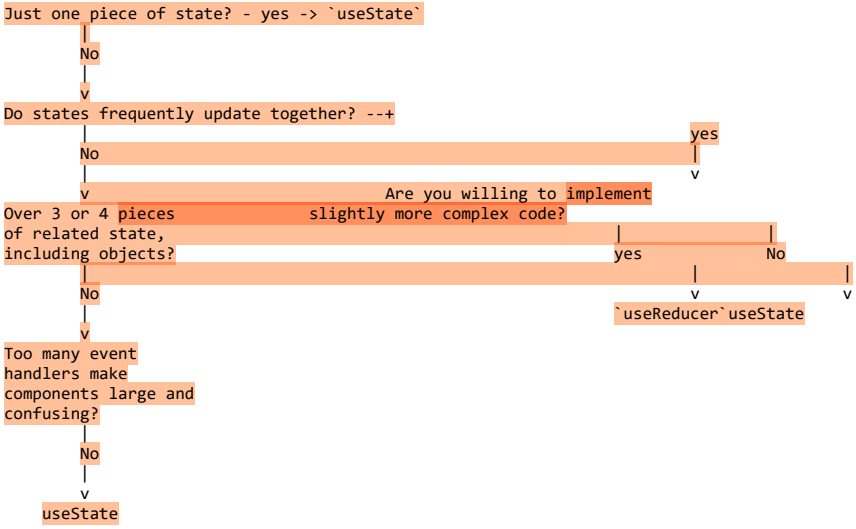
    dispatch({ type: 'startGame' });
    ```

        1. Downside: Slightly more difficult to understand and implement

5. More **difficult** to understand and implement

When to Use useReducer?

1. Flow:



    1. `useState` should remain your **default choice** for managing state
        1. If there are problems, consider using `useReducer`

CHALLENGE #1: Constructing a Bank Account With useReducer

1. MUST DO

Section 17: React Router: Building Single-Page Applications (SPA)

Section Overview

1. Topics:
    1. **React Router**: most important 3rd-party library
        1. Used in almost all React projects
        2. Used to build apps with multiple pages, that feel as smooth and as fluid as mobile apps (SPAs)
    2. Building our first single-page application (**SPA**)
        1. Example: WorldWise
    3. Styles with **CSS Modules** (new way)

Constructing Our First App With Vite: "WorldWise"

1. `Vite` - used to instantiate a project

    1. Installation:

    ```
    npm create vite@4

    React
    JavaScript
    ```

        1. It a build tool
        2. It has templates for the listed frameworks
        3. Unlike `create-react-app`, we need to manually install dependencies
            1. Installation is much faster that `create-react-app`
                1. There are a lot less packages

    2. **index.html** is outside the **public** folder

        1. The developer who maintains the template thinks that this is the best structure
            1. React doesn't care about the file structure

3. **main.jsx**
    1. Stands for JavaScript JSX
        1. Vite wants the extension to be JSX
        2. Pretty much the same as JS file
    2. Smaller and simpler than **index.js** file of `create-react-app`
4. Running the app:

    ```
    npm run dev
    ```

5. `create-react-app` already comes with important developer tools pre-installed
    1. ESLint (We don't want to build React App without ESLint)
        1. For Vite, we need to install ESLint

            ```
            npm i eslint vite-plugin-eslint eslint-config-react-app --save-dev
            ```

        2. New file: `.eslintrc.json` (to extend eslint with react rules we installed with `eslint-config-react-app`)

            ```
            {
                "extends": "react-app"
            }
            ```

        3. `vite.config.js` - used to config Vite project. We can configure things about development and building the project. We need to add eslint plugin

            ```
            import eslint from 'vite-plugin-eslint'
            ...
            export default defineConfig({
                plugins: [react(), eslint()],
            });
            ```

        4. Application reloads
        5. Testing:

            ```
            function App() {
                let x = 24;

                return <div>Worldwise</div>;
            }

            export default App;
            ```

What is Routing?

1. Routing
    1. With routing, we match **different URLs** to **different UI views** (React components): **routes**
        1. In case of React, we match each URL to a specific React component
            1. Each match between a URL and a component is called a route
            2. When a specific URL is visited, the corresponding React component will be rendered
                1. Example: www.example.com/
                    1. Home component is shown
                2. Example: www.example.com/login
                    1. Login component is shown
                    2. If login is successful, we redirect to www.example.com/app path
    2. This enables users to **navigate between different application screens**, using the browser URL (using links)
    3. Keeps the UI **in sync** with the current browser URL
    4. Note: **The routing only works on the client side**
        1. There is routing on server side
            1. We are not using this in the Client-side React applications
        2. Most front-end frameworks have the client-side routing capabilities baked into the framework
            1. React depends on third-party packages for many different functionalities (routing is one of them)
                1. In React, it is handled by a third-party package called **React Router** (most important and most used)
                    1. Must learn because it allows us to build **single-page applications** (routing is fundamental)

Routing and Single-Page Applications (SPAs)

Single-Page Applications (SPA)

1. Application that is **executed entirely on the client** (browsers)
2. **Routes:** different URLs correspond to different views (components)
    1. SPA Running on Client

        ```
        User clicks router link (special link)
                |
                v
        URL is changed (In React, react-router package does this job)
                |
                v
        DOM is updated (as a result of URL change)
            When URL is changed, React component
            corresponding to the new URL is
            rendered
        ```

        1. In SPAs, **JavaScript** (React) is used to update the page (DOM)
            1. The page is updated by JS
                1. The page is never reloaded (the entire app exists as one page without hard reloads)

        2. The whole cycle can be repeated
            1. Each time a user clicks on a router link, it will change the URL, and the component on the screen (without reloading the page)
3. Feels like a **native app** (Desktop app, or mobile app)
    1. Fantastic UX
4. Additional data **might be loaded** from a web API
    1. SPA can communicate with a server to fetch data it needs
5. We cannot load a completely new page (Otherwise, it cannot be called as SPA)
    1. All React Apps are SPAs (they are never reloaded)
6. For big applications:
    1. Rely on URLs
    2. Rely on routing capabilities

Implementing Main Pages and Routes

1. Installing React Router:

```
npm i react-router-dom@6
```

    1. There are two big ways to declare routes
        1. Traditional way:
            1. We can define routes in a declarative way
                1. We can use special components that React router gives to define routes in JSX
        2. React matched the URL with the `path` that was provided in the JSX, and selects the one that matches
            1. When we manually change the URL, the page gets reloaded

Linking Between Routes With and

1. Traditional way:

```
<a href="/pricing">Pricing</a>
```

    1. Whole page gets reloaded
        1. Check the Network tab
2. SPA way:

```
import { Link } from "react-router-dom";

function HomePage() {
  return (
    <div>
      <h1>Home Page</h1>
      <Link to="/pricing">Pricing</Link> {/*  **(M)** Only DOM gets updated */}
    </div>
  );
}

export default HomePage;
```

3. We can bookpark any page, or share the URL of the page with anybody.
4. If we use `NavLink` instead of `Link`, The element gets a `class="active"` property, which can be used to style the active link differently

```
<a aria-current="page" class="active" href="/pricing">Pricing</a>
```

5. Code: HomePage.jsx

```
import { Link } from "react-router-dom";
import PageNav from "../components/PageNav";

function HomePage() {
  return (
    <div>
      <PageNav />
      <h1>Home Page</h1>
      <Link to="/pricing">Pricing</Link> {/*  **(M)** Only DOM gets updated */}
    </div>
  );
}

export default HomePage;
```

6. Code: PageNav.jsx

```
import { NavLink } from "react-router-dom";

function PageNav() {
  return (
    <nav>
      <ul>
        <li>
          <NavLink to="/">Home</NavLink>
        </li>
        <li>
          <NavLink to="/pricing">Pricing</NavLink>
        </li>
        <li>
          <NavLink to="/product">Product</NavLink>
```

```
            </li>
          </ul>
        </nav>
      );
    }

    export default PageNav;
```

Styling Options For React Applications

Styling Options in React

1. Why are there so many different ways of styling a React App?
    1. A philosophy of React App is to be un-opinionated in regards to many common aspects of building web-applications
        1. One such aspect is styling
            1. Styling option: React doesn't care about styling
                1. We have different styling options
                    1. Most of them are provided by third-party libraries
2. Different options:
    1. Styling option:
        1. Inline CSS
            1. Where?
                1. JSX elements
            2. How?
                1. `style` prop
            3. Scope
                1. JSX element
                    1. Only to a particular JSX element it is applied to (locally scoped)
            4. Based on
                1. CSS
        2. CSS or Sass file
            1. Where?
                1. External file
            2. How?
                1. `className` prop
            3. Scope
                1. Entire app
                    1. Proplems: Global
                        1. Any JSX element could use any of the classes in the external CSS file
                            1. Example: We don't know which components use which classes, if we update classes, they will be unnecessary repurcussions.
                            2. Example: If we add a new class with the name of a class that already exists, there will be clashes between classes
                    2. Solution: CSS must never be global, but must be scoped to individual components
            4. Based on
                1. CSS
        3. CSS Modules
            1. Where?
                1. One external file per component
            2. How?
                1. `className` prop
            3. Scope
                1. Component
                    1. No other component can use them
                        1. Makes the components modular and re-usable
                            1. Better reflects React's separation of concerns
            4. Based on
                1. CSS
        4. CSS-in-JS (library) (Styled components)
            1. Where?
                1. External file or component file
                    1. We write CSS inside a JS file. In the same file we define a component
                        1. React components are created with styles directly applied to them which can be used just like any other component
                            1. Embraces React philosophy - A component must contain all the information related to its appearance
            2. How?
                1. Constructs new component
            3. Scope
                1. Component
            4. Based on
                1. JavaScript
        5. Utility-first CSS (tailwindcss)
            1. Where?
                1. JSX elements
                    1. We use pre-defined utility classes to define individual styles to use
                        1. Flexbox
                        2. To make layouts responsive
                        3. To make hover effects
                        4. (to design the entire UI without leaving the JSX markup)
            2. How
                1. `className` prop
            3. Scope
                1. JSX element
            4. Based on
                1. CSS
        6. Alterative to styling with CSS: UI libraries like MUI, Chakra UI, Mantine, etc. (We don't have to write any CSS)
            1. Libraries contain pre-built, and pre-styled components that are common in most web-applications (not ideal for beginners)

Using CSS Modules

1. CSS modules comes out of the box with `create-react-app` and `vite` (nothing to install)
2. One external CSS file per component
    1. Convention: `<component-name>.module.css`
    2. Including the the component file:

        ```
        import styles from './PageNav.module.css';

        // ...

        <nav className={styles.nav}>
        ```

        1. Internals: The class is converted to:

            ```
            <nav class="_nav_hqhhq_1">
            ```

            1. classname + random id (unique to the module)
                1. We can give the same classname in different modules
3. Global CSS: global reset, font, ...
    1. We can include an external CSS file
        1. No classnames exist
    2. Global classnames will not work

        ```
        <h1 className="test">Home Page</h1>
        ```

        1. Solution:

            ```
            :global(.test) {
                // ...
            }
            ```
    3. The following doesn't work

        ```
        .nav .active {
            // ...
        }
        ```

        1. Solution:

            ```
            .nav :global(.active) {
                // ...
            }
            ```

            1. `:global(...)` is useful when working with classes provided by external sources
                1. Otherwise, css modules will append a random id

                    ```
                    .active_gasdf_1
                    ```
4. Composing classes
    1. Not covered here

Building the Pages

1. The images in `assets` folder are directly imported into JS code
2. For css modules, we need to write classnames using camelCase and not kabab-case

Building the App Layout

1. We can have a folder for each component
2. Most of the code related to components can be inside the components

Nested Routes and Index Route

1. Nested routes
    1. We need them when we want a part of the UI by a part of the URL

        ```
        /cities
        ```

        1. Shows list of cities
    2. It is nested route, if the path decides what component is rendered inside another component
    3. Nested routes
        1. App.jsx

            ```
            <Route path="app" element={<AppLayout />}>
              {" "}
              {/* parent path is not required */}
              <Route path="cities" element={<p>List of cities</p>} />
              <Route path="countries" element={<p>Countries</p>} />
              <Route path="form" element={<p>Form</p>} />
            </Route>
            ```

        2. Sidebar.jsx

```
function Sidebar() {
  return (
    <div className={styles.sidebar}>
    // ...

      <Outlet /> {/* Similar to children prop */}

    // ...
```

4. Instead of using `useState` to store the active tab info, we let the React router to store the state and URL path
    1. Overall navigation (including sub-navigations) are managed using React router

Implementing the Cities List

1. Proptypes:

```
import PropTypes from "prop-types";

// ...

Message.propTypes = {
  message: PropTypes.string.isRequired,
};
```

Implementing the Countries List

1. We can have all imports from third-party libraries on top, and other imports below

Storing State in the URL

The URL for State Management

1. The URL is an excellent place to store UI state and an alternative to `useState` in some situations!
    1. Example: open/closed panels, currently selected list item, list sorting order, applied list filters
2. Why?
    1. Easy way to store state in a **global place**, accessible to **all components** in the app
        1. We can read the value from the URL, wherever the component is in the component tree
    2. Good way to **"pass" data** from one page into the next page
        1. Without storing the data in some temporary place inside the app
    3. Makes it possible to **bookmark and share** the page with the exact UI state it had at the time
        1. Example: We might filter products by color, price, etc...
            1. We can share the page with someone, and same filters will be applied (great UX)
3. Implementation:

```
www.example.com/app/cities/lisbon?lat=38.728&lng=-9.141
```

    1. path - state that is not useful for state management
    2. params - `/lisbon`
        1. Useful to pass data to next page
    3. query string - `lat=38.728&lng=-9.141`
        1. Useful to store global state that should be accessible everywhere

Example: Params and Query String

1. `lisbon` - A link can contain this param and it gets passed to the next page whenever the user clicks on the link
2. `lat=38.728&lng=-9.141` - They correspond to specific positions on the map
3. **City name** and **GPS location** were retrieved from the **URL** instead of application state!

Dynamic Routes With URL Parameters

1. New route:

```
<Route path="cities/:id" element={<City />} />
```

2. Link to the route:

```
<Link className={styles.cityItem} to={`${id}`}>
  <span className={styles.emoji}>{emoji}</span>
  <h3 className={styles.name}>{cityName}</h3>
  <time className={styles.date}>({formatDate(date)})</time>
  <button className={styles.deleteBtn}>&times;</button>
</Link>
```

3. Retrieve passed data using `useParam` hook

```
const { id } = useParams();
```

Reading and Setting a Query String

Programmatic Navigation with useNavigate

1. To move to a new URL without the user having to click on any link
    1. Use-case: After a user submits a form (programmatic navigation can be used)

2. Code: Form:

```
<div className={styles.buttons}>
  <Button type="primary">Add</Button>
  <Button
    type="back"
    onClick={(e) => {
      e.preventDefault();
      navigate(-1);
    }}
  >
    {" "}
    {/**Number of steps back */}
    &larr; Back
  </Button>
</div>
```

3. Code: Button

```
import styles from "./Button.module.css";
import PropTypes from "prop-types";

function Button({ children, onClick, type }) {
  return (
    <button onClick={onClick} className={`${styles.btn} ${styles[type]}`}>
      {children}
    </button>
  );
}

Button.propTypes = {
  children: PropTypes.node.isRequired,
  onClick: PropTypes.func.isRequired,
  type: PropTypes.string.isRequired,
};

export default Button;
```

Programmatic Navigation with

1. `<Navigate />` - not so much used.
    1. One important use-case: Inside nested routes
        1. Immediately navigate to a particular URL
    2. **It is a redirect**

```
<Route index element={<Navigate to="cities" />} />
<Route
  path="cities"
  element={<CityList cities={cities} isLoading={isLoading} />}
/>
```

        1. We cannot use `navigate` function coming from `useNavigate` hook
            1. We use declarative JSX `<Navigate ...>` to redirect (`navigate` is imperative)

Section 18: Advanced State Management: The Context API

Section Overview

1. Topics:
    1. Context API **patterns**
        1. Powerful pattern
        2. Apply
    2. State management **deep dive**
        1. New types of state & tools to handle them
    3. Including an interactive **map**

CHALLENGE #1: Understand "The Atomic Blog" App

1. Study code and data flow of `12-atomic-blog`
    1. Open it up in VS Code
        1. How it works
        2. What it does
        3. Open terminal

           `npm install`
        4. What was used to instantiate the project

           `react-scripts`

What is the Context API?

A Solution to Prop Drilling

1. Task: Passing state into multiple deeply nested child components
    1. Solution 1: Pass props down

        1. Cumbersome and inconvenient
            1. Prop drilling
    2. Solution 2: Component composition using `children` prop
        1. It doesn't always solve the problem
    3. Solution 3: Directly passing a state from parent component to deeply nested child component
        1. **Context API**
            1. Allows a component anywhere in the tree to read state that a context shares

What is Context API?

1. Context API:
    1. System to pass data throughout the app **without manually passing props** down the component tree
    2. Allows us to **"broadcase" global state** to the entire app (available to all child components of a certain context)
        1. **Provider:** Special React component that gives all child components access to `value`
            1. Can sit anywhere in the component tree
                1. Convention to place it at the top (`App` component)
        2. **value:** data that we want to make available (usually state and function)
            1. It is passed into provider
            2. value contains one or more state variables & some setter functions
        3. **Consumers:** all components that read the provided context `value`
            1. They are components that subscribe to the context
                1. They can read the value from the context
            2. Any number of consumers can be provided with a context
    3. Whenever a context value is updated, all consumers subscribed to the context are re-rendered
        1. Provider will immediately notify consumers about the value change, and will re-render the components
    4. We can have as many contexts as we want and place them in the component tree

Constructing and Providing a Context

1. We usually construct one context per state domain
    1. One context for posts
    2. One context for search query
2. Code:

```
// 1) CREATION OF CONTEXT
const PostContext = createContext(); // we can pass default value, but it cannot change over time. So we pass `null` or not pass anything.
// It is a component, so upper-case letter

// ...

// 2) PROVIDE VALUE TO CHILD COMPONENTS
<PostContext.Provider
  value={{
    posts: searchedPosts,
    onClearPosts: handleClearPosts,
    onAddPost: handleAddPost,
    searchQuery,
    setSearchQuery,
  }}
>
// ...
</PostContext.Provider>
```

Consuming the Context

1. Code:

```
const { searchQuery, setSearchQuery } = useContext(PostContext);
```

2. The context api make the components much more independent and standalone
    1. We can re-use `List` anywhere now

```
<footer>
    <List />
    // ...
</footer>
```

        1. We didn't have to pass any props first into the `footer` and then to the `List`

Advanced Pattern: A Custom Provider and Hook

1. Implementation of advanced patterns
    1. We can remove all state and state updating logic from a component and place it in a custom provider component.
2. Code: PostProvider.js

```
import { createContext, useContext, useState } from "react";
import { faker } from "@faker-js/faker";

function createRandomPost() {
  return {
    title: `${faker.hacker.adjective()} ${faker.hacker.noun()}`,
    body: faker.hacker.phrase(),
  };
}

const PostContext = createContext();

function PostProvider({ children }) {
```

```
  const [posts, setPosts] = useState(() =>
    Array.from({ length: 30 }, () => createRandomPost())
  );
  const [searchQuery, setSearchQuery] = useState("");

  const searchedPosts =
    searchQuery.length > 0
      ? posts.filter((post) =>
          `${post.title} ${post.body}`
            .toLowerCase()
            .includes(searchQuery.toLowerCase())
        )
      : posts;

  function handleAddPost(post) {
    setPosts((posts) => [post, ...posts]);
  }

  function handleClearPosts() {
    setPosts([]);
  }

  return (
    <PostContext.Provider
      value={{
        posts: searchedPosts,
        onClearPosts: handleClearPosts,
        onAddPost: handleAddPost,
        searchQuery,
        setSearchQuery,
      }}
    >
      {children}
    </PostContext.Provider>
  );
}

function usePosts() {
  const context = useContext(PostContext);
  if (context === undefined)
    throw new Error("PostContext was used outside of the PostProvider");
  return context;
}

export { PostProvider, usePosts };
```

3. Code: App.js

```
<PostProvider>
  <Header />
  <Main />
  <Archive />
  <Footer />
</PostProvider>
```

Thinking in React: Advanced State Management

Review: What is State Management?

1. State management: Giving each piece of state the right home
    1. **When** to use state
    2. **Types** of state **(accessibility)**: local vs. global
    3. From lecture "Fundamentals of State Management". You can keep using the flowchart
2. This lecture:
    1. **Types** of state (**domain**): UI vs remote
    2. **Where** to place each piece of state
    3. **Tools** to manage all types of state

Types of State

1. Classification:
    1. State Accessibility (Check if this component was rendered **twice**, should a state update in one of the components reflect in the other one? No - Local State, Yes - Global State)
        1. Local State
            1. Needed only by **one or few components**
                1. Only accessible inside the component it was defined in + child components
        2. Global State
            1. Might be needed by **many components** (in different parts of the tree)
                1. Can be made accessible in every single component in the tree
    2. State Domain (they are managed in a different way)
        1. Remote State
            1. All application data **loaded from a remote server** (usually using an API)
                1. Usually resides on a server that is loaded into the application
            2. Usually obtained in an **asynchronous** way
                1. Because data might need to be re-fetched an updated frequently
                    1. For large-scale applications, data needs to be
                        1. Cached
                        2. Re-validated
                        3. ...
            3. We need some specialized tools for the above
        2. UI State
            1. Everything else
                1. Theme
                2. List filters
                3. Form data

4. Open panels
5. ...

2. Usually **synchronous** and stored in the application
    1. Doesn't interact with any server
3. Easy to handle with the tools we know of
    1. `useState`
    2. `useReducer`

State Placement Options

1. Where to physically place state?
    1. Options:
        1. Local component:
            1. Tools
                1. `useState`
                2. `useReducer`
                3. `useRef` (it doesn't re-render a component)
            2. When to use?
                1. Local state
        2. Parent component (of all components that need the state)
            1. Tools
                1. `useState`
                2. `useReducer`
                3. `useRef`
            2. When to use?
                1. Lifting up state
        3. Context (A better solution for the above)
            1. Tools
                1. Context API + `useState` or `useReducer`
                    1. Context itself doesn't manage state
                        1. It only gives access of state to the tree which is managed using other mechanisms
            2. When to use?
                1. Global state (preferably UI state)
                    1. Not for remote state
        4. 3rd-party library
            1. Tools
                1. Redux
                2. React Query
                3. SWR
                4. Zustand
                5. ...
            2. When to use?
                1. Global state (remote or UI states)
        5. URL
            1. Tools
                1. React Router
            2. When to use?
                1. Global state
                    1. Easily sharable or bookmarkable
                2. Passing between pages
        6. Browser (doesn't re-render any components, but it is technically application state)
            1. Tools
                1. Local storage
                2. Session storage
                3. ...
            2. When to use?
                1. Storing data in user's browser

State Management Tool Options

1. How to manage different types of state in practice?
    1. Local UI State
        1. `useState`
        2. `useReducer`
        3. `useRef`
    2. Global UI State
        1. Context API + `useState`/`useReducer` (good enough)
        2. Redux, Zustand, Recoil, etc. (if we want more performance)
        3. React Router (for more performance)
    3. Local Remote State
        1. `fetch` + `useEffect` + `useState`/`useReducer` (small applications only)
            1. `fetch` inside `useEffect` loads data from a remote location
            2. `useState`, or `useReducer` can be used to store the state fetched
    4. Global Remote State
        1. Context API + `useState`/`useReducer`
        2. Redux, Zustand, Recoil, etc.
        3. Tools highly specialized in handling remote state (**They have caching and automatic re-fetching (to deal with async nature of remote state)**)
            1. React Query
            2. SWR
            3. RTK Query

Back to "WorldWise": Constructing a CitiesContext

1. Context API
    1. For small applications
        1. Where performance is not an issue

Consuming the CitiesContext

Finishing the City View

1. Code: BackButton.jxs

```
import { useNavigate } from "react-router-dom";
import Button from "./Button";

function BackButton() {
  const navigate = useNavigate();

  return (
    <div>
      <Button
        type="back"
        onClick={(e) => {
          e.preventDefault();
          navigate(-1);
        }}
      >
        {" "}
        {/**Number of steps back */}
        &larr; Back
      </Button>
    </div>
  );
}

export default BackButton;
```

Including a Map With the Leaflet Library

1. Third-party package

```
npm i react-leaflet leaflet
```

    1. `react-leaflet` is built on `leaflet`

2. React Leaflet: https://react-leaflet.js.org/docs/start-setup/

Displaying City Markers on Map

1. Leaflet adds classes which can be used for styling
    1. Make them global with `.global(...)` since we are using CSS modules.

Interacting With the Map

1. In Leaflet map, we need a custom component to implement centering
2. Capturing position:

```
function DetectClick() {
  const navigate = useNavigate();
  useMapEvents({
    click: (e) => {
      // console.log(e); // Sent by leaflet library
      navigate(`form?lat=${e.latlng.lat}&lng=${e.latlng.lng}`);
    },
  });
}
```

Setting Map Position With Geolocation

1. Code: useGeolocation.js

```
import { useState } from "react";

export function useGeolocation(defaultPosition = null) {
  const [isLoading, setIsLoading] = useState(false);
  const [position, setPosition] = useState(defaultPosition);
  const [error, setError] = useState(null);

  function getPosition() {
    if (!navigator.geolocation)
      return setError("Your browser does not support geolocation");

    setIsLoading(true);
    navigator.geolocation.getCurrentPosition(
      (pos) => {
        setPosition({
          lat: pos.coords.latitude,
          lng: pos.coords.longitude,
        });
        setIsLoading(false);
      },
      (error) => {
        setError(error.message);
        setIsLoading(false);
      }
    );
  }

  return { isLoading, position, error, getPosition };
}
```

2. Code: Map.jsx

```
import styles from "./Map.module.css";
import { useNavigate, useSearchParams } from "react-router-dom";
import {
  MapContainer,
  TileLayer,
  Marker,
  Popup,
  useMap,
  useMapEvents,
} from "react-leaflet";
import { useCities } from "../contexts/CitiesContext";
import PropTypes from "prop-types";
import { useEffect, useState } from "react";
import { useGeolocation } from "../hooks/useGeolocation";
import Button from "./Button";

function Map() {
  const navigate = useNavigate();
  const { cities } = useCities();
  const [mapPosition, setMapPosition] = useState([40, 0]);
  const [searchParams] = useSearchParams();
  const {
    isLoading: isLoadingPosition,
    position: geolocationPosition,
    getPosition,
  } = useGeolocation();

  const mapLat = searchParams.get("lat");
  const mapLng = searchParams.get("lng");

  useEffect(
    function () {
      if (mapLat && mapLng) setMapPosition([mapLat, mapLng]);
    },
    [mapLat, mapLng]
  );

  useEffect(
    function () {
      if (geolocationPosition)
        setMapPosition([geolocationPosition.lat, geolocationPosition.lng]);
    },
    [geolocationPosition]
  );

  return (
    <div className={styles.mapContainer} onClick={() => navigate("form")}>
      <Button type="position" onClick={getPosition}>
        {isLoadingPosition ? "Loading..." : "Use your position"}
      </Button>
      <MapContainer
        center={mapPosition}
        zoom={6}
        scrollWheelZoom={true}
        className={styles.map}
      >
        <TileLayer
          attribution='&copy; <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a> contributors'
          url="https://{s}.tile.openstreetmap.fr/hot/{z}/{x}/{y}.png"
        />
        {cities.map((city) => (
          <Marker
            position={[city.position.lat, city.position.lng]}
            key={city.id}
          >
            <Popup>
              <span>{city.emoji}</span> <span>{city.cityName}</span>
            </Popup>
          </Marker>
        ))}

        <ChangeCenter position={mapPosition} />
        <DetectClick />
      </MapContainer>
    </div>
  );
}

function ChangeCenter({ position }) {
  const map = useMap();
  map.setView(position);
  return null;
}

function DetectClick() {
  const navigate = useNavigate();
  useMapEvents({
    click: (e) => {
      // console.log(e); // Sent by leaflet library
      navigate(`form?lat=${e.latlng.lat}&lng=${e.latlng.lng}`);
    },
  });
}

ChangeCenter.propTypes = {
  position: PropTypes.array.isRequired,
};

export default Map;
```

Fetching City Data in the Form

Constructing a New City

1. We grab whatever we need from npm

   ```
   npm i react-datepicker
   ```

2. For small applications, we synchronize application state with the UI using `useEffect` - UI in sync with remote state

    1. For big applications, we use React Query
        1. Whenever we add something new to the remote state, the data will get automatically re-fetched into our application

3. Handler functions can be async functions

Deleting a City

Advanced State Management System: Context + useReducer

1. Converting state management into a reducer
    1. Common with Context API

Adding Fake Authentication: Setting Up Context

Adding Fake Authentication: Implementing "Login"

Adding Fake Authentication: Protecting a Route

CHALLENGE #2: Refactoring "React Quiz" to Context API

Section 19: Performance Optimization and Advanced useEffect

Section Overview

1. Topics:
    1. Analyzing wasted renders
    2. Optimizing performance
        1. To understand React apps better and make apps fast and snappy
    3. Deep dive into `useEffect`
        1. Advanced section

Performance Optimization and Wasted Renders

1. What can be optimized and how

Performance Optimization Tools

1. Techniques:
    1. Prevent wasted renders
        1. `memo`
            1. To memoize components
        2. `useMemo`
            1. To memoize objects
        3. `useCallback`
            1. To memoize functions
        4. Passing elements as `children` or regular prop
            1. To prevent them from being re-rendered (?)
    2. Improve app speed/responsiveness (to make it 100% fluid and without delays)
        1. `useMemo`
        2. `useCallback`
        3. `useTransition`
    3. Reduce bundle size
        1. Using fewer 3rd-party packages
        2. Code splitting and lazy loading
2. We don't have to use all the techniques all the time
    1. We can use whenever a situation calls for it
        1. We will see in which situations, we need each of the tools
3. The list of tools and techniques is not exhaustive
    1. We are already doing many optimizations by following best practices
        1. Not defining components inside other components

When Does a Components Instance Re-Render?

1. A component instance only gets re-rendered in 3 different situations:
    1. State changes
    2. Context changes (that the component is subscribed to)
    3. Parent re-renders
2. Prop changes technically do to re-render components
    1. props change only when parent re-renders
    2. Since parent re-renders, children (which receive the props) re-render anyway
3. Remember:
    1. A render does not mean that the DOM actually gets updated
        1. It just means that component function gets called
            1. This can be an expensive operation
                1. React will construct new VDOM, do diffing, and reconciliation
                    1. If DOM doesn't change, it causes **wasted render**
4. **Wasted Render**
    1. A render that didn't produce any change in the DOM

1. All calculations were for nothing
    1. Usually not a problem because React is fast
        1. Only a problem when they happen **too frequently** or when the **component is very slow**
            1. Application feels laggy and unresponsive

The Profiler Developer Tool

1. Profiler
    1. We can analyze renders and re-renders
        1. Which components have rendered
        2. Why the components rendered
        3. How long the renders took
    2. Open Dev Tools > Profiler
        1. Click on "Start profiling"
    3. Settings
        1. Profiler
            1. Record why each component rendered while profiling (state update, context update, parent re-render)
        2. Reset the setting when we reload the page
        3. Start profiler
            1. Do something on the page
        4. Stop profiler
            1. Flamegraph
                1. Gray: Did not render while the application re-render
                2. More yellow: Longer to re-render
                    1. Hover shows:
                        1. How long it took to re-render
                        2. Why did this render?
                3. We can see number of commits (a re-render) on the top right corner
            2. Ranked
                1. Topmost component took the longest time

A Surprising Optimization Trick With children

1. To prevent some components from re-rendering
    1. Gives insight into how react works iternally
2. Test.js

```
function Counter({ children }) {
    const [count, setCount] = useState(0);
    return (
        <div>
            <h1>Slow counter?!?</h1>
            <button onClick={() => setCount((c) => c + 1)}>Increase: {count}</button>
            {children}
        </div>
    );
}

// ...
<Counter>
    <SlowComponent />
</Counter>
```

    1. The component which is passed as `children` is already rendered before it is passed into the component
        1. The component is not affected by the state change in the outer component
    2. The following would work as well

```
<Counter showThis={<SlowComponent />} />
```

Uderstanding memo

What is Memoization?

1. **Memoization**: Optimization technique that executes a pure function once, and saves the result in memory. If we try to execute the function again with teh **same arguments as before**, the previously saved result will be returned, **without executing the function again**
    1. We can use this technique to optimize React applications
        1. Memoize **components** with `memo`
        2. Memoize **objects** with `useMemo`
        3. Memoize **functions** with `useCallback`
2. Advantages:
    1. Prevent wasted renders
    2. Improve ap speed/responsiveness

The Memo Function

1. `memo`
    1. Memoized component: Used to construct a component that will **not re-render when its parent re-renders**, as long as the **props stay the same between renders** (compents are defined using functions)
        1. Regular behavior (no memo)
            1. If a component re-renders, the child components re-render
        2. Memoized child with `memo`: A memoized child does not re-render if it receives the same props. If props change, the memoized child needs to re-render
    2. **Only affects props!** A memoized component will still re-render when its **own state changes** or when a **context that it's subscribed to changes**
        1. The componet has new data to show
    3. Use-case:

1. Only makes sense when the component is **heavy** (slow rendering), **re-renders often**, and does so **with the same props**

memo in Practice

1. Memoized version:

```
const Archive = memo(function Archive({ show }) {
    // ...
});
```

    1. It is very heavy
    2. It re-renders very often
    3. It re-renders with the same props

2. Passing an object

```
const archiveOptions = {
    show: false,
    title: 'Pos: Archive in addition to the main post'
}

//

const Archive = memo(function Archive ({ archiveOptions }) {
    // ...
});
```

    1. Memoization doesn't work in this case

Understanding useMemo and useCallback

An Issue with Memo

1. If React, everyting is **re-created on every render** (including objects and functions) (whenever a component is re-rendered)
2. In JavaScript, two objects or functions that look the same, **are actually different**

```
{} != {}
```

    1. They are unique

3. If objects or functions are passed as props, the child component will always see them as **new props on each re-render**
    1. **If props are different between re-renders, memo will not work**
4. Solution:
    1. We need to memoize objects and functions, to make them stable (preserve) between re-renders (memoized {} == memoized {})
        1. Implementation:
            1. `useMemo`
                1. Used to memoize values (`useMemo`) **between renders**
            2. `useCallback`
                1. Used to memoize functions (`useCallback`) **between renders**
            3. Values passed into `useMemo` and `useCallback` will be stored in memory ("cached") and **returned in subsequent re-renders, as long as dependencies** ("inputs") **stay the same**
                1. `useMemo` and `useCallback` have a **dependency array** (like `useEffect`): Whenever one **dependency changes**, the value will be **re-created** (it will no longer be returned from the cache)
        2. Understanding
            1. Regular behavior (no `useMemo`)
                1. Components re-renders -trigger-> New value [creation]
            2. Memoizing a value with `useMemo`
                1. Components re-renders -trigger-> Cache value is returned. No new value.
                    1. True if dependencies in the depedendency array don't change
                    2. If they change, a new value is created
        3. If props are objects or functions:
            1. Three big use-cases:
                1. Memoizing props to prevent wasted renders (together with `memo`)
                    1. Hence we have `useMemo` and `useCallback` hooks
                2. Memoizing values to avoid expensive re-calculations on every re-render
                    1. Example: A derived state calculated from 100,000 components
                        1. React doesn't have to do this expensive calculation
                            1. Using `useMemo`
                3. Memoizing values that are used in dependency array of another hook
                    1. For example to avoid infinite `useEffect` loops
            4. Only use them for one of the three **use cases!**

useMemo in Practice

1. Code:

```
const archiveOptions = useMemo(() => {
    return {
        show: false,
        title: `Post archive in addition to ${posts.length} main posts`,
    };
}, [posts.length]); // callback is called on initial render
```

    1. Function can have other intensive calculations.
        1. The returned object is saved in the cache
    2. Dependency array
        1. `[]` - it is computed only once in the beginning (never re-computed again)

2. `[posts.length]`: Dependency - only when the `length` changes
    1. It is better to have only primitives in the dependency array

useCallback in Practice

1. Code:

```
<Archive archiveOptions={archiveOptions} onAddPost={handleAddPost} />

// ...

const Archive = memo(fuction Archive({ archiveOptions, onAddPost }) {
    // ...
}
```

2. Code: Memoized function

```
const handleAddPost = useCallback(function handleAddPost(post) {
    // ...
});
```

    1. It will not call the function immediately, but it will memoize the function
        1. The function is memoized
    2. It is better to find slow components and optimize them using `useCallback`
    3. React team is working on a compiler that can automatically memoize if required behind the scenes.
    4. How does it work?
        1. Initial render: returns the passed function.
        2. Subsequent renders:
            1. If dependencies have not changed: returns already stored function from the last render
            2. If dependencies have changed: returns the function passed during this render

3. `set...` always have a stable identity

    1. It will not change on renders
        1. They are automatically memoized
        2. We don't have to pass them as dependencies in `memo`, `useMemo`, and `useCallback`

Optimizing Context Re-Renders

1. Optimize context only if the following three things are true at the same time

    1. State in a context needs to change all the time
    2. Context has many consumers
    3. App is slow and laggy

2. Two options for optimization for a context

    1. Use `{children}` OR
        1. They are not automatically re-rendered
            1. They are first created and then passed
    2. Memoize the direct descendants of the context
        1. Wrap components inside a `memo`

3. Examples:

    1. `PostProvider` is the child of the `App` component

        1. When `App` component re-renders, the `PostProvider` re-renders as well

            1. The object will be re-created

                1. Implies that the context value has changed

                    1. Therefore all components that depend on the context are re-rendered

                        1. Solution: Memoize the object

```
const value = useMemo(() => {
    return {
      posts: searchedPosts,
      onClearPosts: handleClearPosts,
      onAddPost: handleAddPost,
      searchQuery,
      setSearchQuery,
    };
  }, [searchedPosts, searchQuery]);

  return <PostContext.Provider value={value}>{children}</PostContext.Provider>;
```

4. If we have a lot of variables in a context value, if any one of them changes, the context changes

    1. Solution: One context per state
        1. Example:
            1. Post context
            2. SearchQuery context
                1. All components that consume this context are re-rendered but do not affect components that do not depend on this context

Back to the "WorldWise" App

1. Go to "Ranked" tab

2. Memoize a function to avoid infinite loop

3. Example:

```
function City() {
  const { id } = useParams();
  const { getCity, currentCity, isLoading } = useCities();

  useEffect(
    function () {
      getCity(id); // it can get recreated
    },
    [id, getCity]
  );
// ...
```

`CitiesContext.jsx`

```
const getCity = useCallback(
    async function getCity(id) {
      if (Number(id) === currentCity.id) return;

      dispatch({ type: "loading" });
      try {
        const res = await fetch(`${BASE_URL}/cities/${id}`);
        const data = await res.json();
        dispatch({ type: "city/loaded", payload: data });
      } catch (e) {
        dispatch({
          type: "rejected",
          payload: "There was an error loading city...",
        });
      }
    },
    [currentCity.id]
  );
```

Optimizing Bundle Size With Code Splitting

The Bundle and Code Splitting

1. On initial load, server sends a huge JS file to client
    1. **Bundle**: JavaScript file containing the **entire application code**. Downloading the bundle will load **the entire app at once**, turning it into a SPA
        1. It is produced by a tool like Webpack (inside create-react-app) or Vite
        2. SPA runs entirely on the client
            1. Whenever a URL changes in the app, the client renders entire React component but without loading any files from the server
                1. JS code is already there in the bundle
    2. **Bundle size**: Amount of JS users have to download to start using the app. One of the most important things to be optimized, so that the bundle takes **less time to download**
    3. **Code splitting**: Splitting bundle into multiple parts that can be downloaded over time ("Lazy loading")
        1. As they become necessary for the application
2. Implementation
    1. There are many ways to split
        1. We can split at route level
            1. `lazy()` **(M)**
                1. A feature of React
                    1. Vite or Webpack automatically split bundle when they see `lazy` function
            2. Bundle without lazy loading

               ```
               npm run build
               ```

            3. Example:

               ```
               const HomePage = lazy(() => import("./pages/HomePage"));
               const Product = lazy(() => import("./pages/Product"));
               const Pricing = lazy(() => import("./pages/Pricing"));
               const Login = lazy(() => import("./pages/Login"));
               const AppLayout = lazy(() => import("./pages/AppLayout"));
               const PageNotFound = lazy(() => import("./pages/PageNotFound"));
               ```

                1. We want loading spinner while the lazy loading is happening
                2. `<Suspense fallback={<SpinnerFullPage />}>...</Suspense>`
                    1. It makes component suspended while it is downloaded, and displays spinner (fallback)
                    2. If page has arrived, the children will be rendered
                    3. If we re-visit the same page, it will be available already

Don't Optimize Prematurely!

1. Don't optimize prematurely
    1. Don't optimize anything if there is nothing to optimize...
        1. If the application is performing just fine:
            1. Don't wrap all components in `memo()`
            2. Don't wrap all values in `useMemo()`
            3. Don't wrap all functions in `useCallback()`
        2. Don't optimize context if it's not slow and doesn't have many consumers
    2. Memoizing as slight hit on performance. If we add many of them, it will impact performance
        1. Code also becomes unreadable and too messy
2. Do
    1. Find performance bottlenecks using the Profiler and visual inspection (laggy UI)

1. Fix those real performance issues
    1. Memoize expensive re-renders
    2. Memoize expensive calculations
    3. Optimize context if it has many consumers and changes often
        1. Memoize context value + child components (direct child components of the provider)
            1. Setup separate contexts if required
    4. Implement code splitting + lazy loading for SPA routes

useEffect Rules and Best Practices

UseEffect Dependency Array Rules

1. Every **state variable, prop, and context value** used inside the effect **MUST** be included in the dependency array
2. All **reactive values** must be included! That means any **function** or **variable** that reference **any other** reactie value
    1. Reactive value:
        1. state
        2. prop
        3. context value
        4. any other value that references a reactive value
    2. Examples:

```
const [number, setNumber] = useSate(5);
const [duration, setDuration] = useState(0);
const mins = Math.floor(duration);
const secs = (duration - mins) * 60;

const formatDur = function () {
    return `${min}:${secs} < 10 ? '0' : ''}${secs}`;
};

useEffect(
    function () {
        document.title = `${number}-exercise workout (${formatDur()})`;
    },
    [number, formatDur]
);
```

        1. Stale closures
3. Dependencies choose themselves: **NEVER** ignore the `exhaustive-deps` ESLint rule!
    1. We must include all reactive values in the list
4. Do **NOT** use **objects** or **arrays** as dependencies (objects are recreated on each render, and React sees new objects as **different**, `{} !== {}`)
    1. Effect compares using `===` operator
        1. Objects will have different references each time they are recreated
            1. How to use objects as dependencies?

Removing Unnecessary Dependencies

1. Including too many dependencies can make the effect run too often and cause problems
    1. Solution: Strategies to make some dependencies unnecessary, so we can remove them
        1. Removing function dependencies
            1. Move function **into the effect**
                1. It is no longer a dependency
            2. If you need the function in multiple places, **memoize it** (`useCallback`)
            3. If the function doesn't reference any reactive values, move it **out of the component**
                1. It is not a dependency
                    1. Function is not recreated on every render
2. Removing Object Dependencies
    1. Instead of including the entire object, include **only the properties you need** (primitive values)
    2. If that doesn't work, use the same strategies as for functions (**moving** or **memoizing** object)
3. Other Strategies
    1. If you have **multiple related reactive values** as dependencies, try using a **reducer** (`useReducer`)
        1. Usually used to get rid of problems due to dependencies
    2. You don't need to include `setState` (from `useState`) and `dispatch` (from `useReducer`) in the dependencies, as **React guarantees them to be stable** across renders

When Not to Use an Effect

1. Effects should be used as a **last resort**, when no other solution makes sense. React calls them an "escape hatch" to step outside of React
    1. Three cases where effects are overused:
        1. **Responding to a user event** An event handler function should be used instead
            1. Even if handler has a side effect
        2. **Fetching data on component mount** This is fine in small apps, but in real-world app, a library like React Query should be used.
            1. React Query: Professional data-fetching library
        3. **Synchronizing state changes with one another** (setting state based on another state variable). (multiple re-renders can get created)
            1. Solution: Try to use derived state and event handlers

CHALLENGE #1: Fix Performance Issues in "Workout Timer"

1. `13-workout-timer` - `starter`

Setting State Based on Other State Updates

1. Code:

```
useEffect(
  function () {
    setDuration((number * sets * speed) / 60 + (sets - 1) * durationBreak);
  },
  [number, sets, speed, durationBreak]
);

function handleInc() {
  setDuration((duration) => Math.floor(duration) + 1);
}

function handleDec() {
  setDuration((duration) => (duration > 1 ? Math.floor(duration) - 1 : 0));
}
```

    1. If we have many state variables that influence the value of another variable, we can implement `useEffect`

Using Helper Functions in Effects

1. Code:

```
const playSound = useCallback(
function () {
  if (!allowSound) return;
  const sound = new Audio(clickSound);
  sound.play();
},
[allowSound]
);

useEffect(
  function () {
    setDuration((number * sets * speed) / 60 + (sets - 1) * durationBreak);
    playSound();
  },
  [number, sets, speed, durationBreak, playSound]
);

function handleInc() {
  setDuration((duration) => Math.floor(duration) + 1);
  playSound();
}

function handleDec() {
  setDuration((duration) => (duration > 1 ? Math.floor(duration) - 1 : 0));
  playSound();
}
```

2. Fix:

```
useEffect(
  // One effect for each side-effect
  function () {
    const playSound = function () {
    if (!allowSound) return;
    const sound = new Audio(clickSound);
    sound.play();
  };

  playSound();
  },
  [duration, allowSound]
); // voluntarily declared, even though we are not using the variable

function handleInc() {
  setDuration((duration) => Math.floor(duration) + 1);
}

function handleDec() {
  setDuration((duration) => (duration > 1 ? Math.floor(duration) - 1 : 0));
}
```

Closures in Effects

1. Stale closures
2. Why does `useEffect` needs the dependency array? Why can't it run automatically whenever the dependencies change?
    1. Closures: A function captures all its lexical scope from the place it was defined at the time the function was created.
        1. When a function is created, it closes over the effects of that lexical environment at the time
            1. It will always have access to the variables from the place where they were defined.
        2. Hooks rely on closures
3. Example:
    1. A closure in a function closes over props, and state
        1. If a function is not recreated, it has access to initial snapshot of state and props
    2. If we use empty dependency array, the function will never be recreated, so it will not have access to the new snapshot
        1. If we change state variables, effect is not executed and we don't get fresh values
            1. It is a stale closure
                1. Function has captured a value when the variable value was something else
                    1. Solution: We need to give it in the dependency array

3. Code:

```
useEffect(function () {
  document.title = `Your ${number}-exercise workout`;
}, []);

// Fix

useEffect(function () {
  document.title = `Your ${number}-exercise workout`;
}, [number]);
```

    1. Effect captures the state values when the function is executed. (current snapshot)

Section 20: Redux and Modern Redux Toolkit (With Thunks)

Section Overview

1. Topics:
    1. We'll learn Redux based on `useReducer`
        1. Redux in isolation
    2. **Modern** Redux Toolkit
    3. API requests with **Thunks**
        1. Inside Redux

Introduction to Redux

What is Redux?

1. Redux
    1. 3rd-party library to manage **global state** in a web application
    2. It is a **standalone** library, but easy to integrate with React apps using `react-redux` library
        1. Redux is tightly linked to React, but easy to connect using `react-redux` library
2. Feature
    1. All global state is stored in one **globally accessible store**, which is easy to update using "**actions**" (like `useReducer`)
        1. It implements the same mechanism as `useReducer`
3. Functionality
    1. Global store is updated
        1. All consuming components re-render
    2. It is like combining `useReducer` and context API
        1. Context API might seem like a replacement for redux
4. Two "versions": (1) Classic Redux, (2) Modern Redux Toolkit
    1. Both are compatible
        1. We will learn both

Do You Need to Learn Redux?

1. Historically, Redux was used in most React apps for all global state. Today, that has changed, because there are many alternatives. **Many apps don't need Redux anymore**, unless they need **a lot of global UI state**
    1. Might not need to learn Redux
        1. Reasons it is included in the course
            1. Redux can be hard to learn, and this courses teaches it well (most requested by students)
            2. You will encounter Redux code in your job, so you should understand it
            3. Some apps do require Redux (or a similar library)
2. Use Global State management library only if
    1. There are lots of state that updates frequently (UI State (that which is not fetched from server. No API involved), Global State)
        1. For Remote State & Global State (we have better & sophisticated tools)
            1. React Query
            2. SWR
            3. RTK Query
    2. In real-life, most Global state is Remote state
        1. Hence we don't need Redux anymore

The Mechanism of the `useReducer` Hook

1. Flow:

```
Event Handler in Component
    |
    action
        type = 'deposit'
        payload = 50
    | (object that contains information
    | on how the reducer should update state)
    v
dispatch
    |
    v
reducer (function)
    | take action, payload, current state
    | and calculates the next state
    Next State
    |
    v
Re-render (the component that originated the state transition)
```

2. Redux:

```
Event Handler in Component
    |
```

```
    Action creator function
    |   action
    |           type = 'deposit'
    |           payload = 50
    |   (object that contains information
    |   on how the reducer should update state)
    v
dispatch
    | to the store (not to reducer)
    v
Store
    | All global state lives in this centralized
    | container. It's the **single source of truth**
    | of global state in the app
    | It contains one or more reducers (a pure function)
    Next State
    |
    v
Re-render (the component that originated the state transition)
```

1. Each reducer is a pure function that calculates the next state (state transition) based on the **action** and the **current state**. Usually one reducer per **app feature** (e.g. shopping cart + user data + theme)
2. Why multiple reducers?
    1. One reducer per application feature or per data domain
        1. To keep things separated
            1. Example:
                1. Shopping cart reducer
                2. User data reducer
                3. App color theme reducer
3. Any component that consumes the updated state will get re-rendered by React
4. Action Creator Function: (optional feature of Redux)
    1. Used to automate the process of writing actions
        1. Instead of writing action objects by hand, we construct functions that do them automatically
            1. Advantages:
                1. Keeps all actions in one central place
                    1. Reduces bugs
                        1. Developers don't have to remember action `type` strings

3. Redux cycle
    1. We call action creator
    2. Action creator dispatches an action
    3. The action will reach the store where the correct reducer will pick it up and update the state according to the instructions
    4. Then a re-render of UI is triggered
4. Goal:
    1. Make the state update logic separate from the rest of the application
5. Example:
    1. Deposit into your bank account
        1. We go to person at the desk (dispatcher)
        2. Vault (redux store)

Constructing a Reducer: Bank Account

1. New React app
    1. `npx create-react-app@5 redux-intro`
2. In Redux reducer, we pass the initial state as the default state (to make it the state at the beginning)
3. store.js

```
const initialState = {
  balance: 0,
  loan: 0,
  loanPurpose: "",
};

function reducer(state = initialState, action) {
  switch (action.type) {
    case "account/deposit":
      return { ...state, balance: state.balance + action.payload };
    case "account/withdraw":
      return { ...state, balance: state.balance - action.payload };
    case "account/requestLoan":
      if (state.loan > 0) return state;
      // LATER
      return {
        ...state,
        loan: action.payload,
        balance: state.balance + action.payload,
      };
    case "account/payLoan":
      return {
        ...state,
        loan: 0,
        loanPurpose: "",
        balance: state.balance - state.loan,
      };
    default:
      return state; // For Redux
  }
}
```

Constructing a Redux Store

1. Install: `npm install redux`
2. Code:

```
import { createStore } from "redux"; // not used this way anymore

const initialState = {
  balance: 0,
  loan: 0,
  loanPurpose: "",
};

function reducer(state = initialState, action) {
  switch (action.type) {
    case "account/deposit":
      return { ...state, balance: state.balance + action.payload };
    case "account/withdraw":
      return { ...state, balance: state.balance - action.payload };
    case "account/requestLoan":
      if (state.loan > 0) return state;
      // LATER
      return {
        ...state,
        loan: action.payload.amount,
        loanPurpose: action.payload.purpose,
        balance: state.balance + action.payload.amount,
      };
    case "account/payLoan":
      return {
        ...state,
        loan: 0,
        loanPurpose: "",
        balance: state.balance - state.loan,
      };
    default:
      return state; // For Redux
  }
}

const store = createStore(reducer);

store.dispatch({ type: "account/deposit", payload: 500 });
console.log(store.getState());
store.dispatch({ type: "account/withdraw", payload: 200 });
console.log(store.getState());

store.dispatch({
  type: "account/requestLoan",
  payload: { amount: 1000, purpose: "Buy a car" },
});
console.log(store.getState());

store.dispatch({ type: "account/payLoan" });
console.log(store.getState());
```

Working with Action Creators

1. Action creators are functions that return actions
    1. Redux works even without them
2. Code:

```
function deposit(amount) {
  return { type: "account/deposit", payload: amount };
}

function withdraw(amount) {
  return { type: "account/withdraw", payload: amount };
}

function requestLoan(amount, purpose) {
  return {
    type: "account/requestLoan",
    payload: { amount: amount, purpose: purpose },
  };
}

function payLoan() {
  return { type: "account/payLoan" };
}

store.dispatch(deposit(500));
console.log(store.getState());

store.dispatch(withdraw(200));
console.log(store.getState());

store.dispatch(requestLoan(1000, "Buy a cheap car"));
console.log(store.getState());

store.dispatch(payLoan());
console.log(store.getState());
```

    1. We can have the action types in separate variables and re-use the variables

Adding More State: Customer

1. We need to keep as much logic as possible in the reducer but not side-effects.

2. Code:

```
import { combineReducers, createStore } from "redux"; // not used this way anymore

const initialStateAccount = {
  balance: 0,
  loan: 0,
  loanPurpose: "",
};

const initialStateCustomer = {
  fullName: "",
  nationalID: "",
  createdAt: "",
};

function accountReducer(state = initialStateAccount, action) {
  switch (action.type) {
    case "account/deposit":
      return { ...state, balance: state.balance + action.payload };
    case "account/withdraw":
      return { ...state, balance: state.balance - action.payload };
    case "account/requestLoan":
      if (state.loan > 0) return state;
      // LATER
      return {
        ...state,
        loan: action.payload.amount,
        loanPurpose: action.payload.purpose,
        balance: state.balance + action.payload.amount,
      };
    case "account/payLoan":
      return {
        ...state,
        loan: 0,
        loanPurpose: "",
        balance: state.balance - state.loan,
      };
    default:
      return state; // For Redux
  }
}

function createCustomer(fullName, nationalID) {
  return {
    type: "customer/createCustomer",
    payload: {
      fullName,
      nationalID,
      createdAt: new Date().toISOString(),
    },
  };
}

function updateName(fullName) {
  return { type: "account/updateName", payload: fullName };
}

function customerReducer(state = initialStateCustomer, action) {
  switch (action.type) {
    case "customer/createCustomer":
      return {
        ...state,
        fullName: action.payload.fullName,
        nationalID: action.payload.nationalID,
        createdAt: action.payload.createdAt,
      };
    case "customer/updateName":
      return { ...state, fullName: action.payload };
    default:
      return state;
  }
}

const rootReducer = combineReducers({
  account: accountReducer,
  customer: customerReducer,
}); // names of states
const store = createStore(rootReducer);

store.dispatch({ type: "account/deposit", payload: 500 });
console.log(store.getState());
store.dispatch({ type: "account/withdraw", payload: 200 });
console.log(store.getState());

store.dispatch({
  type: "account/requestLoan",
  payload: { amount: 1000, purpose: "Buy a car" },
});
console.log(store.getState());

store.dispatch({ type: "account/payLoan" });
console.log(store.getState());

function deposit(amount) {
  return { type: "account/deposit", payload: amount };
}

function withdraw(amount) {
  return { type: "account/withdraw", payload: amount };
}
```

```
function requestLoan(amount, purpose) {
  return {
    type: "account/requestLoan",
    payload: { amount: amount, purpose: purpose },
  };
}

function payLoan() {
  return { type: "account/payLoan" };
}

store.dispatch(deposit(500));
console.log(store.getState());

store.dispatch(withdraw(200));
console.log(store.getState());

store.dispatch(requestLoan(1000, "Buy a cheap car"));
console.log(store.getState());

store.dispatch(payLoan());
console.log(store.getState());

store.dispatch(createCustomer("Jonas Schmedtmann", "322323"));
console.log(store.getState());

store.dispatch(deposit(250));
console.log(store.getState());
```

Professional Redux File Structure: State Slices

1. It is not practical to have too many reducers in a single file
    1. Traditionally - too much jumping around
        1. One reducers folder
            1. One file per reducer
        2. One action creators folder
            1. One file per action creator
    2. Modern: Organize by features
        1. Account - folder
            1. Balance
            2. Operations
        2. Customer - folder
            1. Creation
            2. Display
2. Slice:
    1. A piece or part of the total state
        1. Entire state lives in the store
            1. We take one slice of the state

Back to React! Connecting our Redux App with React

1. Inject store into the application
2. Code:

```
import React from "react";
import ReactDOM from "react-dom/client";
import { Provider } from "react-redux";
import "./index.css";
import App from "./App";

import store from "./store"; // simply runs the to level code

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <Provider store={store}>
      <App />
    </Provider>
  </React.StrictMode>
);
```

    1. Each component in the application can read data from the store and dispatch actions to the store.
        1. Similar to context API
            1. Broadcasting global state into every component

3. `useSelector` - we must do as much data manipulation as possible to get the data that we need

    1. `useSelector`: **It defines a subscription to the store**
        1. Whenever the store changes, the component subscribed to the store gets re-rendered.
            1. Redux implements performance optimizations (similar to the ones we used for context api)
                1. If state changes, component re-renders

Dispatching Actions from Our React App

1. Code:

```
import { useState } from "react";
import { useDispatch, useSelector } from "react-redux";
import { deposit, payLoan, requestLoan, withdraw } from "./accountSlice";

function AccountOperations() {
  const [depositAmount, setDepositAmount] = useState("");
  const [withdrawalAmount, setWithdrawalAmount] = useState("");
  const [loanAmount, setLoanAmount] = useState("");
  const [loanPurpose, setLoanPurpose] = useState("");
```

```
  const [currency, setCurrency] = useState("USD");

  const dispatch = useDispatch();
  const {
    loan: currentLoan,
    loanPurpose: currentLoanPurpose,
    balance,
  } = useSelector((store) => store.account);

  function handleDeposit() {
    if (!depositAmount) return;
    dispatch(deposit(depositAmount));
    setDepositAmount("");
  }

  function handleWithdrawal() {
    if (!withdrawalAmount) return;
    dispatch(withdraw(withdrawalAmount));
    setWithdrawalAmount("");
  }

  function handleRequestLoan() {
    if (!loanAmount || !loanPurpose) return;
    dispatch(requestLoan(loanAmount, loanPurpose));
    setLoanAmount("");
    setLoanPurpose("");
  }

  function handlePayLoan() {
    dispatch(payLoan());
  }

  return (
    <div>
      <h2>Your account operations</h2>
      <div className="inputs">
        <div>
          <label>Deposit</label>
          <input
            type="number"
            value={depositAmount}
            onChange={(e) => setDepositAmount(+e.target.value)}
          />
          <select
            value={currency}
            onChange={(e) => setCurrency(e.target.value)}
          >
            <option value="USD">US Dollar</option>
            <option value="EUR">Euro</option>
            <option value="GBP">British Pound</option>
          </select>

          <button onClick={handleDeposit}>Deposit {depositAmount}</button>
        </div>

        <div>
          <label>Withdraw</label>
          <input
            type="number"
            value={withdrawalAmount}
            onChange={(e) => setWithdrawalAmount(+e.target.value)}
          />
          <button onClick={handleWithdrawal}>
            Withdraw {withdrawalAmount}
          </button>
        </div>

        <div>
          <label>Request loan</label>
          <input
            type="number"
            value={loanAmount}
            onChange={(e) => setLoanAmount(+e.target.value)}
            placeholder="Loan amount"
          />
          <input
            value={loanPurpose}
            onChange={(e) => setLoanPurpose(e.target.value)}
            placeholder="Loan purpose"
          />
          <button onClick={handleRequestLoan}>Request loan</button>
        </div>

        {currentLoan > 0 && (
          <div>
            <span>
              Pay back ${currentLoan} ({currentLoanPurpose})
            </span>
            <button onClick={handlePayLoan}>Pay loan</button>
          </div>
        )}
      </div>
    </div>
  );
}

export default AccountOperations;
```

The Legacy Way of Connecting Components to Redux

1. Connect API - Complicated
2. Code:

```
import { connect } from "react-redux";

function formatCurrency(value) {
  return new Intl.NumberFormat("en", {
    style: "currency",
    currency: "USD",
  }).format(value);
}

function BalanceDisplay({ balance }) {
  return <div className="balance">{formatCurrency(balance)}</div>;
}

function mapStateToProps(state) {
  return {
    balance: state.account.balance,
  };
}

export default connect(mapStateToProps)(BalanceDisplay); // returns a component. This was required because there were no hooks, and there wasn't
```

Redux Middleware and Thunks

1. Where to make an **asynchronous API call** (or any other async operation) in Redux?
    1. Store
        1. No asynchronous operations
            1. It only does synchronous operations and updates the state
        2. Reducers need to be pure functions
            1. No side-effects
2. A Flow:
    1. Component -> dispatch -> store
        1. Fetch data in component
        2. Dispatch action to store
    2. Can make asynchronous operations and then disptach
    3. Fetching data in components is not ideal
        1. Reason:
            1. We want to keep components clean and free from data fetching
            2. We want our data-fetching logic encapsulated somewhere and not spread all over the application
3. Where then?
    1. Solution: Middlewhere
        1. A function that sits between dispatching the action and the store.
            1. Allows us to run code **after** dispatching [an action], but **before** [action] reaching the reducer in the store.
                1. We can do something with the action before it gets to the reducer
                    1. Perfect for asynchronous code
                    2. API calls, timers, logging, etc.
                        1. Pausing action
                        2. Cancelling action
                        3. ...
                    3. The place for side effects.
4. Where does middleware come from?
    1. We can write middleware ourselves, but we use a third-party package
        1. Popular middleware - Redux thunk
5. How does thunks work?
    1. Action that is dispatched will first get into the middleware (thunk)
    2. In thunk, we can fetch data, or do some other async operation
    3. Once we receive the data, we populate the action's payload, and dispatch the action to the the store.
        1. Defers dispatching to the future
    4. The state gets immediately updated

Making an API Call with Redux Thunks

1. We deposit money in a foreign currency, which will call an external API to do the conversion.
2. Steps:
    1. We install middleware package
        1. `npm i redux-thunk` **(M)**
        2. `import thunk from 'redux-thunk'`
        3. Add thunk middleware to the store

        ```
        const store = createStore(rootReducer, applyMiddleware(thunk));
        ```
    2. We apply middleware to store
    3. We use middleware in action creator functions
3. In action creator, we return a function (instead of action)
    1. When dispatching, we dispatch a function instead of a value
        1. The function we dispatch is the thunk
    2. If we return a function from action creator, Redux knows that it is the async action that we want to execute before dispatching anything to the store
        1. The function that redux calls internally gets access to `dispatch` function, and the current state through `getState` function
    3. A component doesn't know that there is currency conversion happening behind the scenes. It is all handled in a redux slice (a centralized place for the feature).

The Redux DevTools

1. Installation:

1. Install google chrome extension
    1. Search: Redux DevTools
2. Install npm package
    1. `npm i redux-devtools-extension` **(M)**
3. Import: store.js

```
import { composeWithDevTools } from 'redux-devtools-extension';

// ...

const store = create(rootReducer, composeWithDevTools(applyMiddleware(thunk)));
```

2. Testing:
    1. Dispatch actions
        1. Left side has action dispatched
        2. Right side: Action object
        3. Right side: State from dispatching object
    2. We can jump to different states
        1. The UI reflects the state we jumped to
    3. We can move slider to transition to different states
    4. We can manually dispatch actions
        1. We don't need buttons

What is Redux Toolkit (RTK)?

1. It is the more modern way to write redux

What is Redux Toolkit?

1. Redux Toolkit
    1. The **modern and preferred** way to write Redux code
        1. Redux team recommends this way over classic method
            1. Why?
                1. It is an **opinionated** approach, forcing us to use Redux best practices
                    1. It is built on top of classic redux
        2. 100% compatible with "classic" Redux, allowing us to **use them together**
            1. We can use classic redux in one part
            2. We can use the modern redux in another part
    2. Advantages:
        1. Allows us to write **a lot less code** to achieve the same result (less "bolierplate")
            1. Bolerplate - it only sets things up but doesn't do anything meaningful
                1. Setting up middleware & developer tools
    3. Gives us 3 big things (but there are many more...):
        1. We can write code that "mutates" state inside reducers (will be converted to **immutable** logic behind the scenes by "Immer" library)
            1. Biggest advantage
                1. Reduces complexity if we have a complex state
        2. Action creators are **automatically** created (from reducers)
            1. Helps writing less code
            2. Can result in additional work in some situations
        3. **Automatic** setup of thunk middleware and DevTools

Constructing the Store with RTK

1. Installation:

```
npm i @reduxjs/toolkit // **(M)**
```

    1. `@reduxjs` - namespace
        1. The author of the namespace can publish multiple packages under the namespace
2. Use `configureStore` instead of `createStore`
    1. `configureStore` function wraps around `createStore` and adds a few functionalities.
        1. It does many things automatically for us
            1. Combines reducers
            2. Add thunk middleware
            3. Set up developer tools
3. Code: store.js

```
import accountReducer from "./features/accounts/accountSlice";
import customerReducer from "./features/customers/customerSlice";
import { configureStore } from "@reduxjs/toolkit";

const store = configureStore({
  reducer: {
    account: accountReducer,
    customer: customerReducer,
  },
});

export default store;
```

4. RTK also helps in writing state-slices as well

Constructing the Account Slice

1. code: `accountState.js` slice

```
import { createSlice } from '@reduxjs/toolkit';
```

    1. `createSlice`
        1. Automatically constructs action creators from reducers
        2. It makes writing reducers a lot easier
            1. No switch statement
            2. Default case is automatically handled
        3. We can mutate state inside reducers
            1. The biggest advantage

2. Default action creators accept only one argument which becomes payload

    1. Solution: prepare data before it reaches reducer (accountSlice.js)

```
requestLoan: {
  prepare(amount, purpose) {
    return {
      payload: { amount, purpose }, // New payload for action creator
    };
  },
  reducer(state, action) {
    if (state.loan > 0) return; // we don't have to return state
    state.loan = action.payload.amount;
    state.loanPurpose = action.payload.loanPurpose;
    state.balance += action.payload.amount;
  },
},
```

    2. Solution: Pass all arguments in an object

Back to Thunks

1. `createAsyncThunk` provided by RTK

    1. Extra work

        1. Solution: Re-use action creator function

            1. Thunks are automatically provided in the RTK (no other installation)

            2. Code:

```
const accountSlice = createSlice({
  name: "account",
  initialState,
  reducers: {
    deposit(state, action) {
      state.balance += action.payload; // gets converted to immutable logic
      state.isLoading = false;
    },
    withdraw(state, action) {
      state.balance -= action.payload;
    },
    requestLoan: {
      prepare(amount, purpose) {
        return {
          payload: { amount, purpose }, // New payload for action creator
        };
      },
      reducer(state, action) {
        if (state.loan > 0) return; // we don't have to return state
        state.loan = action.payload.amount;
        state.loanPurpose = action.payload.loanPurpose;
        state.balance += action.payload.amount;
      },
    },
    payLoan(state, action) {
      state.balance -= state.loan;
      state.loan = 0;
      state.loanPurpose = "";
    },
    convertingCurrency(state) {
      state.isLoading = true;
    },
  },
});

export const { withdraw, requestLoan, payLoan } = accountSlice.actions;

// Name of slice/name of reducer convention needs to be followed. Redux figures out that this is the action creator for the redu

export function deposit(amount, currency) {
  if (currency === "USD") return { type: "account/deposit", payload: amount };

  return async function (dispatch, getState) {
    dispatch({ type: "account/convertingCurrency" });
    // API call
    const res = await fetch(
      `https://api.freecurrencyapi.com/v1/latest?apikey=fca_live_AR8p14dOxX9mUbXFGb4J5iOXzZyqwYUOfAqRI5z5&&base_currency=${curre
    );
    const data = await res.json();
    console.log(data);

    const converted = Number(amount) * Number(data.data.USD);
    console.log(converted);
```

```
      // return action
      dispatch({ type: "account/deposit", payload: converted });
    };
  }

  export default accountSlice.reducer;
```

Creating the Customer Slice

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  fullName: "",
  nationalID: "",
  createdAt: "",
};

const customerSlice = createSlice({
  name: "customer",
  initialState,
  reducers: {
    createCustomer: {
      prepare(fullName, nationalID) {
        return { payload: {
          fullName,
          nationalID,
          createdAt: new Date().toISOString(), // side effects can be executed inside a prepare function, but not in a reducer } };
      },
      reducer(state, action) {
        state.fullName = action.payload.fullName;
        state.nationalID = action.payload.nationalID;
        state.createdAt = action.payload.createdAt;
      },
    },
    updateName(state, action) {
      state.fullName = action.payload.fullName;
    },
  },
});

export const { createCustomer, updateName } = customerSlice.actions; // action creators are created and exported

export default customerSlice.reducer;
```

Redux vs. Context API

1. Context API vs Redux
    1. Context API + useReducer - looked at as complete replacement for Redux
        1. True in certain situations, but not in others
2. Differences:
    1. Context API + useReducer
        1. Built into React
        2. Easy to set up a **single context**
            1. Pass state value into context & provide it to app
        3. Additional state "slice" quires new context **set up from scratch** ("provider hell" in App.js)
            1. Provider hell - many context providers in main app component
        4. **No** mechanism for async operations (don't use for remote state)
        5. Performance optimization is **pain**
            1. Requires some work
        6. Only React DevTools
    2. Redux
        1. Requires additional packages (more work, large bundle size)
        2. More work to set up **initially**
        3. Once set up, it's easy to construct **additional state "slices"**
            1. A new slice file & add reducers
                1. No need for another provider and another custom hook
        4. Supports **middleware** for async operations
            1. A mechanism to handle async tasks inside the state management tool (don't use for remote state)
        5. Performance is optimized **out of the box**
            1. Includes minimizing wasted renders
        6. Excellent DevTools

Whent to Use Context AI or Redux?

1. Context API + useReducer
    1. Use the Context API for global state management in **small** apps
    2. When you just need to share a value that **doesn't change often** [Color theme, perferred language, authenticated user, ...]
        1. Reason? No need to optimize context in this situation
    3. When you need to solve a simple **prop drilling** problem
    4. When you need to manage state in a **local sub-tree** of the app
        1. State that is global to a sub-part of the app (not to the entire app)
            1. Example: In compound component pattern
2. Redux
    1. Use Redux for global state management in **large** apps
    2. When you have lots of global UI state that needs to be **updated frequently** (because Redux is optimized for this) [Shopping cart, current tabs, complex filters or search, ...]
        1. Reason? Redux is heavily optimized for frequent updates (not very common)
    3. When you have **complex state** with nested objects and arrays (because you can mutate state with Redux Toolkit) (not very common)
3. There is no right answer that fits every projec. It all depends on the project needs

Section 21: Part 4: PROFESSIONAL REACT DEVELOPMENT [2 PROJECTS]

Introduction to Part 4

1. Significant shift
    1. Leverage React + huge library ecosystem to develop two full-blown professional web-applications.
        1. React Query
        2. Supabase
        3. Tailwind
    2. Features:
        1. Authentication
        2. Data filtering & pagination
        3. Beautiful charts
        4. Darkmode
        5. Context menus and modals
        6. ...
2. Goal:
    1. To empower to build high quality apps completely on our own

Useful Resources for Part 4

1. Resources to refer to while studying part 4 (for additional insights and deeper divers)
    1. React Libraries for 2023
        1. Written in 2023 (will be up to date)
    2. Styled-components best practices
        1. Josh W. Comeau
    3. A Thorough Analysis of CSS-in-JS
    4. Practical React Query
        1. Series from React Query's maintainer
        2. Initiated in 2020
    5. React Query meets React Router
    6. Picking the right React compoent pattern
    7. Bulletproof-react: A simple, scalable, and powerful architecture for building production-ready React applications
2. Library documentation
    1. Tailwind CSS: Installation with Vite
    2. styled-components
    3. Supabase Javascript Client Library
    4. TanStack (React) Query v4
    5. Recharts
    6. date-fns

Section 22: React Router with Data Loading (v6.4+)

Section Overview

1. Topics
    1. React Router's modern **data loading** capabilities
    2. How to **plan** a professional project
        1. That's what this course is all about

Setting Up a New Project: "Fast React Pizza Co."

1. Using Vite
2. New project
    1. `npm create vite@4` **(M)**
        1. Project name: fast-react-pizza
            1. React
            2. JavaScript
    2. `npm install`
    3. `npm i eslint vite-plugin-eslint eslint-config-react-app --save-dev`
    4. `.eslintrc.json`
        1. We want to use rules for react apps
        2. Code

```
{
  "extends": "react-app"
}
```

    5. `vite.config.js`

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import eslint from "vite-plugin-eslint";

// https://vitejs.dev/config/
export default defineConfig({
  plugins: [react(), eslint()],
});
```

        1. We need to add plugin
3. Delete

1. App.css
2. index.css
3. Assets
4. content of App.jsx

```
function App() {
    return <div>Hello Vite!</div>;
}

export default App;
```

Application Planning

1. How to plan a professional React application step-by-step

The Project: Fast React Pizza Co

1. Fast React Pizza Co
    1. Now the same restaurant (business) needs a simple way of allowing customers to **order pizzas and get them delivered** to their home
    2. We were hired to build the application front-end

How to Plan and Build a React Application

1. From the earlier "thinking in React" lecture:
    1. Break the desired UI into **components**
    2. Build a **static** version (no state yet)
    3. Think about **state management + data flow**
2. This works well for small apps with **one page and a few features**
3. In **real-world apps**, we need to adapt this process
4. Steps:
    1. Gather application **requirements and features**
    2. Divide the application into **pages**
        1. Think about the **overall** and **page-level** UI
        2. Break the desired UI into **components** (from earlier)
        3. Design and build a **static** version (no state yet) (from earlier)
            1. Optional (but acts as a guideline)
    3. Divide the application into **feature categories**
        1. Place features into a feature categories
            1. To organize code in a logical way
        2. Think about **state management + data flow** (from earlier)
            1. For each feature category
                1. What data it needs
                2. Where do we store the data
    4. Decide on what **libraries** to use (technology decisions)
5. The above is just a rough overview. In the real-world, things are never this linear

Project Requirements from the Business

1. Very simple application, where users can order **one or more pizzas from a menu**
    1. We need to start by gathering a list of requirements like this
2. Requires **no user accounts** and no login: users just input their names before using the app
3. The pizza menu can change, so it should be **loaded from an API** (already done)
4. Users can add multiple pizzas to a **cart** before ordering
5. Ordering requires just the **user's name**, **phone number**, and **address**
6. If possible, **GPS location** should also be provided, to make delivery easier
7. User's can **mark their order as "priority"** for an additional 20% of the cart price
8. Orders are made by **sending a POST request** with the order data (user data + selected pizzas) to the API
9. Payments are made on delivery, so **no payment processing** is necessary in the app
10. Each order will get a **unique ID** that should be displayed, so the **user can later look up their order** based on the ID
11. Users should be able to mark their order as "priority" order **even after it has been placed**
    1. From the requiremets, we can understand the featurs we need to implement

Features + Pages

1. Feature categories (extract from requirements)
    1. User (everything related to user)
    2. Menu (everything realted to loading and displaying menu)
    3. Cart (user will be able to take one or more items from menu and add them to cart and also update quantities)
    4. Order (Many features are related to this topic. Making an order, fetching orders, ...)
2. All features can be placed into one of the categories
    1. So this is what the app will essentially be about
3. From feature categoies, we can come up with necessary pages to implement
    1. Homepage (/)
        1. User connects to this
            1. User will input name
    2. Pizza menu (/menu)
        1. Menu
    3. Cart (/cart)
        1. Cart
    4. Placing a new order (/order/new) (or /newOrder)
        1. Order
    5. Looking up an order (/order/:orderID)
        1. Order

4. Feature categories helps us understand what pages we need in the app

State Management + Technology Decisions

1. State "Domain" / "Slices" (term is from Redux)
    1. They usually map quite nicely to the app features
        1. User slice
        2. Menu slice
        3. Cart slice
        4. Order slice
2. Type of State (classification)
    1. User slice - Global UI state (no accounts, so stays in app, needs to be accessible in many different components in the tree) (technology depends on this)
    2. Menu - Global remote state (menu is fetched from API)
    3. Cart - Global UI state (no need for API, just stored in app)
        1. Not stored in remote DB
    4. Order - Global remote state (fetched and submitted to API)
        1. They live on the server
3. Technology stack
    1. Routing - React Router (The standard for React SPAs)
    2. Styling - TailwindCSS (Trendy way of styling applications that we want to learn)
        1. Trendy
    3. Remote state management: React Router
        1. New way of fetching data right inside React Router (v6.4+) that is worth exploring ("render-as-you-fetch" instead of "fetch-on-render"). Not really state management, as it doesn't persist state (in a place that is easy to access from anywhere)
        2. Nice way to work
    4. UI State management: Redux
        1. State is fairly complex.
        2. Redux has many advantages for UI state.
        3. Also, we want to practice Redux a bit more.
4. The above is just one of many tech stacks

Setting Up a Professional File Structure

1. Structure
    1. Feature base structure
        1. `src/features` folder
            1. One folder for each feature categories
                1. Contain all JS files necessary to make each feature work
        2. `src/ui` folder
            1. Re-usable UI components
                1. Buttons, inputs, ...
                    1. Presentational & no side-effects
        3. `src/services` folder
            1. Re-usable code for interacting with an API
        4. `src/utils` folder
            1. Helper functions to re-use in multiple places of the application
                1. Stateless - no side-effects
                    1. Date manipulation
                    2. Number manipulation
        5. For complex applications
            1. `src/contexts` - re-usable contexts
            2. `src/hooks` - re-usable hooks
            3. `src/pages` - re-usable pages
        6. Copy files to feature categories

A New Way of Implementing Routes

1. React v6.4 introduced a new way to define routes (powerful for fetching data into pages, and submitting data from forms)

2. Install React router

```
npm i react-router-dom@6
```

    1. Open documentation: https://reactrouter.com/en/main
        1. Routers/Picking a Router
        2. Routers/createBrowserRouter

3. New APIs we can use:

    1. Data Loaders
    2. Data Actions
    3. Data Fetchers

Building the App Layout

1. Header & Overview (how many items are in the cart, and a link)

    1. The other parts change

2. Code: AppLayout.jsx

```
import { Outlet } from "react-router-dom";
import CartOverview from "../features/cart/CartOverview";
import Header from "./Header";

function AppLayout() {
  return (
    <div>
```

```
        <Header />
        <main>
          <Outlet />
        </main>
        <CartOverview />
      </div>
    );
  }

  export default AppLayout;
```

Fetching Data with React Router "Loaders": Pizza Menu

1. Loaders - powerful data loading feature
    1. If there is a function that fetches data from an API, a loader runs the function to fetch the data as soon as we navigate to the route
        1. The data is then provided to the page component using a custom hook
2. Steps:
    1. Construct loader
        1. It is conventional to keep the loader inside the page which needs the data
    2. Provide loader
    3. Provide data to the page
        1. Using custom hook
3. React Router fetches data as it starts rendering the current route
    1. Before, we used fetch on render (after we render the component)
4. Code:

Displaying a Loading Indicator

1. `useNavigation` - Helps us know whether application (entire application - if one of the pages is in a given state) is
    1. Loading
    2. Idle
    3. Submitting

Handling Errors with Error Elements

1. Whenever an error is thrown in a loader, action, or when rendering a component, we can render error element (instead of actual pages)
2. Define error element in the parent route, because errors bubble up to the parent route
3. We can get error message using a custom hook
4. Code:

```
import { createBrowserRouter, RouterProvider } from "react-router-dom";
import Home from "./ui/Home";
import Error from "./ui/Error";
import Menu, { loader as menuLoader } from "./features/menu/Menu";
import Cart from "./features/cart/Cart";
import CreateOrder from "./features/order/CreateOrder";
import Order from "./features/order/Order";
import AppLayout from "./ui/AppLayout";

const router = createBrowserRouter([
  {
    element: <AppLayout />, // `path` is not required, because it effectively becomes layout route. It's purpose is to provide layout to the app
    errorElement: <Error />,
    children: [
      {
        path: "/",
        element: <Home />,
      },
      {
        path: "/menu",
        element: <Menu />,
        loader: menuLoader,
        errorElement: <Error />,
      },
      {
        path: "/cart",
        element: <Cart />,
      },
      {
        path: "/order/new",
        element: <CreateOrder />,
      },
      {
        path: "/order/:orderId",
        element: <Order />,
      },
    ],
  },
]);

function App() {
  return <RouterProvider router={router} />;
}

export default App;
```

Fetching Orders

1. Loading individual orders (based on id)
2. We want a search field

Writing Data with React Router "Actions"

1. Actions are used to write data, or mutate data
    1. Actions allows us to manage remote server state
        1. Using action functions & forms
            1. No boilerplate code
                1. No state variables
                2. No handling requests
                3. No preventing defaults
                4. ...

2. Code: CreateOrder.jsx

```
// import { useState } from "react";

import { Form, redirect } from "react-router-dom";
import { createOrder } from "../../services/apiRestaurant";

// https://uibakery.io/regex-library/phone-number
// const isValidPhone = (str) =>
//   /^\+?\d{1,4}?[-.\s]?\(?\d{1,3}?\)?[-.\s]?\d{1,4}[-.\s]?\d{1,4}[-.\s]?\d{1,9}$/.test(
//     str
//   );

const fakeCart = [
  {
    pizzaId: 12,
    name: "Mediterranean",
    quantity: 2,
    unitPrice: 16,
    totalPrice: 32,
  },
  {
    pizzaId: 6,
    name: "Vegetale",
    quantity: 1,
    unitPrice: 13,
    totalPrice: 13,
  },
  {
    pizzaId: 11,
    name: "Spinach and Mushroom",
    quantity: 1,
    unitPrice: 15,
    totalPrice: 15,
  },
];

function CreateOrder() {
  // const [withPriority, setWithPriority] = useState(false);
  const cart = fakeCart;

  return (
    <div>
      <h2>Ready to order? Let&apos;s go!</h2>

      {/* <Form method="POST" action="/order/new"> */
      /* chooses the closes route */}
      <Form method="POST">
        <div>
          <label>First Name</label>
          <input type="text" name="customer" required />
        </div>

        <div>
          <label>Phone number</label>
          <div>
            <input type="tel" name="phone" required />
          </div>
        </div>

        <div>
          <label>Address</label>
          <div>
            <input type="text" name="address" required />
          </div>
        </div>

        <div>
          <input
            type="checkbox"
            name="priority"
            id="priority"
            // value={withPriority}
            // onChange={(e) => setWithPriority(e.target.checked)}
          />
          <label htmlFor="priority">Want to yo give your order priority?</label>
        </div>

        <div>
          <input type="hidden" name="cart" value={JSON.stringify(cart)} />
          <button>Order now</button>
        </div>
      </Form>
    </div>
  );
}

export async function action({ request }) {
  // Intercepts request created when the form is submitted (if it is connected with react router)
  const formData = await request.formData();
  const data = Object.fromEntries(formData); // transforms a list of key/value pairs into an object
  // console.log(data);
```

```
  /*
  const entries = new Map([
    ['foo', 'bar'],
    ['baz', 42],
  ]);

  const obj = Object.fromEntries(entries);

  console.log(obj);
  // Expected output: Object { foo: "bar", "baz": 42 }
  */

  const order = {
    ...data,
    cart: JSON.parse(data.cart),
    priority: data.priority === "on",
  };

  // console.log(order);

  const newOrder = await createOrder(order);

  return redirect(`/order/${newOrder.id}`); // we cannot use hooks, because hooks work only in components. This function returns a new request /
}

export default CreateOrder;
```

3. Code: App.jsx

```
import { createBrowserRouter, RouterProvider } from "react-router-dom";
import Home from "./ui/Home";
import Error from "./ui/Error";
import Menu, { loader as menuLoader } from "./features/menu/Menu";
import Cart from "./features/cart/Cart";
import CreateOrder, {
  action as createOrderAction,
} from "./features/order/CreateOrder";
import Order, { loader as orderLoader } from "./features/order/Order";
import AppLayout from "./ui/AppLayout";

const router = createBrowserRouter([
  {
    element: <AppLayout />, // `path` is not required, because it effectively becomes layout route. It's purpose is to provide layout to the app
    errorElement: <Error />,
    children: [
      {
        path: "/",
        element: <Home />,
      },
      {
        path: "/menu",
        element: <Menu />,
        loader: menuLoader,
        errorElement: <Error />,
      },
      {
        path: "/cart",
        element: <Cart />,
      },
      {
        path: "/order/new",
        element: <CreateOrder />,
        action: createOrderAction,
      },
      {
        path: "/order/:orderId",
        element: <Order />,
        loader: orderLoader,
        errorElement: <Error />,
      },
    ],
  },
]);

function App() {
  return <RouterProvider router={router} />;
}

export default App;
```

Error Handling in Form Actions

1. Code: CreateOrder.jsx

```
// import { useState } from "react";

import { Form, redirect, useActionData, useNavigation } from "react-router-dom";
import { createOrder } from "../../services/apiRestaurant";

// https://uibakery.io/regex-library/phone-number
const isValidPhone = (str) =>
  /^\+?\d{1,4}?[-.\s]?\(?\d{1,3}?\)?[-.\s]?\d{1,4}[-.\s]?\d{1,4}[-.\s]?\d{1,9}$/.test(
    str
  );

const fakeCart = [
  {
    pizzaId: 12,
    name: "Mediterranean",
    quantity: 2,
    unitPrice: 16,
    totalPrice: 32,
```

```
    },
    {
      pizzaId: 6,
      name: "Vegetale",
      quantity: 1,
      unitPrice: 13,
      totalPrice: 13,
    },
    {
      pizzaId: 11,
      name: "Spinach and Mushroom",
      quantity: 1,
      unitPrice: 15,
      totalPrice: 15,
    },
  ];

  function CreateOrder() {
    // const [withPriority, setWithPriority] = useState(false);
    const cart = fakeCart;
    const navigation = useNavigation();
    const isSubmitting = navigation.state === "submitting";

    const formErrors = useActionData(); // Gives access to data returned from the action

    return (
      <div>
        <h2>Ready to order? Let&apos;s go!</h2>

        {/* <Form method="POST" action="/order/new"> */
        /* chooses the closes route */}
        <Form method="POST">
          <div>
            <label>First Name</label>
            <input type="text" name="customer" required />
          </div>

          <div>
            <label>Phone number</label>
            <div>
              <input type="tel" name="phone" required />
            </div>
            {formErrors?.phone && <p>{formErrors.phone}</p>}
          </div>

          <div>
            <label>Address</label>
            <div>
              <input type="text" name="address" required />{" "}
              {/* Used to ensure that form is not submitting without writing any JS */}
            </div>
          </div>

          <div>
            <input
              type="checkbox"
              name="priority"
              id="priority"
              // value={withPriority}
              // onChange={(e) => setWithPriority(e.target.checked)}
            />
            <label htmlFor="priority">Want to yo give your order priority?</label>
          </div>

          <div>
            <input type="hidden" name="cart" value={JSON.stringify(cart)} />
            <button disabled={isSubmitting}>
              {isSubmitting ? "Placing order..." : "Order now"}
            </button>
          </div>
        </Form>
      </div>
    );
  }

  export async function action({ request }) {
    // Intercepts request created when the form is submitted (if it is connected with react router)
    const formData = await request.formData();
    const data = Object.fromEntries(formData); // transforms a list of key/value pairs into an object
    // console.log(data);

    /*
    const entries = new Map([
      ['foo', 'bar'],
      ['baz', 42],
    ]);

    const obj = Object.fromEntries(entries);

    console.log(obj);
    // Expected output: Object { foo: "bar", "baz": 42 }
    */

    const order = {
      ...data,
      cart: JSON.parse(data.cart),
      priority: data.priority === "on",
    };

    const errors = {};
    if (!isValidPhone(order.phone))
      errors.phone =
        "Please give us your correct phone number. We might need it to contact you.";
```

```
  if (Object.keys(errors).length > 0) {
    return errors;
  }

  // console.log(order);

  const newOrder = await createOrder(order);

  return redirect(`/order/${newOrder.id}`); // we cannot use hooks, because hooks work only in components. This function returns a new request /
}

export default CreateOrder;
```

Section 23: [Optional] Tailwind CSS Crash Course: Styling the App

Section Overview

1. Topics:
    1. A complete Tailwind CSS **crash course**
    2. Super **popular** technology
        1. Especially with React
    3. You can **skip** if you're not interested in Tailwind...

What is Tailwind CSS?

1. Tailwind CSS:
    1. A utility-first framework packed with utility classes like `flex`, `text-center`, and `rotate-90` that can be composed to build any design, directly in your markup (HTML or JSX)
        1. **Utility-first CSS approach**:
            1. Writing tiny classes with one single purpose, and then combining them to build entire components, and layouts
                1. Atomic-CSS - another name
            2. Tailwind made it popular
    2. In tailwind, **the classes are already written for us**. So we're not gonna write any new CSS, but instead use some of tailwind's hundreds of classes (to design and build anything)

The Good and Bad About Tailwind

1. The good
    1. **You don't need to think about class names**
        1. We can just focus on writing styles instead of coming up with class-names
    2. **No jumping between files to write markup and styles**
        1. We do it at the same time, and in one place
    3. We will immediately understand styling in any project that uses tailwind
        1. Easy to collaborate
        2. Can understand own projects after months or years
    4. Tailwind is a design system:
        1. Many design decisions have been taken for you, which makes UIs look beter and more consistent (makes it easier)
            1. Saves a lot of time, e.g. on responsive design (fast and easy to implement)
    5. Docs and VS Code integration are great
        1. Auto-completion in VS Code
2. The bad
    1. Markup (HTML or JSX) looks very unreadable
        1. with lots of class names (you get used to it)
            1. Makes the markup look cluttered
    2. You have to learn a lot of class names (but after a day of usage you know fundamentals)
        1. But they are well-named and intuitive
    3. You need to install and set up tailwind on each new project
        1. It takes 5-10 minutes
    4. Fealing that we're giving up on "vanilla CSS"
        1. It is a thin layer of abstraction
            1. It doesn't makes us feel that we're completely removed from CSS

Setting Up Tailwind CSS

1. Search: Tailwind CSS documentation (on Google)
    1. Getting Started
        1. Installation
            1. v3.x
            2. Framework Guides
                1. Vite (or search: tailwind install vite)

                ```
                npm install -D tailwindcss@3 postcss autoprefixer

                npx tailwindcss init -p # tailwind & postcss config files
                ```

                2. Add content: tailwind.config.js

                ```
                content: [
                    "./index.html",
                    "./src/**/*.{js,ts,jsx,tsx}",
                ]
                ```

                3. index.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

1. Preflight - documentation (base styles)

4. home.jsx

```
<h1 className="text-xl text-yellow-500 font-semibold">
```

2. VS Code
    1. Tailwind CSS Intellisense
    2. Tailwind prettier extension (search in Google)
        1. Tailwindlabs github

        ```
        npm install -D prettier prettier-plugin-tailwindcss # orders classes as recommended
        ```

        2. Prettier config: `prettier.config.cjs`

        ```
        module.exports = {
            plugins: [require('prettier-plugin-tailwindcss')],
            singleQuote: true,
        };

        // OR

        // .prettierrc
        {
          "plugins": ["prettier-plugin-tailwindcss"]
        }
        ```

Working with Color

1. Tailwind docs
    1. Customization
        1. Colors
            1. Color palettes are predefined
                1. Select one grey
                2. Select base color
                    1. `text-yellow-500` - maps to color
                        1. 500 - shade (50 - 950)
                3. Search: `text` (ctrl + k)
            2. Background color: `bg-<name>-<intensity>`

Styling Text

1. We can use tailwind classes inside HTML as well.

The Box Model: Spacing, Borders, and Display

1. Spacing
    1. Margin
    2. Padding
    3. Box-model
        1. Borders
        2. Display
2. Adding spacing between elements
    1. On parent: `space-x-300`

Responsive Design

1. Min-width media queries - mobile-first approach (recommended for tailwind)
    1. Default classes are mobile-first classes
        1. `sm:px-6` - value starts getting applied when the small screen size ends
            1. We can customize screen sizes
                1. Helps change the design at the point when design starts looking wierd.

Using Flexbox

1. Make a flex container, align items.
2. Classes

```
flex
text-align
justify-between
justify-center
```

Using CSS Grid

1. Grid:

```
grid
grid-rows-2
grid-rows-[auto_1fr_auto]
gap-y-4
h-screen
```

Styling Buttons: Element States and Transitions

1. Code: CreateOrder

```
<button ...
    className="inline-block rounded-full bg-yellow-400 px-4 py-3 font-semibold uppercase tracking-wide text-stone-800 transition-colors duration
>
```

Styling Form Elements

1. Code:

```
<div>
  <label>Address</label>
  <div>
    <input
      className="w-full rounded-full border border-stone-200 px-4 py-2 text-sm transition-all duration-300 placeholder:text-stone-400 focus:outl
      type="text"
      name="address"
      required
    />{' '}
    {/* Used to ensure that form is not submitting without writing any JS */}
  </div>
</div>

<div>
  <input
    className="h-6 w-6 accent-yellow-400 focus:outline-none focus:ring focus:ring-yellow-400 focus:ring-offset-2"
    type="checkbox"
    name="priority"
    id="priority"
    // value={withPriority}
    // onChange={(e) => setWithPriority(e.target.checked)}
  />
  <label htmlFor="priority">Want to yo give your order priority?</label>
</div>
```

Reusing Styles with @apply

1. We can construct an old-school class by composing many tailwind classes together
2. Composition should not be the norm
    1. We will end up writing classes in the old-school way
        1. Use-case: Only if there are too many classes and we do not want to construct a new component
            1. It is better to construct a React component instead (which can be re-used)
3. Code:

```
@layer components {
  .input {
    @apply w-full rounded-full border border-stone-200 px-4 py-2 text-sm transition-all duration-300 placeholder:text-stone-400 focus:outline-no
  }
}

// ...
<input className="input" type="text" name="customer" required />
```

Reusing Styles with React Components

1. New components can embed tailwind styles, which enables us to re-use the components, and their styles.

Absolute Positioning, z-index, and More

1. Code: Loader

```
function Loader() {
  return (
    <div className="absolute inset-0 flex items-center justify-center bg-slate-200/20 backdrop-blur-sm">
      {/* `inset-0` top, left, right, bottom = 0 */}
      <div className="loader"></div>
    </div>
  );
}

export default Loader;
```

2. Code: index.css

```
@layer components {
  .input {
    @apply w-full rounded-full border border-stone-200 px-4 py-2 text-sm transition-all duration-300 placeholder:text-stone-400 focus:outline-no
  }

  /* https://dev.to/afif/i-made-100-css-loaders-for-your-next-project-4eje */
  .loader {
    width: 45px;
    aspect-ratio: 0.75;
    --c: no-repeat linear-gradient(theme(colors.stone.800) 0 0);
    background:
      var(--c) 0% 50%,
      var(--c) 50% 50%,
      var(--c) 100% 50%;
    background-size: 20% 50%;
    animation: loading 1s infinite linear;
  }
```

```
      @keyframes loading {
        20% {
          background-position:
            0% 0%,
            50% 50%,
            100% 50%;
        }
        40% {
          background-position:
            0% 100%,
            50% 0%,
            100% 50%;
        }
        60% {
          background-position:
            0% 50%,
            50% 100%,
            100% 0%;
        }
        80% {
          background-position:
            0% 50%,
            50% 50%,
            100% 100%;
        }
      }
    }
```

Configuring Tailwind: Custom Font Family

1. tailwind.config.js

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],
  theme: {
    // Overrides everything
    fontFamily: {
      sans: 'Roboto Mono, monospace',
    },
    extend: {
      // Adds to texisting properties
      colors: {
        pizza: '#123456',
      },
      fontSize: {
        huge: ['80rem', { lineHeight: '1' }],
      },
      height: {
        screen: '100dvh', // dynamic viewport height units
      },
    },
  },
  plugins: [],
};
```

Styling the Menu

Styling the Cart

Styling the Order Form

1. Don't immediately go for a new component, if we need to re-use styles
2. Never use `width` when using `flex` property

Styling the Order Overview

Section 24: Adding Redux and Advanced React Router

Section Overview

1. Topics:
    1. Implementing a **shopping cart**
    2. **Advanced** parts of React Router
        1. Data loading
    3. **Real-world** use cases of Redux
        1. Pre-requisite: Redux

Modeling the "User" State with Redux Toolkit

1. We can use Redux for global UI state
2. Code: userSlice.js

```
import { createSlice } from '@reduxjs/toolkit';

const initialState = {
  username: '',
};

const userSlice = createSlice({
  name: 'user',
  initialState,
  reducers: {
    updateName(state, action) {
```

```
      state.username = action.payload;
    },
  },
});

export const { updateName } = userSlice.actions;

export default userSlice.reducer;
```

3. Code: UserName.jsx

```
import { useSelector } from 'react-redux';

function UserName() {
  const username = useSelector((state) => state.user.username);

  if (!username) return null;

  return (
    <div className="hidden text-sm font-semibold md:block">{username}</div>
  );
}

export default UserName;
```

Reading and Updating the User State

1. Always try to derive state from other states.

Modeling the "Cart" State

Adding Menu Items to the Cart

Building the Cart Overview with Redux Selectors

Building the Cart Page

Deleting Cart Items

Updating Cart Quantities

1. Calling a reducer from another reducer:

```
decreaseItemQuantity(state, action) {
  const item = state.cart.find((item) => item.pizzaId === action.payload);
  item.quantity--;
  item.totalPrice = item.quantity * item.unitPrice;

  if (item.quantity === 0) cartSlice.caseReducers.deleteItem(state, action);
}
```

Using the Cart for New Orders

1. To dispatch actions in a normal function (not a component)

```
store.dispatch(clearCart);
```

Redux Thunks with createAsyncThunk

1. `createAsyncThunk` function
    1. To use geolocation - to get GPS position and address
    2. It is RTK native way of using Thunks
2. Code: userSlice.js (Thunks)

```
import { createAsyncThunk, createSlice } from '@reduxjs/toolkit';
import { getAddress } from '../../services/apiGeocoding';

function getPosition() {
  return new Promise(function (resolve, reject) {
    navigator.geolocation.getCurrentPosition(resolve, reject);
  });
}

// async function fetchAddress() {
//   // 1) We get the user's geolocation position
//   const positionObj = await getPosition();
//   const position = {
//     latitude: positionObj.coords.latitude,
//     longitude: positionObj.coords.longitude,
//   };

//   // 2) Then we use a reverse geocoding API to get a description of the user's address, so we can display it the order form, so that the user
//   const addressObj = await getAddress(position);
//   const address = `${addressObj?.locality}, ${addressObj?.city} ${addressObj?.postcode}, ${addressObj?.countryName}`;

//   // 3) Then we return an object with the data that we are interested in
//   return { position, address };
// }

const initialState = {
  username: '',
  status: 'idle',
  position: {},
```

```
  address: '',
  error: '',
};

const userSlice = createSlice({
  name: 'user',
  initialState,
  reducers: {
    updateName(state, action) {
      state.username = action.payload;
    },
  },
  extraReducers: (builder) =>
    builder
      .addCase(fetchAddress.pending, (state) => (state.status = 'loading'))
      .addCase(fetchAddress.fulfilled, (state, action) => {
        state.position = action.payload.position;
        state.address = action.payload.address;
        state.status = 'idle';
      })
      .addCase(fetchAddress.rejected, (state, action) => {
        state.status = 'error';
        state.error = action.error.message; // gets automatically placed
      }),
});

// RTK way of constructing thunk
// 'user/fetchAddress' - action
// second arg: Promise, the code executed as soon as the action is dispatched
// The function produces 3 action states
// 1 - Pending Promise state
// 2 - Fulfilled Promise state
// 3 - Rejected Promise state
export const fetchAddress = createAsyncThunk(
  'user/fetchAddress',
  async function () {
    // 1) We get the user's geolocation position
    const positionObj = await getPosition();
    const position = {
      latitude: positionObj.coords.latitude,
      longitude: positionObj.coords.longitude,
    };

    // 2) Then we use a reverse geocoding API to get a description of the user's address, so we can display it the order form, so that the user
    const addressObj = await getAddress(position);
    const address = `${addressObj?.locality}, ${addressObj?.city} ${addressObj?.postcode}, ${addressObj?.countryName}`;

    // 3) Then we return an object with the data that we are interested in
    // Payload of the FULFILLED state
    return { position, address };
  },
);

export const { updateName } = userSlice.actions;

export default userSlice.reducer;
```

Integrating Geolocation

Fetching Data Without Navigation: useFetcher

1. Fetching data without moving to another page
    1. Data from another route (without causing navigation)
        1. Example: Loading menu data in order page

            `useFetcher()` # returns a fetcher

2. Code: Order.jsx

```
const fetcher = useFetcher();
useEffect(
  function () {
    if (!fetcher.data && fetcher.state === 'idle') fetcher.load('/menu'); // `fetcher` can be in different states. Default state: `idle`
  },
  [fetcher],
);
```

    1. We might use this hook quite often if we need data from a different page

Updating Data Without Navigation

1. Example: Updating priority after the order is placed.
2. Revalidation:
    1. It means, if data has changed as a result of an action it will re-fetch the data in the background, and re-render the page with the new data
        1. `fetcher.Form` - we can update data without a navigation
3. Code: UpdateOrder.jsx

```
import { useFetcher } from 'react-router-dom';
import Button from '../../ui/Button';
import PropTypes from 'prop-types';
import { updateOrder } from '../../services/apiRestaurant';

function UpdateOrder({ order }) {
  const fetcher = useFetcher();
```

```
  console.log(order);
  return (
    <fetcher.Form method="PATCH" className="text-right">
      {' '}
      {/* will not navigate away, it submits form, and re-validate the page */}
      <Button type="primary">Make priority</Button>
    </fetcher.Form>
  );
}

UpdateOrder.propTypes = {
  order: PropTypes.object.isRequired,
};

export async function action({ request, params }) {
  console.log('update');
  console.log(request);
  console.log(params);

  const data = { priority: true };

  await updateOrder(params.orderId, data);
  return null;
}

export default UpdateOrder;
```

4. Code: App.jsx

```
import { createBrowserRouter, RouterProvider } from 'react-router-dom';
import Home from './ui/Home';
import Error from './ui/Error';
import Menu, { loader as menuLoader } from './features/menu/Menu';
import Cart from './features/cart/Cart';
import CreateOrder, {
  action as createOrderAction,
} from './features/order/CreateOrder';
import Order, { loader as orderLoader } from './features/order/Order';
import AppLayout from './ui/AppLayout';
import { action as updateOrderAction } from './features/order/UpdateOrder';

const router = createBrowserRouter([
  {
    element: <AppLayout />, // `path` is not required, because it effectively becomes layout route. It's purpose is to provide layout to the app
    errorElement: <Error />,
    children: [
      {
        path: '/',
        element: <Home />,
      },
      {
        path: '/menu',
        element: <Menu />,
        loader: menuLoader,
        errorElement: <Error />,
      },
      {
        path: '/cart',
        element: <Cart />,
      },
      {
        path: '/order/new',
        element: <CreateOrder />,
        action: createOrderAction,
      },
      {
        path: '/order/:orderId',
        element: <Order />,
        loader: orderLoader,
        errorElement: <Error />,
        action: updateOrderAction,
      },
    ],
  },
]);

function App() {
  return <RouterProvider router={router} />;
}

export default App;
```

Section 25: Setting Up Our Biggest Project + Styled Components

Section Overview

1. Topics
    1. Plan and build this **stunning** application
        1. Using many professional libraries from React ecosystem
            1. React Query
            2. React forms
            3. React Router
            4. Backend (with database)
                1. Using Supabase
    2. Styling with **Styled Components** in this section

Application Planning

The Project: The Wild Oasis

1. Requirement
    1. The Wild Oasis is a small boutique **hotel** with 8 luxurious wooden cabins
    2. They need a custom-built application to manage everything about the hotel: **bookings**, **cabins**, and **guests**
    3. This is the **internal application** used inside the hotel to **check in guests as they arrive**
    4. They have nothing right now, so they **also need the API**
        1. We need to model the data & store it
        2. Application functions as a backoffice that manages the data and API
            1. API -> Internal Hotel Management App
            2. API -> Customer-facing Website to Book Stays on their own
    5. Later they will probably want a **customer-facing** website as well, where customers will be able to **book stays**, using the same API

Review: How to Plan a React Application

1. Gather application **requirements and features**
2. Divide the application into **pages**
    1. Think about UI
    2. How we divide into components
3. Divide the application into **feature categories**
    1. Features from requirements can be gathered into categories
4. Decide on what **libraries** to use (technology decisions)

Project Requirements form the Business

1. Users of the app are hotel employees. They need to be logged into the application to perform tasks (**Authentication**)
2. New users can only be signed up inside the applications (to guarantee that only actual hotel employees can get accounts) (**Authentication**)
3. Users should be able to upload an avatar, and change their name and password (**Authentication**)
4. App needs a table view with all cabins, showing the cabin photo, name, capacity, price, and current discount (**Cabins**)
5. Users should be able to update or delete a cabin, and to construct new cabins (including uploading a photo) (**Cabins**)
6. App needs a table view with all bookings, showing arrival and departure dates, status, and paid amount, as well as cabin and guest data (**Bookings**)
7. The booking status can be "unconfirmed" (booked but not yet checked in), "checked in", or "checkout out". The table should be filterable by this important status. (**Bookings**)
8. Other booking data includes: number of guests, number of nights, guest observations, whether they booked breakfast, breakfast price (**Bookings**)
9. Users should be able to delete, check in, or check out a booking as the guest arrives (no editing necessary for now) (**Check In/Out**)
10. Bookings may not have been paid yet on guest arrival. Therefore, on check in, users need to accept payment (outside the app), and then confirm that payment has been recieved (inside the app). (**Check In/Out**)
11. On check in, the guest should have the ability to add breakfast for the entire stay, if they hadn't already. (**Check In/Out**)
12. Guest data should contain: full name, email, national ID, nationality, and a country flag for easy identification. (**Guests**)
13. The initial app screen should be a dashboard, to display important information for the last 7, 30, or 90 days. (**Dashboard**)
    1. A list of guests checking in and out on the current day. Users should be able to perform these tasks from here.
    2. Statistics on recent bookings, sales, check ins, and occupancy rate.
    3. A chart showing all daily hotel sales, showing both "total" sales and "extras" sales (only breakfast at the moment)
    4. A chart showing statistics on stay durations, as this is an important metric for the hotel
14. Users should be able to define a few application-wide settings: breakfast price, min and max nights/booking, max guests/booking (**Settings**)
15. App needs a dark mode (**Settings**)

Features + Pages

1. Feature Category:
    1. Bookings
    2. Cabins
    3. Guests
    4. Dashboard
    5. Check in and out
    6. App settings
    7. Authentication
2. Necessary pages:
    1. Dashboard `/dashboard`
    2. Bookings `/bookings`
    3. Cabins `/cabins`
        1. Edit & delete cabins
    4. Guests
        1. Nothing in the requirements
    5. Booking check in `/checkin/:bookingID`
    6. App settings `/settings`
    7. User sign up `/users`
        1. Existing users can register new ones
    8. Login `/login`
        1. Default homepage
3. We will discuss state later. Most of it will be **global**
4. We will discuss APIs later

Client-Side Rendering (CSR) OR Server-Side Rendering (SSR)?

1. React team has been advocating moving back to server (NextJS)
    1. CSR with Plain React (Vanilla React)
        1. Use to build **single**-page applications (SPAs)
        2. All HTML is rendered in the **client**
            1. All done by JS
                1. JS needs to be downloaded even before the application starts running
                    1. Bad for performance if users
                        1. Are using a low-end device
                        2. Have low internet connection

3. **One perfect use case:** apps that are used "internally" as tools inside companies (not downloaded by many users), that are entirely hidden behind a login (no need for SEO)
2. SSR with Famework (NextJS, Remix)
    1. Use to build **multi**-page applications (SPAs)
    2. Some HTML is rendered in the **server**
        1. **More performant**, as less JS needs to be downloaded
    3. The React team is moving more and more in this direction
        1. Not true for all applications

Technology Decisions

1. Routing - React Router - The standard for React SPAs
2. Styling - Styled Components - Very popular way of writing component-scoped CSS, right inside JS. A technology worth learning
    1. It is used inside components
    2. Used by
        1. IMDB
        2. Spotify
        3. Coinbase
        4. ...
3. State management - React Query
    1. All state will be remote state
    2. Application will implemet an easy to interface to remote data that will be used in server
    3. We load and mutate data a lot
    4. The best way of managing remote state, with features like **caching**, **automatic re-fetching**, **pre-fetching** (pagination), **offline support**, etc.
    5. Alternatives are SWR and RTK Query, but this is the most popular
4. UI State management - Context API
    1. There is almost no UI state needed in this app, so one simple context with `useState` will be enough. No need for Redux.
5. Form management - React Hook Form
    1. Handling bigger forms can be a lot of work, such as manual state creation and error handling (library takes care of that). A library can simplify all this
6. Other tools
    1. React icons
    2. React hot toast
    3. Recharts (beautiful charts)
    4. date-fns (date manipulation)
    5. Supabase (build API, store remote state)

Setting Up the Project: "The Wild Oasis"

1. Creation of vite project
2. Open it in VS Code
3. Install dependencies (`npm install`)
4. Configure ES Lint (`npm i --save-dev vite-plugin-eslint eslint-config-react-app eslint`)
    1. Add `.eslintrc.json`
    2. Update `vite.cnfig.js`
5. Start the app: `npm run dev`
6. Structure:
    1. pages
        1. All pages
        2. One component file per route
            1. Each of the pages will not have any side effects, but delegate all functionality to component associated with the feature
                1. Pages created once and forgotten

Introduction to Styled Components

1. Allow us to write CSS right inside JS component files

    1. Procedure:
        1. We take a regular HTML element & using `styled` function, we construct a brand new React component with some CSS styles applied to it
        2. We can then use the new React component instead of the regular HTML element

2. Installation:

    ```
    npm i styled-components
    ```

    1. VS Code extension: vscode-styled-components
        1. For highlighting

3. Code:

    ```
    const H1 = styled.h1`
      font-size: 30px;
      font-weight: 600;
    `; // ES6 - tagged template literals. It returns a new component
    // A randomly named class is created, and added styles into it
    /*

    <h1 class="sc-gTRrQi sgasdf">The Wild Oasis</h1>

    .sgasdf{
      font-size: 30px;
      font-weight: 600;
    }

    It doesn't apply globally, and there won't be name collisions.
    The CSS is available is available only for this component
    */
    ```

    1. The styled components receive the same props that regular HTML or JSX elements can receive

```
const Button = styled.button`
  font-size: 1.4rem;
  padding: 1.2rem 1.6rem;
  font-weight: 500;
  border: none;
  border-radius: 7px;
  background-color: purple;
  color: white;
`;

function App() {
  return (
    <div>
      <H1>The Wild Oasis</H1>
      <Button onClick={() => alert("Check in")}>Check in</Button>
    </div>
  );
}
```

4. How to style existing components?
    1. If the component is called `App`, we can construct a `StyledApp` for a `div` inside the component.

Global Styles with Styled Components

1. We construct a Gobal styled component (instead of using `index.css`)
2. Code: App.css

```
function App() {
  return (
    <>
      <GlobalStyles /> {/* Doesn't accept children */}
      <StyledApp>
        <H1>The Wild Oasis</H1>
        <Button onClick={() => alert("Check in")}>Check in</Button>
        <Button onClick={() => alert("Check out")}>Check in</Button>

        <Input type="number" placeholder="Number of guests" />
      </StyledApp>
    </>
  );
}
```

3. Code: GlobalStyles.js

```
import { createGlobalStyle } from "styled-components";

const GlobalStyles = createGlobalStyle`

/* Colors adapted from https://tailwindcss.com/docs/customizing-colors */

:root {
  /* Indigo */
  --color-brand-50: #eef2ff;
  --color-brand-100: #e0e7ff;
  --color-brand-200: #c7d2fe;
  --color-brand-500: #6366f1;
  --color-brand-600: #4f46e5;
  --color-brand-700: #4338ca;
  --color-brand-800: #3730a3;
  --color-brand-900: #312e81;

  /* Grey */
  --color-grey-0: #fff;
  --color-grey-50: #f9fafb;
  --color-grey-100: #f3f4f6;
  --color-grey-200: #e5e7eb;
  --color-grey-300: #d1d5db;
  --color-grey-400: #9ca3af;
  --color-grey-500: #6b7280;
  --color-grey-600: #4b5563;
  --color-grey-700: #374151;
  --color-grey-800: #1f2937;
  --color-grey-900: #111827;

  --color-blue-100: #e0f2fe;
  --color-blue-700: #0369a1;
  --color-green-100: #dcfce7;
  --color-green-700: #15803d;
  --color-yellow-100: #fef9c3;
  --color-yellow-700: #a16207;
  --color-silver-100: #e5e7eb;
  --color-silver-700: #374151;
  --color-indigo-100: #e0e7ff;
  --color-indigo-700: #4338ca;

  --color-red-100: #fee2e2;
  --color-red-700: #b91c1c;
  --color-red-800: #991b1b;

  --backdrop-color: rgba(255, 255, 255, 0.1);

  --shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.04);
  --shadow-md: 0px 0.6rem 2.4rem rgba(0, 0, 0, 0.06);
  --shadow-lg: 0 2.4rem 3.2rem rgba(0, 0, 0, 0.12);

  --border-radius-tiny: 3px;
  --border-radius-sm: 5px;
  --border-radius-md: 7px;
  --border-radius-lg: 9px;
```

```
  /* For dark mode */
  --image-grayscale: 0;
  --image-opacity: 100%;
}

*,
*::before,
*::after {
  box-sizing: border-box;
  padding: 0;
  margin: 0;

  /* Creating animations for dark mode */
  transition: background-color 0.3s, border 0.3s;
}

html {
  font-size: 62.5%;
}

body {
  font-family: "Poppins", sans-serif;
  color: var(--color-grey-700);

  transition: color 0.3s, background-color 0.3s;
  min-height: 100vh;
  line-height: 1.5;
  font-size: 1.6rem;
}

input,
button,
textarea,
select {
  font: inherit;
  color: inherit;
}

button {
  cursor: pointer;
}

*:disabled {
  cursor: not-allowed;
}

select:disabled,
input:disabled {
  background-color: var(--color-grey-200);
  color: var(--color-grey-500);
}

input:focus,
button:focus,
textarea:focus,
select:focus {
  outline: 2px solid var(--color-brand-600);
  outline-offset: -1px;
}

/* Parent selector, finally 😃 */
button:has(svg) {
  line-height: 0;
}

a {
  color: inherit;
  text-decoration: none;
}

ul {
  list-style: none;
}

p,
h1,
h2,
h3,
h4,
h5,
h6 {
  overflow-wrap: break-word;
  hyphens: auto;
}

img {
  max-width: 100%;

  /* For dark mode */
  filter: grayscale(var(--image-grayscale)) opacity(var(--image-opacity));
}

/*
FOR DARK MODE

--color-grey-0: #18212f;
--color-grey-50: #111827;
--color-grey-100: #1f2937;
--color-grey-200: #374151;
--color-grey-300: #4b5563;
--color-grey-400: #6b7280;
--color-grey-500: #9ca3af;
--color-grey-600: #d1d5db;
--color-grey-700: #e5e7eb;
```

```
--color-grey-800: #f3f4f6;
--color-grey-900: #f9fafb;

--color-blue-100: #075985;
--color-blue-700: #e0f2fe;
--color-green-100: #166534;
--color-green-700: #dcfce7;
--color-yellow-100: #854d0e;
--color-yellow-700: #fef9c3;
--color-silver-100: #374151;
--color-silver-700: #f3f4f6;
--color-indigo-100: #3730a3;
--color-indigo-700: #e0e7ff;

--color-red-100: #fee2e2;
--color-red-700: #b91c1c;
--color-red-800: #991b1b;

--backdrop-color: rgba(0, 0, 0, 0.3);

--shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.4);
--shadow-md: 0px 0.6rem 2.4rem rgba(0, 0, 0, 0.3);
--shadow-lg: 0 2.4rem 3.2rem rgba(0, 0, 0, 0.4);

--image-grayscale: 10%;
--image-opacity: 90%;
*/
`;

export default GlobalStyles;
```

4. Why use variables?
    1. Helps us update them independently
        1. Helps with dark-mode
5. Styled components provides similar variables using a mechanism called themes
    1. We can inject design tokens like the above into application
        1. The mechanism was introduced before css variables became popular in modern css
            1. This is better
    2. However the documentation is in Styled components - Go to Themes

```
<ThemeProvider theme={theme}>
    ...
<color: ${props => props.theme.main};
```

6. We can leave styles used only in a particular component inside the component

Styled Component Props and the "css" Function

1. Code: Heading.jsx

```
import styled, { css } from "styled-components";

// const test = css`
//   text-align: center;
//   ${10 > 5 && "background-color: yellow"}
// `; // css function. For syntax highlighting

// const Heading = styled.h1`
//   // This is a template literal
//   /* font-size: ${10 > 5 ? "30px" : "5px"}; */
//   font-size: 20px;
//   font-weight: 600;
//   background-color: yellow;
//   /* ${test} */
// `;

const Heading = styled.h1`
  ${(props) =>
    props.as === "h1" &&
    css`
      font-size: 3rem;
      font-weight: 600;
    `}

  ${(props) =>
    props.as === "h2" &&
    css`
      font-size: 2rem;
      font-weight: 600;
    `}

  ${(props) =>
    props.as === "h3" &&
    css`
      font-size: 2rem;
      font-weight: 500;
    `}

  line-height: 1.4;
`; // ES6 - tagged template literals. It returns a new component
// A randomly named class is created, and added styles into it
/*

<h1 class="sc-gTRrQi sgasdf">The Wild Oasis</h1>

.sgasdf{
```

```
  font-size: 30px;
  font-weight: 600;
}

It doesn't apply globally, and there won't be name collisions.
The CSS is available is available only for this component
*/

export default Heading;
```

2. Code: App.jsx

```
function App() {
  return (
    <>
      <GlobalStyles /> {/* Doesn't accept children */}
      <StyledApp>
        <Heading as="h1">The Wild Oasis</Heading> {/* We can give any name */}
        <Heading as="h2">Check in and out</Heading>
        <Button onClick={() => alert("Check in")}>Check in</Button>
        <Button onClick={() => alert("Check out")}>Check in</Button>
        <Heading as="h3">Form</Heading>{" "}
        {/* `as` specifies the HTML element to be rendered as */}
        <Input type="number" placeholder="Number of guests" />
      </StyledApp>
    </>
  );
}
```

Building More Reusable Styled Components

1. Code: App.jsx

```
import styled from "styled-components";
import GlobalStyles from "./styles/GlobalStyles";
import Button from "./ui/Button";
import Input from "./ui/Input";
import Heading from "./ui/Heading";
import Row from "./ui/Row";

const StyledApp = styled.div`
  /* background-color: orangered; */
  padding: 20px;
`;

function App() {
  return (
    <>
      <GlobalStyles /> {/* Doesn't accept children */}
      <StyledApp>
        <Row>
          <Row type="horizontal">
            <Heading as="h1">The Wild Oasis</Heading>{" "}
            {/* We can give any name */}
            <div>
              <Heading as="h2">Check in and out</Heading>
              <Button onClick={() => alert("Check in")}>Check in</Button>
              <Button
                variation="secondary"
                size="small"
                onClick={() => alert("Check out")}
              >
                Check in
              </Button>
            </div>
          </Row>
          <Row type="vertical">
            <Heading as="h3">Form</Heading>{" "}
            {/* `as` specifies the HTML element to be rendered as */}
            <form>
              <Input type="number" placeholder="Number of guests" />
            </form>
          </Row>
        </Row>
      </StyledApp>
    </>
  );
}

export default App;
```

2. Code: Button.jsx

```
import styled, { css } from "styled-components";

const sizes = {
  small: css`
    font-size: 1.2rem;
    padding: 0.4rem 0.8rem;
    text-transform: uppercase;
    font-weight: 600;
    text-align: center;
  `,
  medium: css`
    font-size: 1.4rem;
    padding: 1.2rem 1.6rem;
    font-weight: 500;
  `,
  large: css`
    font-size: 1.6rem;
    padding: 1.2rem 2.4rem;
    font-weight: 500;
```

```
    `,
  };

  const variations = {
    primary: css`
      color: var(--color-brand-50);
      background-color: var(--color-brand-600);

      &:hover {
        background-color: var(--color-brand-700);
      }
    `,
    secondary: css`
      color: var(--color-grey-600);
      background: var(--color-grey-0);
      border: 1px solid var(--color-grey-200);

      &:hover {
        background-color: var(--color-grey-50);
      }
    `,
    danger: css`
      color: var(--color-red-100);
      background-color: var(--color-red-700);

      &:hover {
        background-color: var(--color-red-800);
      }
    `,
  };

  const Button = styled.button`
    border: none;
    border-radius: var(--border-radius-sm);
    box-shadow: var(--shadow-sm);

    ${(props) => sizes[props.size]}
    ${(props) => variations[props.variation]}
  `;

  Button.defaultProps = {
    variation: "primary",
    size: "medium",
  };

  export default Button;
```

Setting Up Pages and Routes

1. Installing React router:

   ```
   npm i react-router-dom@6
   ```

2. Icons for links

   ```
   npm i react-icons
   ```

   1. It imports many popular icon sets
      1. Heroicons 2
         1. `import { IconName } from "react-icons/hi2";`

Building the App Layout

Building the Sidebar and Main Navigation

Section 26: Supabase Crash Course: Building a Back-End!

Section Overview

1. Topics:
   1. **Plan** application data
      1. All data we need
      2. Constructing tables that we need
   2. Model **relationships** between data tables
   3. Load data into the app via **Supabase API**

What is Supabase?

Supabase: Service that allows developers to easily **construct a back-end with a Postgres database** 1. Automatically construct a **database** and **API** so we can easily request and receive data from the server. 2. No back-end development needed 3. Perfect to get up and running **quickly**! 4. Not just an API: Supabase also comes with easy-to-use **user authentication** and **file storage** 1. Accessible using API or JS library 2. It enables building

Constructing a New Database

1. supabase.com
   1. Firebase alternative & open-source
      1. Free - 2 projects
         1. Pauses project if not used for a week (data is retained)
2. Sign up
3. Start a project
4. A new organization
5. Features:
   1. Table editor

        1. New Table
    2. Authentication
    3. Upload
    4. API Docs
    5. Project settings

Modeling Application State

Modeling State

1. We need to think about state slices at a much higher level
    1. We think about state at application feature level and page level
2. State "Domains"/"Slices"
    1. Feature Categories: (each feature needs their own state)
        1. Bookings
            1. **Bookings** - state
        2. Cabins
            1. **Cabins** - state
        3. Guests
            1. **Guests** - state
        4. Dashboard
            1. Bookings
                1. Dashboards display a few statistics about recent bookings
        5. Check in and out
            1. Bookings:
                1. Updating the bookings from checkout to checked in
        6. App settings
            1. **Settings**
        7. Authentication
            1. **Users**
3. All the state will be **global remote state**, stored within supabase
    1. Managed using React Query
    2. There will be one **table** for each state "slice" in the **database**

The Bookings Table

1. Booking:
    1. It is about:
        1. **Guest** renting a **Cabin**
    2. A booking needs information about what **guest** is booking which **cabin**: we need to **connect** them
    3. Supabase uses a Postgres DB, which is SQL (relational DB). So we **join** tables using **foreign keys**
    4. Guest's `id` (primary key) is stored in Booking as `guestId` (foreign key)
    5. Cabin's `id` (primary key) is stored in Booking as `cabinId` (foreign key)

Constructing Tables

1. Cabins table
    1. Enable row level security
    2. Columns:
        1. id
        2. created_at
        3. name
            1. text
            2. No default value
        4. maxCapacity (how many people can rent a cabin)
            1. int2 (2-byte integer)
        5. regularPrice
            1. int2
        6. discount
            1. int2
        7. description
            1. text
        8. image
            1. text (URL)
                1. Stored in storage buckets
2. Insert row:
    1. name: 001
    2. maxCapacity: 2
    3. regularPrice: 250
    4. discount: 50
    5. description: Small luxury cabin in the woods
    6. image: null
3. guests table
    1. fullName: text
    2. email: text
    3. nationalID: text
    4. nationality: text
    5. countryFlag: text
4. Insert row:
    1. fullName: Jonas Schmedtmann
    2. email: test@email.com
    3. nationalID: gasdfdsf
    4. nationality: german
    5. countryFlag:
5. settings table (special table which has only one row)

1. minBookingLength: int2
2. maxBookingLength: int2
3. maxGuestsPerBooking: int2
4. breakfastPrice: float4

6. Insert row:
    1. maxBookingLength: 3
    2. maxBookingLength: 90
    3. maxGuestsPerBooking: 8
    4. breakfastPrice: 15

Relationships Between Tables

1. Bookings table
    1. startDate: timestamp
    2. endDate: timestamp
    3. numNights: int2 (auto-calculated)
    4. numGuests: int2
    5. cabinPrice: float4
    6. extrasPrice: float4
    7. totalPrice: float4
    8. status: text
    9. hasBreakfast: bool
    10. isPaid: bool
    11. observations: text (when they arrived, ...)
    12. cabinID: Edit foreign key relation
        1. Select a teable to reference to: cabins
        2. Select a column from `public.cabins` to reference to: `id`
    13. guestID: Edit foreign key relation
        1. Select a table to reference to: guests
        2. Select a column from `public.guests` to reference to: `id`
2. Insert row:
    1. startDate: any date
    2. endDate: any date
    3. numNights: 4
    4. numGuests: 2
    5. cabinPrice: 300
    6. extrasPrice: 120
    7. totalPrice: 420
    8. status: unconfirmed
    9. hasBreakfast: TRUE
    10. isPaid: TRUE
    11. observations: I will arrive at 10pm
    12. cabinId: Select the row
    13. guestId: Select the row

Adding Security Policies (RLS)

1. API Docs
    1. API docs are automatically created for the tables
    2. cabins
        1. copy JS code
        2. READ ALL ROWS
            1. Project API Key
            2. Copy Curl code
            3. Paste in terminal
                1. Row level security is enabled so no data
                    1. Row level security tables
                        1. They prevent anyone who owns a particular key (Bearer token) to do whatever they want
2. RLS policies
    1. Authentication
        1. Policies
            1. New Policy
                1. Get started quickly
                    1. Enable read access to everyone

```
CREATE POLICY "policy_name"
ON public.cabins
FOR SELECT USING (
    true
);
```

                        1. Use this template
                        2. Review
                        3. Save policy
            2. Enable access to other tables

Connecting Supabase with Our React App

1. API Docs

1. Introduction
    1. All instructions are given to connect
        1. Client Libraries
            1. Install

```
npm install --save @supabase/supabase-js
```

                1. services/supabase.js
                    1. Copy and paste initializing code

2. Code: Cabins.jsx

```
import { useEffect } from "react";
import Heading from "../ui/Heading";
import Row from "../ui/Row";
import { getCabins } from "../services/apiCabins";

function Cabins() {
  useEffect(function () {
    getCabins().then((data) => console.log(data));
  }, []);

  return (
    <Row type="horizontal">
      <Heading as="h1">All cabins</Heading>
      <p>TEST</p>
    </Row>
  );
}

export default Cabins;
```

3. Code: apiCabins.js

```
import supabase from "./supabase";

export async function getCabins() {
  const { data, error } = await supabase.from("cabins").select("*");

  if (error) {
    console.error(error);
    throw new Error("Cabins could not be loaded");
  }

  return data;
}
```

Setting up Storage Buckets

1. Storage
    1. New Bucket:
        1. Name of bucket: avatars
        2. Public bucket
            1. RLS policies are required
    2. New Bucket:
        1. Name of bucket: cabin-images
        2. Public bucket
            1. src/data/cabins
                1. Drag and drop
                    1. Get URL
    3. Add the URL in the table

Section 27: React Query: Managing Remote State

Section Overview

1. Topics:
    1. **Remote state** management
    2. **React Query** will take over data fetching and storage (of remote data)
        1. Simplifies state management

What is React Query?

1. Integration of React query for all data fetching, and remote state management.

What is React Query?

1. Powerful library for managing **remote (server) state** (to load into application)
    1. What React is missing
2. Many features that allow us to write a **lot less code**, while also **making the UX a lot better**
3. Useful features:
    1. Data is stored in a cache
        1. To be re-used in different points of the application

```
fetch("api.com/cabins"); // Component A
```

1. API fetches data and stores it in cache
2. Component can use it
3. If component B wants to use the data, not API request is necessary, the data is provided from the cache
    1. No loading spinner is required for subsequent components

2. Automatic loading and error states
3. Automatic re-fetching to keep state synced
    1. After timeout
    2. Leave browser window and come back
    3. State updated
        1. Example: If cabin is updated, the updated data is automatically fetched
4. Pre-fetching
    1. To fetch data that will be used later, but before displaying it on the screen
        1. Example: Pagination
            1. We can pre-fetch the next page
5. Easy remote state mutation (updating)
    1. Tools built into React query
6. Offline support
    1. If user is offline, cached data can be used

4. Needed because remote state is **fundamentally** different from regular (UI) state
    1. Asynchronous
    2. Usually shared by many users
        1. Data in browsers can get out of sync with the remote data stored on a server
    3. (other special needs)
5. Other libraries (don't work as well, and not as popular)
    1. RTK Query (Remote state solution integrated into Redux Took Kit)
    2. SWR

Setting up React Query

1. Installation:

    ```
    npm i @tanstack/react-query@4
    ```

    1. The library is no longer called `react-query` because it supports other frameworks like Svelte & Vue.
    2. The official name is `tanstack`

2. Steps:
    1. Construct a place where the data lives
    2. Provide the data to the application
3. Code: App.jsx

    ```
    const queryClient = new QueryClient({
      defaultOptions: {
        queries: {
          staleTime: 60 * 1000,
        },
      },
    }); // Sets up cache

    function App() {
        return(
            <QueryClientProvider client={queryClient}>
                <GlobalStyles />
                <BrowserRouter>
                    ...
                </BrowserRouter>
            </QueryClientProvider>
        )
    }
    ```

4. React Query dev tools:

    ```
    npm i @tastack/react-query-devtools@4
    ```

    1. App.jsx

        ```
        <QueryClientProvider ...>
            <ReactQueryDevtools initialIsOpen={false} />
        ```

        1. `QueryClientProvider` - helps having data in one place, and providing it to the whole component tree

Make Sure to Use React Query v4!

1. Use the following dependency:

    ```
    npm i @tanstack/react-query@4
    ```

1. To use React Query v5
    1. `isLoading` is now `isPending`
    2. `cacheTime` is called `gcTime`

Fetching Cabin Data

1. Instead of fetching in `useEffect`
2. Install `date-fns`

```
npm i date-fns
```

3. Code: App.js

```
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      // staleTime: 60 * 1000,
      staleTime: 0,
    },
  },
}); // Sets up cache
```

4. Code: Cabins.jsx

```
import CabinTable from "../features/cabins/CabinTable";
import Heading from "../ui/Heading";
import Row from "../ui/Row";

function Cabins() {
  return (
    <>
      <Row type="horizontal">
        <Heading as="h1">All cabins</Heading>
        <p>Filter / Sort</p>
      </Row>

      <Row>
        <CabinTable />
      </Row>
    </>
  );
}

export default Cabins;
```

Mutations: Deleting a Cabin

3. Code: CabinRow.jsx

```
const queryClient = useQueryClient();

const { isLoading: isDeleting, mutate } = useMutation({
    // mutationFn: (id) => deleteCabin(id),
    mutationFn: deleteCabin,
    onSuccess: () => {
      alert("Cabin successfully deleted");
      queryClient.invalidateQueries({
        queryKey: ["cabins"],
      });
    },
    onError: (err) => alert(err.message), // If there is an error React Query tries to fetch multiple times.
}); // **(M)**

// mutate - callback function that we can connect to the button

return (
    <TableRow role="row">
      {/* ... */}
      <button onClick={() => mutate(cabinId)} disabled={isDeleting}>
        Delete
      </button>
    </TableRow>
);
```

Displaying Toasts (Notifications)

1. Another library:

```
npm i react-hot-toast
```

2. Code: App.jsx

```
    <Toaster
    position="top-center"
    gutter={12}
    containerStyle={{ margin: "8px" }}
    toastOptions={{
      success: {
        duration: 3000,
      },
      error: {
        duration: 5000,
      },
      style: {
        fontSize: "16px",
        maxWidth: "500px",
        padding: "16px 24px",
        backgroundColor: "var(--color-grey-0)",
```

```
          color: "var(--color-grey-700)",
        },
      }}
    />
</QueryClientProvider>
```

3. Code: CabinRow.jsx

```
const { isLoading: isDeleting, mutate } = useMutation({
  // mutationFn: (id) => deleteCabin(id),
  mutationFn: deleteCabin,
  onSuccess: () => {
    toast.success("Cabin successfully deleted");
    queryClient.invalidateQueries({
      queryKey: ["cabins"],
    });
  },
  onError: (err) => toast.err(err.message), // If there is an error React Query tries to fetch multiple times.
}); // **(M)**
```

Introducing Another Library: React Hook Form

1. Form setup: It only handles form submission, or form errors (no pre-built components)
    1. Installation:

       ```
       npm i react-hook-form@7
       ```

        1. No need for controlled elements
            1. No state variable for them
        2. The library can handle the inputs
            1. Generates event-handlers
            2. Handles submit

Constructing a New Cabin

1. Code: CreateCabinForm.jsx

```
function CreateCabinForm() {
  const { register, handleSubmit } = useForm();
  // `register` - to register inputs into the hook

  function onSubmit(data) {
    console.log(data);
  }

  return (
    <Form onSubmit={handleSubmit(onSubmit)}>
      <FormRow>
        <Label htmlFor="name">Cabin name</Label>
        <Input type="text" id="name" {...register("name")} />{" "}
        {/* adds `onChange`, and `onBlur` functions */}
      </FormRow>

      <FormRow>
        <Label htmlFor="maxCapacity">Maximum capacity</Label>
        <Input type="number" id="maxCapacity" {...register("maxCapacity")} />
      </FormRow>

      <FormRow>
        <Label htmlFor="regularPrice">Regular price</Label>
        <Input type="number" id="regularPrice" {...register("regularPrice")} />
      </FormRow>

      <FormRow>
        <Label htmlFor="discount">Discount</Label>
        <Input
          type="number"
          id="discount"
          defaultValue={0}
          {...register("discount")}
        />
      </FormRow>

      <FormRow>
        <Label htmlFor="description">Description for website</Label>
        <Textarea
          type="number"
          id="description"
          defaultValue=""
          {...register("description")}
        />
      </FormRow>

      <FormRow>
        <Label htmlFor="image">Cabin photo</Label>
        <FileInput id="image" accept="image/*" />
      </FormRow>

      <FormRow>
        {/* type is an HTML attribute! */}
        <Button variation="secondary" type="reset">
          Cancel
        </Button>
        <Button>Add cabin</Button>
      </FormRow>
    </Form>
  );
}
```

Handling Form Errors

1. Code: FormRow.jsx

```
import styled from "styled-components";
import PropTypes from "prop-types";

const StyledFormRow = styled.div`
  display: grid;
  align-items: center;
  grid-template-columns: 24rem 1fr 1.2fr;
  gap: 2.4rem;

  padding: 1.2rem 0;

  &:first-child {
    padding-top: 0;
  }

  &:last-child {
    padding-bottom: 0;
  }

  &:not(:last-child) {
    border-bottom: 1px solid var(--color-grey-100);
  }

  &:has(button) {
    display: flex;
    justify-content: flex-end;
    gap: 1.2rem;
  }
`;

const Label = styled.label`
  font-weight: 500;
`;

const Error = styled.span`
  font-size: 1.4rem;
  color: var(--color-red-700);
`;

function FormRow({ label, error, children }) {
  return (
    <StyledFormRow>
      {label && <Label htmlFor={children.props.id}>{label}</Label>}
      {children}
      {/* adds `onChange`, and `onBlur` functions */}
      {error && <Error>{error}</Error>}
    </StyledFormRow>
  );
}

FormRow.propTypes = {
  label: PropTypes.string.isRequired,
  error: PropTypes.string,
  children: PropTypes.object.isRequired,
};

export default FormRow;
```

2. Code: CreateCabinForm.jsx

```
import Input from "../../ui/Input";
import Form from "../../ui/Form";
import Button from "../../ui/Button";
import FileInput from "../../ui/FileInput";
import Textarea from "../../ui/Textarea";
import { useForm } from "react-hook-form";
import { useMutation, useQueryClient } from "@tanstack/react-query";
import { createCabin } from "../../services/apiCabins";
import toast from "react-hot-toast";
import FormRow from "../../ui/FormRow";

function CreateCabinForm() {
  const { register, handleSubmit, reset, getValues, formState } = useForm();
  // `register` - to register inputs into the hook

  const { errors } = formState;
  console.log(errors);

  const queryClient = useQueryClient();

  const { mutate, isLoading: isAdding } = useMutation({
    mutationFn: createCabin,
    onSuccess: () => {
      toast.success("New cabin successfully created!");
      queryClient.invalidateQueries({ queryKey: ["cabins"] });
      reset();
    },
    onError: (err) => toast.error(err.message),
  });

  function onSubmit(data) {
    mutate(data);
  }

  function onError(errors) {
    console.log(errors);
  }

  return (
```

```
      <Form onSubmit={handleSubmit(onSubmit, onError)}>
        <FormRow label="Cabin name" error={errors?.name?.message}>
          <Input
            type="text"
            id="name"
            disabled={isAdding}
            {...register("name", {
              required: "This field is required",
            })}
          />
          {/* adds `onChange`, and `onBlur` functions */}
        </FormRow>

        <FormRow label="Maximum capacity" error={errors?.maxCapacity?.message}>
          <Input
            type="number"
            id="maxCapacity"
            disabled={isAdding}
            {...register("maxCapacity", {
              required: "This field is required",
              min: {
                value: 1,
                message: "Capacity should be at least 1",
              },
            })}
          />
        </FormRow>

        <FormRow label="Regular price" error={errors?.regularPrice?.message}>
          <Input
            type="number"
            id="regularPrice"
            disabled={isAdding}
            {...register("regularPrice", {
              required: "This field is required",
              min: {
                value: 1,
                message: "Price should be at least 1",
              },
            })}
          />
        </FormRow>

        <FormRow label="Discount" error={errors?.discount?.message}>
          <Input
            type="number"
            id="discount"
            disabled={isAdding}
            defaultValue={0}
            {...register("discount", {
              required: "The field is required",
              validate: (value) =>
                value <= getValues().regularPrice ||
                "Discount should be less than regular price",
            })}
          />
        </FormRow>

        <FormRow
          label="Description for website"
          error={errors?.description?.message}
        >
          <Textarea
            type="number"
            id="description"
            disabled={isAdding}
            defaultValue=""
            {...register("description", {
              required: "This field is required",
            })}
          />
        </FormRow>

        <FormRow label="Cabin photo">
          <FileInput id="image" accept="image/*" />
        </FormRow>

        <FormRow>
          {/* type is an HTML attribute! */}
          <Button variation="secondary" type="reset">
            Cancel
          </Button>
          <Button disabled={isAdding}>Add cabin</Button>
        </FormRow>
      </Form>
    );
  }

  export default CreateCabinForm;
```

Uploading Images to Supabase

1. supabase.com
    1. API Docs
        1. Introduction
            1. Javascript
                1. Upload a file
    2. Storage
        1. Policies
            1. cabin-images

1. New policy
    1. Full customization
        1. Enable everything for everyone

2. Code: apiCabins.js

```
export async function createCabin(newCabin) {
  const imageName = `${Math.random()}-${newCabin.image.name}`.replaceAll(
    "/",
    ""
  );
  const imagePath = `${supabaseUrl}/storage/v1/object/public/cabin-images/${imageName}`;
  // https://pwvrahphsegoojprrclx.supabase.co/storage/v1/object/public/cabin-images/cabin-001.jpg

  const { data, error } = await supabase
    .from("cabins")
    .insert([{ ...newCabin, image: imagePath }])
    .select();

  if (error) {
    console.log(error);
    throw new Error("Cabin could not be created");
  }

  console.log(data);

  // 2. Upload image
  const { error: storageError } = await supabase.storage
    .from("cabin-images")
    .upload(imageName, newCabin.image);

  // 3. Delete the cabin if there was an error uploading image
  if (storageError) {
    console.log(storageError);
    await supabase.from("cabins").delete().eq("id", data.id);
    throw new Error(
      "Cabin image could not be uploaded and the cabin was not created"
    );
  }
}
```

Editing a Cabin

Abstracting React Query into Custom Hooks

1. Code: useCabins.js

```
import { useQuery } from "@tanstack/react-query";
import { getCabins } from "../../services/apiCabins";

export function useCabins() {
  const {
    isLoading,
    data: cabins,
    // error,
  } = useQuery({
    queryKey: ["cabins"], // uniquely identifies data we are going to query
    queryFn: getCabins, // Function which is used for querying. Needs to return a promise
  });
  // console.log(x); // We get data, error, states (isLoading, isPaused, isStale, ..., status: "loading")

  return { isLoading, cabins };
}
```

2. Code: useCreateCabins.js

```
import { useMutation, useQueryClient } from "@tanstack/react-query";
import toast from "react-hot-toast";
import { createEditCabin } from "../../services/apiCabins";

export function useCreateCabin() {
  const queryClient = useQueryClient();

  const { mutate: addCabin, isLoading: isAdding } = useMutation({
    mutationFn: createEditCabin,
    onSuccess: () => {
      toast.success("New cabin successfully created!");
      queryClient.invalidateQueries({ queryKey: ["cabins"] });
    },
    onError: (err) => toast.error(err.message),
  });

  return { isAdding, addCabin };
}
```

3. Code: useDeleteCabins.js

```
import { useMutation, useQueryClient } from "@tanstack/react-query";
import toast from "react-hot-toast";
import { deleteCabin as deleteCabinApi } from "../../services/apiCabins";

export function useDeleteCabin() {
  const queryClient = useQueryClient();

  const { isLoading: isDeleting, mutate: deleteCabin } = useMutation({
    // mutationFn: (id) => deleteCabin(id),
    mutationFn: deleteCabinApi,
    onSuccess: () => {
      toast.success("Cabin successfully deleted");
      queryClient.invalidateQueries({
        queryKey: ["cabins"],
```

```
      });
    },
    onError: (err) => toast.err(err.message), // If there is an error React Query tries to fetch multiple times.
  }); // **(M)**

  // mutate - callback function that we can connect to the button

  return { isDeleting, deleteCabin }; // extracting logic that contains hook into a custom hook. We had two hooks combined into a custom hook
}
```

4. Code: useEditCabins.js

```
import { useMutation, useQueryClient } from "@tanstack/react-query";
import { createEditCabin } from "../../services/apiCabins";
import toast from "react-hot-toast";

export function useEditCabin() {
  const queryClient = useQueryClient();

  const { mutate: editCabin, isLoading: isEditing } = useMutation({
    mutationFn: ({ newCabinData, id }) => createEditCabin(newCabinData, id),
    onSuccess: () => {
      toast.success("New cabin successfully edited!");
      queryClient.invalidateQueries({ queryKey: ["cabins"] });
    },
    onError: (err) => toast.error(err.message),
  });

  return { isEditing, editCabin };
}
```

5. Code: CabinTable.jsx

```
function CabinTable() {
  const { isLoading, cabins } = useCabins(); // re-usable
```

6. Code: CabinRow.jsx

```
function CabinRow({ cabin }) {
  const [showForm, setShowForm] = useState(false);
  const { isDeleting, deleteCabin } = useDeleteCabin();
```

7. Code: CreateCabinForm.jsx

```
function CreateCabinForm({ cabinToEdit = {} }) {
  const isWorking = isAdding || isEditing;

  const { isAdding, addCabin } = useCreateCabin();
  const { isEditing, editCabin } = useEditCabin();
```

Duplicating Cabins

1. Code: CabinRow.jsx

```
function CabinRow({ cabin }) {
  const [showForm, setShowForm] = useState(false);
  const { isDeleting, deleteCabin } = useDeleteCabin();
  const { isAdding, addCabin } = useCreateCabin();

  const {
    id: cabinId,
    name,
    maxCapacity,
    regularPrice,
    discount,
    image,
    description,
  } = cabin;

  function handleDuplicate() {
    addCabin({
      name: `Copy of ${name}`,
      maxCapacity,
      regularPrice,
      discount,
      image,
      description,
    });
  }

  return (
    <>
      <TableRow role="row">
        <Img src={image} />
        <Cabin>{name}</Cabin>
        <div>Fits up to {maxCapacity} guests</div>
        <Price>{formatCurrency(regularPrice)}</Price>
        {discount ? (
          <Discount>{formatCurrency(discount)}</Discount>
        ) : (
          <span>&mdash;</span>
        )}
        <div>
          <button onClick={handleDuplicate} disabled={isAdding}>
            <HiSquare2Stack />
          </button>
          <button onClick={() => setShowForm((show) => !show)}>
            <HiPencil />
          </button>
          <button onClick={() => deleteCabin(cabinId)} disabled={isDeleting}>
            <HiTrash />
          </button>
```

```
        </div>
      </TableRow>
      {showForm && <CreateCabinForm cabinToEdit={cabin} />}
    </>
  );
}
```

Fetching Applications Settings

1. RLS security policies - Read and Update
2. Code: useSettings.js

```
import { useQuery } from "@tanstack/react-query";
import { getSettings } from "../../services/apiSettings";

export function useSettings() {
  const {
    isLoading,
    error,
    data: settings,
  } = useQuery({
    queryKey: ["settings"],
    queryFn: getSettings,
  });

  return { isLoading, error, settings };
}
```

3. Code: UpdateSettingsForm.jsx

```
import Form from "../../ui/Form";
import FormRow from "../../ui/FormRow";
import Input from "../../ui/Input";
import Spinner from "../../ui/Spinner";
import { useSettings } from "./useSettings";

function UpdateSettingsForm() {
  const {
    isLoading,
    settings: {
      minBookingLength,
      maxBookingLength,
      maxGuestsPerBooking,
      breafastPrice,
    } = {},
  } = useSettings();

  if (isLoading) return <Spinner />;

  return (
    <Form>
      <FormRow label="Minimum nights/booking">
        <Input type="number" id="min-nights" defaultValue={minBookingLength} />
      </FormRow>
      <FormRow label="Maximum nights/booking">
        <Input type="number" id="max-nights" defaultValue={maxBookingLength} />
      </FormRow>
      <FormRow label="Maximum guests/booking">
        <Input
          type="number"
          id="max-guests"
          defaultValue={maxGuestsPerBooking}
        />
      </FormRow>
      <FormRow label="Breakfast price">
        <Input
          type="number"
          id="breakfast-price"
          defaultValue={breafastPrice}
        />
      </FormRow>
    </Form>
  );
}

export default UpdateSettingsForm;
```

Updating Application Settings

1. Code: useUpdateSetting.js

```
import { useMutation, useQueryClient } from "@tanstack/react-query";
import toast from "react-hot-toast";
import { updateSettingApi } from "../../services/apiSettings";

export function useUpdateSetting() {
  const queryClient = useQueryClient();

  const { mutate: updateSetting, isLoading: isUpdating } = useMutation({
    mutationFn: updateSettingApi,
    onSuccess: () => {
      toast.success("Setting successfully edited!");
      queryClient.invalidateQueries({ queryKey: ["settings"] });
    },
    onError: (err) => toast.error(err.message),
  });

  return { isUpdating, updateSetting };
}
```

2. Code: UpdateSettingsForm.jsx

```
import Form from "../../ui/Form";
import FormRow from "../../ui/FormRow";
import Input from "../../ui/Input";
import Spinner from "../../ui/Spinner";
import { useSettings } from "./useSettings";
import { useUpdateSetting } from "./useUpdateSetting";

function UpdateSettingsForm() {
  const {
    isLoading,
    settings: {
      minBookingLength,
      maxBookingLength,
      maxGuestsPerBooking,
      breafastPrice,
    } = {},
  } = useSettings();

  const { isUpdating, updateSetting } = useUpdateSetting();

  if (isLoading) return <Spinner />;

  function handleUpdate(e, field) {
    const { value } = e.target.value;
    console.log(value);
    if (!value) return;
    updateSetting({ [field]: value });
  }

  return (
    <Form>
      <FormRow label="Minimum nights/booking">
        <Input
          type="number"
          id="min-nights"
          defaultValue={minBookingLength}
          disabled={isUpdating}
          onBlur={(e) => handleUpdate(e, "minBookingLength")}
        />
      </FormRow>
      <FormRow label="Maximum nights/booking">
        <Input
          type="number"
          id="max-nights"
          defaultValue={maxBookingLength}
          disabled={isUpdating}
          onBlur={(e) => handleUpdate(e, "maxBookingLength")}
        />
      </FormRow>
      <FormRow label="Maximum guests/booking">
        <Input
          type="number"
          id="max-guests"
          defaultValue={maxGuestsPerBooking}
          disabled={isUpdating}
          onBlur={(e) => handleUpdate(e, "maxGuestsPerBooking")}
        />
      </FormRow>
      <FormRow label="Breakfast price">
        <Input
          type="number"
          id="breakfast-price"
          defaultValue={breafastPrice}
          disabled={isUpdating}
          onBlur={(e) => handleUpdate(e, "breafastPrice")}
        />
      </FormRow>
    </Form>
  );
}

export default UpdateSettingsForm;
```

Section 28: Advanced React Patterns

Section Overview

1. Topics:
    1. Advanced patterns used by **senior React engineers** (in most professional code bases)
    2. Render props
    3. Higher-order components
    4. **Compound components**
    5. Reusable modal window and context menu (we build)
    6. **Unique** content! (we build)

An Overview of Reusability in React

1. How to reuse different types of code in React
2. How do advanced patterns fit into the picture

How to Reuse Code in React?

1. I need to reuse:
    1. UI
        1. Components & props

1. Use props as a component API, to enable custom behavior.
    1. To customize how it works, and what it looks like
    2. Can be:
        1. Stateless
        2. Stateful
        3. Structural
    3. We can pass content or other components
        1. Using `children` prop
            1. To customize the component's content
2. Stateful Logic
    1. Logic that contains at least one React hook
    2. Custom hooks
        1. Using any number of any other hooks
3. Non-stateful logic
    1. JS functions will suffice
4. **To combine visuals & stateful logic**
    1. More advanced patterns
        1. **Render props pattern**
            1. They are not React features
                1. They are clever ways to use React that have emerged over time to solve certain patterns
            2. For complete control over **what** the component renders, by passing in a function that tells the component what to render. Was more commn before hooks, but still useful.
                1. The function tells the component, what and how to render
                    1. We can reuse logic that has UI (some JSX attached to it) while giving component more ability to recieve more JSX
                        1. This is different from `children` prop
5. **Compound component pattern**
    1. Compound - multiple components that play together to construct one big super-component
        1. For very self-contained components that need/want to manage their own state (internally). Compound components are like fancy super-components
            1. The state is not necessary to exist in the parent component that uses the compound component

Setting Up an Example

1. Code-sandbox
    1. https://codesandbox.io/s/react-render-props-starter-7tomkd
    2. Understand the implementation
        1. We want to re-use the logic and UI for both products & companies
            1. `children` only allows us to pass in only content
                1. However, in this case, we want to pass content & how the items need to be rendered
                    1. Solution: Render props pattern

The Render Props Pattern

1. A prop called `render` can be passed
    1. It is a function that a component uses to know:
        1. What it must render
        2. How it must render
2. Example:

```
function List({ title, items, render }) {
    // ...
    {isOpen && <ul className="List">{displayItems.map(render)}</ul>}
    // ...
}

<List title="Product" items={products} render={(product) => {
    <ProductItem key={product.productName} product={product} />
}} />
```

    1. We inverted the control of how it must render to the user of the component
        1. Inversion of control
    2. `displayItems.map` only calls the function passed for each item in the array
        1. It doesn't know what it is rendering
            1. It makes it easy to use it for other render props

```
<List
    title="companies"
    items={companies}
    render={(company) => (
        <CompanyItem key={company.companyName} company={company} defaultVisibility={false} />
    )}
/>
```

            1. We were able to re-use collapsing logic
            2. We were able to re-use showing fewer or more items
                1. We were able to re-use stateful logic & UI
    3. `render` - it must be a function
        1. It was the main way of sharing stateful logic across multiple components
            1. After React hooks was introduced, the pattern is not used much
                1. It is used in certain complex situations
                    1. Custom hooks helps us share logic

A Look at Higher-Order Components (HOC)

1. It is less used
    1. Some libraries expose HOCs (hence useful to know, but not know how to write them)
2. Example: We got the `ProductList` from a third-party vendor, and we cannot change it
    1. Solution: HOC to enhance or improve the existing component
        1. What is HOC?
            1. **A component that takes in another component and returns a new component that is an enhanced version of the original component**
                1. Example:

```
export default function withToggles(WrappedComponent) { // `with` is convention
    // ...
    {isOpen && <WrappedComponent {...props} items={displayItems} />

    <button onClick={() => setIsCollapsed(...) />
}

export default App() {
    const ProductListWithToggles = withToggles(ProductList); // If we pass the original component, it returns an enhanced
    // ...
    <div className='col-2'>
        {/*<ProductList title='Products HOC' items={products} />*/}
        <ProductListWithToggles title='Products HOC' items={products} />
    </div>
}
```

The Compound Component Pattern

1. We can construct a set of related components
    1. Use-cases:
        1. Modal windows
        2. Pagination
        3. Tables
        4. ...
2. Imlementation:
    1. A parent component is implemented
    2. Child components are implemented, that belong to the parent, and make sense when used together with parent component
        1. Example: HTML Select & Option elements
            1. Select - implements select box
            2. Option - implements each of the select options inside the select box
                1. It can only be used inside `Select` element.
    3. Advantages:
        1. Highly flexible & highly re-usable components
        2. Very expressive APIs
        3. No props
3. Example:
    1. Prop explosion:

```
<Counter
    iconIncrease="+"
    iconDecrease="-"
    label="My NOT so flexible counter"
    hideLabel={false}
    hideIncrease={false}
    hideDecrease={false}
/>
```

        1. Solution:

```
<Counter>
    <Counter.Decrease icon="-" />
    <Counter.Count />
    <Counter.Increase icon="+" />
    <Counter.Label>My super flexible counter</Counter.Label> {/* We could delete this if not required */}
</Counter>
```

            1. We can re-order
            2. We can delete some and keep the others
            3. ...
                1. How to implement this, since we cannot pass any props?
                    1. Using Context API (the biggest use-case of context)
4. Code:

```
import {
  createContext,
  useContext,
  useState,
} from "react/cjs/react.production.min";

// 1. Creation of a context
const CounterContext = createContext();

// 2. Creation of a parent component
```

```
function Counter() {
  const [count, setCount] = useState(0);
  const increase = () => setCount((c) => c + 1);
  const decrease = () => setCount((c) => c - 1);

  return (
    <CounterContext.provider value={{ count, increase, decrease }}>
      <span>{children}</span> {/* children get access to the three values */}
    </CounterContext.provider>
  );
}

// 3. Creation of child components to help implementing the common task
function Count() {
  const { count } = useContext(CounterContext);
  return <span>{count}</span>;
}

function Label({ children }) {
  return <span>{children}</span>;
}

function Increase({ icon }) {
  const { increase } = useContext(CounterContext);
  return <button onClick={increase}>{icon}</button>;
}

function Decrease() {
  const { decrease } = useContext(CounterContext);
  return <button onClick={decrease}>{icon}</button>;
}

// 4. Add child components as properties to parent component
Counter.Count = Count;
Counter.Label = Label;
Counter.Increase = Increase;
Counter.Decrease = Decrease;

export default Counter;
```

1. Another usage:

```
<div>
    <Counter>
        <Counter.Decrease icon="<" />
        <Counter.Count />
        <Counter.Increase icon=">" />
    </Counter>
</div>
```

Building a Modal Window Using a React Portal

1. Re-usable modal component
2. React Portal:
    1. A feature that allows us to render an element outside of parent component's DOM structure while keeping the element in the original position of the component tree
        1. Render a component in any place inside the DOM tree while leaving the component inside the React component tree.
            1. Use-case: For elements to stay on top of other components
                1. Tool-tips
                2. Modal windows
                3. Menus
                4. ...
    2. Why portal?
        1. To avoid conflicts with css property `overflow` set to `hidden`
            1. If a developer uses it somewhere else where `overflow: hidden` is set on the parent, the modal will get cut off
                1. Solution: We render modal completely outside the rest of the DOM. (On top of the DOM tree)
3. Problems:
    1. The way we opened the modal

Converting the Modal to a Compound Component

1. State:
    1. We don't want component which uses the modal to be responsible for maintaining the piece of state, and keeping track of whether the modal is open or not.
        1. `Modal` should keep the state internally (encapsulated)
            1. We should simply be able to pass content & display inside the modal
    2. `cloneElement`
        1. uncommon - can lead to fragile code
            1. Don't overuse
        2. Let you construct a new React element using another element as a starting point

           `const clonedElement = cloneElement(element, props, ...children);`
            1. Usage: overriding props of an element
2. Code: Modal.jsx

```
import styled from "styled-components";
import PropTypes from "prop-types";
import { HiXMark } from "react-icons/hi2";
```

```
import { createPortal } from "react-dom";
import { cloneElement, createContext, useContext, useState } from "react";

const StyledModal = styled.div`
  position: fixed;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  background-color: var(--color-grey-0);
  border-radius: var(--border-radius-lg);
  box-shadow: var(--shadow-lg);
  padding: 3.2rem 4rem;
  transition: all 0.5s;
`;

const Overlay = styled.div`
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100vh;
  background-color: var(--backdrop-color);
  backdrop-filter: blur(4px);
  z-index: 1000;
  transition: all 0.5s;
`;

const Button = styled.button`
  background: none;
  border: none;
  padding: 0.4rem;
  border-radius: var(--border-radius-sm);
  transform: translateX(0.8rem);
  transition: all 0.2s;
  position: absolute;
  top: 1.2rem;
  right: 1.9rem;

  &:hover {
    background-color: var(--color-grey-100);
  }

  & svg {
    width: 2.4rem;
    height: 2.4rem;
    /* Sometimes we need both */
    /* fill: var(--color-grey-500);
    stroke: var(--color-grey-500); */
    color: var(--color-grey-500);
  }
`;

const ModalContext = createContext();

function Modal({ children }) {
  const [openName, setOpenName] = useState("");

  const close = () => setOpenName("");
  const open = setOpenName;

  return (
    <ModalContext.Provider value={{ openName, close, open }}>
      {children}
    </ModalContext.Provider>
  );
}

function Open({ children, opens: opensWindowName }) {
  const { open } = useContext(ModalContext);

  return cloneElement(children, { onClick: () => open(opensWindowName) }); // clones the element with new props
}

function Window({ children, name }) {
  const { openName, close } = useContext(ModalContext);

  if (name !== openName) return null;

  return createPortal(
    <Overlay>
      <StyledModal>
        <Button onClick={close}>
          <HiXMark />
        </Button>
        <div>{cloneElement(children, { onCloseModal: close })}</div>
      </StyledModal>
    </Overlay>,
    document.body
  ); // **(M)**. The returned element is a child of `document.body`. The modal is still in the same place in the Component tree.
}

Modal.Open = Open;
Modal.Window = Window;

Window.propTypes = {
  children: PropTypes.object,
  name: PropTypes.string,
};

Modal.propTypes = {
  children: PropTypes.array,
  onClose: PropTypes.func,
};
```

```
export default Modal;
```

3. Code: AddCabin.jsx

```
import Button from "../../ui/Button";
import CreateCabinForm from "./CreateCabinForm";
import Modal from "../../ui/Modal";
import CabinTable from "./CabinTable";

function AddCabin() {
  return (
    <Modal>
      {/* State is maintained here, inside the Modal */}
      <Modal.Open opens="cabin-form">
        <Button>Add new cabin</Button>
      </Modal.Open>
      <Modal.Window name="cabin-form">
        <CreateCabinForm />
      </Modal.Window>

      <Modal.Open opens="table">
        <Button>Show table</Button>
      </Modal.Open>
      <Modal.Window name="table">
        <CabinTable />
      </Modal.Window>
    </Modal>
  );
}

export default AddCabin;
```

Detecting a Click Outside the Modal

1. Code: useOutsideClick.js

```
import { useEffect, useRef } from "react";

export function useOutsideClick(handler, listenCapturing = true) {
  const ref = useRef();

  useEffect(() => {
    function handleClick(e) {
      if (ref.current && !ref.current.contains(e.target)) handler();

      // ref.current: DOM node that references `StyleModal` element will be stored here
      // contains: returns true if the current element contains the passed element
    }

    document.addEventListener("click", handleClick, listenCapturing); // `true`: event will be handled only in capturing phase (when it moves down the

    return () => document.removeEventListener("click", handleClick);
  }, [handler, listenCapturing]);

  return ref;
}
```

Confirming Cabin Deletions

1. We need to be asked if cabin needs to be deleted.

Building a Reusable Table

1. Using compound component pattern

Applying the Render Props Pattern

1. Code: CabinTable.jsx

```
function CabinTable() {
  const { isLoading, cabins } = useCabins(); // re-usable

  if (isLoading) return <Spinner />;

  return (
    <Table columns="0.6fr 1.8fr 2.2fr 1fr 1fr 1fr">
      ...
      <Table.Body
        data={cabins}
        render={(cabin) => <CabinRow cabin={cabin} key={cabin.id} />}
      />
    </Table>
  );
}
```

2. Code: Table.jsx

```
...
function Body({ data, render }) {
  if (!data.length) <Empty>No data to show at the moment</Empty>;

  return <StyledBody>{data.map(render)}</StyledBody>;
}
...
```

Building a Reusable Context Menu

1. When a button is clicked, a small context menu appears with the actions of duplicating, editing, and deleting.

Section 29: [Optional] Implementing More Features: Authentication, Dark Mode, Dashboard, etc

Section Overview

1. Topics:
    1. Building real-world app features
    2. Filter, sort, pagination
    3. Dark mode
    4. Dashboard with charts
    5. Authentication and authorization
        1. Only logged in users can access the application and data
            1. Powered by Supabase

Client-Side Filtering: Filtering Cabins

1. Ability to filter cabin data on the client side - make it re-usable (to be used in the future)
2. Cabin table must be filterable
    1. Only cabins with discount
    2. Only cabins without a discount
3. URL - filter criteria is stored here

Client-Side Sorting: Sorting Cabins

1. Dropdown menu
    1. Which field we want to sort data by

Building the Bookings Table

Uploading Sample Data

API-Side Filtering: Filtering Bookings

1. We want to do filtering on the server side (API side)
    1. API should send only the bookings that have a particular status
2. Code: useBookings.js

```
import { useQuery } from "@tanstack/react-query";
import { getBookings } from "../../services/apiBookings";
import { useSearchParams } from "react-router-dom";

export function useBookings() {
  // It flexibly reads filter value from the URL
  const [searchParams] = useSearchParams();

  // FILTER
  const filterValue = searchParams.get("status");
  const filter =
    !filterValue || filterValue === "all"
      ? null
      : { field: "status", value: filterValue };
  // { field: "totalPrice", value: 5000, method: "gte" };

  const {
    isLoading,
    data: bookings,
    error,
  } = useQuery({
    queryKey: ["bookings", filter], // uniquely identifies data we are going to query. Whenever filter changes, React Query re-fetches data (dependenc
    queryFn: () => getBookings({ filter }), // Function which is used for querying. Needs to return a promise
  });
  // console.log(x); // We get data, error, states (isLoading, isPaused, isStale, ..., status: "loading")

  return { isLoading, error, bookings };
}
```

API-Side Sorting: Sorting Bookings

Building a Reusable Pagination Component

API-Side Pagination: Paginating Bookings

Prefetching with React Query

1. What is prefetching
    1. Fetching some data that we know might become necessary before we need the data to render it on the UI
        1. We fetch the next page before it is displayed
            1. If page=7, we have page=8 in cache
                1. It loads is on the fly otherwise
2. Code: useBookings.js

```
  // PRE-FETCHING'
  const pageCount = Math.ceil(count / PAGE_SIZE);

  if (page < pageCount)
    queryClient.prefetchQuery({
      queryKey: ["bookings", filter, sortBy, page + 1],
      queryFn: () => getBookings({ filter, sortBy, page: page + 1 }),
    });
```

```
if (page > 1)
  queryClient.prefetchQuery({
    queryKey: ["bookings", filter, sortBy, page - 1],
    queryFn: () => getBookings({ filter, sortBy, page: page - 1 }),
  });
```

3. Infinite Queries - for infinite scroll
    1. React Query feature - Documentation

Building the Single Booking Page

1. Rule: A page should not fetch data and should not have any side-effects (makes pages folder a lot cleaner)

Checking in a Booking

Adding Optional Breakfast

Checking Out a Booking (+ Fixing a Small Bug)

Deleting a Booking

1. `onSettled` - always executed whether success or not.

Authentication: User Login with Supabase

1. Supabase
    1. Authentication tab
        1. Providers
            1. Email provider - enabled by default
                1. Deactivate Confirm email
            2. Github
            3. Azure
            4. ...
        2. Users
            1. Add user
                1. Email: ...
                2. Password: ...
                3. Create user
        3. API Docs
            1. Authentication
                1. JWT is used
            2. User Management
                1. Sign up
                2. Sign in
                    1. Log in With Email/Password
                3. OAuth
                4. Reset
                5. Updating
                6. Logging out
    2. Supabase changes the auth token in local storage
2. Authorization - protecting routes from access to non-authenticated users

Authorization: Protecting Routes

1. Each time we reload a page, we want to fetch the user and verify

User Logout

Fixing an Important Bug

1. `queryClient.setQueryData(["user"], user.user);`

Building the Sign Up Form

1. Since the application is used only by internal staff, so users can be signed up in the application

User Sign Up

1. Turn on Confirm email
    1. Sign up works only if they have confirmed the email address
        1. Email Templates:
        2. URL Configuration (need to be updated when deployed to a production server)
            1. Site URL: http://localhost:5173/dashboard (for redirection)
            2. Redirect URL: http://localhost:5173
    2. temp-mail.org - for temporary emails
        1. Copy the email
            1. We get emails into inbox
    3. New user
        1. New email address
        2. Any password
            1. New user gets created in Users table
                1. Waiting for verification
                    1. We need to open the email and click on confirm your email

Authorization on Supabase: Protecting Database (RLS)

1. Row Level Security policies
    1. Malicious user can figure out URLs from the HTML pages
        1. Solution: RLS
            1. Policies
                1. Apply all to authenticated users only
                    1. Target role: authenticated

Building The App Header

Updating User Data and Password

1. Storage
    1. Policies
        1. Custom access
            1. Policy name: Allow access for auth users
            2. Select, Insert, Update, Delete
            3. Authenticated (users)
        2. We can change policies to allow only authenticated users

Implementing Dark Mode with CSS Variables

Building the Dashboard Layout

Computing Recent Bookings and Stays

Displaying Statistics

Displaying a Line Chart with the Recharts Library

1. Recharts: `npm i recharts@2`
    1. Go through the documentation

Displaying a Pie Chart

Displaying Stays for Current Day

Error Boundaries

1. Errors that might occur during React rendering
    1. Getting white screen in production is unaccaptable
        1. Solution: Error Boundaries
            1. Like try-catch for React (quite hard to use in React - Use class components)
                1. Solution: `npm i react-error-boundary`
                    1. Error boundaries only catch errors while React is rendering
                        1. Don't catch: event handler errors, bugs in async code are not caught

Final Touches + Fixing Bugs

1. `e.stopPropagation()`
    1. The `stopPropagation()` method of the `Event` interface prevents further propagation of the current event in the capturing and bubbling phases.
2. React Dev tools don't appear in production
3. Suggestions:
    1. New bookings from the application
    2. Edit booking
    3. Add checkin and checkout times
    4. Implement changing prices for cabins for each day
    5. Add restaurant
        1. A new page - register a bill value when customer goes to restaurant
    6. Checkout can lead to a page that generates PDF invoice and mail to user

Section 30: Deployment with Netlify and Vercel

Section Overview

1. Topics:
    1. Let's finish by deploying our project
    2. Top hosting providers: Netlify and Vercel
        1. Upload prod build to Netlify
    3. Git and GitHub
    4. Continuous deployment (with Vercel - Similar to what real companies do)

Deploying to Netlify

1. Application bundle

    1. Vite takes all the files and bundle them into a single file which is deployed to production (and other related files)

```
npm run build
```

    2. If the bundle is too big, we could use code-splitting, but the application is behind a login and used internally, so it is not required.
    3. Generates: `dist` folder

2. https://netlify.com
    1. VS Code
        1. `dist/netlify.toml` (also copy into the main `public` folder (it is automatically copied to the `dist` folder))

```
[[redirects]]
from = "/*"
to = "/index.html"
status = 200
```

    2. Login with Github
        1. Add new site
            1. Deploy manually
                1. Drag and drop `disk` folder
    3. Site overview
        1. Site settings: the-wild-oasis (it must be available)
    4. We can buy our own domain name
        1. Domain settings
    5. Deploy:
        1. Deploys - Drag and drop

Setting Up a Git and GitHub Repository

Deploying to Vercel

1. Free for small projects. Good hosting provider for React projects
2. Netlify
    1. New site
        1. Import an existing project
            1. Github Account
            2. Select the project
3. https://vercel.com
    1. For Next.js and React.js apps
    2. Sign up
        1. Continue with GitHub
            1. Add new
                1. Select git repo from the list
                    1. Import
                        1. Framework - Vite
                        2. Build and Output Settings
                            1. Build command - npm run build (override is not required) - Vercel will do it
                                1. Each time we commit and push to GitHub, vercel will detect and run the command and deploy
                        3. Deploy
                            1. Continue to Dashboard
                                1. Domain - Vercel chooses based on the name of the repo
            2. Settings
    3. Preferred way to deploy - makes is easy to push changes.

Section 31: PART 5: FULL-STACK REACT WITH NEXT.JS [1 PROJECT]

Introduction to Part 5

1. Taking React to the server
2. Taking advantage of cutting edge features
    1. Using Next.js
3. Topics
    1. Wind Oasis website
        1. Server side rendering
        2. File-based routing
        3. Suspense
        4. Server components
4. Almost mandatory for React JS developers to master Next.js

Useful Resources for Part 5

1. High quality resources while studying this part
    1. 28 advanced Nest.js features that everyone should know
        1. A must read for everyone after finishing the course
    2. How React server components work
        1. React Server Components - RSC
            1. Few more visualizations
    3. RSC DevTools
        1. Nice way of visualizing RSC payload in Chrome's DevTools
    4. Next.js Image Component Tutorial
        1. Going deep into `<Image />` component
    5. Understanding Next.js cache
        1. Caching article
    6. Static Site Generation (SSG) Next.js documentation page
    7. High-quality open-source Next.js app for learning

        1. Codebase to learn a lot from
    2. Framework/Library documentation
        1. Next.js documentation
        2. React Server Components documentation
            1. Important section
        3. Auth.js documentation

Section 32: Overview of Next.js with the "App" Router

Section Overview

1. Full-stack server-side rendered React Apps with Next.js
2. Topics
    1. What is **server-side rendering (SSR)** and why do we need it?
    2. What is **Next.js**?
        1. App router
    3. Deep dive into **React Server Components**
        1. Diagrams

Download Fresh Starter Files + Slides!

1. https://www.udemy.com/course/the-ultimate-react-course/learn/lecture/38736100#content
    1. ultimate-react-course

An Overview of Server-Side Rendering (SSR)

1. What is SSR and when should we use it

Review: The Rise of Single-Page Applications

1. SSR:
    1. Old days
        1. Webpages were rendered on the server and sent to the browser
    2. Modern days
        1. A shift to SSR for certain types of apps
        2. Powered by
            1. Next.js
            2. Remix
            3. Nuxt.js
            4. SwelteKit
            5. SolidStart
        3. The frameworks combine both aspects of SSR and CSR
            1. Blending both
2. CSR:
    1. Modern way
        1. Webpages are rendered on the browser
            1. Frameworks to do that
                1. React
                2. Vue
                3. Swelte
                4. Angular
        2. We will continue with this for many types of applications

Client-Side Rendering (CSR) vs Server-Side Rendering (SSR)

1. CSR
    1. HTML is rendered on the **client** (the user's computer) using JS
        1. Generated on the client (user's computer)
            1. React
    2. **Problems**
        1. Slower initial page loads
            1. Bigger JS bundle needs to be downloaded before app starts running
                1. It must be completely downloaded until anyting happens on the page
            2. Data is fetched after components mount (apps require them)
                1. After they are rendered on the client
                2. Request waterfall
                    1. **Slows down the entire experience** - main criticism
        2. SEO can be problematic (if it is a concern)
            1. Content is not rendered until after JS is executed and data is fetched
                1. **Search engines might find a blank page, when they try to index the site**
            2. **It is getting better with SPAs now**
    3. **Advantage**
        1. Highly interactive (UX):
            1. All the code and content has already been loaded (except some data)
                1. SSP application feel
        2. **SPAs**: Perfect for building highly interactive web apps
        3. **Apps that don't need SEO**
            1. Apps that are used "internally" as tools inside companies
            2. Apps that are entirely hidden behind a login (not accessible to averyone)
2. SSR
    1. HTML is rendered on the **server** (the developer's computer)
        1. Server sends already generated website to the client whenever requested
            1. Rendering is transfered to developer's computer
    2. Done by

        1. PHP
        2. Next.js
    3. **Advantages**
        1. Faster initial page loads
            1. Less JS needs to be downloaded and executed
                1. Client doesn't need JS to render the HTML (already rendered on the server)
            2. Data is fetched (on the server) before HTML is rendered
                1. Data is incorporated into the page that is sent to browser
        2. **Less interactive**
            1. Pages might be downloaded on demand and require full page reloads
                1. Navigating from page to page requires server to render a new page each time
                    1. Next.js blurs the line
                        1. **SSR pages get hidrated on the client to become interactive later**
        3. **SEO-friendly**
            1. Content is easier for search engines to index
                1. Pre-generated content is much easier for search engines (Google) to index
            2. If this is super important, then SSR can be chosen
                1. **Content-driven websites or apps where SEO is essential:** E-commerce, blogs, news, marketing websites, etc. (SSR is a great choice)
                    1. SEO is crucial for users to get onto the site
        4. Two types of SSR:
            1. Static: HTML generated at **build time** (often called Static Site Generation, or SSG)
                1. Once development is done, static HTML, CSS, and JS files are generated which are deployed on a web server
                    1. Webserver will not re-generate market each time. It will send the markup generated by the developer in the beginning
            2. Dynamic: HTML generated each time **server receives new request** (some call only this SSR)
                1. Generates new pages for each user
                    1. When underlying data changes often
                        1. Some think this is true SSR

Typical Timeline of CSR vs. SSR With Data Fetching

1. CSR
    1. Server
        1. Step 1: Empty page (HTML, CSS, JS (heavy bundle - entire app logic)) sent to client
        2. Step 4: Fetch data (from the same or different server)
    2. Client
        1. Step 2: Download JS bundle
        2. Step 3: Render spinner (for data fetch)
            1. First paint (FCP) - a metric
                1. Something is painted onto the screen
        3. Step 5: Render app with data
            1. Largest Contentful Paint (LCP) - a metric
                1. Time when all the relevant content is shown to the user
2. SSR
    1. Server
        1. Step 1: Fetch data (relevant)
            1. Server initiates data fetching (before app is rendered)
        2. Step 2: Render app with data
            1. Rendering is moved from client to the server
                1. Useful for content-heavy sites
    2. Client
        1. Step 3: Whole finished product is sent to client (HTML, CSS, JS)
            1. First paint (FCP)
            2. Content paint (LCP) - happens here (much earlier than in CSR)
                1. Hence initial page load is much faster
        2. Step 4: Download JS Bundle
            1. Downloaded and executed
        3. Step 5: Hydration
            1. It is a process where static HTML becomes interactive by adding JS to it

Experiment: Manual SSR With React DOM + Node.js

1. Nodemon - to restart server whenever there are changes
    1. Now it is built into node
        1. `node --watch server.js`
    2. To support JSX:
        1. `npm i -D @babel/core @babel/preset-env @babel/preset-react @babel/register`
        2. `npm i react react-dom`
    3. Issue with server side rendered React:
        1. The controls (buttons, links) don't work - not interactive
2. Code:

```
<!DOCTYPE html>
<html lang="en">
  <!-- index.html -->

  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>

  <body>
    <div id="root">%%%CONTENT%%%</div>
  </body>
</html>
```

```
// server.js
const { readFileSync } = require("fs");
```

```
const { createServer } = require("http");
const { parse } = require("url");
const { renderToString } = require("react-dom/server");
const React = require("react");

const pizzas = [
  {
    name: "Focaccia",
    price: 6,
  },
  {
    name: "Pizza Margherita",
    price: 10,
  },
  {
    name: "Pizza Spinaci",
    price: 12,
  },
  {
    name: "Pizza Funghi",
    price: 12,
  },
  {
    name: "Pizza Prosciutto",
    price: 15,
  },
];

function Home() {
  return (
    <div>
      <h1>🍕 Fast React Pizza Co.</h1>
      <p>This page has been rendered with React on the server 🤯</p>

      <h2>Menu</h2>
      <ul>
        {pizzas.map((pizza) => (
          <MenuItem pizza={pizza} key={pizza.name} />
        ))}
      </ul>
    </div>
  );
}

function Counter() {
  const [count, setCount] = React.useState(0);
  return (
    <div>
      <button onClick={() => setCount((c) => c + 1)}>+1</button>
      <span>{count}</span>
    </div>
  );
}

function MenuItem({ pizza }) {
  return (
    <li>
      <h4>
        {pizza.name} (${pizza.price})
      </h4>
      <Counter />
    </li>
  );
}

const htmlTemplate = readFileSync(`${__dirname}/index.html`, "utf-8");

const server = createServer((req, res) => {
  const pathName = parse(req.url, true).pathname;

  if (pathName === "/") {
    const renderedReact = renderToString(<Home />);
    const html = htmlTemplate.replace("%%%CONTENT%%%", renderedReact);
    res.writeHead(200, { "Content-Type": "text/html" });
    res.end(html);
  } else if (pathName === "/test") {
    res.end("TEST");
  } else {
    res.end("The URL cannot be found");
  }
});

server.listen(8000, () => console.log("Listening for requests on port 8000"));

// start.js
require("@babel/register")({ extensions: [".js", ".jsx"] }); // allows using JSX
require("./server.js");

{
  "name": "manual-ssr",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js"
  },
  "author": "",
  "license": "ISC",
  "devDependencies": {
    "@babel/core": "^7.26.0",
    "@babel/preset-env": "^7.26.0",
    "@babel/preset-react": "^7.25.9",
    "@babel/register": "^7.25.9"
```

```
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  }
}
```

The Missing Piece: Hydration

1. Hydration
    1. What is hydration?
        1. React component tree - on server (written using Next.js)
            1. SSRd HTML (HTML is generated on server side)
            2. Rendered webpage HTML (client side) - LCP (Content paint)
                1. It is just HTML 2. Solution: Hydration
                    1. Adds back the **interactivity and event handlers** that were lost when HTML was server-side rendered - rendering HTML on the server will remove all the interactivity.
                    2. Analogy - Watering the "dry" HTML with the "water" of interactivity and event handlers
                    3. The rendered webpage should work exactly like the React app we started with
        2. Implementation:
            1. The React bundle is sent as well (HTML page also downloads React bundle)
            2. The bundle hydrates the rendered static DOM of the web-page
                1. React builds the component tree on the client and compares it with the actual SSRd DOM [currently on the page]: **They must be the same** so React can adopt it (the existing DOM)!
                    1. React then attach events to existing DOM and fire of existing effects
                        1. It tries to adopt already existing DOM.
                            1. It continues and finishes the process of SSR
                                1. End result is exact same React App we started with but on the client - Page then becomes interactive (if page changes after hydration, it would be bad user experience) - if there is difference, we get hydration error (not common)
                                    1. Common reasons for hydration errors:
                                        1. Incorrect nesting of HTML elements (using `div` inside a `p` say)
                                        2. Different data used for rendering (on server and client)
                                        3. Using browser-only APIs
                                            1. APIs that exist only in the browser - `window`, `localstorage`
                                        4. Side effects
                                            1. Incorrect use of side-effects
                                        5. ...

Implementing Hydration

1. Instead of converting the entire JS into string and send all at once, frameworks like Next.js can stream the content piece by piece to the client.
2. Code:

```
// client.js
ReactDOM.hydrateRoot(document.getElementById("root"), <Home />); // trees from server and client must be the same

// Module bundler takes care of injecting this into client
// Content appears before the controls
const pizzas = [
  {
    name: "Focaccia",
    price: 6,
  },
  {
    name: "Pizza Margherita",
    price: 10,
  },
  {
    name: "Pizza Spinaci",
    price: 12,
  },
  {
    name: "Pizza Funghi",
    price: 12,
  },
  {
    name: "Pizza Prosciutto",
    price: 15,
  },
];

function Home() {
  return (
    <div>
      <h1>🍕 Fast React Pizza Co.</h1>
      <p>This page has been rendered with React on the server 🤯</p>

      <h2>Menu</h2>
      <ul>
        {pizzas.map((pizza) => (
          <MenuItem pizza={pizza} key={pizza.name} />
        ))}
      </ul>
    </div>
  );
}

function Counter() {
  const [count, setCount] = React.useState(0);
  return (
    <div>
      <button onClick={() => setCount((c) => c + 1)}>+1</button>
      <span>{count}</span>
    </div>
  );
```

```
}

function MenuItem({ pizza }) {
  return (
    <li>
      <h4>
        {pizza.name} (${pizza.price})
      </h4>
      <Counter />
    </li>
  );
}

// server.js
const { readFileSync } = require("fs");
const { createServer } = require("http");
const { parse } = require("url");
const { renderToString } = require("react-dom/server");
const React = require("react");

const pizzas = [
  {
    name: "Focaccia",
    price: 6,
  },
  {
    name: "Pizza Margherita",
    price: 10,
  },
  {
    name: "Pizza Spinaci",
    price: 12,
  },
  {
    name: "Pizza Funghi",
    price: 12,
  },
  {
    name: "Pizza Prosciutto",
    price: 15,
  },
];

function Home() {
  return (
    <div>
      <h1>🍕 Fast React Pizza Co.</h1>
      <p>This page has been rendered with React on the server 🤯</p>

      <h2>Menu</h2>
      <ul>
        {pizzas.map((pizza) => (
          <MenuItem pizza={pizza} key={pizza.name} />
        ))}
      </ul>
    </div>
  );
}

function Counter() {
  const [count, setCount] = React.useState(0);
  return (
    <div>
      <button onClick={() => setCount((c) => c + 1)}>+1</button>
      <span>{count}</span>
    </div>
  );
}

function MenuItem({ pizza }) {
  return (
    <li>
      <h4>
        {pizza.name} (${pizza.price})
      </h4>
      <Counter />
    </li>
  );
}

const htmlTemplate = readFileSync(`${__dirname}/index.html`, "utf-8");
const clientJS = readFileSync(`${__dirname}/client.js`, "utf-8");

const server = createServer((req, res) => {
  const pathName = parse(req.url, true).pathname;

  if (pathName === "/") {
    const renderedReact = renderToString(<Home />);
    const html = htmlTemplate.replace("%%%CONTENT%%%", renderedReact);
    res.end(html);
  } else if (pathName === "/client.js") {
    res.writeHead(200, { "Content-Type": "application/javascript" });
    res.end(clientJS);
  } else {
    res.end("The URL cannot be found");
  }
});

server.listen(8000, () => console.log("Listening for requests on port 8000"));

<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
  <head>
```

```
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>

  <body>
    <div id="root">%%%CONTENT%%%</div>

    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
    <script
      crossorigin
      src="https://unpkg.com/react@18/umd/react.development.js"
    ></script>
    <script
      crossorigin
      src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
    ></script>
    <script type="text/babel" src="client.js"></script>
  </body>
</html>
```

What is Next.js?

1. Next.js
    1. The React Framework for the Web - Vercel
        1. A meta-framework built on top of React: we still use components, props, react hooks, etc. 2. It is a framework on top of another framework
        2. **Opinionated way of building React apps**: Set of conventions and best practices regarding routing, data fetching etc.
            1. The conventions are given by Engineers at Vercel
                1. We can avoid boilerplate code & start building app
                2. Fixed set of rules makes building apps as a team a lot easier
                3. If everyone knows Next.js and follows the convention, then applications get built faster, fewer bugs, better developer experience
        3. Allows us to build **complex full-stack** web apps and sites
            1. Data fetching
            2. Data mutations on the server
        4. Allows us to use **cutting-edge React** features that need to be integrated into a framework: Suspense, Server Components, Server Actions, streaming, etc.
            1. Nearly impossible to integrate without a framework
            2. The pieces are part of the React team's full-stack architecture vision (requires a framework to implement)

The Next.JS Key Ingredients

1. Server-Side Rendering (Dynamic and Static)
    1. Dynamic or static can be **selected** for each route
2. File-based routing conventions (file-based convention)
    1. **Folders** as routes
        1. Folder name is route-name
    2. Special **files** for pages, layouts, loaders, errors, etc.
3. Data fetching and mutation on the server (within React app running on a server)
    1. Fetching data directly in **Server Components**
    2. Mutations in **Server Actions**
4. Optimizations
    1. Images
    2. Fonts
    3. SEO
    4. Preloading (automatic route preloading)

Two Flavours of Next.JS: "APP" and "PAGES" Router

1. Modern Next.JS: "App" Router (completely different from the other)
    1. Introduced in Next.js 13.4 (2023)
    2. Recommend for **new projects**
        1. Implements **React's full-stack architecture**: Server Components, Server Actions, Streaming, etc.
            1. Available only in the modern App router
    3. Easy fetching with `fetch()` right in components (even in server-components)
    4. Extremely easy to construct **layouts**, loaders, error boundaries, etc.
    5. More advanced routing (parallel routing, intercepting routes, etc.)
    6. Better DX (Developer Experience) and UX (because of server components)
    7. **Caching** is very aggressive and confusing
        1. It many times does things we don't want
    8. Steep learning curve (but it's React - new React features)
        1. More capable framework
2. Legacy Next.JS: "Pages" Router
    1. The first Next.js router since v1 (2016)
    2. Still supported and updated in the future
    3. Overall more **simple and easy** to learn
    4. Simple things like **layouts** are confusing to implement
    5. Data fetching using Next.js-specific APIs such as **getStaticProps** and **getServerSideProps** (not native React functions. They only work at page level and not at individual React component level)
3. We're gonna learn the "app" router from the start. At the end, there is a section on the fundamentals of the "pages" router (for older Next.js project)

Setting up a Next.js Project

1. `npx create-next-app@14 the-wild-oasis-website`
    1. All defaults except TS
2. Next.JS is also a Node.JS app (uses it)
3. Starting: `npm run dev`

Frequent Next.js Updates + Documentation

1. Keeping up with framework is an essential skill for web developer
    1. Next.js documentation - must read as a professional developer
    2. to update Next.JS version:

```
npm install next@latest react@latest react-dom@latest eslint-config-next@latest
```

    3. Next.js blog
        1. New versions are announced here

Defining Routes and Pages

1. Each page in a react router is a page component that we export from a page.js file
2. Custom labels for pages in VS Code:
    1. Settings
        1. custom labels:
           Add Items
            1. `**/app/**/page.js: PAGE ${dirname}`
3. Conventions:
    1. `page.js`
    2. `loading.js`
    3. `error.js`

Navigating Between Pages

1. `Link` - applies a few optimization techniques
    1. Prefetches all routes linked on a page (in production)
    2. Each page is downloaded separately as separate chunk
    3. Each page we visit is cached in the browser (stored temporarily)
        1. Not re-fetched again
2. Next JS also provides some React hooks for programmatic navigation
    1. They don't work in page components (they are for server components)
3. Next.js can cache pages and can serve old pages.

Constructing a Layout

1. layout.js
    1. The root layout contains the HTML returned to client.
        1. HTML and body are mendatory
        2. Whatever is in each page is what is returned from the RootLayout
        3. The rest of the content is returned by `children` prop
            1. It will always be the current page
        4. Navigation part of the layout will not be re-rendered as we navigate through the application
    2. We can export page metadata from the layout
        1. Page metadata - page's title say

```
export const metadata = {
  title: "The Wild Oasis",
};
```

        2. We can export variables - there are conventions

What are React Server Components? (RSC - Part 1)

1. RSC - biggest and hardest in React

Why React Server Components?

1. UI = f(state) - function of state changing over time
    1. If 100% Client-Side - As we keep updating state, the app keeps re-rendering (showing differet parts of UI and layout)
        1. We "hack" this by storing fetched **data** in **state** too
        2. Advantages:
            1. Apps are super interactive
            2. Components - re-usable packages
        3. Disadvantages:
            1. Requires lots of JS to be downloaded
                1. Devs needs to constantly think about it
            2. Client-server data waterfalls
                1. Happens when client on a page needs to fetch different data one after another
                    1. When data from one component depends on data fetched in another component (large apps)
    2. 100% Server-Side
        1. PHP - no interactivity
        2. Disadvantages:
            1. No components
            2. Easy and fast to **fetch all data** (from data source)
            3. Close to the **data source**
                1. No need to build API to communicate between front-end and backend
            4. Needs to ship **0 kb** of JS - faster page load for users
            5. UI = f(data)
                1. The data that determines the content of the UI
2. UI = f(data, state) - How to achieve this?
    1. Closer to what a real-world app is
    2. How to achieve a gradient?
        1. What if we could take the best of both worlds?

1. Having React components both on server and on client
    1. Be either interactive or be close to data source and ship on JS at all
        1. Solution: RSC

What are React Server Components?

1. Back to pure React
    1. React Server Components (RSC)
        1. A new **full-stack architecture** for React apps
            1. It introduces server as a first-class citizen in React apps
        2. Introduces the server as an integral part of React component trees: **server components**
            1. React tree extends all the way to the server like a bridge that closes the gap between client and server
                1. By introducing Server Components
                    1. RSC - name of the whole paradigm/architecture
                        1. Server Component - Name of new component type introduced by RSC
                            1. These components are only rendered on the server and never on the client
                                1. They fetch data on the server
                                2. They make part of UI = f(data)
                                3. Don't make it into the bundle (0 kb)
                                    1. Enables us to build **back-end also with React**
2. Regular components are called client components
3. We write **frontend code** next to **backend code** in a natural way that "feels" like regular React
    1. We can also compose server and client components together
        1. Full-stack apps entirely controlled by React
4. RSC is **NOT** active by default in new React apps (e.g. Vite apps): it needs to be **implemented** by a framework like **Next.js** ("app router")
    1. Any app we build in the Next.js `app` folder will use the new RSC architecture in which SCs become default components
        1. Default in apps that use the RSC architecture (like Next.js)
            1. Client components become opt-in
                1. Defined with `"use client"` directive at the top of the module

An Example + The Server-Client Boundary

1. Sidebar, header, main elements are not interactive
    1. They could be rendered on server using SCs without shipping JS
    2. Structure:
        1. App
            1. Sidebar (not interactive)
            2. Header (not interactive)
                1. Avatar (not interactive) - can be fetched on the server & avoid client/server waterfall
                2. DarkMode (interactive) - cannot be rendered on the server
            3. Main (not interactive)
                1. Table (not interactive)
                2. CabinRow (fetches all cabin data and rendering it) (non interactive)
                    1. Menu (interactive) - "use client"
                        1. Duplicate (interactive) - "use client" is not required because it is already in the client-side boundary (it is under a client component)
                        2. Edit (interactive)
                        3. Delete (interactive)
                3. SortBy (interactive)
        2. Component tree contains both backend and frontend code (mixed tog
2. Server-Client Boundary
    1. Split point between server and client code
    2. Client sub-tree. Child modules need no "use client"
        1. Subtrees are only executed in the browser

Server Components vs Client Components

1. Client components
    1. "use client" needs to be used at the top of the module
    2. Only these can have state/hooks
    3. Lifting state up - possible
    4. Props - yes
    5. Data fetching - Possible, preferably with 3rd party library (React Query say)
    6. Can import (component module imports another module using `import`)
        1. Can import only client components (can't go back in the client-server boundary)
    7. Can render (one component calls another one - uses inside it's JSX body)
        1. Client components can render both client components and server components as long as they are **passed as props**
    8. When re-render?
        1. On **state** change (own state or parent state)
2. Server components
    1. Default components
    2. They cannot be stateful and cannot use any hooks
    3. Lifting state up - not possible
    4. Props - Yes (must be serializable (must be convertable to a format that can be transfered) when passed to client components. **No functions** or classes (because they are no serializable))
    5. Data fetching - Preferred. Use async/await in component (in the top-level code, too - doesn't work in client-side component)
        1. First-class way of fetching data
        2. To pass data to client, we pass them as props
    6. Can import (component module imports another module using `import`)
        1. They can import both client and server components
    7. Can render (one component calls another one - uses inside it's JSX body)
        1. Can render client and server components
    8. When re-render?
        1. Yes. On URL Change (navigation)
            1. When user navigates to the URL that has a server component in question
                1. Because server components are tied to specific routes in the framework

2. When URL changes to a route, all server components associated with the route re-render
    1. They might fetch data again

A Simple New Mental Model

1. "Traditional" React
    1. Components -display-> View
    2. View -interaction-> updated state
    3. updated state -re-render-> component
    4. Components -fetch data-> stored state
    5. updated state -re-render-> component
2. React with RSC
    1. Client components:
        1. Components -display-> View
        2. View -interaction-> updated state
        3. updated state -re-render-> component
        4. Components -fetch data-> stored state
        5. updated state -re-render-> component
    2. Server components
        1. Components -display-> View
        2. Components -fetch data-> to display in View / send props to client components
            1. Props are bridge across client/server boundary
        3. View -navigation (updates URL)-re-renders-> Component
            1. Triggers:
                1. Component -fetch data-> View (update)

The Good and Bad of the RSC Architecture

1. The good:
    1. We can write React components on the front-end and on the back-end
        1. We can encapsulate server-side concerns into components as well
            1. We can compose entire full-stack apps with **React components alone** (+ server actions (for mutations - later))
                1. **One single codebase** for front and back-end
                    1. API is not required in many situations
                    2. Server components have **more direct and secure access** to the data source (no API, no exposing API keys, etc.)
                        1. We can grab data from a database in a server component, and send already rendered page to browser (instead of sending only the data as json)
                        2. Pure client-side app needs to worry about securely storing API keys
        2. **Eliminate client-server waterfalls** by fetching all the data needed for a page **at once** before sending it to the client (not each component)
            1. Not once piece after another
            2. Can be streamed in one-go
        3. **Disappearing code**: server components ship no JS, so they can import huge libraries "for free" (without increasing bundle size)
            1. Good for libraries used only for rendering data
                1. Libraries that connect to CMS
                2. Libraries for embedding things like tweets
                3. Libraries for syntax highlighting etc.
2. The bad:
    1. Makes React **more complex**
        1. More things to **learn and understand**
            1. After understanding RSC, they are easy to use and reason about
                1. No state
                2. No effects
                3. No stale closures
                4. No memoization
    2. Things like **Context API** don't work in server components (like other hooks)
    3. **More decisions** to make: "Should this be a client or a server component?", "Should I fetch this data on the server or the client?", etc.
    4. Somtimes you still need to **build an API** (for example if you also have a mobile app)
        1. Not a problem with large teams
    5. Can only be used within a **framework**
        1. Alternative: Setting up a Vite app for it is very complex

Fetching Data in a Page

1. Each page in a Next.js application is a server-component
    1. Pages need to be server components
        1. Navigation component is a server component (purely rendered on server)
2. Code: app/cabins/page.js

```
import Navigation from "../components/Navigation";

export default async function Page() {
  // React function as asyn function is new

  const res = await fetch("https://jsonplaceholder.typicode.com/users");
  const data = await res.json(); // data is not sent to the client, but is fetched on server and the rendered HTML page is sent back. Data is cached o

  console.log(data); // prints on terminal where server is running

  return (
    <div>
      {/* <Navigation /> */}
      <h1>Cabins page</h1>

      <ul>
        {data.map((user) => (
          <li key={user.id}>{user.name}</li>
        ))}
      </ul>
    </div>
```

```
  );
}
```

Adding Interactivity with Client Components

1. All components (both client and server) are first rendered on the server and then sent to the client
    1. The client components and the hydrated
2. Code: Counter.js

```
"use client";

import { useState } from "react";

export default function Counter({ users }) {
  const [count, setCount] = useState(0);

  console.log(users);

  return (
    <div>
      <p>There are {users.length} users</p>
      <button onClick={() => setCount((c) => c + 1)}>{count}</button>;
    </div>
  );
}
```

3. Code: app/cabins/page.js

```
// import { useState } from "react";
import Counter from "../components/Counter";
import Navigation from "../components/Navigation";

export default async function Page() {
  // React function as asyn function is new

  const res = await fetch("https://jsonplaceholder.typicode.com/users");
  const data = await res.json(); // data is not sent to the client, but is fetched on server and the rendered HTML page is sent back. Data is cached o

  console.log(data); // prints on terminal where server is running

  // useState(23); // Doesn't work

  return (
    <div>
      {/* <Navigation /> */}
      <h1>Cabins page</h1>

      <ul>
        {data.map((user) => (
          <li key={user.id}>{user.name}</li>
        ))}
      </ul>

      <Counter users={data} />
    </div>
  );
}
```

Displaying a Loading Indicator

1. When data is loaded, it is streaming from server to the client
    1. `renderToString` is not used
        1. `renderToReadableStream` is used by Next JS
            1. Needs JS to be enabled on the browser
2. We can stream individual components using suspense
    1. If a page had 20 components, and if only one component is fetching data, the whole page might be in loading state
        1. Solution: Suspense (later)
3. Code: loading.js

```
export default function Loading() {
  // name does not matter.
  // This is global loader that works at any nesting level
  return <p>LOADING DATA</p>; // Appears while data is getting loaded and then gets swapped with the actual data
}
```

How RSC Works Behind the Scenes (RSC - Part 2)

A Quick Review of Rendering in React

1. "Traditional" React (without RSC) (Skipping, rendering, diffing, reconciliation, ...)
    1. Components
    2. Tree of component instances (Component tree) -Render->
        1. Render (it is just calling component functions)
            1. Result: React element (it is JS object that contains all info necessary to construct DOM elements for the corresponding instance)
    3. React element tree ("Virtual DOM") - Commit to DOM ->
        1. Constructing DOM elements necessary to represent the different elements in React Element Tree ("Virtual DOM")
    4. DOM Elements (HTML)

How RSC Works Behind the Scenes

1. Component Tree
    1. Server Component (SC), Client Component (CC)
    2. Steps of rendering:
        1. SCs are rendered on Server

1. Result in React Elements
    1. They contain the output of SCs
        1. They only contain info on how the DOM for each component will look like (they don't contain the code itself to render the component)
            1. The code from SC has disappeared on the server (reason why we cannot use state, because funtions like `useState` and `useEffect` will disappear as the component is rendered)
                1. The React elements need to be sent to the client - They have to be serializable - functions are not serializable
                    1. We cannot keep state because there is no fiber-tree on the server - even if existed, it cannot be sent to the client
3. Client Components
    1. Client components will not be rendered on the server
    2. The component tree contains a place-holder
        1. Holes where CCs will eventually be rendered
        2. Each placeholder contains:
            1. Serialized props passed from SC to CC
            2. URL to **script** with actual component code
                1. It is necessary because CCs can later to executed on the client
        3. A script with component code and URL pointing to it is complex
            1. It is powered by bundler the framework is using (not by React itself)
4. RSC Payload
    1. Virtual DOM of SC + Trees of CC (un-rendered, placeholders)
    2. The data-structure is sent to the client
        1. JSON like datastructure (not exactly JSON. Data-format came up by React team by splitting data into multiple rows. Helps stream the data easily from server to client)
        2. All are strings (serialized)
        3. URLs or placeholders
5. After the RSC payload is received by client, the client components are rendered into React elements to fill the holes
    1. Complete final VDOM exists at this point that can be comitted to the DOM.
6. Rendering is done in two different environments. First on server, and then on client

2. Why RSC payload? Why not render SCs as HTML?
    1. React describes UI as data, and not as finished HTML
        1. The reason why VDom and Fiber tree exist
            1. Helps React correctly react to re-renders of server components
                1. How?
                    1. Whenever a SC re-renders and produces a new React Element, the Element can be merged seamlessly with the VDOM on the client
                    2. Whenever SC re-renders, a new RSC payload is sent to the client
                    3. React is able to **merge** ("reconcile") the **current tree** on the client with a **new tree** coming from the server (using the mechanisms that exist in React)
                    4. As a result, UI **state can be preserved** when a SC re-renders, instead of completely re-generating the page as HTML
                        1. If HTML is sent, the entire UI needs to be replaced with the new HTML
                            1. All UI state will be lost - bad UX
                                1. RSC payload avoids this issue

Traditional React

```
Component tree
    |
    render
    |
    v
"Virtual DOM"
    |
    commit
    |
    v
DOM Elements
```

React with RSC

```
Component tree of SC and CC
    |
    Render SC
    |
    v
"Virtual DOM" of SC + Component trees of CC (un-rendered) (RSC payload)
    RSC Payload - For each CC in Tree: (contains all data that React needs to render CC)
        1. "Hole" where CC will render
        2. Serialized props from SC
        3. URL to script with code
    |
    Send "RSC Payload" to client
    |
    Render CC
    |
    v
Complete "Virtual DOM"
    |
    Commit
DOM Elements
```

```
1. Steps **don't wait** for one another (not sequential and blocking). Completed render work (on server) is **streamed** to client (and integrated sea
    1. More fluid and faster for the user
2. UI = f(data, state) - not 100% correct
    1. UI = f(data)(state) (data is input of the hypothetical function, and state is the input to the function returned by the hypothetical function)
```

RSC vs. SSR: How are They Related? (RSC - Part 3)

1. SSR & RSC are not the same thing
    1. They interact with each other

Review: Server-Side Rendering (SSR)

1. We'll be talking about **dynamic SSR** (HTML generated at **runtime**)
2. Process:

```
Server                                  Client

Component tree -JS---+> Hydrated (adds interactivity to HTML)
    |                |          |
    render           |          v
    |                | Interactive React App (DOM is generated on the server)
    v                |
HTML -------shipped--+
```

    1. **SSR**: "Just take this component tree, render it as HTML, and send that HTML to the browser"
    2. Also send the React code to make the HTML **interactive**

The Relationship Between SRC and SSR

1. Process:

```
Server                                  Client

What about RSC?
```

    1. **RSC** vs. **SSR**
        1. RSC is **NOT** the same as SSR: they are separate technologies
        2. RSC does **NOT** replace SSR
            1. To complement it (to work with it)
        3. They usually work together:
            1. Frameworks can combine them
                1. Next.JS
        4. SSR works when it is coupled with RSC. ALL components are pre-rendered on the server
            1. **SSR**: "Just take this component tree, render it as HTML, and send that HTML to the browser"
        5. Both client and server components are **initially rendered on the server** when SSR is used

2. Rendering process with Next.JS:

```
Component tree of SC and CC
    |
    Render SC
    |
    v
"Virtual DOM" of SC + Component trees of CC (un-rendered) (RSC payload)
    RSC Payload - For each CC in Tree: (contains all data that React needs to render CC)
        1. "Hole" where CC will render
        2. Serialized props from SC
        3. URL to script with code
    |
    Send "RSC Payload" to client
    |
    Render CC
    |
    v
Complete "Virtual DOM"
    |
    Commit
DOM Elements
```

    1. React Server & React Client
        1. They are just two different environment on the Server
            1. Two different parts of RSC protocol
        2. React Server
            1. It doesn't need to be an actual web-server. Simply "another computer"
    2. **Result**: RSC does NOT require a running web server! Components could run only **once at build time** (static site generation)
        1. Developer can read files from the file-system during development process (on own computer)
    3. React Client:
        1. It doesn't need to be a traditional web-browser
        2. It is only the part of the protocol that consumes the rendered react app
            1. Consuming - To render the app as HTML, instead of DOM elements
                1. SSR takes the entire component tree and renders it as HTML & sends to browser
                    1. Both server components and client components are rendered on the server that Next.JS provides in the two environments on the React server components protocol
                        1. React Server
                        2. React Client
    4. Both React Client and React Server environments run on the server on **initial render**
        1. Output is HTML (not DOM nodes)
            1. Sent to browser
        2. Also works like this when coupled with RSC. ALL components are pre-rendered on the server
    5. Simplified process:

        1. It is similar to the initial diagram

```
Component tree (SC and CC)--JS-+--> Hydrate
    |                          |              |
    render -- RSC Payload -----+              v
    |                          |       Interactive React App
    v                          |
HTML --------------------------+
```

        1. JS - many different chunks (code-splitting and streaming is used)
        2. RSC Payload - Sent with JS so that React has the entire **component tree** on the client, **not just HTML**. Necessary to preserve UI state on future SC re-renders
        3. Only CCs are interactive
        4. One React app is on Client following the RSC architecture

    2. SSR happens only on **initial render**. On re-renders, client components only render on the **actual client**

        1. React server is then just a web-server where Next.JS is running
        2. React client is then just the web-browser

    3. When SC re-renders, a new RSC payload is generated and sent to the client

        1. To preserve the existing UI state

3. Take-aways

    1. Both SCs and CCs are rendered on the server on the initial SSR
    2. After that, SCs run on web-server, and CCs run on web-browser

Section 33: Starting to Build the "Wild Oasis" Website

Section Overview

1. Topics:
    1. Next.js project structure
        1. Clean structure
        2. Different pages
    2. Implementing pages
    3. Font and image **optimization** techniques
    4. Adding metadata and favicon
    5. **Nested** layouts
        1. With multiple `layout.js` files

Project Planning: "The Wild Oasis" Customer Website

The Project: The Wild Oasis Website

1. **Remember**: "The Wild Oasis" is a small boutique **hotel** with 8 luxurious wooden cabins
    1. We built their application to manage everything about the hotel: **bookings**, **cabins**, and **guests**
    2. We also build the **API** using Supabase
    3. Now they need a **customer-facing website** where guests can learn about the hotel, browse all cabins, reserve a cabin, and define and update their profile (before even booking or checking into hotel)
    4. Updating data in the internal app should update the website, so we use the same DB and API

Project Requirements From the Business

1. Users of the app are potenial guests and actual guests (those who planned reservation)
2. Guests should be able to learn all about the Wild Oasis Hotel - **About**
3. Guests should be able to get information about each cabin and see booked dates - **Cabins**
4. Guests should be able to filter the cabins by their maximum guest capacity - **Cabins**
    1. For perfect fit for themselves
5. Guests should be able to reserve a cabin for a certain date range - **Reservations**
6. Reservations are not paid online. Payments will be made at the property upon arrival. Therefore, new reservations should be set to "unconfirmed" (booked but not yet checked in) - **Reservations**
7. Guests should be able to view all their past and future reservations - **Reservations**
8. Guests should be able to update or delete a reservation - **Reservations**
9. Guests need to sign up and log in before they can reserve a cabin and perform any operation (delete or update reservation) - **Authentication**
10. On sign up, each guest should get a profile in the DB - **Authentication**
11. Guests should be able to set and update basic data about their profile to make check-in at the hotel faster - **Profile**

Features + Pages

1. Homepage - `/`
2. About
    1. About page: `/about`
3. Cabins
    1. Cabin overview: `/cabins/`
    2. Cabin detail: `/cabins/:cabinId`
4. Reservations
    1. Cabin detail - used for reservation
    2. Reservation list - `/account/reservations`
    3. Edit reservation - `/account/reservation/edit`
5. Authentication
    1. Login - `/login` (and sign-up)
6. Profile
    1. Update profile - `/account/profile`

Technology Decisions

1. Framework - Next.JS
    1. The most popular React meta-framework. Handles routing, SSR, data fetching and even remote state management (in a way...), therefore replacing many tools we had to include before
        1. Less decisions to take
            1. React Router is not required

        2. React Query is not required
2. UI State management - Context API
    1. We might still need global UI state in Next.js app. For that, we can use the Context API, Redux, or any of the other solutions. In this case the Context API will be enough.
3. DB/API - supabase
    1. We'll use the data and API we already built in the first "Wild Oasis" project. If you skipped that project, please go back to the "Supabase" section to set everything up
4. Styling - tailwindcss
    1. Modern way of writing CSS. Extremely easy to integrate into Next.js. Most styles and markup will be pre-written anyway in this project.
        1. Tailwind is automatically set up by Next.js starter

Project Organization

1. `_components` - opt out of routing
    1. No router is created for the folder

Styling with Tailwind CSS

1. With fresh install, we get `globals.css` with the following content:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

2. Tailwind:
    1. Utility classes
    2. Predefined colors
    3. Predefined values
        1. Good values for good design
    4. Customizing tailwind:

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  content: [
    "./pages/**/*.{js,ts,jsx,tsx,mdx}",
    "./components/**/*.{js,ts,jsx,tsx,mdx}",
    "./app/**/*.{js,ts,jsx,tsx,mdx}",
  ],
  theme: {
    extend: {
      colors: {
        primary: {
          50: "#E1E8EF",
          100: "#D4DEE7",
          200: "#B7C7D7",
          300: "#99B0C7",
          400: "#7C99B6",
          500: "#5E82A6",
          600: "#4C6B8A",
          700: "#3C546C",
          800: "#2C3D4F",
          900: "#1B2631",
          950: "#141C24",
        },
        accent: {
          50: "#FAF5F0",
          100: "#F4ECE1",
          200: "#E8D6BF",
          300: "#DDC2A2",
          400: "#D2AF84",
          500: "#C69963",
          600: "#B78343",
          700: "#926835",
          800: "#6C4D28",
          900: "#4B351B",
          950: "#382814",
        },
      },
    },
  },
  plugins: [],
};
```

3. CSS Modules - automatically supported by Next.js
4. Styled Components - supported by Next.js (tailwind is more modern)
5. Adding icons:

```
npm i @heroicons/react
```

Adding Page Metadata and Favicon

1. `export const metadata` - Next.js convention
    1. The constant with `title` becomes the title of the HTML
    2. All metadata should be defined in `metadata` constant
    3. `meatadata` exported from a page overrides the global `metadata` defined in `layout.js`
2. Placing favicon:
    1. Just place `favicon.ico` or just an image file called `icon` in the `app` folder
        1. `app/icon.png` (or any image type)
            1. `<link rel="icon" href="/icon.png?sga798a7sgsdf09sd" type="image/png" sizes="163x163">`
                1. Automatically sized

Loading and Optimization Fonts

1. Fonts performance optimization
    1. Automatically self-host Google fonts, without the font being downloaded from Google
        1. It will prevent layout shifts, improve performance, and privacy
            1. If we use fonts from Google fonts, it might be a problem with privacy. It may not be good for GDBR regulation.
            2. May not be good for performance, because the font files may need to be downloaded from a Google server.
                1. Best to have the files on own server
                    1. Downloading will be faster
                    2. Prevents layout shifts
2. Code: layout.js

```
import Logo from "@/app/_components/Logo";
import Navigation from "@/app/_components/Navigation";
import "@/app/_styles/globals.css";

import { Josefin_Sans } from "next/font/google";
// import { Josefin_Sans } from "next/font/local"; // for local fonts

const josefin = Josefin_Sans({
  subsets: ["latin"],
  display: "swap",
});
// swap - first default faunt is used. Once the Josephin font is downloaded, it is swapped.

export const metadata = {
  // title: "About",
  title: {
    template: "%s / The Wild Oasis",
    default: "Welcome / The Wild Oasis",
  },
  description:
    "Luxurious cabin hotel, located in the heart of the Italian Dolomites, surrounded by beautiful mountains and dark forests",
}; // description - becomes `meta` tag on the page. It is for SEO
// <meta name="description" content="Luxurious cabin hotel, located in the heart of the Italian Dolomites, surrounded by beautiful mountains and dark

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body
        className={`${josefin.className} bg-primary-950 text-primary-100 min-h-screen`}
      >
        <header>
          <Logo />
          <Navigation />
        </header>
        <main>{children}</main>
        <footer>Copyright by The Wild Oasis</footer>
      </body>
    </html>
  );
}
```

Improving the Navigation and Root Layout

1. Tailwind: `antialiased` - grayscale antialiasing
    1. It controls font smoothing of an element

    ```
    antialiased:

    -webkit-font-smoothing: antialiased;
    -moz-osx-font-smoothing: grayscale;
    ```

2. Code:

```
import { Josefin_Sans } from "next/font/google";
// import { Josefin_Sans } from "next/font/local"; // for local fonts

const josefin = Josefin_Sans({
  subsets: ["latin"],
  display: "swap",
});
// swap - first default faunt is used. Once the Josephin font is downloaded, it is swapped.


<body
        className={`${josefin.className} antialiased ...`}
>
      ...
</body>
```

Optimizing Images with Next.js Component

1. Images - Images are one of the biggest factors contributing to page-size
    1. Huge impact on loading speeds
2. Next.js - `Image` component - produces optimized image
    1. It will automatically server correct size images in modern format (webp)
        1. It does it only on demand (when necessary)
            1. `srcset` is used - multiple images for different screen sizes
    2. Prevents layout shifts
        1. Forces us to specify height and width
    3. Automatically lazy-loads, only when they enter the viewport
        1. Complicated to implement manually
        2. `loading="lazy"`
3. `Image` - it can be used in many different ways. The above is only one way

1. If we import image first, height and width are not required (original size is retained). We can also change the quality

Building the Home Page

1. Images can be generated using Midjourney (AI)
2. Code: page.js

```
import Image from "next/image";
import Link from "next/link";
import bg from "@/public/bg.png";

export default function Page() {
  return (
    <main className="mt-24">
      <Image
        src={bg}
        fill
        placeholder="blur"
        className="object-cover object-top"
        alt="Mountains and forests with two cabins"
      />

      <div className="relative z-10 text-center">
        <h1 className="text-8xl text-primary-50 mb-10 tracking-tight font-normal">
          Welcome to paradise.
        </h1>
        <Link
          href="/cabins"
          className="bg-accent-500 px-8 py-6 text-primary-800 text-lg font-semibold hover:bg-accent-600 transition-all"
        >
          Explore luxury cabins
        </Link>
      </div>
    </main>
  );
}
```

Building the About Page with Responsive Images

1. No layout shift is an important metric for Google (SEO)
2. Code: about/page.js

```
import image1 from "@/public/about-1.jpg";

...

      <div className="col-span-2">
        <Image
          src={image1}
          placeholder="blur"
          alt="Family sitting around a fire pit in front of cabin"
        />
      </div>

      <div className="relative aspect-square col-span-2">
        <Image
          src="/about-2.jpg"
          fill
          className="object-cover"
          alt="Family that manages The Wild Oasis"
        />
      </div>
```

Adding Nested Routes and Pages

Adding a Nested Layout

1. Code: app/about/layout.js

```
import SideNavigation from "../_components/SideNavigation";

export default function Layout({ children }) {
  return (
    <div className="grid grid-cols-[16rem_1fr] h-full gap-12">
      <SideNavigation />
      <div>{children}</div>
    </div>
  );
}
```

Section 34: Data Fetching, Caching, and Rendering

Section Overview

1. Getting data and delivering it to the users in the best possible way
    1. Techniques
        1. Data streaming
        2. Caching
        3. Rendering
2. Topics:
    1. How to **work with data** effectively in Next.js
    2. **Streaming** with Suspense and loading.js
    3. Static **vs.** dynamic rendering
        1. When to use them
    4. **Caching** mechanisms

1. To improve website performance

Setting up Supabase

1. Supabase install

```
npm i @supabase/supabase-js
```

2. Environment variables
    1. They are only available in the application they are running

    2. `.env.local`

```
SUPABASE_URL=https://pwvrahphsegoojprrclx.supabase.co
SUPABASE_KEY=ddd # only available on the server
NEXT_PUBLIC_SOME_VAR=23 # leaks to the client
```

        1. Row-level security - takes care even if secret leaks
        2. service_role secret - bypeasses Row Level Security
3. supabase.js

```javascript
import { createClient } from "@supabase/supabase-js";

export const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_KEY
);
```

    4. Code: data-service.js

```
import { supabase } from "./supabase";
```

    5. Vercel's key/value data storage
        1. We can read data directly in the components without using any API layer

Fetching and Displaying Cabin List

1. When we are fetching images from external sources, we need to configure Next.js with the path

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  images: {
    remotePatterns: [
      {
        protocol: "https",
        hostname: "pwvrahphsegoojprrclx.supabase.co",
        port: "",
        pathname: "/storage/v1/object/public/cabin-images/**",
      },
    ],
  },
};

export default nextConfig;
```

```
<div className="flex-1 relative">
  <Image
    src={image}
    fill
    alt={`Cabin ${name}`}
    className="object-cover flex-1 border-r border-primary-800"
  />
</div>
```

Streaming Route Segments with loading.js File

1. `cabins/loading.js` - will apply to this route and all sub-routes (it overrides the higher level one)
    1. Having a `loading.js` file activates streaming (anywhere)
        1. JS must be enabled in Browser

What is React Suspense?

1. Suspense

    1. Built-in React component that we can use to catch/isolate components (or entire subtrees) that are **not ready to be rendered** ("suspending")

        1. They are doing async work
        2. The components or subtrees are suspending
        3. It is like a catch block in a try-catch statement
            1. Instead of catching errors, it catches components that are suspending

    2. What causes a component to be **suspending**?

        1. Fetching data (with a spported library - that supports suspense)
        2. Loading code (with React's lazy loading feature)
        3. Use-case of suspense - data-fetching
            1. Data-fetching libraries like React-Query or Next.js support suspense

    3. Example:

```
App
    Nav
    Suspense (Built-in React component - to wrap Products component)
        Products (component in Next.js that will fetch data (will be suspending - while it fetches the product data)
```

```
        List
    Filter
```

1. Native way to support **asynchronous** operations in a **declarative** way (no more `isLoading` states and render logic)
    1. Declarative - using components instead of imperative render logic
        1. Makes components simpler and cleaner

4. Steps:

    1. While rendering, **suspending** component is found (Product in this case)
    2. Go to **closest Suspense parent** ("boundary") and discard already rendered `children`
        1. It separates the suspending subtree from the rest of the app
    3. Display **fallback** component/JSX
        1. While async operation is happening in the background (usually a spinner)
        2. After async work is done
    4. Render subtree under suspense boundary (with the product data in this case)

5. **Important**: Components do **NOT** automatically suspend just because an async operation is happening inside them. Integrating async operations with `Suspense` is hard, so we use libraries (React Query, Next.js, Remix, etc.)

    1. It is extremely complex and integrating async operations with suspense is very hard
    2. Works with `React.Lazy`

A Look Behind the Scenes

1. Fiber Tree Representation

```
App
|
child
|
Nav -Sibling- Suspense
              |
              Products ----- Filter
              |
              List
```

1. Fiber tree - Internal representation of each component instance and DOM element of our app
    1. Comparing with VDom - Fiber tree is a mutable data-structure that is never destroyed (perfect place to store state, hooks, ...)
    2. It enables concurrent features
        1. Allows React pausing rendering with suspense
    3. Reconciliation phase:
        1. Behind the scenes, the child tree below the `Suspense` component is moved into another built-in component called `Activity` (available only in fiber-tree)
        2. The fallback `Spinner` is placed as a sibling to the `Activity`
            1. Fallback is initially hidden.
                1. Because, mode on `Activity` is set to `Visible` (`Suspense` children are visible) (`Activity = 'Visible'`)
        3. When component (`Products`) suspends, the mode is set to `Hidden` (`Activity = 'Hidden'`)
            1. Fallback (`Spinner`) will be rendered, and the subtree will be hidden
        4. When the async task finishes, `Activity` is set to `Visible` and the final subtree gets rendered
        5. If the component gets suspended againt, the fallback gets rendered again and the process gets repeated
            1. The subtree is always kept in the fiber-tree (elements are not destroyed)
                1. Consequences - All state is **preserved** in subsequent suspending and unsuspending phases
                    1. Constraint - the fallback will be rendered if the component suspends again
                        1. Fallback will **NOT** be shown again if the `Suspense` trigger is wrapped in a **transition** (React's `startTransition` function). In Next.js, that's the case with **page navigations** (all page navigations are wrapped in transitions. If a component refetches data as a result of page transition, fallback will not rendered again (not what we want)). We can **reset** the `Suspense` boundary with a unique **key** prop (resets suspense boundary)
    4. How does `Suspense` actually know that a component is suspending? (it's child component)

        1. Behind the scenes, the suspended component throws a Promise

```
throw new Promise(...)
```

            1. This triggers the suspense boundary to render the fallback
            2. This is a notification to the closest suspense boundary

2. Fiber tree with during reconciliation

```
App
|
child
|
Nav -Sibling- Suspense
              |     (`Activity = 'Visible'|'Hidden'`)
              Activity ----- Spinner ('Hidden' - rendered)
              | ('Visible' - rendered)
              Products ----- Filter
              |
              List
```

Streaming UI with Suspense: Cabin List

1. Rule: Move data fetching as close to the component as possible which needs it
    1. For more granular data-fetching strategy
2. `Suspense` must be outside the component that does the asynchronous work (`CabinList` is async component)
3. Code: cabins/page.js

```
import { Suspense } from "react";
import CabinList from "../_components/CabinList";
```

```
import Spinner from "../_components/Spinner";

export const metadata = {
  title: "Cabins",
};

export default function Page() {
  // CHANGE

  return (
    <div>
      <h1 className="text-4xl mb-5 text-accent-400 font-medium">
        Our Luxury Cabins
      </h1>
      <p className="text-primary-200 text-lg mb-10">
        Cozy yet luxurious cabins, located right in the heart of the Italian
        Dolomites. Imagine waking up to beautiful mountain views, spending your
        days exploring the dark forests around, or just relaxing in your private
        hot tub under the stars. Enjoy nature&apos;s beauty in your own little
        home away from home. The perfect spot for a peaceful, calm vacation.
        Welcome to paradise.
      </p>

      <Suspense fallback={<Spinner />}>
        {/* overrides loading.js */}
        <CabinList />
      </Suspense>
    </div>
  );
}

import { Suspense } from "react";
import CabinList from "../_components/CabinList";
import Spinner from "../_components/Spinner";

export const metadata = {
  title: "Cabins",
};

export default function Page() {
  // CHANGE

  return (
    <div>
      <h1 className="text-4xl mb-5 text-accent-400 font-medium">
        Our Luxury Cabins
      </h1>
      <p className="text-primary-200 text-lg mb-10">
        Cozy yet luxurious cabins, located right in the heart of the Italian
        Dolomites. Imagine waking up to beautiful mountain views, spending your
        days exploring the dark forests around, or just relaxing in your private
        hot tub under the stars. Enjoy nature&apos;s beauty in your own little
        home away from home. The perfect spot for a peaceful, calm vacation.
        Welcome to paradise.
      </p>

      <Suspense fallback={<Spinner />}>
        {/* overrides loading.js */}
        <CabinList />
      </Suspense>
    </div>
  );
}
```

Dynamic Route Segments: Building the Cabin Page

1. Dynamic - the route value can be anything - filled at request time/pre-rendered
2. Code:

```
export default function Page({ params }) {
// params = {cabinId: '1'}
```

4. Code: cabins/[cabinId]

```
import { getCabin } from "@/app/_lib/data-service";
import { EyeSlashIcon, MapPinIcon, UsersIcon } from "@heroicons/react/24/solid";
import Image from "next/image";

export default async function Page({ params }) {
  const cabin = await getCabin(params.cabinId);

  const { id, name, maxCapacity, regularPrice, discount, image, description } =
    cabin;

  // console.log(params);

  return (
    <div className="max-w-6xl mx-auto mt-8">
      <div className="grid grid-cols-[3fr_4fr] gap-20 border border-primary-800 py-3 px-10 mb-24">
        <div className="relative scale-[1.15] -translate-x-3">
          <Image
            fill
            className="object-cover"
            src={image}
            alt={`Cabin ${name}`}
          />
        </div>

        <div>
          <h3 className="text-accent-100 font-black text-7xl mb-5 translate-x-[-254px] bg-primary-950 p-6 pb-1 w-[150%]">
            Cabin {name}
          </h3>
```

```
          <p className="text-lg text-primary-300 mb-10">{description}</p>

          <ul className="flex flex-col gap-4 mb-7">
            <li className="flex gap-3 items-center">
              <UsersIcon className="h-5 w-5 text-primary-600" />
              <span className="text-lg">
                For up to <span className="font-bold">{maxCapacity}</span>{" "}
                guests
              </span>
            </li>
            <li className="flex gap-3 items-center">
              <MapPinIcon className="h-5 w-5 text-primary-600" />
              <span className="text-lg">
                Located in the heart of the{" "}
                <span className="font-bold">Dolomites</span> (Italy)
              </span>
            </li>
            <li className="flex gap-3 items-center">
              <EyeSlashIcon className="h-5 w-5 text-primary-600" />
              <span className="text-lg">
                Privacy <span className="font-bold">100%</span> guaranteed
              </span>
            </li>
          </ul>
        </div>
      </div>

      <div>
        <h2 className="text-5xl font-semibold text-center">
          Reserve today. Pay on arrival.
        </h2>
      </div>
    </div>
  );
}
```

Generating Dynamic Metadata

1. Code: [cabinId]/page.js

```
// export const metadata = {
//   title: "Cabin",
// };

export async function generateMetadata({ params }) {
  // Next.js waits for the following function to complete before streaming the UI to the client. Head tags are streamed first
  const { name } = await getCabin(params.cabinId);
  return {
    title: `Cabin ${name}`,
  };
}
```

Error Handling: Setting Up Error Boundaries

1. `error.js` - we can have nested error boundaries
    1. Works only in rendering. Callback function errors will not be caught by error boundary
    2. Might not catch errors in the root layout (layout.js)
        1. `global-error.js`
            1. Replaces the entire layout (even the root layout)
                1. It needs its own HTML and Body tags
            2. Nest.js documentation has details about conventions
2. Code: error.js

```
"use client";

export default function Error({ error, reset }) {
  return (
    <main className="flex justify-center items-center flex-col gap-6">
      <h1 className="text-3xl font-semibold">Something went wrong!</h1>
      <p className="text-lg">{error.message}</p>

      <button
        className="inline-block bg-accent-500 text-primary-800 px-6 py-3 text-lg"
        onClick={reset}
      >
        Try again
      </button>
    </main>
  );
}
```

Error Handling: "Not Found" Errors

1. Code: _lib/data-server.js

```
import { eachDayOfInterval } from "date-fns";
import { supabase } from "./supabase";
import { notFound } from "next/navigation";

/////////////
// GET

export async function getCabin(id) {
  const { data, error } = await supabase
    .from("cabins")
    .select("*")
    .eq("id", id)
    .single();
```

```
  // For testing
  // await new Promise((res) => setTimeout(res, 1000));

  if (error) {
    console.error(error);
    notFound(); // navigates to the error page
  }

  return data;
}
...
```

2. Code: cabins/[cabinId]/not-found.js

```
import Link from "next/link";

function NotFound() {
  return (
    <main className="text-center space-y-6 mt-4">
      <h1 className="text-3xl font-semibold">
        This cabin could not be found :(
      </h1>
      <Link
        href="/cabins"
        className="inline-block bg-accent-500 text-primary-800 px-6 py-3 text-lg"
      >
        Back to all cabins
      </Link>
    </main>
  );
}

export default NotFound;
```

Different Types of SSR: Static vs. Dynamic Rendering

Server-Side Rendering in Next.JS

1. Next.js is a **React** framework, so rendering is done by React, following the rules we learned earlier
2. **Remember**: Both **Server** and **Client** components are rendered on the server on the initial render
3. In Next.js, the server-side rendering work is **split by routes**
    1. Uses React and React DOM libraries to render each route one by one on the server
        1. Each route can be either **static** (also called pre-rendered), or **dynamic**
            1. It is not the entire app that is rendered statically or dynamically. It is each route individually
4. There is also **Partial Pre-Rendering** (PPR) which mixes dynamic and static rendering in the same route (more later)

Static vs Dynamic Rendering

1. Static Rendering
    1. HTML is generated at **build time** (developer triggers the rendering), or **periodically in the background** by re-fetching data (ISR, more on this later)
        1. ISR - Incremental Static Regeneration (periodic rendering)
    2. Useful when data **doesn't change often** and is **not personolized** to user (e.g. product page)
        1. Product page doesn't need any data that belongs to or generated by the user
    3. **Default rendering strategy** in Next.js (even when a page or component fetches data)
    4. When deployed to **Vercel**, each static route is automatically hosted on a CDN (company that built Next.js farmework)
    5. If all routes of an app are static, the entire app can be **exported as a static site** (SSG)
2. Dynamic Rendering
    1. HTML is generated at **request time** (a new version for each new request that reaches the server) - User triggers the rendering
    2. Makes sense if:
        1. The **data changes frequently** and is **personalized** to the user (e.g. shopping cart - each user has a different shopping cart)
        2. Rendering a route requires information that **depends on request** (e.g. search params from URL, cookie, header, etc.)
    3. A route **automatically switches to dynammic** rendering in certain conditions (next slide)
        1. Static pages are way faster (don't have to be rebuilt for every request)
    4. When deployed to **Vercel**, each dynamic route becomes a serverless function

When Next.JS Switches to Dynamic Rendering

1. Usually, developers **don't directly choose** whether a route should be static or dynamic. Next.js will **automatically switch to dynamic** rendering in the following scenarios:
    1. The route has a **dynamic segment** (page uses **params**)
        1. They can be known only at request time
    2. **searchParams** (query params from URL) are used in the page component
        1. `/product?quantity=123`
    3. Incoming **headers()** or **cookies()** are used in any of the route's server components
    4. An **uncached data request** is made in any of the route's server components
        1. We can influence the caching behavior
2. This is necessary because any of the values **can not be known** by Next.js at build time
    1. Route must be rendered anew for each request
3. We can also **force** Next.js to render a route dynamically:
    1. `export const dynamic = 'force-dynamic';` from page.js
    2. `export const revalidate = 0;` from page.js
    3. `{ cache: 'no-store' }` added to a `fetch` request in any of the route's server components
    4. `noStore()` in any of the route's server components

Some Terminology You Might Need

1. **Content Delivery Network (CDN)**: A network of servers located around the globe that cache and deliver a website's **static content** (HTML, CSS, JS, images) from as close as possible to each user.
    1. Data doesn't need to travel the entire planet from the host to the user
    2. Hosting providers like: Vercel, Netlify, Render.com will host static assets on global CDN

2. **Serverless Computing**: With the serverless computing model, we can run application code, usually back-end code, without managing the server ourselves. Instead, we can just run single functions on a cloud provider (AWS, Vercel, ...): **serverless functions**. The server in initialized and active only for the duration the serverless function is running, unlike a traditional Node.js app where the server is constantly running. Remember: each dynamic route becomes a serverless function
    1. Each dynamic route becomes a serverless function on vercel
    2. Next.js is not a Node.js app always running in Vercel
    3. If there is huge traffic to a serverless function, Vercel will automatically allocate more resources to handle the additional load
    4. Static content will be generated during build-time and hosted on a CDN
3. **The "edge"**: "As close as possible to the user". A CDN is part of an "edge" network, but there is also **serverless "edge" computing**. This is serverless computing that does not happen on a central server, but on a network that's distributed around the globe, as close as possible to the user (like a CDN but for running code). Important: we can select certain routes to run on the edge when deployed to Vercel.
    1. They become faster
4. **Incremental Static Regenaration (ISR)**: A Next.js feature that allows developers to update the content of a **static page**, in the background, even after the website has already been built and deployed. This happens by **re-fetching the data** of a component or entire route after a certain interval (we define in code). More on this later.

Analyzing Rendering in Our App

1. `npm run build`

```
Route (app)                              Size     First Load JS
┌ ○ /                                    526 B          99.5 kB
├ ○ /_not-found                          155 B          87.2 kB
├ ○ /about                               728 B          92.9 kB
├ ○ /account                             155 B          87.2 kB
├ ○ /account/profile                     155 B          87.2 kB
├ ○ /account/reservations                155 B          87.2 kB
├ ○ /cabins                              185 B          99.2 kB
├ ƒ /cabins/[cabinId]                    295 B          92.5 kB
└ ○ /icon.png                            0 B               0 B
+ First Load JS shared by all            87.1 kB
  ├ chunks/117-23b09b1911886961.js       31.6 kB
  ├ chunks/fd9d1056-aa94ea5c2eabf904.js  53.6 kB
  └ other shared chunks (total)          1.86 kB


○  (Static)   prerendered as static content
ƒ  (Dynamic)  server-rendered on demand

└ ○ /icon.png                            0 B               0 B
+ First Load JS shared by all            87.1 kB
  ├ chunks/117-23b09b1911886961.js       31.6 kB
  ├ chunks/fd9d1056-aa94ea5c2eabf904.js  53.6 kB
  └ other shared chunks (total)          1.86 kB


○  (Static)   prerendered as static content
ƒ  (Dynamic)  server-rendered on demand

  ├ chunks/117-23b09b1911886961.js       31.6 kB
  ├ chunks/fd9d1056-aa94ea5c2eabf904.js  53.6 kB
  └ other shared chunks (total)          1.86 kB


○  (Static)   prerendered as static content
ƒ  (Dynamic)  server-rendered on demand

  └ other shared chunks (total)          1.86 kB


○  (Static)   prerendered as static content
ƒ  (Dynamic)  server-rendered on demand

○  (Static)   prerendered as static content
ƒ  (Dynamic)  server-rendered on demand

ƒ  (Dynamic)  server-rendered on demand
```

Making Dynamic Pages Static with generateStaticParams

1. If we tell Next.js the set of IDs we know already, then Next.js can render the pages statically
    1. Then the entire site is a static site, and we can host on any static hostic provider
2. Code: cabins/[cabinId]/page.js

```
export async function generateStaticParams() {
  const cabins = await getCabins();

  const ids = cabins.map((cabin) => {
    cabinId: String(cabin.id);
  }); // `cabinId` is the name of the folder

  return ids;
}
```

3. Build: `npm run build`

```
Route (app)                              Size     First Load JS
┌ ○ /                                    526 B          99.5 kB
├ ○ /_not-found                          155 B          87.2 kB
├ ○ /about                               728 B          92.9 kB
├ ○ /account                             155 B          87.2 kB
├ ○ /account/profile                     155 B          87.2 kB
├ ○ /account/reservations                155 B          87.2 kB
├ ○ /cabins                              185 B          99.2 kB
├ ● /cabins/[cabinId]                    295 B          92.5 kB
└ ○ /icon.png                            0 B               0 B
+ First Load JS shared by all            87.1 kB
  ├ chunks/117-23b09b1911886961.js       31.6 kB
```

```
  ├ chunks/fd9d1056-aa94ea5c2eabf904.js  53.6 kB
  └ other shared chunks (total)          1.86 kB


○  (Static)  prerendered as static content
●  (SSG)     prerendered as static HTML (uses getStaticProps)

●  (SSG)     prerendered as static HTML (uses getStaticProps)
●  (SSG)     prerendered as static HTML (uses getStaticProps)

●  (SSG)     prerendered as static HTML (uses getStaticProps)
●  (SSG)     prerendered as static HTML (uses getStaticProps)
```

Static Site Generation (SSG)

1. `next.config.js`

   `output: "export",`

   `OR`

   `output: "standalone"`

2. `npm run build`
    1. `.txt` file is created for each page.
        1. It is the RSC payload
            1. Required for client-side navigation
        2. JS, CSS are bundled
        3. Images were not optimized
            1. On Dev server:
                1. Images are optimized on Vercel server using image optimization API
            2. Optims to optimize image
                1. No optimization using `img` instead of `Image`
                2. Using custom loader such as Cloudinary to optimize images
                    1. Resources: next.js static export

Partial Pre-Rendering

Partial Pre-Rendering (PPR): A Whole New Way of Rendering

1. **Idea/problem**: Most pages don't need to be 100% static or 100% dynamic
    1. Example: If the route is static but displays logged in user name
        1. If the route is rendered dynamically, it is waste
            1. Most of the route except the dynamic part can be rendered at build-time and cached in CDN
                1. Solution: PPR
2. New rendering strategy that combines **static** and **dynamic** rendering in the same route (it is between fully static and fully dynamic)
3. How does it work?
    1. A static (pre-rendered) shell is served immediately from a CDN, leaving holes for dynamic content
        1. Whenever user visits the page
    2. Server in the meantime renders dynamic content (which takes time as compared to the static shell)
        1. The slower **dynamic** content is streamed in as it's rendered on the server
            1. Filling the holes
4. Result: Even faster pages that can mostly be delivered from the edge (CDN) even when there are small dynamically rendered parts on the page
    1. Example: 80% static content, 20% dynamic content
        1. Using Partial Prerendering

How to Use Partial Pre-Rendering

1. As of Next.js 14, PPR is **highly experimental** and should not be used in production
    1. PPR needs to be **turned on** in config file
        1. Static rendering is default rendering mechanism
        2. If PPR is enabled, by default, as much as possible of any route will be **statically rendered**, constructing a **static shell**
            1. Dynamic parts (components) should be placed inside **Suspense boundaries**
    2. There are no new APIs to learn for PPR
        1. These boundaries tell Next.js that **anything within the boundary is dynamic**
            1. The boundary prevents the dynamic part (e.g. reading a header or making a non-cached fetch request) from spreading onto the entire route
                1. Suspense boundary isolates dynamic components or subtrees that will be dynamically rendered
        2. We provide a **static fallback** to be shown while the dynamic part is rendered (by passing it to the suspense component)
            1. Dynamic components or sub-trees are **inserted** into the static shell as they become available

How Next.js Caches Data

1. **Caching** in Next.JS
    1. Next.JS Caching
        1. **Caching**: Storing fetched or computed data in a temporary location for future access, instead of having to re-fetch or re-compute the data every time it's needed
        2. Next.js caches **very aggressively**: everything that is possible to cache, is cached
            1. Fetched data
            2. Visited routes
        3. Next.js provides APIs for **cache revalidation**: removing data from the cache and updating it with **fresh data** (re-fetched or re-computed)
        4. Makes Next.js apps **more performant** and **saves costs** (computing and data access costs)
            1. If paying each time we access data using an API (if we fetch less often, we pay much less)
            2. Problem: Aggressive caching
                1. **Caching always ON by default**: strange and unexpected behavior in some situations (page displaying stale data on the client). Some caches can't be turned off (annoying)
        5. **Very confusing**: Many different Next.js APIs effect and control caching (difficult to grasp especially for the beginners)
2. Topics:

1. 4 Caching Mechanisms

The Caching Mechanisms

1. Request Memoization (Server)
    1. What data?
        1. Data fetched with **similar GET requests** (same url and options in `fetch` function)
    2. How long?
        1. **One page request** (one render, by one user)
            1. Data is cached and reused exactly during one page render
                1. When a route fetches the same data in multiple places in the component tree during one render, only one Network request is made
    3. Enables
        1. No need to fetch at the top of tree: the **same** fetch in multiple components only makes **one request**
            1. Example: If we fetch products from 5 different components, Next.js gets the data from the API once, and not 5 times
    4. Works only with native `fetch` function and when the requests are exactly the same (same URL and options object)
    5. It is a React feature
        1. Works only in React component tree
            1. Not in route handlers or server actions
    6. How to revalidate?
        1. N.A. - data is not persisted anywhere
    7. How to opt out?
        1. `AbordController` with `fetch` function
2. Data Cache (Server)
    1. What data?
        1. Data fetched in a **route** or a **single fetch request**
    2. How long?
        1. **Indefinietely**, even acress re-deploys (can revalidate (clear and re-fetch) or opt out)
            1. Data is available across multiple requests from different users
                1. If there are million users sending the same request, Next.js sends only one Network request (every user gets the exact same data - static data - data that feeds into static pages - data used to statically render routes)
    3. Enables
        1. Data for static pages + ISR when revalidated (the whole page is re-generated on revalidation)
            1. ISR is possible with this cache and revalidation of the cache
    4. Advantages:
        1. Prevents too many requests to original data source
        2. Most configurable
    5. Disadvantages:
        1. Most confusing
    6. How to revalidate? (revalidating Data Cache also revalidates Full Route Cache)
        1. **Time-based (automatic) for all data on page**:
            1. `export const revalidate = <time>;` (page.js)
                1. Data will be re-fetched
        2. **Time-based (automatic) for one data request**: (revalidating only a certain fetch request)
            1. `fetch('...', { next: { revalidate: <time> } })`
        3. **On-demand (manual)**:
            1. `revalidatePath` or `revalidateTag`
                1. Called to revalidate on a certain path or certain tag (?)
    7. How to opt out? (The below options force page to become dynamic, which also opts our of the **Full Route Cache**)
        1. Entire page:
            1. `export const revalidate = 0;` (page.js) (always re-validate)
                1. Turns the cache off
        2. Entire page:
            1. `export const dynamic = 'force-dynamic';` (page.js)
                1. Turns off the data cache (since the cache is only for static data)
        3. Individual request:
            1. `fetch('...', { cache: 'no-store' })`
        4. Individual server component: `noStore()` (calling it inside of it)
3. Full Route Cache (Server)
    1. What data?
        1. Entire **static** pages (HTML and RSC payload) (full route cache)
            1. At build-time
    2. How long?
        1. Until the "Data cache" is **invalidated** (or app is re-deployed)
            1. If underlying data changes, the page needs to be re-gerenerated and stored in the cache again to reflect the latest data
            2. Cleared when new version of the application is deployed
    3. Enables
        1. Static pages
            1. Storage mechanism for static routes
4. Router Cache (Client)
    1. What data?
        1. Pre-fetched and visited pages (while navigating around the application): **static** and **dynamic** [routes] (in Browser. The browser doesn't know how the route was generated at all)
            1. Idea: Pages stored in memory allows almost instance access (gives the use the feel of a single page application with no hard reloads)
    2. 30 sec dynamic / 5 min static (throughout one user session)
    3. Enables
        1. SPA-like navigation (instant navigation and no full reloads)
            1. Pages are not requested from the server again as user navigates back and forth (results in stale data being displayed on the page)
                1. Dynamic pages - 30 seconds
                2. Static pages - 5 minutes
        2. There is no way to revalidate the cache
            1. Unless user performs hard reload/closes and re-opens
    4. How to revalidate?
        1. `revalidatePath` or `revalidateTag` **in SA** - manually revalidating data cache
            1. SA - Server Action (later)
            2. Intutition: If we declare the data in cache as old (by revalidating it), the pages that are cache in user's browser that depend on the data should be refreshed with fresh data

    2. `router.refresh` - forcing reload
    3. `cookie.set` or `cookie.delete` in SA
5. How to opt out?
    1. Not possible (makes no sense)
        1. Can be problematic - Users always want up to date date. We cannot guarantee that with this cache
        2. If data on a page is updated by other than revalidation, (without cache knowing about the udate), we cannot do anything about it
            1. Example: If vanilla React app updates data
5. This is the behavior in **production** mode. Caching doesn't work in development (we always get fresh data while building the apps)

Experimenting with Caching and ISR

1. Caching is disabled in Dev environment
    1. Need to simulate prod environment

        ```
        npm run build
        npm start
        ```

        1. Script:

            ```
            "prod": "next build && next start"
            ```

        2. `npm run prod`
            1. The site is rendered as static site
        3. If data is changed in DB, the data doesn't get refreshed on the page
            1. That is because of data cache and full route cache
                1. Route is cached with the data that was in place when the page was statically generated (this will be used until we re-validate the data)
            2. We don't have to go to DB if we don't have to query from db again
2. We want middle ground
    1. Incremental Static Regeneration
        1. Re-generates static page and fetch fresh data from time to time

            ```
            export const revalidate = <number-of-seconds>; // regerated after every <number-of-seconds>
            ```

        2. If the cabin prices change once per day, we can set it to once per hour. Once per minute makes sure maximum for 1 minute we have stale data
            1. If user wants to book the cabin, then we want the fresh data
            2. It is the first user who visits the page after a value has changed, will regenerate the page in the background for the next user (next visitor will get the result)
3. We can use revalidation at the component level or at the individual fetch level

CHALLENGE #1: Fetching the Number of Cabins

1. Code: about/page.js

```
export const revalidate = 84600;

export default async function Page() {
  const cabins = await getCabins();
  ...
  <p>
    Our {cabins.length}
```

Section 35: Client and Server Interactions

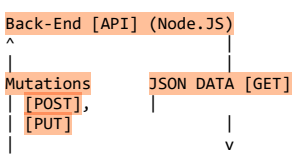
Section Overview

1. How client and server portions of Next.js interact with each other
2. Topics:
    1. Composing **full-stack** apps
    2. Rendering **server components** in client components
        1. When it makes sense
    3. **Sharing state** between server and client
        1. In both directions
            1. URL
            2. Context API
    4. URL and React Context API
    5. How to **think** about data fetching
        1. How and where to fetch and stream different pieces of related data on a page

Blurring the Boundary Between Server and Client (RSC - Part 4)

The Server-Client Boundary: Front-End vs. Back-End

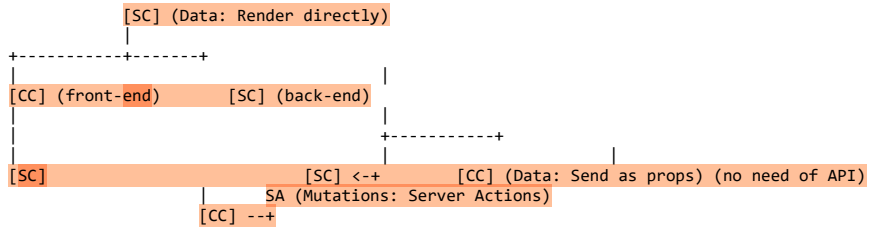
1. Traditional



```
Front-End (React)
```

1. They may have different code-bases
2. They might be hosted on different platforms
    1. Very clear server-client boundary
3. Communication happens via an API
4. Once JSON arrives from the back-end, the front-end takes over (completely)
    1. Backend is not involved in rendering the data

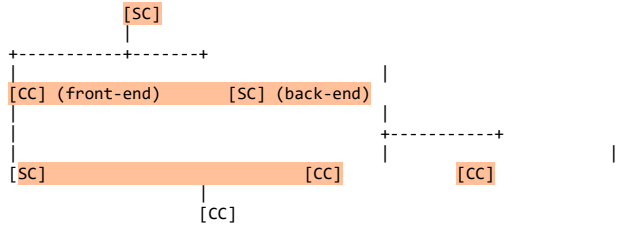
2. Next.JS with RSC + SA



1. No clear separation between front-end and back-end anymore
    1. Lines are completely blurred (lines are flexible and dynamic)
2. "Knitting": pieces of server and client code interweave (composability)
    1. Powerful mechanism to compose server & client into full-stack app with single code-base
3. Allow us to build true **full-stack applications** in just one codebase
    1. Seperation is not always good
    2. No longer need to build API
4. **No need** for an intermediary API many times
5. SA - replaces POST, PUT, DELETE, ...

Importing vs. Rendering

1. Component tree:

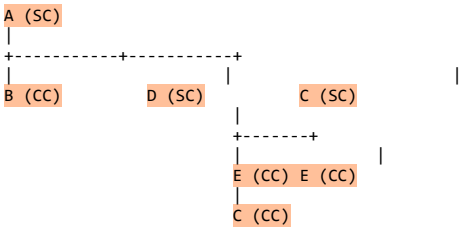


1. A parent component renders a child component that it is connected to
2. A client component renders a server component that is is connected to (how?)
    1. Important trick
        1. Possible if **server** component is passed as **prop** (`children` or other regular prop)
            1. Example:

```
import B from 'B';
import C from 'C';
import D from 'D';

function A() {
    return (
        <div>
            <B>
                <D>Hey </D>
            </B>
        </div>
    );
}
```

        2. D has already been executed here... (on the server)
            1. Component A imports D too
                1. Dependency tree: (modules imported by other modules)



                1. A imports D because or else, it cannot be passed as a prop
                2. In the dependency tree, the client/server boundary is established (not in the component tree
                    1. Client components cannot import server components but only client components
                        1. It is not possible to go back over the client/server boundary and import server components from the client

1. Client components can render server components as long as they have been passed as props
3. In the example:
    1. A imports both B and D
    2. A executes D, before passing the result as prop to B (client component)
        1. Instance of D is already created on server and then passed as prop to client
    4. Server components can import an render all components

2. A component can be both SC and CC

    1. A component is just a definition (a blueprint) and can be used for creation of multiple component instances

        1. If no "use client" directive, it becomes SC instance as soon as it is used
            1. Example: When A has imported C, it becomes an SC instance

    2. If a client component imports a server component, a CC instance is created

```
E (CC) "use client"
|
C (CC)
```

        1. C is imported by E (CC), so C becomes a CC

    3. If SC is imported and used in CC, a CC instance is created

Client Components in Server Components

1. Using CC in SC
    1. If there is a long text, and if we want a button to expand the text on demand
        1. Tip: We have to move a CC as low as possible in the component tree
            1. Because, all child components of a CC are CCs (they will be rendered on a client)
                1. Leaves are better
2. Code: TextExpander.js

```
"use client";

import Logo from "./Logo";

...

<Logo /> {/* This becomes a CC instance even though it is defined as SC */}
```

3. `hydrateRoot()`
    1. Component is first rendered on the server & then hydrated
4. `Suspense`
    1. `Suspended: true` - shows loading spinner

Highlighting Current Side Navigation Link

1. Code: SideNavigation.js

```
"use client";

import {
  CalendarDaysIcon,
  HomeIcon,
  UserIcon,
} from "@heroicons/react/24/solid";
import SignOutButton from "@/app/_components/SignOutButton";
import Link from "next/link";
import { usePathname } from "next/navigation";

const navLinks = [
  {
    name: "Home",
    href: "/account",
    icon: <HomeIcon className="h-5 w-5 text-primary-600" />,
  },
  {
    name: "Reservations",
    href: "/account/reservations",
    icon: <CalendarDaysIcon className="h-5 w-5 text-primary-600" />,
  },
  {
    name: "Guest profile",
    href: "/account/profile",
    icon: <UserIcon className="h-5 w-5 text-primary-600" />,
  },
];

function SideNavigation() {
  const pathName = usePathname(); // Works only in client components. The component is switched to CC to use React hook.

  return (
    <nav className="border-r border-primary-900">
      <ul className="flex flex-col gap-2 h-full text-lg">
        {navLinks.map((link) => (
          <li key={link.name}>
            <Link
              className={`py-3 px-5 hover:bg-primary-900 hover:text-primary-100 transition-colors flex items-center gap-4 font-semibold text-primary-2
                pathName === link.href ? "bg-primary-900" : ""
              }`}
              href={link.href}
            >
              {link.icon}
              <span>{link.name}</span>
            </Link>
```

```
          </li>
        ))}

        <li className="mt-auto">
          <SignOutButton />
        </li>
      </ul>
    </nav>
  );
}

export default SideNavigation;
```

Sharing State Between Client and Server: The URL

1. How to pass data from client to server?
2. Filter component - to filter by cabin capacity
    1. How to share state from client to server
        1. Store state in URL
        2. In SC, we can read the data from the URL
            1. `searchParams` is only available in the `page.js` (not in SCs)
                1. `searchParams` can only be known at the runtime - The page cannot be statically rendered (dynamically rendered)
                    1. `revalidate` doesn't work
3. A navigation (new URL) in Next.js is wrapped in a **React transition**
    1. In transition, `Suspense` will not hide the content that has already been rendered earlier
        1. Next.js will wait and swap it out as new content comes in
            1. Fix: Pass in a unique key in `Suspense`

```
<Suspense fallback={<Spinner />} key={filter}> {/* Because `filter` is unique */}
  ...
</Suspense>
```

4. Recap:
    1. Sharing state from client to server
        1. Place it in the URL
            1. Makes pages sharable and bookmarkable
    2. `router.replace` - used to navigate to the new URL
    3. Whenever `searchParams` changes (as a result of URL changing), the server component will re-render (when there is navigation)
        1. As a result, child components also re-render (just like in client-side React)
            1. Re-fetching data might occur
        4. `Suspense` needs a unique key to show fallback
5. Code: Filter.js

```
"use client";

import { usePathname, useRouter, useSearchParams } from "next/navigation";

function Filter() {
  const searchParams = useSearchParams();
  const router = useRouter();
  const pathname = usePathname();

  const activeFilter = searchParams.get("capacity") ?? "all";

  function handleFilter(filter) {
    const params = new URLSearchParams(searchParams);
    params.set("capacity", filter); // doesn't change URL bar
    router.replace(`${pathname}?${params.toString()}`, { scroll: false }); // doesn't allow page to scroll to the top
  }

  return (
    <div className="border border-primary-800 flex">
      <Button
        filter="all"
        handleFilter={handleFilter}
        activeFilter={activeFilter}
      >
        All cabins
      </Button>
      <Button
        filter="small"
        handleFilter={handleFilter}
        activeFilter={activeFilter}
      >
        1&mdash;3 guests
      </Button>
      <Button
        filter="medium"
        handleFilter={handleFilter}
        activeFilter={activeFilter}
      >
        4&mdash;7 guests
      </Button>
      <Button
        filter="large"
        handleFilter={handleFilter}
        activeFilter={activeFilter}
      >
        8&mdash;12 guests
      </Button>
    </div>
```

```
  );
}

function Button({ filter, handleFilter, activeFilter, children }) {
  // also becomes a client component
  return (
    <button
      className={`px-5 py-2 hover:bg-primary-700 ${
        filter === activeFilter ? "bg-primary-700 text-primary-50" : ""
      }`}
      onClick={() => handleFilter(filter)}
    >
      {children}
    </button>
  );
}

export default Filter;
```

3. Code: cabins/page.js

```
import { Suspense } from "react";
import CabinList from "../_components/CabinList";
import Spinner from "../_components/Spinner";
import Filter from "../_components/Filter";

// export const revalidate = 3600; // value cannot be computed. It must be a direct value
export const revalidate = 15;

export const metadata = {
  title: "Cabins",
};

export default function Page({ searchParams }) {
  // CHANGE

  console.log(searchParams);

  const filter = searchParams?.capacity ?? "all";

  return (
    <div>
      <h1 className="text-4xl mb-5 text-accent-400 font-medium">
        Our Luxury Cabins
      </h1>
      <p className="text-primary-200 text-lg mb-10">
        Cozy yet luxurious cabins, located right in the heart of the Italian
        Dolomites. Imagine waking up to beautiful mountain views, spending your
        days exploring the dark forests around, or just relaxing in your private
        hot tub under the stars. Enjoy nature&amp;apos;s beauty in your own little
        home away from home. The perfect spot for a peaceful, calm vacation.
        Welcome to paradise.
      </p>

      <div className="flex justify-end mb-8">
        <Filter />
      </div>

      <Suspense fallback={<Spinner />} key={filter}>
        {/* overrides loading.js */}
        <CabinList filter={filter} />
      </Suspense>
    </div>
  );
}
```

Advanced: Server Components in Client Components

1. Guest profile
    1. Selector with all countries in the World
2. Code: UpdateProfile.js

```
"use client";

import { useState } from "react";

function UpdateProfileForm({ children }) {
  const [count, setCount] = useState();

  // CHANGE
  const countryFlag = "pt.jpg";
  const nationality = "portugal";

  return (
    <form className="bg-primary-900 py-8 px-12 text-lg flex gap-6 flex-col">
      <div className="space-y-2">
        <label>Full name</label>
        <input
          disabled
          className="px-5 py-3 bg-primary-200 text-primary-800 w-full shadow-sm rounded-sm disabled:cursor-not-allowed disabled:bg-gray-600 disabled:t
        />
      </div>

      <div className="space-y-2">
        <label>Email address</label>
        <input
          disabled
          className="px-5 py-3 bg-primary-200 text-primary-800 w-full shadow-sm rounded-sm disabled:cursor-not-allowed disabled:bg-gray-600 disabled:t
        />
      </div>

      <div className="space-y-2">
```

```
        <div className="flex items-center justify-between">
          <label htmlFor="nationality">Where are you from?</label>
          <img
            src={countryFlag}
            alt="Country flag"
            className="h-5 rounded-sm"
          />
        </div>

        {children}
      </div>

      <div className="space-y-2">
        <label htmlFor="nationalID">National ID number</label>
        <input
          name="nationalID"
          className="px-5 py-3 bg-primary-200 text-primary-800 w-full shadow-sm rounded-sm"
        />
      </div>

      <div className="flex justify-end items-center gap-6">
        <button className="bg-accent-500 px-8 py-4 text-primary-800 font-semibold hover:bg-accent-600 transition-all disabled:cursor-not-allowed disab
          Update profile
        </button>
      </div>
    </form>
  );
}

export default UpdateProfileForm;
```

3. Code: account/profile/page.js

```
import SelectCountry from "@/app/_components/SelectCountry";
import UpdateProfileForm from "@/app/_components/UpdateProfileForm";

export const metadata = {
  title: "Update profile",
};

export default function Page() {
  // CHANGE
  const countryFlag = "pt.jpg";
  const nationality = "portugal";

  return (
    <div>
      <h2 className="font-semibold text-2xl text-accent-400 mb-4">
        Update your guest profile
      </h2>

      <p className="text-lg mb-8 text-primary-200">
        Providing the following information will make your check-in process
        faster and smoother. See you soon!
      </p>

      <UpdateProfileForm>
        <SelectCountry
          name="nationality"
          id="nationality"
          className="px-5 py-3 bg-primary-200 text-primary-800 w-full shadow-sm rounded-sm"
          defaultCountry={nationality}
        />
        {/* It becomes a React element and passed to the client */}
      </UpdateProfileForm>
    </div>
  );
}
```

Data Fetching Strategies for the Reservation Section

1. Reservation on each cabin page
    1. We need to fetch data
        1. We can do it in multiple pieces of data in multiple places
            1. Different strategies - on a single page
2. Clients should only be passed minimum amount of data (only the required amount of data)
3. We need to think about data fetching strategies when building real applications
4. Code: Cabin.js

```
import Image from "next/image";
import TextExpander from "./TextExpander";
import { EyeSlashIcon, MapPinIcon, UsersIcon } from "@heroicons/react/24/solid";

function Cabin({ cabin }) {
  const { id, name, maxCapacity, regularPrice, discount, image, description } =
    cabin;

  return (
    <div className="grid grid-cols-[3fr_4fr] gap-20 border border-primary-800 py-3 px-10 mb-24">
      <div className="relative scale-[1.15] -translate-x-3">
        <Image
          fill
          className="object-cover"
          src={image}
          alt={`Cabin ${name}`}
        />
      </div>

      <div>
        <h3 className="text-accent-100 font-black text-7xl mb-5 translate-x-[-254px] bg-primary-950 p-6 pb-1 w-[150%]">
          Cabin {name}
```

```
        </h3>

        <p className="text-lg text-primary-300 mb-10">
          <TextExpander>{description}</TextExpander>
        </p>

        <ul className="flex flex-col gap-4 mb-7">
          <li className="flex gap-3 items-center">
            <UsersIcon className="h-5 w-5 text-primary-600" />
            <span className="text-lg">
              For up to <span className="font-bold">{maxCapacity}</span> guests
            </span>
          </li>
          <li className="flex gap-3 items-center">
            <MapPinIcon className="h-5 w-5 text-primary-600" />
            <span className="text-lg">
              Located in the heart of the{" "}
              <span className="font-bold">Dolomites</span> (Italy)
            </span>
          </li>
          <li className="flex gap-3 items-center">
            <EyeSlashIcon className="h-5 w-5 text-primary-600" />
            <span className="text-lg">
              Privacy <span className="font-bold">100%</span> guaranteed
            </span>
          </li>
        </ul>
      </div>
    </div>
  );
}

export default Cabin;
```

Code: cabins/[cabinId]/page.js

```
import Cabin from "@/app/_components/Cabin";
import Reservation from "@/app/_components/Reservation";
import Spinner from "@/app/_components/Spinner";
import { getCabin, getCabins } from "@/app/_lib/data-service";
import { Suspense } from "react";

// export const metadata = {
//   title: "Cabin",
// };

export async function generateMetadata({ params }) {
  // Next.js waits for the following function to complete before streaming the UI to the client. Head tags are streamed first
  const { name } = await getCabin(params.cabinId);
  return {
    title: `Cabin ${name}`,
  };
}

export async function generateStaticParams() {
  const cabins = await getCabins();

  const ids = cabins.map((cabin) => {
    cabinId: String(cabin.id);
  }); // `cabinId` is the name of the folder

  return !ids || ids.length === 0 ? [{ cabinId: "not-found" }] : ids;
}

export default async function Page({ params }) {
  // blocking waterfall
  const cabin = await getCabin(params.cabinId);
  // const settings = await getSettings();
  // const bookedDates = await getBookedDatesByCabinId(params.cabinId);

  // Better solution is to have separate components, each fetching its own data and streaming the components as they are ready

  // console.log(cabin);

  // console.log(params);
  // console.log(`SEARCH PARAMS: ${searchParams}`);

  return (
    <div className="max-w-6xl mx-auto mt-8">
      {/* <Cabin cabin={cabin} /> */}
      <div>
        <h2 className="text-5xl font-semibold text-center mb-10 text-accent-400">
          Reserve {cabin.name} today. Pay on arrival.
        </h2>

        <Suspense fallback={<Spinner />}>
          <Reservation cabin={cabin} />
        </Suspense>
      </div>
    </div>
  );
}
```

Using the Context API for State Management

1. Change URL for each data range selection would cause a new navigation
    1. Whenever we navigate to a different page, it would re-render the server component - this will re-fetch all the data on the page
2. Lifting state up
    1. A parent component fetches the data and passes it on to child components

3. Context API - only works for client components (hooks are used)
    1. A provider must be placed deep down in the component tree to avoid unnecessary re-renders
        1. Steps:
            1. A client component for context API needs to be craated

```
"use client";

import { createContext, useContext, useState } from "react";

const ReservationContext = createContext();

const initialState = { from: undefined, to: undefined };

function ReservationProvider({ children }) {
  const [range, setRange] = useState(initialState);

  return (
    <ReservationContext.Provider value={{ range, setRange }}>
      {children}
    </ReservationContext.Provider>
  );
}

function useReservation() {
  const context = useContext(ReservationContext);

  if (context === undefined)
    throw new Error("Context was used outside provider");

  return context;
}

export { ReservationProvider, useReservation };
```

            2. Use the provider in the global `layout.js`
                1. We can pass server components as children to the provider

```
<div className="flex-1 px-8 py-12 grid">
  <main className="max-w-7xl mx-auto w-full">
    <ReservationProvider>{children}</ReservationProvider>
  </main>
</div>
```

4. If state changes, client component re-renders
    1. If a component re-renders, all its child components re-render
5. Code: ReservationContext.js

```
"use client";

import { createContext, useContext, useState } from "react";

const ReservationContext = createContext();

const initialState = { from: undefined, to: undefined };

function ReservationProvider({ children }) {
  const [range, setRange] = useState(initialState);

  const resetRange = () => setRange(initialState);

  return (
    <ReservationContext.Provider value={{ range, setRange, resetRange }}>
      {children}
    </ReservationContext.Provider>
  );
}

function useReservation() {
  const context = useContext(ReservationContext);

  if (context === undefined)
    throw new Error("Context was used outside provider");

  return context;
}

export { ReservationProvider, useReservation };
```

6. Code: ReservationReminder.js

```
"use client";

import { XMarkIcon } from "@heroicons/react/24/solid";
import { format } from "date-fns";
import { useReservation } from "./ReservationContext";

function ReservationReminder() {
  const { range, resetRange } = useReservation();

  if (!range.from || !range.to) return null;

  return (
    <div className="fixed bottom-6 left-1/2 -translate-x-1/2 py-5 px-8 rounded-full bg-accent-500 text-primary-800 text  font-semibold shadow-xl shado
      <p>
        <span>👋</span> Don&apos;t forget to reserve your dates <br /> from{" "}
        {format(new Date(range.from), "MMM dd yyyy")} to{" "}
```

```
        {format(new Date(range.to), "MMM dd yyyy")}
      </p>
      <button
        className="rounded-full p-1 hover:bg-accent-600 transition-all"
        onClick={resetRange}
      >
        <XMarkIcon className="h-5 w-5" />
      </button>
    </div>
  );
}

export default ReservationReminder;
```

Constructing an API Endpoint with Route Handlers

1. API endpoint from Next.JS using Route Handlers
    1. Not as important as it was with pages router
        1. Now we use Server Actions for mutations
    2. API Endpoints can be constructed using Route handlers
        1. `route.js` - convention file
            1. It can be in any folder that doesn't have page.js yet
                1. Since no HTML is returned
                    1. JSON data is returned
                2. The route handler is executed and returns JSON data
            2. It can have one or more functions each corresponding to HTTP verbs
            3. `Response` - Interface of Fetch API
                1. `Response.json()` - Returns a new `Response` object for returning the provided JSON encoded data
2. Code: api/cabins/[cabinId]

```
import { getBookedDatesByCabinId, getCabin } from "@/app/_lib/data-service";

export async function GET(request, { params }) {
  const { cabinId } = params;

  try {
    const [cabin, bookedDates] = await Promise.all([
      getCabin(cabinId),
      getBookedDatesByCabinId(cabinId),
    ]);
    return Response.json({ cabin, bookedDates });
  } catch (e) {
    return Response.json({ message: "Cabin not found" });
  }

  console.log(request, params);

  return Response.json({ text: "text" }); // Response - web-standard also implemented in browsers. MDN documentation
}

// export async function POST() {}
```

Section 36: Authentication with NextAuth (Auth.js)

Section Overview

1. Topics:
    1. **NextAuth** library (Auth.js) for authentication
        1. Multiple providers
    2. Google provider
    3. **Middleware** for authorization
    4. Accounts in **Supabase** DB
        1. For each newly signed up user

Setting up NextAuth

1. Google: NextAuth (now Auth.js)
    1. Library that simplifies authentication in a Next.JS application (not its own auth service)
        1. Supabase has auth service
    2. Alternative
        1. Clerk - DB with users, pre-made components, ... (easier)
            1. Needs payment after certain point
        2. Lucia
2. Step:
    1. .env.local

```
NEXTAUTH_URL=http://localhost:3000/
NEXTAUTH_SECRET=<secret-generated-using-https://generate-secret.vercel.app/>
```

    2. Login to Google Developer Console
        1. https://console.cloud.google.com/cloud-resource-manager
        2. Create Project
            1. the wild oasis
            2. Create
        3. Select the project settings

4. APIs & Services
    1. OAuth consent screen
        1. External (Internal - Google workspace user)
            1. App Info
                1. The Wild Oasis
                2. User support email
                3. App Logo
                    1. logo.png
                4. Developer contact info: email
                5. Save and Continue
            2. Save and Continue
            3. Add Users
                1. email
                    1. To test the app
            4. Save and Continue
    2. Credentials
        1. Create Credentials
            1. OAuth Client ID
            2. Application type: Web application
            3. Authorized JavaScrit origins
                1. http://localhost:3000/
            4. Authorized redirect URIs
                1. http://localhost:3000/api/auth/callback/google
            5. Create
        2. Download JSON
        3. Copy Client Id
            1. .env.local:

```
AUTH_GOOGLE_ID=<client-id>
```

        4. Copy Client Secret
            1. .env.local

```
AUTH_GOOGLE_SECRET=<client-secret>
```

    3. Install the library

```
npm i next-auth@beta
```

3. To handle whatever we want in the route
    1. Code: api/auth/[...nextauth]/route.js

```
http://localhost:3000/api/auth/signin
http://localhost:3000/api/auth/signout
http://localhost:3000/api/auth/providers
```

    2. route.js

```
export { GET, POST } from "@/app/_lib/auth";
```

    3. _lib/auth.js

```
import NextAuth from "next-auth";
import Google from "next-auth/providers/google";

const authConfig = {
  providers: [
    Google({
      clientId: process.env.AUTH_GOOGLE_ID,
      clientSecret: process.env.AUTH_GOOGLE_SECRET,
    }),
  ],
};

export const {
  auth,
  handlers: { GET, POST },
} = NextAuth(authConfig);
```

    4. `http://localhost:3000/api/auth/signin`
        1. You get: Sign in with Google
4. `auth()` makes entire route dynamic because it uses cookies and headers
    1. It reads cookies from incoming requests
        1. Cookies are known at runtime
            1. In `Navigation` makes every route dynamic
5. It is better to deal with user login in the server
6. Authorization
    1. Only logged-in users can visit a route
        1. Guest area say

Getting the User Session

1. Code: Reservation.js

```
import { auth } from "../_lib/auth";
import { getBookedDatesByCabinId, getSettings } from "../_lib/data-service";
import DateSelector from "./DateSelector";
import LoginMessage from "./LoginMessage";
import ReservationForm from "./ReservationForm";

async function Reservation({ cabin }) {
  const [settings, bookedDates] = await Promise.all([
    getSettings(cabin.id),
    getBookedDatesByCabinId(cabin.id),
  ]);

  const session = await auth();

  return (
    <div className="grid grid-cols-2 border border-primary-800 min-h-[400px]">
      <DateSelector
        settings={settings}
        bookedDates={bookedDates}
        cabin={cabin}
      />
      {session?.user ? (
        <ReservationForm cabin={cabin} user={session.user} />
      ) : (
        <LoginMessage />
      )}
    </div>
  );
}

export default Reservation;
```

2. Code: LoginMessage.js

```
import Link from "next/link";

function LoginMessage() {
  return (
    <div className="grid bg-primary-800 ">
      <p className="text-center text-xl py-12 self-center">
        Please
        <Link href="/login" className="underline text-accent-500">
          login
        </Link>
        to reserve this
        <br /> cabin right now
      </p>
    </div>
  );
}

export default LoginMessage;
```

What is Middleware in Next.js?

1. Middleware
    1. It is a program that sits between two things
        1. Next.js - Middleware is a function that sits between the request and the response
            1. It allows us to run some code based on the request but before the response is completed
2. Middleware Details
    1. By default, middleware runs **before every route** in a project, but we can specify which paths using a **matcher**
        1. It always runs after the request, but before the route the user is visiting is rendered and sent back
    2. **Analogy**: Chunk of code that's in every page.js component (route)
        1. It is in one central place that runs before routes
    3. Only **one** middleware function: needs to be exported from **middleware.js** (or .ts) in the project **root folder** (not `app` folder)
3. Use Cases:
    1. Read and set cookies and headers
    2. Authentication and authorization
    3. Server-side analytics
    4. Redirect based on geolocation (for intnl)
    5. A/B testing
4. Middleware needs to **produce a response**: Two ways
    1. Redirects or rewrites
    2. Send response directly (usually JSON) to client
        1. Route is bypassed
            1. Use-Case: Send data response as JSON like API (we have route handlers as alternatives)

Protecting Routes with NextAuth Middleware

1. Code: middleware.js

```
import { NextResponse } from "next/server";

export function middleware(request) {
  console.log(request);

  return NextResponse.redirect(new URL("/about", request.url));
}

export const config = {
  matcher: ["/account", "/cabins"],
};
```

2. `auth` also acts as a middleware

    1. Code: middleware.js

```
import { auth } from "@/app/_lib/auth";
export const middleware = auth;

export const config = {
  matcher: ["/account"],
};
```

3. Code: auth.js

```
import NextAuth from "next-auth";
import Google from "next-auth/providers/google";

const authConfig = {
  providers: [
    Google({
      clientId: process.env.AUTH_GOOGLE_ID,
      clientSecret: process.env.AUTH_GOOGLE_SECRET,
    }),
  ],
  callbacks: {
    authorized({ auth, request }) {
      return !!auth?.user;
    },
  },
};

export const {
  auth,
  handlers: { GET, POST },
} = NextAuth(authConfig);
```

Building a Custom Sign in Page

1. We can keep the entire authentication flow on the server (no client-side authentication)

    1. Much better UX

        1. Code: auth.js

```
export const {
  auth,
  signIn,
  signOut,
  handlers: { GET, POST },
} = NextAuth(authConfig);
```

2. Server Actions are used to act interactivity to server components

3. Code: SignInButton.js

```
import { signInAction } from "../_lib/actions";

export const metadata = {
  title: "Login",
};

function SignInButton() {
  return (
    <form action={signInAction}>
      <button className="flex items-center gap-6 text-lg border border-primary-300 px-10 py-4 font-medium">
        <img
          src="https://authjs.dev/img/providers/google.svg"
          alt="Google logo"
          height="24"
          width="24"
        />
        <span>Continue with Google</span>
      </button>
    </form>
  );
}

export default SignInButton;
```

4. Code: actions.js

```
"use server";

import { signIn } from "./auth";

export async function signInAction() {
  await signIn("google", { redirectTo: "/account" }); // We can obtain provider dynamically
}
```

5. Code: auth.js

```
"use server";

import { signIn } from "./auth";

export async function signInAction() {
  await signIn("google", { redirectTo: "/account" }); // We can obtain provider dynamically
}
```

Building a Custom Sign Out Button

1. Server actions can also be called from a client, but they only get executed on the server
2. Code: actions.js

```
export async function signOutAction() {
  await signOut({ redirectTo: "/" });
}
```

3. Code: SignOutButton.js

```
import { ArrowRightOnRectangleIcon } from "@heroicons/react/24/solid";
import { signOutAction } from "../_lib/actions";

function SignOutButton() {
  return (
    <form action={signOutAction}>
      <button className="py-3 px-5 hover:bg-primary-900 hover:text-primary-100 transition-colors flex items-center gap-4 font-semibold text-primary-20
        <ArrowRightOnRectangleIcon className="h-5 w-5 text-primary-600" />
        <span>Sign out</span>
      </button>
    </form>
  );
}

export default SignOutButton;
```

Constructing a New Guest on First Sign In

1. Steps:
    1. Callback: singIn in config
        1. It runs before the actual sign-up process runs
            1. A middleware that performs operations associated with signing in process (after user puts in credentials but before actually loggin in to the application)
2. Code: auth.js

```
async session({ session, user }) {
  const guest = await getGuest(session.user.email);
  session.user.guestId = guest.id;
  return session;
},
```

1. Runs afer signIn callback and each time session is checked out (when we call `auth` function)
2. `session` - object
3. Why can't we do it inside `signIn`?
    1. Because `session` is not created yet.
        1. It is ready in the `session` callback

Section 37: Mutations with Server Actions + Modern React Hooks

Section Overview

1. Topics:
    1. **Server actions** for data mutations
        1. In RSCs
    2. **Implementing features**: Constructing, updating, and deleting a reservation + updating profile
    3. New cutting-edge **React hooks**
        1. For performance and interactivity
            1. `useFormStatus`
            2. `useTransition`
            3. `useOptimistic`

What are Server Actions?

1. Server Actions:
    1. Goal: To build interactive full-stack applications with React
        1. Solution: RSC architecture
            1. data Fetching: Server Components (most fundamental part of RSC architecture)
            2. Mutations: Server Actions
    2. The missing piece in the RSC architecture that enables **interactive full-stack applications** (CRUD)
    3. Async functions that run **exclusively on the server**, allowing us to perform **data mutations**
    4. Created with the "use server" directive at the top of the function or an entire module
        1. An async function in a **server** component.
            1. Can be used in component or passed to a **client** component (unlike functions)
                1. Can be used directly in a server component OR
                2. Can be passed as a prop in a client component
                    1. We cannot usually pass functions as props to client components
                        1. Server actions is an exception
        2. Standalone file: exported functions (multiple server actions) become server actions that can be imported into **any** component **[recommended]**
            1. All mutations are stored in a central place (it is okay to have multiple files)
        3. "use server" is only for server actions (not for SCs)
            1. It bridges the gap from client to server (allowing client to talk to the server)
                1. Like an API call
    5. Behind the scenes: Next.js constructs an **API endpoint** (with **URL** (each its own URL)) for each server action. Whenever a server action is called, a **POST request** is made to its URL (the function itself **never** reaches client)
        1. The code stays always on the server
            1. It is safe to connect to DBs, use secret keys, ...
        2. When a server action is invoked, it gets translated to a POST request with data automatically serialized
            1. The function abstracts API endpoint details
    6. We don't have to manually construct APIs or route handlers to mutate data
    7. Unlike server components, server actions actually require a **running web server**
        1. SCs can run at build-time

        2. Web server is required for the endpoints
    8. Server Actions can be called from:
        1. `action` attribute in a `<form>` element (in server and client components)
            1. The form can be in SC or CC
                1. No additional code on the front-end
            2. It sends all data in a serialized form to the API endpoint behind the scenes
        2. Event handlers (only client components)
        3. `useEffect` (only client components)
    9. In Server Actions, we can:
        1. Perform **data mutations** (creation, updation, deletion)
        2. **Update the UI with new data**: Revalidate cache with `revalidatePath` and `revalidateTag` (manually on demand)
            1. SAs are tightly integrated with Next.js caching and revalidation system
        3. Work with cookies
        4. Many more... (almost any code)
    10. Note: We need to ensure that the user is authorized to perform the action and we must assume that inputs are unsafe
    11. SAs cannot be defined in CCs

Updating the Profile Using a Server Action

1. `form` with `action` containing SA doesn't reload the page
2. `form` also passes all form data using native `formData` API
3. Code: actions.js

```
"use server";

import { auth, signIn, signOut } from "./auth";
import { supabase } from "./supabase";

export async function updateGuest(formData) {
  const session = await auth();

  if (!session) throw new Error("You must be logged in"); // caught by closest error boundary `Error.js`

  const nationalID = formData.get("nationalID");
  const [nationality, countryFlag] = formData.get("nationality").split("%");

  if (!/^[a-zA-Z0-9]{6,12}$/.test(nationalID))
    throw new Error("Please provide a valid national ID");

  const updateData = { nationality, countryFlag, nationalID };

  const { data, error } = await supabase
    .from("guests")
    .update(updateData)
    .eq("id", session.user.guestId);

  if (error) throw new Error("Guest could not be updated");
}

export async function signInAction() {
  await signIn("google", { redirectTo: "/account" });
}

export async function signOutAction() {
  await signOut({ redirectTo: "/" });
}
```

4. Code: account/profile/page.js

```
import SelectCountry from "@/app/_components/SelectCountry";
import UpdateProfileForm from "@/app/_components/UpdateProfileForm";
import { auth } from "@/app/_lib/auth";
import { getGuest } from "@/app/_lib/data-service";

export const metadata = {
  title: "Update profile",
};

export default async function Page() {
  const session = await auth();
  const guest = await getGuest(session.user.email);

  return (
    <div>
      <h2 className="font-semibold text-2xl text-accent-400 mb-4">
        Update your guest profile
      </h2>

      <p className="text-lg mb-8 text-primary-200">
        Providing the following information will make your check-in process
        faster and smoother. See you soon!
      </p>

      <UpdateProfileForm guest={guest}>
        <SelectCountry
          name="nationality"
          id="nationality"
          className="px-5 py-3 bg-primary-200 text-primary-800 w-full shadow-sm rounded-sm"
          defaultCountry={guest.nationality}
        />
      </UpdateProfileForm>
    </div>
  );
}
```

Manual Cache Revalidation

1. Clear cache and refill it

1. Time base revalidation - after certain defined time
2. On demand revalidation - re-fetch right away (whenever we need it)

2. Code: actions.js

```
export async function updateGuest(formData) {
    ...
    revalidatePath("/account/profile");
}
```

Displaying a Loading Indicator: The useFormStatus Hook

1. New React hook: `useFormStatus`
    1. It must be used in a component that is rendered inside a form
        1. A new component is created that is used inside the form
2. Code: ReservationForm.js

```
function ReservationForm({ cabin, user }) {
    ...
    <div className="flex justify-end items-center gap-6">
      <Button />
    </div>
    ...
}

function Button() {
  const { pending } = useFormStatus();

  return (
    <button
      className="bg-accent-500 px-8 py-4 text-primary-800 font-semibold hover:bg-accent-600 transition-all disabled:cursor-not-allowed disabled:bg-gra
      disabled={pending}
    >
      {pending ? "Updating..." : "Update profile"}
    </button>
  );
}
...
```

Building the Guest's Reservations Page

1. Code: reservations/page.js

```
import ReservationList from "@/app/_components/ReservationList";
import { auth } from "@/app/_lib/auth";
import { getBookings } from "@/app/_lib/data-service";

export const metadata = {
  title: "Reservations",
};

export default async function Page() {
  const session = await auth();
  const bookings = await getBookings(session.user.guestId);

  return (
    <div>
      <h2 className="font-semibold text-2xl text-accent-400 mb-7">
        Your reservations
      </h2>

      {bookings.length === 0 ? (
        <p className="text-lg">
          You have no reservations yet. Check out our{" "}
          <a className="underline text-accent-500" href="/cabins">
            luxury cabins &rarr;
          </a>
        </p>
      ) : (
        <ReservationList bookings={bookings} />
      )}
    </div>
  );
}
```

Deleting a Reservation

1. To define a server action in SC or CC:

```
function deleteReservation() {
  "use server";
  // code
}
```

2. `revalidateTag` - We can add a tag to some data. Only the data with tag will be revalidated
    1. `fetch` function needs to be used
3. Code: actions.js

```
...
export async function deleteReservation(bookingId) {
  const session = await auth();

  if (!session) throw new Error("You ust be logged in");

  const guestBookings = await getBookings(session.user.guestId);
  const guestBookingIds = guestBookings.map((booking) => booking.id);

  if (!guestBookingIds.includes(bookingId))
    throw new Error("You are not allowed to delete this booking");
```

```
  const { error } = await supabase
    .from("bookings")
    .delete()
    .eq("id", bookingId);

  if (error) {
    console.error(error);
    throw new Error("Booking could not be deleted");
  }

  revalidatePath("/account/reservations");
}
...
```

Another Loading Indicator: The useTransition Hook

1. `useTransition` - concurrent feature released in React 18
    1. Allows us to mark a state update as transition
        1. When it is marked using `useTransition` hook, the update will happen without blocking the UI (stays responsive during a re-render and indicates that a state transition is happening)
            1. Updates can be done in the background so that the UI stays responsive
    2. The hook can be used mark a server action as a transition
        1. To get indication that something is happening in the background
2. The hook is used for `buttons`
3. Code: deleteReservation.js

```
"use client";

import { TrashIcon } from "@heroicons/react/24/solid";
import { deleteReservation } from "../_lib/actions";
import { useTransition } from "react";
import SpinnerMini from "./SpinnerMini";

function DeleteReservation({ bookingId }) {
  const [isPending, startTransition] = useTransition();

  function handleDelete() {
    if (confirm("Are you sure you want to delete this reservation?"))
      startTransition(() => deleteReservation(bookingId)); // marks the function passed as a transition. While this work is pending, `isPending` becom
  }

  return (
    <button
      onClick={handleDelete}
      className="group flex items-center gap-2 uppercase text-xs font-bold text-primary-300 flex-grow px-3 hover:bg-accent-600 transition-colors hover
    >
      {isPending ? (
        <>
          <TrashIcon className="h-5 w-5 text-primary-600 group-hover:text-primary-800 transition-colors" />
          <span className="mt-1">Delete</span>
        </>
      ) : (
        <span className="mx-auto">
          <SpinnerMini />
        </span>
      )}
    </button>
  );
}

export default DeleteReservation;
```

CHALLENGE #1: Updating a Reservation

Removing Reservations Immediately: The useOptimistic Hook

1. Optimistic UI - Technique to improve percieved performance
    1. We assume that a certain async operation is successful before it even finished (while it is working on the background)
2. Example: Remove the reservation from the UI as soon as Delete button is clicked
    1. If the operation didn't succeed, the data will be revalidated and fetched
        1. Makes app feel faster and more responsive
            1. Avoids too many spinner
            2. Snappy experience
3. `useOptimistic`
    1. Allows us to show new state while an async operation is happening
4. Code: ReservationList.js

```
"use client";

import { deleteReservation } from "../_lib/actions";
import ReservationCard from "./ReservationCard";
import { useOptimistic } from "react";

function ReservationList({ bookings }) {
  const [optimisticBookings, optimisticDelete] = useOptimistic(
    bookings,
    (curBookings, bookingId) => {
      return curBookings.filter((booking) => booking.id !== bookingId);
    }
  ); // two states. Actual state, optimistic state. Initial state & updating function
  // `bookings` - current state that will be rendered initially
  // `() => {}` - state update function that determines the next optimistic state. Takes current state & new info required to compute the optimistic s
  // `optimisticDelete` - Sumilar to the dispatch function. It triggers the optimistic operation

  async function handleDelete(bookingId) {
    // async operation that runs in the background
    optimisticDelete(bookingId); // optimistic operation (optimistically deleting first)
```

```
    await deleteReservation(bookingId); // actual operation
  }

  return (
    <ul className="space-y-6">
      {optimisticBookings.map((booking) => (
        <ReservationCard
          booking={booking}
          key={booking.id}
          onDelete={handleDelete}
        />
      ))}
    </ul>
  );
}

export default ReservationList;
```

Back to the Cabin Page: Finishing the Date Selector

1. Code: DateSelector.js

```
"use client";

import {
  differenceInDays,
  isPast,
  isSameDay,
  isWithinInterval,
} from "date-fns";
import { DayPicker } from "react-day-picker";
import "react-day-picker/dist/style.css";
import { useReservation } from "./ReservationContext";

function isAlreadyBooked(range, datesArr) {
  return (
    range.from &&
    range.to &&
    datesArr.some((date) =>
      isWithinInterval(date, { start: range.from, end: range.to })
    )
  );
}

function DateSelector({ settings, cabin, bookedDates }) {
  const { range, setRange, resetRange } = useReservation();

  const displayRange = isAlreadyBooked(range, bookedDates) ? {} : range;

  const { regularPrice, discount } = cabin;
  const { minBookingLength, maxBookingLength } = settings;

  const numNights = differenceInDays(displayRange.to, displayRange.from);

  const cabinPrice = numNights * (regularPrice - discount);

  return (
    <div className="flex flex-col justify-between">
      <DayPicker
        className="pt-12 place-self-center"
        mode="range"
        onSelect={setRange}
        selected={displayRange}
        min={minBookingLength + 1}
        max={maxBookingLength}
        fromMonth={new Date()}
        fromDate={new Date()}
        toYear={new Date().getFullYear() + 5}
        captionLayout="dropdown"
        numberOfMonths={2}
        disabled={(curDate) =>
          isPast(curDate) ||
          bookedDates.some((date) => isSameDay(date, curDate))
        }
      />

      <div className="flex items-center justify-between px-8 bg-accent-500 text-primary-800 h-[72px]">
        <div className="flex items-baseline gap-6">
          <p className="flex gap-2 items-baseline">
            {discount > 0 ? (
              <>
                <span className="text-2xl">${regularPrice - discount}</span>
                <span className="line-through font-semibold text-primary-700">
                  ${regularPrice}
                </span>
              </>
            ) : (
              <span className="text-2xl">${regularPrice}</span>
            )}
            <span className="">/night</span>
          </p>
          {numNights ? (
            <>
              <p className="bg-accent-600 px-3 py-2 text-2xl">
                <span>&times;</span> <span>{numNights}</span>
              </p>
              <p>
                <span className="text-lg font-bold uppercase">Total</span>{" "}
                <span className="text-2xl font-semibold">${cabinPrice}</span>
              </p>
            </>
          ) : null}
        </div>
```

```
        {range.from || range.to ? (
          <button
            className="border border-primary-800 py-2 px-4 text-sm font-semibold"
            onClick={resetRange}
          >
            Clear
          </button>
        ) : null}
      </div>
    </div>
  );
}

export default DateSelector;
```

Constructing a New Reservation

1. Code: actions.js

```
"use server";

import { revalidatePath } from "next/cache";
import { auth, signIn, signOut } from "./auth";
import { supabase } from "./supabase";
import { getBookings } from "./data-service";
import { redirect } from "next/navigation";

export async function updateGuest(formData) {
  const session = await auth();

  if (!session) throw new Error("You must be logged in"); // caught by closest error boundary `Error.js`

  const nationalID = formData.get("nationalID");
  const [nationality, countryFlag] = formData.get("nationality").split("%");

  if (!/^[a-zA-Z0-9]{6,12}$/.test(nationalID))
    throw new Error("Please provide a valid national ID");

  const updateData = { nationality, countryFlag, nationalID };

  const { data, error } = await supabase
    .from("guests")
    .update(updateData)
    .eq("id", session.user.guestId);

  if (error) throw new Error("Guest could not be updated");

  revalidatePath("/account/profile");
}

export async function deleteBooking(bookingId) {
  const session = await auth();

  if (!session) throw new Error("You ust be logged in");

  const guestBookings = await getBookings(session.user.guestId);
  const guestBookingIds = guestBookings.map((booking) => booking.id);

  if (!guestBookingIds.includes(bookingId))
    throw new Error("You are not allowed to delete this booking");

  const { error } = await supabase
    .from("bookings")
    .delete()
    .eq("id", bookingId);

  if (error) {
    console.error(error);
    throw new Error("Booking could not be deleted");
  }

  revalidatePath("/account/reservations");
}

export async function createBooking(bookingData, formData) {
  // bound data is sent as `bookingData`, and the remaining form data is sent as `formData`
  const session = await auth();

  if (!session) throw new Error("You ust be logged in");

  // Object.entries(formData.entries); // for huge form data

  const newBooking = {
    ...bookingData,
    guestId: session.user.guestId,
    numGuests: formData.get("numGuests"),
    observations: formData.get("observations").slice(0, 1000),
    extrasPrice: 0,
    totalPrice: bookingData.cabinPrice,
    isPaid: false,
    hasBreakfast: false,
    status: "unconfirmed",
  };

  // zod - library to validate input fields

  // console.log(newBooking);

  const { error } = await supabase.from("bookings").insert([newBooking]);

  if (error) throw new Error("Booking could not be created");
```

```
  revalidatePath(`/cabins/${bookingData.cabinId}`); // clears browser cache (router cache), for static sites: data cache, full-route cache (on server)

  redirect("/cabins/thankyou");
}

export async function signInAction() {
  await signIn("google", { redirectTo: "/account" });
}

export async function signOutAction() {
  await signOut({ redirectTo: "/" });
}
```

2. Code: ReservationForm.js

```
"use client";

import { differenceInDays } from "date-fns";
import { useFormStatus } from "react-dom";
import { createBooking } from "../_lib/actions";
import { useReservation } from "./ReservationContext";
import { revalidatePath } from "next/cache";
import SubmitButton from "./SubmitButton";

function ReservationForm({ cabin, user }) {
  const { range, resetRange } = useReservation();
  const { maxCapacity, regularPrice, discount, id } = cabin;

  const startDate = range.from;
  const endDate = range.to;

  const numNights = differenceInDays(endDate, startDate);
  const cabinPrice = numNights * (regularPrice - discount);

  const bookingData = {
    startDate,
    endDate,
    numNights,
    cabinPrice,
    cabinId: id,
  };

  const createBookingWithData = createBooking.bind(null, bookingData);

  return (
    <div className="scale-[1.01]">
      <div className="bg-primary-800 text-primary-300 px-16 py-2 flex justify-between items-center">
        <p>Logged in as</p>

        <div className="flex gap-4 items-center">
          <img
            // Important to display google profile images
            referrerPolicy="no-referrer"
            className="h-8 rounded-full"
            src={user.image}
            alt={user.name}
          />
          <p>{user.name}</p>
        </div>
      </div>

      <form
        // action={createBookingWithData}
        action={async (formData) => {
          await createBookingWithData(formData);
          resetRange();
        }}
        className="bg-primary-900 py-10 px-16 text-lg flex gap-5 flex-col"
      >
        <div className="space-y-2">
          <label htmlFor="numGuests">How many guests?</label>
          <select
            name="numGuests"
            id="numGuests"
            className="px-5 py-3 bg-primary-200 text-primary-800 w-full shadow-sm rounded-sm"
            required
          >
            <option value="" key="">
              Select number of guests...
            </option>
            {Array.from({ length: maxCapacity }, (_, i) => i + 1).map((x) => (
              <option value={x} key={x}>
                {x} {x === 1 ? "guest" : "guests"}
              </option>
            ))}
          </select>
        </div>

        <div className="space-y-2">
          <label htmlFor="observations">
            Anything we should know about your stay?
          </label>
          <textarea
            name="observations"
            id="observations"
            className="px-5 py-3 bg-primary-200 text-primary-800 w-full shadow-sm rounded-sm"
            placeholder="Any pets, allergies, special requirements, etc.?"
          />
        </div>

        {/* <div className="flex justify-end items-center gap-6">
          <Button />
        </div> */}
```

```
        <div className="flex justify-end items-center gap-6">
          {!startDate && !endDate ? (
            <p className="text-primary-300 text-base">
              Start by selecting dates
            </p>
          ) : (
            <SubmitButton pendingLabel="Reserving...">
              Reserver now
            </SubmitButton>
          )}
        </div>
      </form>
    </div>
  );
}

function Button() {
  const { pending } = useFormStatus();

  return (
    <button
      className="bg-accent-500 px-8 py-4 text-primary-800 font-semibold hover:bg-accent-600 transition-all disabled:cursor-not-allowed disabled:bg-gra
      disabled={pending}
    >
      {pending ? "Updating..." : "Update profile"}
    </button>
  );
}

export default ReservationForm;
```

Section 38: Deployment with Vercel

Section Overview

1. Topics:
    1. Deploy "The Wild Oasis" website to Vercel
    2. Set up environment variables and authentication credentials (a little challenging)

Setting up the GitHub Repository

1. `git init`
2. `git add A`
3. `git commit -m "Initial commit"`
4. Github: `the-wild-oasis-website`
    1. No gitignore, README, ...
5. Pushing the code:

```
git remote add origin https://github.com/bmutthoju/the-wild-oasis-website.git
git branch -M main
git push -u origin main
```

Deploying to Vercel

1. https://vercel.com/pricing
    1. Cheaper alternatives (https://opennext.js.org/)
        1. Cloudflare
        2. Netlify
    2. Nextjs.com - Vercel
        1. Has optimizations out of the box
            1. Image optimization
            2. Cache
            3. ...
2. Deployment
    1. New project
        1. Select project
        2. Environment Veriables
            1. Copy all in `.env.local`
            2. Paste
        3. Deploy
    2. Continue to dashboard
        1. Domain

Updating Environment Variables and OAuth Credentials

1. Copy URL
    1. Settings > Environment Variables
        1. `NEXTAUTH_URL` - paste the URL
    2. Needs redeployment
        1. Env variables are loaded at the beginning when the app is deployed
        2. Redeploy
2. Google Developer Console
    1. APIs & Services
        1. Credentials
            1. Authorized JavaScript origins:
                1. URIs
                    1. ADD URI: paste the URL
            2. Authorized redirect URIs
                1. URIs
                    1. ADD URI: paste the URL + `/api/auth/callback/google`
        2. Save

3. Features of Vercel
    1. Web Analytics
        1. Enable
            1. Requires a few changes in the code
                1. Github push will automatically redeploy the application
    2. Speed insights
    3. Logs
        1. Every request made to the page

Section 39: [OPTIONAL] Legacy Next.js: The "Pages" Router

Section Overview

Setting up Another Project

Routes, Pages, and Navigation

Dynamic Routes

Constructing a Layout with a Custom _App

Constructing Pages

Defining Page Title and Favicon

Fetching Data with getStaticProps (SSG)

Fetching Data with getServerSideProps (SSR)

API Routes

Handling Form Submissions

Section 40: The End!

Where to Go from Here

1. Tips:
    1. Practice, Practice, Practice!
        1. Dive deeper
    2. Think of creative project ideas
        1. Start simple and gradually increase complexity
    3. Copy other apps
    4. Write as much code as possible
        1. Can be shown to potential employers
    5. Never stop learning
        1. New trends
            1. To know what's going on in community
                1. Free newsletter (news, tutorials, ...)
                    1. jonas.io
    6. Other skills
        1. TS
        2. Motion - animation
        3. Testing
        4. Material UI - component library